

EJEMPLAR DE LA OBRA — CÓDIGO FUENTE COMPLETO

Plataforma Digital de Pedidos

Autor y titular: Jose Miguel Gonzalez Domingo

Código fuente completo: 130 ficheros, 14257 líneas

```

/* ===== */
/* FICHERO: montadito-magic-flow/src/pages/ClientApp.tsx */
/* ===== */
import { useState, useCallbck, useEffect, useRef } from 'react';
import { useSearchParams } from 'react-router-dom';
import { X } from 'lucide-react';
import { ClientHeader } from '@components/client/ClientHeader';
import { LocalSelector } from '@components/client/LocalSelector';
import { useActiveLocal } from '@lib/active-local';
import { CategoryGrid } from '@components/client/CategoryGrid';
import { SectionTabs } from '@components/client/SectionTabs';
import { ProductCard } from '@components/client/ProductCard';
import { MontyRuedaCard } from '@components/client/MontyRuedaCard';
import { CartSheet } from '@components/client/CartSheet';
import { AlcoholWarningSheet } from '@components/client/AlcoholWarningSheet';
import { AllergenFilterSheet } from '@components/client/AllergenFilterSheet';
import { AllergenLegalNoticeFloat } from '@components/client/AllergenLegalNotice';
import { OrderTracking } from '@components/client/OrderTracking';
import { MenuChatBot } from '@components/client/MenuChatBot';
import { WelcomeScreen } from '@components/client/WelcomeScreen';
import { MontyLoader } from '@components/client/MontyLoader';
import { useCategories, useProducts } from '@hooks/use-menu';
import { useAllergens } from '@hooks/use-allergens';
import { useLocalRestricciones, buildRestriccionFilter } from '@hooks/use-local-restricciones';
import { useAgotados } from '@hooks/use-agotados';
import { useCartStore } from '@lib/cart-store';
import { useAllergenFilter } from '@lib/allergen-filter';
import { supabase } from '@integrations/supabase/client';
import { EmbeddedCheckout } from '@components/client/EmbeddedCheckout';
import { toast } from 'sonner';
import { registerPushServiceWorker, usePushSubscription } from '@hooks/use-push-notifications';

const WELCOME_KEY = 'monty-welcome-seen';
const ACTIVE_ORDER_KEY = 'monty-active-order';
const BEBIDAS_CATEGORY_NAME = 'Bebidas';
const INITIAL_STAFF_STATUS = 'pendiente_pago' as const;

const MONTADITO_SECTIONS = [
  'De la casa',
  'Clásicos',
  'Imprescindibles',
  'Especiales',
  'MontyCookie',
  'MontyDinas',
  'MontyPerros',
  'MontyBurgers',
  'MontyPizzas',
  'MontyGourmet',
] as const;

const BEBIDAS_SECTIONS = [
  // Jarras y cervezas arriba del todo
  'Jarras Heladas',
  'Cerveza Premium',
  'Cerveza en Botella',
  // Luego las bebidas clásicas
  'Clásicas',
  // Resto
  'Energéticas',
  'Tardeo Chill',
  'Tardeo Premium',
  'Vino',
  'Café e Infusiones',
] as const;

const APERITIVOS_SECTIONS = [
  'Aperitivos',
  'Para picar',
] as const;

type ActiveOrder = {
  pedidoId: string;
  numeroPedido: number;
  hasCocina: boolean;
  hasBebidas: boolean;
  sessionId?: string;
  total?: number;
};

const loadActiveOrder = (): ActiveOrder | null => {
  try {
    const raw = localStorage.getItem(ACTIVE_ORDER_KEY);
    if (!raw) return null;
    const parsed = JSON.parse(raw);
    // Only pedidoId is required - numeroPedido can be 0 when the DB trigger hasn't fired yet
    if (parsed?.pedidoId) {
      return {
        pedidoId: parsed.pedidoId,
        numeroPedido: parsed.numeroPedido ?? 0,
        hasCocina: parsed.hasCocina ?? true,
      };
    }
  }

```

```

        hasBebidas: parsed.hasBebidas ?? false,
        sessionId: parsed.sessionId,
        total: parsed.total,
    };
}
if (parsed?.cocina || parsed?.bebidas) {
    const ref = parsed.cocina || parsed.bebidas;
    return {
        pedidoId: ref.pedidoId,
        numeroPedido: ref.numeroPedido,
        hasCocina: !!parsed.cocina,
        hasBebidas: !!parsed.bebidas,
        sessionId: ref.sessionId,
    };
}
return null;
} catch {
    return null;
}
};

const ClientApp = () => {
    const [searchParams] = useSearchParams();

    // Escape hatch: ?reset=1 limpia localStorage y recarga.
    // Útil cuando el app queda bloqueado por un pedido corrupto en storage.
    if (searchParams.get('reset') === '1') {
        localStorage.removeItem(ACTIVE_ORDER_KEY);
        window.location.replace(window.location.pathname);
        return null;
    }

    const localSlugFromUrl = searchParams.get('local');
    const mesaNum = searchParams.get('mesa');

    const activeLocal = useActiveLocal((s) => s.local);
    const setActiveLocal = useActiveLocal((s) => s.setLocal);

    const [activeCategory, setActiveCategory] = useState<string | null>(null);
    const [activeSection, setActiveSection] = useState<string | null>(null);
    const [cartOpen, setCartOpen] = useState(false);
    const [welcomeOpen, setWelcomeOpen] = useState(false);
    const [showLocalSelector, setShowLocalSelector] = useState(false);
    const [orderState, setOrderState] = useState<ActiveOrder | null>(() => {
        // Recuperación sincrónica en el primer render para evitar flash de LocalSelector
        // cuando iOS borra localStorage tras el redirect cross-domain de Apple Pay.
        const urlParams = new URLSearchParams(window.location.search);
        if (urlParams.get('pago') !== 'ok') return null;
        const saved = loadActiveOrder();
        if (saved) return saved;
        const pid = urlParams.get('pid');
        const num = urlParams.get('num');
        const sid = urlParams.get('sid');
        if (!pid || num === null) return null;
        const recovered: ActiveOrder = {
            pedidoId: pid,
            numeroPedido: parseInt(num, 10),
            hasCocina: urlParams.get('cocina') !== '0',
            hasBebidas: urlParams.get('bebidas') === '1',
            sessionId: sid ?? undefined,
        };
    });
    localStorage.setItem(ACTIVE_ORDER_KEY, JSON.stringify(recovered));
    return recovered;
});
const [pendingCheckout, setPendingCheckout] = useState<ActiveOrder | null>(null);
const [resumableOrder, setResumableOrder] = useState<ActiveOrder | null>(null);
// Si venimos del redirect de Apple Pay (?pago=ok), empezamos sin recovering
// para que MontyLoader NUNCA bloquee la pantalla. El efecto pago=ok restaura el pedido.
const [recovering, setRecovering] = useState(() => {
    const urlParams = new URLSearchParams(window.location.search);
    if (urlParams.get('pago') === 'ok') return false;
    return !!loadActiveOrder()?.pedidoId;
});
const [isCheckedOut, setIsCheckingOut] = useState(false);
const [filterOpen, setFilterOpen] = useState(false);
const { data: allAllergens = [] } = useAllergens();
const excluded = useAllergenFilter((s) => s.excluded);
const toggleExcluded = useAllergenFilter((s) => s.toggle);

// Refs para el handler de back button (evita closures obsoletos)
const pendingCheckoutRef = useRef<ActiveOrder | null>(pendingCheckout);
const activeCategoryRef = useRef<string | null>(activeCategory);
const activeSectionRef = useRef<string | null>(activeSection);
const handlePaymentCancelRef = useRef<() => void>(() => {});
useEffect(() => { pendingCheckoutRef.current = pendingCheckout; }, [pendingCheckout]);
useEffect(() => { activeCategoryRef.current = activeCategory; }, [activeCategory]);
useEffect(() => { activeSectionRef.current = activeSection; }, [activeSection]);

// Botón atrás del móvil – mapea a navegación interna
// Siempre hay una entrada en el historial para interceptar el back.
// Después de cada navegación se re-inserta la guardia para que el
// siguiente back también sea interceptado (sin esto, solo funciona una vez).

```

```

useEffect(() => {
  window.history.pushState({ step: 'root' }, '');

  const onPopState = () => {
    if (pendingCheckoutRef.current) {
      // Opción A: volver al menú con el pedido recuperable, sin cancelar
      setResumableOrder(pendingCheckoutRef.current);
      setPendingCheckout(null);
      setActiveCategory(null);
      setActiveSection(null);
      window.history.pushState({ step: 'root' }, '');
    } else if (activeSectionRef.current) {
      setActiveSection(null);
      window.history.pushState({ step: 'category' }, '');
    } else if (activeCategoryRef.current) {
      setActiveCategory(null);
      window.history.pushState({ step: 'root' }, '');
    } else {
      window.history.pushState({ step: 'root' }, '');
    }
  };

  window.addEventListener('popstate', onPopState);
  return () => window.removeEventListener('popstate', onPopState);
}, []);

useEffect(() => { registerPushServiceWorker(); }, []);

// Recuperar sesión al arrancar: verificar en DB el estado real del pedido guardado
useEffect(() => {
  const saved = loadActiveOrder();
  if (!saved?.pedidoId) { setRecovering(false); return; }

  // Venimos del redirect de Apple Pay / Google Pay (/?pago=ok).
  // Llamamos confirm-payment para que el pedido salga de pendiente_pago inmediatamente
  // sin depender del timing del webhook de Stripe.
  const params = new URLSearchParams(window.location.search);
  if (params.get('pago') === 'ok') {
    const paymentIntentId = params.get('payment_intent');
    if (paymentIntentId && saved.pedidoId && saved.sessionId) {
      supabase.functions.invoke('confirm-payment', {
        body: { pedido_id: saved.pedidoId, session_id: saved.sessionId, payment_intent_id: paymentIntentId },
      }).catch((e) => console.error('[confirm-payment redirect]', e));
    }
    setOrderState(saved);
    setRecovering(false);
    return;
  }

  // Timeout de seguridad: si la query DB tarda más de 5 s, mostramos el
  // pedido igualmente. Evita la pantalla beige infinita por red lenta.
  const safetyTimer = window.setTimeout(() => {
    setOrderState(saved);
    setRecovering(false);
  }, 5000);

  (async () => {
    try {
      // RLS exige la cabecera de sesión: sin ella la lectura devuelve 0 filas
      // y el pedido (ya pagado!) se interpretaba como inexistente y se borraba.
      const { data: pedido, error } = await supabase
        .from('pedidos')
        .select('estado')
        .eq('id', saved.pedidoId)
        .setHeader('x-session-id', saved.sessionId ?? '')
        .maybeSingle();
      // Regla de oro: NUNCA borrar el pedido local ante una lectura ambigua
      // (error de red / RLS). Solo se descarta ante un estado terminal explícito.
      if (error || !pedido) {
        setOrderState(saved);
      } else if (pedido.estado === 'cancelado' || pedido.estado === 'entregado') {
        localStorage.removeItem(ACTIVE_ORDER_KEY);
      } else if (pedido.estado === 'pendiente_pago') {
        // Con activeLocal null (iOS borró localStorage tras redirect Apple Pay),
        // ir directo a OrderTracking evita quedarse bloqueado en LocalSelector.
        // OrderTracking muestra "Pago pendiente" + botón "Reintentar pago".
        if (!activeLocal) {
          setOrderState(saved);
        } else {
          setResumableOrder(saved);
        }
      } else {
        setOrderState(saved);
      }
    } catch {
      // Sin red: mostrar seguimiento (OrderTracking se autocorriga al reconectar)
      setOrderState(saved);
    } finally {
      window.clearTimeout(safetyTimer);
      setRecovering(false);
    }
  })();

```

```

// eslint-disable-next-line react-hooks/exhaustive-deps
}, []);

// If a `local` slug is provided in the URL (e.g. QR), bind that local automatically.
useEffect(() => {
  if (!localSlugFromUrl) return;
  if (activeLocal) return;
  (async () => {
    const { data } = await supabase
      .from('locales')
      .select('id, nombre, ciudad, direccion')
      .eq('slug', localSlugFromUrl)
      .eq('activo', true)
      .maybeSingle();
    if (data) setActiveLocal(data as any);
  })();
}, [localSlugFromUrl, activeLocal, setActiveLocal]);

useEffect(() => {
  if (!activeLocal) return;
  if (!localStorage.getItem(WELCOME_KEY)) {
    setWelcomeOpen(true);
  }
}, [activeLocal]);

// Al volver de Stripe (Apple Pay redirect), mostrar feedback y restaurar el pedido.
useEffect(() => {
  const pago = searchParams.get('pago');
  if (!pago) return;
  if (pago === 'ok') {
    toast.success('¡Pago realizado con éxito!');

    // Stripe añade payment_intent al return_url en redirects (Apple Pay, Google Pay).
    // Lo usamos para llamar confirm-payment inmediatamente y sacar el pedido de
    // pendiente_pago sin esperar al webhook – esto soluciona la pantalla en blanco.
    const paymentIntentId = searchParams.get('payment_intent');
    const pidFromUrl = searchParams.get('pid');
    const sidFromUrl = searchParams.get('sid');
    if (paymentIntentId && pidFromUrl && sidFromUrl) {
      supabase.functions.invoke('confirm-payment', {
        body: { pedido_id: pidFromUrl, session_id: sidFromUrl, payment_intent_id: paymentIntentId },
      }).catch((e) => console.error('[confirm-payment redirect]', e));
    }

    const saved = loadActiveOrder();
    if (saved) {
      setOrderState(saved);
    } else {
      // Fallback: iOS Safari a veces borra localStorage durante el redirect cross-domain.
      // Recuperamos desde los params que EmbeddedCheckout codificó en el return_url.
      const numFromUrl = searchParams.get('num');
      const cocinaFromUrl = searchParams.get('cocina');
      const bebidasFromUrl = searchParams.get('bebidas');
      if (pidFromUrl && numFromUrl !== null) {
        const recovered: ActiveOrder = {
          pedidoId: pidFromUrl,
          numeroPedido: parseInt(numFromUrl, 10),
          hasCocina: cocinaFromUrl !== '0',
          hasBebidas: bebidasFromUrl === '1',
          sessionId: sidFromUrl ?? undefined,
        };
        localStorage.setItem(ACTIVE_ORDER_KEY, JSON.stringify(recovered));
        setOrderState(recovered);
      }
    }
  } else if (pago === 'cancel') {
    toast.error('Pago cancelado');
    localStorage.removeItem(ACTIVE_ORDER_KEY);
  }
  // limpiar query params
  window.history.replaceState({}, '', window.location.pathname);
}, [searchParams]);

const closeWelcome = () => {
  localStorage.setItem(WELCOME_KEY, '1');
  setWelcomeOpen(false);
};

const { subscribe } = usePushSubscription();

const { data: categories = [], isLoading: loadingCategories } = useCategories();
const { data: rawProducts = [] } = useProducts(activeCategory || undefined);
// Montaditos (para resolver los componentes de las MontyRuedas). Misma queryKey
// que cuando la categoría activa es Montaditos → react-query lo deduplica.
const montaditosCatId = (categories as any[]).find((c) => c.nombre === 'Montaditos')?.id;
const { data: allMontaditos = [] } = useProducts(montaditosCatId);
const ruedaMontaditos = (allMontaditos as any[]).map((p) => ({ id: p.id, numero: p.numero ?? null, nombre: p.nombre }));
const { data: restricciones = [] } = useLocalRestricciones();
const { isProductoAllowed, isSeccionAllowed: isSeccionAllowedDB } = buildRestriccionFilter(restricciones);
const { isProductoAgotado, isIngredienteAgotado } = useAgotados();

const now = new Date();

```

```

const isAfter17 = now.getHours() >= 17;
const CAFE_SECTION = 'Café e Infusiones';

// After 17:00: keep Café e Infusiones visible (DB hides it entirely, we override to show only Agua)
const isSeccionAllowed = (sec: string) => {
  if (isAfter17 && sec === CAFE_SECTION) return true;
  return isSeccionAllowedDB(sec);
};

const products = (rawProducts as any[]).filter((p) => {
  if (!isProductoAllowed(p.id)) return false;
  // After 17:00 en Café e Infusiones: solo Agua y los 3 zumos en botella (B41-B43)
  if (isAfter17 && p.seccion === CAFE_SECTION) {
    const nombre: string = (p.nombre ?? '').normalize('NFD').replace(/[~]/g, '').toLowerCase();
    return nombre.includes('agua') || ['B41', 'B42', 'B43'].includes(p.numero);
  }
  return true;
});
const cart = useCartStore();

const selectedCategory = categories.find((c: any) => c.id === activeCategory);

const showLoader = loadingCategories && !welcomeOpen && !activeLocal;

const handleCheckout = useCallback(async (ageVerified: boolean) => {
  if (cart.items.length === 0) return;
  if (isCheckingOut) return;
  if (!activeLocal) {
    toast.error('Selecciona un local primero');
    setShowLocalSelector(true);
    return;
  }
}

setIsCheckingOut(true);
try {
  const localId = activeLocal.id;
  const sessionId = cart.sessionId?.trim();
  if (!sessionId) throw new Error('Sesión de pedido no disponible');
  // Fetch categories for items in cart to split Bebidas vs cocina with the exact shared rule.
  const productIds = Array.from(new Set(cart.items.map((i) => i.productoId)));
  const { data: prodCats, error: prodErr } = await supabase
    .from('menu_productos')
    .select('id, nombre, categoria_id, menu_categorias(nombre)')
    .in('id', productIds);
  if (prodErr) throw prodErr;

  // Normalize a string: remove diacritics, lowercase, trim
  const norm = (s: string) =>
    s.normalize('NFD').replace(/[~]/g, '').toLowerCase().trim();

  // Aperitivos servidos en barra – matched by substring to be resilient to
  // exact DB name variants (e.g. "Gilda boqueron" / "Gilda anchoa" / "Gildas")
  const isAperitivoBarra = (nombre: string) => {
    const n = norm(nombre);
    return n.includes('aceituna') || n.includes('gilda') || n.includes('cucurucho');
  };

  const isBebida = (productoId: string) => {
    const p = prodCats?.find((x) => x.id === productoId);
    if ((p?.menu_categorias as any)?.nombre === BEBIDAS_CATEGORY_NAME) return true;
    if (p?.nombre && isAperitivoBarra(p.nombre)) return true;
    return false;
  };

  const hasBebidas = cart.items.some((i) => isBebida(i.productoId));
  const hasCocina = cart.items.some((i) => !isBebida(i.productoId));

  // Determinar estados iniciales según las partes del pedido
  const total = cart.items.reduce((s, i) => s + i.precio * i.cantidad, 0);
  const { data: pedido, error: pErr } = await supabase
    .from('pedidos')
    .insert({
      local_id: localId,
      numero_pedido: 0,
      total,
      session_id: sessionId,
      estado: INITIAL_STAFF_STATUS,
      tipo: hasCocina && hasBebidas ? 'mixto' : (hasBebidas ? 'bebidas' : 'cocina'),
      estado_cocina: hasCocina ? INITIAL_STAFF_STATUS : null,
      estado_bebidas: hasBebidas ? INITIAL_STAFF_STATUS : null,
      edad_verificada_cliente: ageVerified,
    } as any)
    .select()
    .setHeader('x-session-id', sessionId)
    .single();
  if (pErr) throw pErr;

  const itemRows = cart.items.map((item) => ({
    pedido_id: pedido.id,
    producto_id: item.productoId,
    cantidad: item.cantidad,
  }

```

```

    precio_unitario: item.precio,
    destino: isBebida(item.productoId) ? 'bebidas' : 'cocina',
    // Tamaño/variante elegido (ej. "Jarra Quijote" / "Jarra Sancho") o, en las
    // MontyRuedas, los montaditos que la componen (uno por línea) para cocina.
    variante: (item.componentes && item.componentes.length
      ? item.componentes.join('\n')
      : item.variantLabel) ?? null,
  }));

const { error: iErr } = await supabase
  .from('pedido_items')
  .insert(itemRows as any)
  .setHeader('x-session-id', sessionId);
if (iErr) throw iErr;

const newOrder: ActiveOrder = {
  pedidoId: pedido.id,
  numeroPedido: pedido.numero_pedido,
  hasCocina,
  hasBebidas,
  sessionId,
  total,
};

// Persistimos para soportar recarga durante el pago
localStorage.setItem(ACTIVE_ORDER_KEY, JSON.stringify(newOrder));
cart.clearCart();
setCartOpen(false);
setPendingCheckout(newOrder);
unsubscribe(sessionId, pedido.id).catch(() => {});

} catch (err) {
  console.error('[checkout error]', err);
  const detail =
    err instanceof Error ? err.message : JSON.stringify(err);
  toast.error('Error al crear el pedido: ${detail}');
} finally {
  setIsCheckingOut(false);
}
}, [cart, activeLocal, isCheckingOut]);

const handlePaymentSuccess = useCallback(() => {
  if (!pendingCheckout) return;
  setOrderState(pendingCheckout);
  setPendingCheckout(null);
}, [pendingCheckout]);

const handlePaymentCancel = useCallback(async () => {
  if (pendingCheckout?.pedidoId && pendingCheckout?.sessionId) {
    try {
      await supabase.rpc('cancel_own_pending_order', { _pedido_id: pendingCheckout.pedidoId } as any)
        .setHeader('x-session-id', pendingCheckout.sessionId);
    } catch (e) { console.error(e); }
  }
  localStorage.removeItem(ACTIVE_ORDER_KEY);
  setPendingCheckout(null);
}, [pendingCheckout]);

// Mantener ref siempre actualizado con la versión más reciente
useEffect(() => { handlePaymentCancelRef.current = handlePaymentCancel; });

const handlePaymentBackToMenu = useCallback(() => {
  if (pendingCheckout) {
    setResumableOrder(pendingCheckout);
    setPendingCheckout(null);
  }
}, [pendingCheckout]);

const handleCancelResumable = useCallback(async () => {
  if (resumableOrder?.pedidoId && resumableOrder?.sessionId) {
    try {
      await supabase.rpc('cancel_own_pending_order', { _pedido_id: resumableOrder.pedidoId } as any)
        .setHeader('x-session-id', resumableOrder.sessionId);
    } catch (e) { console.error(e); }
  }
  localStorage.removeItem(ACTIVE_ORDER_KEY);
  setResumableOrder(null);
}, [resumableOrder]);

const handleRetryFromTracking = useCallback(() => {
  if (orderState) setPendingCheckout(orderState);
}, [orderState]);

const handleOrderComplete = useCallback(() => {
  localStorage.removeItem(ACTIVE_ORDER_KEY);
  setOrderState(null);
}, []);

if (recovering) {
  return <MontyLoader />;
}

```

```

if (pendingCheckout) {
  return (
    <EmbeddedCheckout
      pedidoId={pendingCheckout.pedidoId}
      sessionId={pendingCheckout.sessionId ?? ''}
      numeroPedido={pendingCheckout.numeroPedido}
      hasCocina={pendingCheckout.hasCocina}
      hasBebidas={pendingCheckout.hasBebidas}
      total={pendingCheckout.total ?? 0}
      onSuccess={handlePaymentSuccess}
      onCancel={handlePaymentBackToMenu}
    />
  );
}

if (orderState) {
  return (
    <OrderTracking
      pedidoId={orderState.pedidoId}
      numeroPedido={orderState.numeroPedido}
      hasCocina={orderState.hasCocina}
      hasBebidas={orderState.hasBebidas}
      sessionId={orderState.sessionId}
      onNewOrder={handleOrderComplete}
      onRetryPayment={handleRetryFromTracking}
    />
  );
}

// Gate: no local activo => pantalla de selección obligatoria
if (!activeLocal) {
  return <LocalSelector />;
}

return (
  <div className="min-h-screen bg-background">
    <ClientHeader
      localName={activeLocal.nombre}
      mesa={mesaNum ? parseInt(mesaNum) : undefined}
      onCartClick={() => setCartOpen(true)}
      onChangeLocal={() => setShowLocalSelector(true)}
      onFilterClick={() => setFilterOpen(true)}
      onHome={() => { setActiveCategory(null); setActiveSection(null); setCartOpen(false); setFilterOpen(false); }}
    />

    {resumableOrder && (
      <div className="px-4 pt-3 pb-1 max-w-lg mx-auto">
        <div className="flex items-center justify-between gap-3 rounded-xl border border-amber-400/50 bg-amber-50 dark:bg-amber-950/30 px-4 py-3">
          <div className="min-w-0">
            <p className="text-sm font-semibold text-amber-900 dark:text-amber-200">
              Pedido #{resumableOrder.numeroPedido} pendiente de pago
            </p>
            <p className="text-xs text-amber-700 dark:text-amber-400 mt-0.5">
              Tu pedido sigue reservado. ¿Quieres completar el pago?
            </p>
          </div>
          <div className="flex shrink-0 gap-2">
            <button
              onClick={() => setPendingCheckout(resumableOrder)}
              className="rounded-lg bg-primary px-3 py-1.5 text-xs font-semibold text-primary-foreground"
            >
              Pagar
            </button>
            <button
              onClick={handleCancelResumable}
              className="rounded-lg border border-amber-400 px-3 py-1.5 text-xs font-semibold text-amber-800 dark:text-amber-300"
            >
              Cancelar
            </button>
          </div>
        </div>
      </div>
    )}

    {excluded.length > 0 && (
      <div className="px-4 pt-3 max-w-lg mx-auto">
        <div className="flex items-center gap-1.5 flex-wrap">
          <span className="text-[10px] uppercase tracking-wider text-muted-foreground mr-1">
            Excluyendo:
          </span>
          {excluded.map((code) => {
            const a = allAllergens.find((x) => x.codigo === code);
            if (!a) return null;
            return (
              <button
                key={code}
                onClick={() => toggleExcluded(code)}
                className="flex items-center gap-1 pl-1 pr-2 py-0.5 rounded-full bg-destructive/15 border border-destructive/40 text-destructive text-[11px] font-semibold"
              >
                <span aria-hidden={a.icono}</span>

```



```

        {a.nombre}
        <X className="w-3 h-3" />
      </button>
    );
  }}}
</div>
</div>
})

{!activeCategory ? (
  <>
    <div className="px-4 pt-6 pb-6 max-w-lg mx-auto text-center">
      <h2 className="font-display text-3xl font-black leading-tight text-foreground">
        ¿Qué te <span className="text-primary italic">apetece</span> hoy?
      </h2>
      <p className="text-muted-foreground mt-2 text-sm">
        Elige una sección para empezar
      </p>
    </div>
    <div className="pb-8">
      <CategoryGrid categories={categories} onSelect={setActiveCategory} />
    </div>
  </>
) : (
  <>
    <div className="px-4 pt-6 pb-2 max-w-lg mx-auto flex items-center gap-3">
      <button
        onClick={() => { setActiveCategory(null); setActiveSection(null); }}
        className="shrink-0 w-10 h-10 rounded-full bg-card border border-border flex items-center justify-center text-lg
        hover:bg-surface-elevated transition-colors"
        aria-label="Volver"
      >
        ←
      </button>
      <h2 className="font-display text-2xl font-black leading-tight text-foreground">
        {selectedCategory?.nombre ?? 'Menú'}
      </h2>
    </div>

    {(() => {
      const isMontaditos = selectedCategory?.nombre === 'Montaditos';
      const isBebidas = selectedCategory?.nombre === 'Bebidas';
      const isAperitivos = selectedCategory?.nombre === 'Aperitivos';
      const sectionList: readonly string[] | null = isMontaditos
        ? MONTADITO_SECTIONS
        : isBebidas
        ? BEBIDAS_SECTIONS
        : isAperitivos
        ? APERITIVOS_SECTIONS
        : null;
      const isDrink = isBebidas;

      const isMontyAhorro = selectedCategory?.nombre === 'MontyRuedas';

      if (!sectionList) {
        return (
          <div className="px-4 pb-32 max-w-lg mx-auto">

            <div className="flex flex-col gap-2">
              {products.map((product, i) =>
                isMontyAhorro ? (
                  <MontyRuedaCard
                    key={product.id}
                    product={product as any}
                    montaditos={ruedaMontaditos}
                    isProductoAgotado={isProductoAgotado}
                    index={i}
                  />
                ) : (
                  <ProductCard key={product.id} product={product as any} index={i} hideAllergens={isMontyAhorro} agotado=
                    {isProductoAgotado(product.id)} isIngredienteAgotado={isIngredienteAgotado} />
                ),
              )}
            </div>
          </div>
        );
      }

      const available = sectionList.filter((s) =>
        isSeccionAllowed(s) &&
        (products as any[]).some((p) => p.seccion === s)
      );
      const grouped = sectionList
        .filter((sec) => isSeccionAllowed(sec) && (!activeSection || sec === activeSection))
        .map((sec) => ({
          sec,
          items: (products as any[])
            .filter((p) => p.seccion === sec)
            .sort((a, b) => {
              const na = parseInt((a.numero ?? '0').replace(/\D/g, '')) || 0;
              const nb = parseInt((b.numero ?? '0').replace(/\D/g, '')) || 0;

```

```

        return na - nb;
    }},
    )))
    .filter((g) => g.items.length > 0);
const sinSeccion = (products as any[]).filter((p) => !p.seccion);
let idx = 0;
return (
    <>
        <div className="px-4 pb-3 max-w-lg mx-auto">
            <SectionTabs
                sections={available}
                active={activeSection}
                onSelect={setActiveSection}
            />
        </div>
        <div className="px-4 pb-32 max-w-lg mx-auto">
            <div className="flex flex-col gap-6">
                {grouped.map((g) => (
                    <section key={g.sec}>
                        {!activeSection && (
border-primary">
                            <h3 className="font-display text-base font-black uppercase tracking-wider text-primary mb-2 pl-1 border-1-4
                                <span className="pl-2">{g.sec}</span>
                                </h3>
                            ))}
                            <div className="flex flex-col gap-2">
                                {g.items.map((p) => (
                                    <ProductCard
                                        key={p.id}
                                        product={p}
                                        index={idx++}
                                        variant={isDrink ? 'drink' : 'default'}
                                        agotado={isProductoAgotado(p.id)}
                                        isIngredienteAgotado={isIngredienteAgotado}
                                    />
                                ))}
                            </div>
                        </section>
                    ))}
                {(!activeSection) && sinSeccion.length > 0 && (
border-1-4 border-border">
                    <section>
                        <h3 className="font-display text-base font-black uppercase tracking-wider text-muted-foreground mb-2 pl-1
                            <span className="pl-2">Otros</span>
                        </h3>
                        <div className="flex flex-col gap-2">
                            {sinSeccion.map((p) => (
                                <ProductCard
                                    key={p.id}
                                    product={p}
                                    index={idx++}
                                    variant={isDrink ? 'drink' : 'default'}
                                    agotado={isProductoAgotado(p.id)}
                                    isIngredienteAgotado={isIngredienteAgotado}
                                />
                            ))}
                        </div>
                    </section>
                ))}
            </div>
        </div>
    </>
    );
    })()
    </>
    )}

    <CartSheet
        open={cartOpen}
        onClose={() => setCartOpen(false)}
        onCheckout={handleCheckout}
        isCheckingOut={isCheckingOut}
    />

    <AllergenLegalNoticeFloat />

    <AllergenFilterSheet open={filterOpen} onClose={() => setFilterOpen(false)} />

    <MenuChatBot />
    <AlcoholWarningSheet />

    <WelcomeScreen open={welcomeOpen} onClose={closeWelcome} />
    {showLoader && <MontyLoader message="Preparando el menú" />}

    {showLocalSelector && (
        <div className="fixed inset-0 z-40 bg-background overflow-y-auto">
            <LocalSelector onSelect={() => setShowLocalSelector(false)} />
        </div>
    )}
    </div>
);

```

```

};

export default ClientApp;

/* ===== */
/* FICHERO: montadito-magic-flow/src/pages/Index.tsx */
/* ===== */
// Update this page (the content is just a fallback if you fail to update the page)

// IMPORTANT: Fully REPLACE this with your own code
const PlaceholderIndex = () => {
  // PLACEHOLDER: Replace this entire return statement with the user's app.
  // The inline background color is intentionally not part of the design system.
  return (
    <div className="flex min-h-screen items-center justify-center" style={{ backgroundColor: '#fcfbf8' }}>
      
    </div>
  );
};

const Index = PlaceholderIndex;

export default Index;

/* ===== */
/* FICHERO: montadito-magic-flow/src/pages/NotFound.tsx */
/* ===== */
import { useLocation } from "react-router-dom";
import { useEffect } from "react";

const NotFound = () => {
  const location = useLocation();

  useEffect(() => {
    console.error("404 Error: User attempted to access non-existent route:", location.pathname);
  }, [location.pathname]);

  return (
    <div className="flex min-h-screen items-center justify-center bg-muted">
      <div className="text-center">
        <h1 className="mb-4 text-4xl font-bold">404</h1>
        <p className="mb-4 text-xl text-muted-foreground">Oops! Page not found</p>
        <a href="/" className="text-primary underline hover:text-primary/90">
          Return to Home
        </a>
      </div>
    </div>
  );
};

export default NotFound;

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/AlcoholWarningSheet.tsx */
/* ===== */
import { AnimatePresence, motion } from 'framer-motion';
import { AlertTriangle } from 'lucide-react';
import { useAgeGate } from '@lib/cart-store';

export function AlcoholWarningSheet() {
  const pending = useAgeGate((s) => s.pending);
  const confirm = useAgeGate((s) => s.confirm);
  const cancel = useAgeGate((s) => s.cancel);
  const open = !!pending;

  return (
    <AnimatePresence>
      {open && (
        <>
          <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            className="fixed inset-0 bg-black/70 z-[60]"
            onClick={cancel}
          />
          <motion.div
            initial={{ y: '100%' }}
            animate={{ y: 0 }}
            exit={{ y: '100%' }}
            transition={{ type: 'spring', damping: 30, stiffness: 300 }}
            className="fixed bottom-0 left-0 right-0 z-[61] rounded-t-3xl bg-popover border-t border-border-subtle p-6 pb-8"
            role="alertdialog"
            aria-labelledby="alcohol-title"
          >
            <div className="flex justify-center pt-1 pb-4">
              <div className="w-10 h-1 rounded-full bg-muted-foreground/30" />
            </div>
            <div className="flex flex-col items-center text-center max-w-sm mx-auto">

```

```

        <div className="w-16 h-16 rounded-full bg-destructive/15 flex items-center justify-center mb-4">
          <AlertTriangle className="w-8 h-8 text-destructive" />
        </div>
        <h2 id="alcohol-title" className="font-display text-xl font-bold mb-2">
          Verificación de edad
        </h2>
        <p className="text-sm text-muted-foreground leading-relaxed mb-6">
          La venta de alcohol a menores de 18 años está prohibida por ley en España.
        </p>
        <div className="flex flex-col gap-2 w-full">
          <button
            onClick={confirm}
            className="w-full py-3.5 rounded-2xl bg-destructive text-destructive-foreground font-bold text-sm"
          >
            Confirmo que soy mayor de 18 años
          </button>
          <button
            onClick={cancel}
            className="w-full py-3 rounded-2xl bg-surface text-foreground font-semibold text-sm"
          >
            Cancelar
          </button>
        </div>
      </div>
    </motion.div>
  </>
  </AnimatePresence>
);
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/AllergenFilterSheet.tsx */
/* ===== */
import { motion, AnimatePresence } from 'framer-motion';
import { X, Check } from 'lucide-react';
import { useAllergens } from '@hooks/use-allergens';
import { useAllergenFilter } from '@lib/allergen-filter';
import { AllergenIcon } from './AllergenIcon';
import { AllergenLegalNotice } from './AllergenLegalNotice';

interface Props {
  open: boolean;
  onClose: () => void;
}

export function AllergenFilterSheet({ open, onClose }: Props) {
  const { data: alergenos = [] } = useAllergens();
  const excluded = useAllergenFilter((s) => s.excluded);
  const toggle = useAllergenFilter((s) => s.toggle);
  const clear = useAllergenFilter((s) => s.clear);
  const hideMode = useAllergenFilter((s) => s.hideMode);
  const setHideMode = useAllergenFilter((s) => s.setHideMode);

  return (
    <AnimatePresence>
      {open && (
        <>
          <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            className="fixed inset-0 bg-black/60 z-40"
            onClick={onClose}
          />
          <motion.div
            initial={{ y: '100%' }}
            animate={{ y: 0 }}
            exit={{ y: '100%' }}
            transition={{ type: 'spring', damping: 30, stiffness: 300 }}
            className="fixed bottom-0 left-0 right-0 z-50 max-h-[88vh] rounded-t-3xl bg-popover border-t border-border-subtle flex
flex-col"
          >
            <div className="flex justify-center pt-3 pb-2">
              <div className="w-10 h-1 rounded-full bg-muted-foreground/30" />
            </div>

            <div className="flex items-center justify-between px-6 pb-2">
              <div>
                <h2 className="font-display text-xl font-bold">Filtrar alérgenos</h2>
                <p className="text-xs text-muted-foreground mt-0.5">
                  Selecciona los que quieres <span className="font-semibold text-destructive">excluir</span>.
                </p>
              </div>
              <button onClick={onClose} className="p-2 rounded-full hover:bg-surface transition-colors">
                <X className="w-5 h-5" />
              </button>
            </div>

            <div className="px-6 py-2 flex items-center gap-2 text-xs">
              <span className="text-muted-foreground">Modo:</span>
            </div>
          </motion.div>
        </>
      )}
    </AnimatePresence>
  );
}

```

```

        <button
          onClick={() => setHideMode('warn')}
          className={`px-2.5 py-1 rounded-full border ${hideMode === 'warn' ? 'bg-destructive/20 border-destructive text-destructive' : 'border-border text-muted-foreground'}`}
        >
          Avisar
        </button>
        <button
          onClick={() => setHideMode('hide')}
          className={`px-2.5 py-1 rounded-full border ${hideMode === 'hide' ? 'bg-destructive/20 border-destructive text-destructive' : 'border-border text-muted-foreground'}`}
        >
          Ocultar
        </button>
        <button
          onClick={clear}
          className="ml-auto text-xs text-muted-foreground hover:text-foreground underline"
        >
          Limpiar
        </button>
      </div>

      <div className="flex-1 overflow-y-auto px-6 py-3">
        <div className="grid grid-cols-3 gap-3">
          {alergenos.map((a) => {
            const active = excluded.includes(a.codigo);
            return (
              <button
                key={a.id}
                onClick={() => toggle(a.codigo)}
                className={`relative flex flex-col items-center gap-2 p-3 rounded-2xl border transition-all ${
                  active
                    ? 'border-destructive bg-destructive/10'
                    : 'border-border-subtle bg-surface hover:bg-surface-elevated'
                }`}
              >
                {active && (
                  <span className="absolute top-1.5 right-1.5 w-4 h-4 rounded-full bg-destructive flex items-center justify-center">
                    <Check className="w-2.5 h-2.5 text-destructive-foreground" />
                  </span>
                )}
                <AllergenIcon codigo={a.codigo} nombre={a.nombre} icono={a.icono} size="md" />
                <span className="text-[11px] font-semibold text-center leading-tight">{a.nombre}</span>
              </button>
            );
          })}
        </div>

        <div className="mt-5">
          <AllergenLegalNotice variant="full" />
        </div>
      </div>

      <div className="p-4 border-t border-border-subtle">
        <button
          onClick={onClose}
          className="w-full py-3 rounded-2xl bg-primary text-primary-foreground font-bold text-base"
        >
          Aplicar {excluded.length > 0 ? `(${excluded.length})` : ''}
        </button>
      </div>
    </motion.div>
  </>
)
</AnimatePresence>
);
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/AllergenIcon.tsx */
/* ===== */
import { Tooltip, TooltipContent, TooltipProvider, TooltipTrigger } from '@components/ui/tooltip';

interface Props {
  codigo: string;
  nombre: string;
  icono?: string | null;
  size?: 'sm' | 'md' | 'lg';
  showLabel?: boolean;
}

const sizeMap = {
  sm: 'w-6 h-6 text-[13px]',
  md: 'w-9 h-9 text-lg',
  lg: 'w-12 h-12 text-2xl',
};

export function AllergenIcon({ codigo, nombre, icono, size = 'sm', showLabel = false }: Props) {
  const emoji = icono ?? '🍌';
  const dot = (

```

```

    <span
      aria-label={nombre}
      role="img"
      className={` ${sizeMap[size]} rounded-full flex items-center justify-center shrink-0 bg-[#FEF3C7] text-[#92400E] ring-1 ring-
[#FCD34D]/60 shadow-sm`}
      data-allergen={codigo}
    >
      <span aria-hidden>{emoji}</span>
    </span>
  );

  if (showLabel) {
    return (
      <div className="flex flex-col items-center gap-1 min-w-[56px]">
        {dot}
        <span className="text-[10px] text-muted-foreground text-center leading-tight">{nombre}</span>
      </div>
    );
  }

  return (
    <TooltipProvider delayDuration={200}>
      <Tooltip>
        <TooltipTrigger asChild>{dot}</TooltipTrigger>
        <TooltipContent side="top" className="text-xs">{nombre}</TooltipContent>
      </Tooltip>
    </TooltipProvider>
  );
}

interface RowProps {
  alergenos: { codigo: string; nombre: string; icono?: string | null }[];
  maxVisible?: number;
}

export function AllergenIconRow({ alergenos, maxVisible = 5 }: RowProps) {
  if (!alergenos || alergenos.length === 0) return null;
  const visible = alergenos.slice(0, maxVisible);
  const extra = alergenos.length - visible.length;

  return (
    <div className="flex items-center gap-1 flex-wrap">
      {visible.map((a) => (
        <AllergenIcon key={a.codigo} codigo={a.codigo} nombre={a.nombre} icono={a.icono} size="sm" />
      ))}
      {extra > 0 && (
        <span className="w-6 h-6 rounded-full flex items-center justify-center bg-[#FEF3C7] text-[#92400E] text-[10px] font-bold ring-
1 ring-[#FCD34D]/60">
          +{extra}
        </span>
      )}
    </div>
  );
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/AllergenLegalNotice.tsx */
/* ===== */
import { useState } from 'react';
import { motion, AnimatePresence } from 'framer-motion';
import { Info, X } from 'lucide-react';

interface Props {
  variant?: 'short' | 'full';
  className?: string;
}

export function AllergenLegalNotice({ variant = 'short', className = '' }: Props) {
  if (variant === 'short') {
    return (
      <p className={`text-[11px] text-muted-foreground leading-snug ${className}`}>
        Si tienes alguna alergia o intolerancia alimentaria consulta con el personal del local antes de realizar tu pedido.
      </p>
    );
  }
  return (
    <p className={`text-[11px] text-muted-foreground leading-snug ${className}`}>
      La información sobre alérgenos se proporciona de buena fe y está basada en la información facilitada por nuestros proveedores.
      Aunque tomamos todas las precauciones posibles, no podemos garantizar que los productos estén completamente libres de trazas de
      alérgenos. Si padeces una alergia o intolerancia grave, consulta siempre con el personal del local antes de realizar tu pedido.
      (Reglamento UE 1169/2011)
    </p>
  );
}

export function AllergenLegalNoticeFloat() {
  const [open, setOpen] = useState(false);

  return (
    <>
      <button

```

```

onClick={() => setOpen(true)}
className="fixed bottom-6 right-4 z-30 w-10 h-10 rounded-full bg-surface-elevated border border-border-subtle shadow-lg flex
items-center justify-center text-muted-foreground hover:bg-surface transition-colors"
aria-label="Información sobre alérgenos"
>
<Info className="w-5 h-5" />
</button>

<AnimatePresence>
  {open && (
    <>
      <motion.div
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        exit={{ opacity: 0 }}
        className="fixed inset-0 bg-black/60 z-40"
        onClick={() => setOpen(false)}
      />
      <motion.div
        initial={{ y: '100%' }}
        animate={{ y: 0 }}
        exit={{ y: '100%' }}
        transition={{ type: 'spring', damping: 30, stiffness: 300 }}
        className="fixed bottom-0 left-0 right-0 z-50 max-h-[60vh] rounded-t-3xl bg-popover border-t border-border-subtle flex
flex-col"
      >
        <div className="flex justify-center pt-3 pb-2">
          <div className="w-10 h-1 rounded-full bg-muted-foreground/30" />
        </div>

        <div className="flex items-center justify-between px-6 pb-2">
          <h2 className="font-display text-lg font-bold">Información sobre alérgenos</h2>
          <button
            onClick={() => setOpen(false)}
            className="p-2 rounded-full hover:bg-surface transition-colors"
          >
            <X className="w-5 h-5" />
          </button>
        </div>

        <div className="flex-1 overflow-y-auto px-6 py-4">
          <div className="rounded-2xl border border-border-subtle bg-surface/50 p-4 space-y-3">
            <p className="text-xs text-muted-foreground leading-relaxed">
              La información sobre alérgenos se proporciona de buena fe y está basada en la información facilitada por nuestros
              proveedores. Aunque tomamos todas las precauciones posibles, no podemos garantizar que los productos estén completamente libres de
              trazas de alérgenos.
            </p>
            <p className="text-xs text-muted-foreground leading-relaxed">
              Si padeces una alergia o intolerancia grave, consulta siempre con el personal del local antes de realizar tu
              pedido.
            </p>
            <p className="text-[11px] text-muted-foreground/70 leading-relaxed">
              Reglamento UE 1169/2011 de información alimentaria facilitada al consumidor.
            </p>
          </div>
        </div>

        <div className="p-4 border-t border-border-subtle">
          <button
            onClick={() => setOpen(false)}
            className="w-full py-3 rounded-2xl bg-primary text-primary-foreground font-bold text-base"
          >
            Entendido
          </button>
        </div>
      </motion.div>
    </>
  )}
</AnimatePresence>
);
}

```

```

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/CartSheet.tsx */
/* ===== */
import { useState, useEffect, useMemo } from 'react';
import { motion, AnimatePresence } from 'framer-motion';
import { X, Minus, Plus, AlertTriangle } from 'lucide-react';
import { useCartStore } from '@lib/cart-store';
import { getDrinkImage } from '@lib/drink-image';
import { Monty } from './Monty';
import { AllergenIcon } from './AllergenIcon';
import { AllergenLegalNotice } from './AllergenLegalNotice';

```

```

interface Props {
  open: boolean;
  onClose: () => void;
  onCheckout: (ageVerified: boolean) => void;
  isCheckingOut?: boolean;
}

```

```

}

export function CartSheet({ open, onClose, onCheckout, isCheckedOut = false }: Props) {
  const items = useCartStore((s) => s.items);
  const updateQuantity = useCartStore((s) => s.updateQuantity);
  const removeItem = useCartStore((s) => s.removeItem);
  const total = useCartStore((s) => s.total);
  const hasAlcohol = useCartStore((s) => s.hasAlcohol());
  const [ageVerified, setAgeVerified] = useState(false);

  // Reset el checkbox si el carrito deja de tener alcohol
  useEffect(() => {
    if (!hasAlcohol) setAgeVerified(false);
  }, [hasAlcohol]);

  const cartAllergens = useMemo(() => {
    const map = new Map<string, { codigo: string; nombre: string; icono?: string | null }>();
    items.forEach((it) => {
      (it.alergenos ?? []).forEach((a) => {
        if (!map.has(a.codigo)) map.set(a.codigo, a);
      });
    });
    return Array.from(map.values());
  }, [items]);

  const canCheckout = !hasAlcohol || ageVerified;

  return (
    <AnimatePresence>
      {open && (
        <>
          { /* Backdrop */}
          <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            className="fixed inset-0 bg-black/60 z-40"
            onClick={onClose}
          />

          { /* Sheet */}
          <motion.div
            initial={{ y: '100%' }}
            animate={{ y: 0 }}
            exit={{ y: '100%' }}
            transition={{ type: 'spring', damping: 30, stiffness: 300 }}
            className="fixed bottom-0 left-0 right-0 z-50 max-h-[85vh] rounded-t-3xl bg-popover border-t border-border-subtle flex
flex-col"
          >
            { /* Handle */}
            <div className="flex justify-center pt-3 pb-2">
              <div className="w-10 h-1 rounded-full bg-muted-foreground/30" />
            </div>

            { /* Header */}
            <div className="flex items-center justify-between px-6 pb-4">
              <h2 className="font-display text-2xl font-bold">Tu Pedido</h2>
              <button onClick={onClose} className="p-2 rounded-full hover:bg-surface transition-colors">
                <X className="w-5 h-5" />
              </button>
            </div>

            { /* Items */}
            <div className="flex-1 overflow-y-auto px-6 space-y-4">
              {items.length === 0 ? (
                <div className="flex flex-col items-center justify-center py-10 text-center">
                  <Monty pose="thinking" size={140} animate="float" />
                  <p className="font-display text-lg mt-3 text-foreground">
                    Tu carrito está vacío
                  </p>
                  <p className="text-sm mt-1 text-muted-foreground max-w-xs">
                    Tu carrito está más vacío que una mesa antes del aperitivo 🍷
                  </p>
                </div>
              ) : (
                items.map((item) => {
                  const itemImage = getDrinkImage(item.nombre, item.foto_url);

                  return (
                    <motion.div
                      key={item.id}
                      layout
                      initial={{ opacity: 0, x: -20 }}
                      animate={{ opacity: 1, x: 0 }}
                      exit={{ opacity: 0, x: 20 }}
                      className="flex items-center gap-4 glass-card p-3"
                    >
                      {itemImage && (
                        <img
                          src={itemImage}
                          alt={item.nombre}
                          className="w-16 h-16 rounded-lg object-contain bg-card border border-border-subtle"
                        />
                      )}
                    </motion.div>
                  );
                })
              )}
            </div>
          </>
        )}
    </AnimatePresence>
  );
}

```



```

        loading="lazy"
      />
    )}
    <div className="flex-1 min-w-0">
      <h4 className="font-display font-bold text-sm truncate">
        {item.numero && (
          <span className="text-muted-foreground font-mono mr-1">#{item.numero}</span>
        )}
        {item.nombre}
      </h4>
      <p className="text-gold text-sm font-semibold">{item.precio.toFixed(2)} €</p>
      {item.componentes && item.componentes.length > 0 && (
        <ul className="mt-1 space-y-0.5">
          {item.componentes.map((c, ci) => (
            <li key={ci} className="text-[11px] text-muted-foreground leading-tight">· {c}</li>
          ))}
        </ul>
      )}
    </div>
    <div className="flex items-center gap-3">
      <button
        onClick={() => updateQuantity(item.id, item.cantidad - 1)}
        className="w-8 h-8 rounded-full bg-surface flex items-center justify-center hover:bg-surface-elevated
transition-colors"
      >
        <Minus className="w-3.5 h-3.5" />
      </button>
      <span className="font-bold text-sm w-5 text-center">{item.cantidad}</span>
      <button
        onClick={() => updateQuantity(item.id, item.cantidad + 1)}
        className="w-8 h-8 rounded-full bg-surface flex items-center justify-center hover:bg-surface-elevated
transition-colors"
      >
        <Plus className="w-3.5 h-3.5" />
      </button>
    </div>
  </motion.div>
);
  })
}
</div>

/* Footer */
{items.length > 0 && (
  <div className="p-6 border-t border-border-subtle">
    {cartAllergens.length > 0 && (
      <div className="mb-3 p-3 rounded-xl bg-[#FEF3C7]/15 border border-[#FCD34D]/40">
        <div className="flex items-center gap-2 mb-2">
          <AlertTriangle className="w-3.5 h-3.5 text-[#92400E]" />
          <p className="text-xs font-semibold text-foreground">
            Revisa los alérgenos de tu pedido antes de confirmar
          </p>
        </div>
      </div>
      <div className="flex flex-wrap gap-1.5">
        {cartAllergens.map((a) => (
          <AllergenIcon key={a.codigo} codigo={a.codigo} nombre={a.nombre} icono={a.icono} size="sm" />
        ))}
      </div>
      <div className="mt-2">
        <AllergenLegalNotice variant="short" />
      </div>
    </div>
  )}
  {hasAlcohol && (
    <label className="flex items-start gap-3 mb-4 p-3 rounded-xl bg-destructive/10 border border-destructive/30 cursor-
pointer">
      <input
        type="checkbox"
        checked={ageVerified}
        onChange={(e) => setAgeVerified(e.target.checked)}
        className="mt-0.5 w-4 h-4 accent-destructive shrink-0"
      />
      <span className="text-xs text-foreground leading-snug flex-1">
        <AlertTriangle className="inline w-3.5 h-3.5 text-destructive mr-1 -mt-0.5" />
        Confirmo que soy mayor de 18 años y acepto mostrar mi DNI si el personal me lo solicita al recoger el pedido.
      </span>
    </label>
  )}
  <div className="flex justify-between items-center mb-4">
    <span className="text-muted-foreground">Total</span>
    <span className="font-display text-2xl font-bold text-gold">
      {total().toFixed(2)} €
    </span>
  </div>
  <motion.button
    whileTap={canCheckout && !isCheckedOut ? { scale: 0.97 } : undefined}
    onClick={() => canCheckout && !isCheckedOut && onCheckout(hasAlcohol && ageVerified)}
    disabled={!canCheckout || isCheckedOut}
    className="w-full py-4 rounded-2xl bg-primary text-primary-foreground font-bold text-lg relative overflow-hidden
group disabled:opacity-40 disabled:cursor-not-allowed"
  >
    <span className="relative z-10 flex items-center justify-center gap-2">

```

```

        {isCheckingOut && (
          <span className="w-5 h-5 border-2 border-primary-foreground border-t-transparent rounded-full animate-spin" />
        )}
        {isCheckingOut ? 'Creando pedido...' : 'Realizar Pedido'}
      </span>
      <div className="absolute inset-0 shimmer opacity-0 group-hover:opacity-100 transition-opacity duration-500" />
    </motion.button>
  </div>
  )}
</motion.div>
</>
)}
</AnimatePresence>
);
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/CategoryGrid.tsx */
/* ===== */
import { motion } from 'framer-motion';
import { SECTION_IMAGES } from '@lib/section-images';

interface Category {
  id: string;
  nombre: string;
  logo_url?: string | null;
  imagen_url?: string | null;
}

interface Props {
  categories: Category[];
  onSelect: (id: string) => void;
}

const CATEGORY_BG: Record<string, { bg: string; text: string }> = {
  Montaditos: { bg: 'bg-[#962d1e]', text: 'text-white' },
  MontyRuedas: { bg: 'bg-[#C9B99A]', text: 'text-[#2D2926]' },
  Ensaladas: { bg: 'bg-[#2D6A4F]', text: 'text-white' },
  Aperitivos: { bg: 'bg-[#B8C5D0]', text: 'text-[#2D2926]' },
  Raciones: { bg: 'bg-[#D4A59A]', text: 'text-[#2D2926]' },
  Bebidas: { bg: 'bg-[#1B4332]', text: 'text-white' },
};

const DEFAULT = { bg: 'bg-primary', text: 'text-primary-foreground' };

export function CategoryGrid({ categories, onSelect }: Props) {
  return (
    <div className="grid grid-cols-2 gap-4 px-4 max-w-lg mx-auto">
      {categories.map((cat, i) => {
        const s = CATEGORY_BG[cat.nombre] || DEFAULT;
        const logo = SECTION_IMAGES[cat.nombre] || cat.logo_url || cat.imagen_url;
        return (
          <motion.button
            key={cat.id}
            onClick={() => onSelect(cat.id)}
            initial={{ opacity: 0, y: 20 }}
            animate={{ opacity: 1, y: 0 }}
            transition={{ delay: i * 0.06, type: 'spring', damping: 18 }}
            whileTap={{ scale: 0.95 }}
            whileHover={{ y: -4 }}
            className={`aspect-square rounded-3xl ${s.bg} ${s.text} shadow-lg flex flex-col items-center justify-center gap-2 p-4 transition-shadow hover:shadow-xl`}
          >
            {logo ? (
              <img
                src={logo}
                alt={cat.nombre}
                className="w-20 h-20 object-contain drop-shadow-md"
                loading="lazy"
              />
            ) : (
              <span className="text-5xl" aria-hidden="" />
            )}
            <span className="font-display text-xl font-black tracking-tight text-center leading-tight">
              {cat.nombre}
            </span>
          </motion.button>
        );
      })}
    </div>
  );
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/CategoryTabs.tsx */
/* ===== */
import { motion } from 'framer-motion';

interface Category {
  id: string;

```

```

    nombre: string;
  }

interface Props {
  categories: Category[];
  activeId: string | null;
  onSelect: (id: string | null) => void;
}

const CATEGORY_COLORS: Record<string, { bg: string; text: string; icon: string }> = {
  Montaditos: { bg: 'bg-[#C8102E]', text: 'text-white', icon: '🍷' },
  Aperitivos: { bg: 'bg-[#B8C5D0]', text: 'text-[#2D2926]', icon: '🍷' },
  Raciones: { bg: 'bg-[#D4A59A]', text: 'text-[#2D2926]', icon: '🍷' },
  Montyahorro: { bg: 'bg-[#C9B99A]', text: 'text-[#2D2926]', icon: '🍷' },
  Ensaladas: { bg: 'bg-[#2D6A4F]', text: 'text-white', icon: '🍷' },
  Bebidas: { bg: 'bg-[#2D6A4F]', text: 'text-white', icon: '🍷' },
};

const DEFAULT_COLOR = { bg: 'bg-primary', text: 'text-primary-foreground', icon: '🍷' };

export function CategoryTabs({ categories, activeId, onSelect }: Props) {
  return (
    <div className="flex gap-3 overflow-x-auto pb-2 px-4 scrollbar-hide">
      {categories.map((cat) => {
        const colors = CATEGORY_COLORS[cat.nombre] || DEFAULT_COLOR;
        const isActive = activeId === cat.id;
        return (
          <motion.button
            key={cat.id}
            onClick={() => onSelect(cat.id)}
            whileTap={{ scale: 0.95 }}
            className={`shrink-0 px-5 py-2.5 rounded-full text-sm font-semibold tracking-wide transition-all duration-300 ${
              isActive
                ? `${colors.bg} ${colors.text} shadow-md`
                : 'bg-card text-muted-foreground hover:text-foreground border border-border'
            }`}
          >
            {colors.icon} {cat.nombre}
          </motion.button>
        );
      })}
    </div>
  );
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/ClientHeader.tsx */
/* ===== */
import { MapPin, ShoppingBag, SlidersHorizontal } from 'lucide-react';
import { motion, AnimatePresence } from 'framer-motion';
import { useCartStore } from '@lib/cart-store';
import { useAllergenFilter } from '@lib/allergen-filter';
import logo from '@assets/logo-100montaditos.png';

interface Props {
  localName?: string;
  mesa?: number;
  onCartClick: () => void;
  onChangeLocal?: () => void;
  onFilterClick?: () => void;
  onHome?: () => void;
}

export function ClientHeader({ localName, mesa, onCartClick, onChangeLocal, onFilterClick, onHome }: Props) {
  const itemCount = useCartStore((s) => s.itemCount);
  const excludedCount = useAllergenFilter((s) => s.excluded.length);

  return (
    <header className="sticky top-0 z-30 glass px-4 py-3">
      <div className="flex items-center justify-between gap-2 max-w-lg mx-auto">
        <motion.button
          whileTap={{ scale: 0.92 }}
          onClick={onHome}
          aria-label="Volver al inicio"
          className="shrink-0"
        >
          <img src={logo} alt="100 Montaditos · Inicio" className="h-12" />
        </motion.button>

        {localName && (
          <button
            onClick={onChangeLocal}
            className="flex items-center gap-1.5 px-3 py-1.5 rounded-full bg-surface hover:bg-surface-elevated transition-colors min-w-0"
          >
            <MapPin className="w-3.5 h-3.5 text-primary shrink-0" />
            <span className="text-xs font-semibold truncate max-w-[110px]">
              {localName}
            </span>
          </button>
        )}

        {mesa != null && (
          <span className="text-[10px] text-muted-foreground shrink-0">· M{mesa}</span>
        )}
      </div>
    </header>
  );
}

```

```

    })
  </button>
})

{onFilterClick && (
  <motion.button
    whileTap={{ scale: 0.9 }}
    onClick={onFilterClick}
    aria-label="Filtrar alérgenos"
    className="relative flex items-center gap-1.5 px-3 py-1.5 rounded-full bg-surface hover:bg-surface-elevated transition-
colors shrink-0"
  >
    <SlidersHorizontal className="w-3.5 h-3.5" />
    <span className="text-xs font-semibold">Alérgenos</span>
    {excludedCount > 0 && (
      <span className="absolute -top-1 -right-1 min-w-[18px] h-[18px] px-1 rounded-full bg-destructive text-destructive-
foreground text-[9px] font-bold flex items-center justify-center">
        {excludedCount}
      </span>
    )}
  </motion.button>
)}

<motion.button
  whileTap={{ scale: 0.9 }}
  onClick={onCartClick}
  className="relative p-3 rounded-full bg-surface hover:bg-surface-elevated transition-colors shrink-0"
>
  <ShoppingBag className="w-5 h-5" />
  <AnimatePresence>
    {itemCount() > 0 && (
      <motion.span
        initial={{ scale: 0 }}
        animate={{ scale: 1 }}
        exit={{ scale: 0 }}
        className="absolute -top-1 -right-1 w-5 h-5 rounded-full bg-primary text-primary-foreground text-[10px] font-bold flex
items-center justify-center"
      >
        {itemCount()}
      </motion.span>
    )}
  </AnimatePresence>
</motion.button>
</div>
</header>
);
}

```

```

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/EmbeddedCheckout.tsx */
/* ===== */
import { useEffect, useState } from 'react';
import { Elements, PaymentElement, useElements, useStripe } from '@stripe/react-stripe-js';
import { ArrowLeft, Lock } from 'lucide-react';
import { supabase } from '@integrations/supabase/client';
import { getStripe } from '@lib/stripe';
import { toast } from 'sonner';

```

```

interface Props {
  pedidoId: string;
  sessionId: string;
  numeroPedido: number;
  hasCocina: boolean;
  hasBebidas: boolean;
  total: number;
  onSuccess: () => void;
  onCancel: () => void;
}

```

```

function PayForm({ total, pedidoId, sessionId, numeroPedido, hasCocina, hasBebidas, onSuccess, onCancel }: { total: number; pedidoId:
string; sessionId: string; numeroPedido: number; hasCocina: boolean; hasBebidas: boolean; onSuccess: () => void; onCancel: () => void
}) {
  const stripe = useStripe();
  const elements = useElements();
  const [submitting, setSubmitting] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!stripe || !elements) return;
    setSubmitting(true);
    setError(null);

    const { error: submitError } = await elements.submit();
    if (submitError) {
      setError(submitError.message ?? 'Error al validar el formulario');
      setSubmitting(false);
      return;
    }

    // Codificamos los datos del pedido en el return_url como fallback:

```

```
// iOS Safari a veces borra localStorage durante el redirect cross-domain de Apple Pay.
const returnParams = new URLSearchParams({
  pago: 'ok',
  pid: pedidoId,
  num: String(numeroPedido),
  sid: sessionId,
  cocina: hasCocina ? '1' : '0',
  bebidas: hasBebidas ? '1' : '0',
});
const { error: confirmError, paymentIntent } = await stripe.confirmPayment({
  elements,
  confirmParams: { return_url: `${window.location.origin}/${returnParams.toString()}` },
  redirect: 'if_required',
});

if (confirmError) {
  setError(confirmError.message ?? 'El pago no se ha podido procesar');
  setSubmitting(false);
  return;
}

if (paymentIntent && (paymentIntent.status === 'succeeded' || paymentIntent.status === 'processing')) {
  try {
    await supabase.functions.invoke('confirm-payment', {
      body: { pedido_id: pedidoId, session_id: sessionId, payment_intent_id: paymentIntent.id },
    });
  } catch (e) {
    console.error('confirm-payment fallback failed', e);
  }
  toast.success('¡Pago realizado!');
  onSuccess();
} else {
  setError('Estado de pago inesperado. Inténtalo de nuevo.');
  setSubmitting(false);
}
};

return (
<form onSubmit={handleSubmit} className="flex flex-col gap-5">
  <PaymentElement options={{ layout: 'tabs', paymentMethodOrder: ['apple_pay', 'google_pay', 'card'] }} />
  {error && (
    <p className="text-sm text-destructive bg-destructive/10 border border-destructive/30 rounded-xl px-3 py-2">
      {error}
    </p>
  )}
  <button
    type="submit"
    disabled={!stripe || submitting}
    className="w-full py-4 rounded-2xl bg-primary text-primary-foreground font-bold text-lg disabled:opacity-50">
    {submitting ? 'Procesando...' : `Pagar ${total.toFixed(2)} €`}
  </button>
  <button
    type="button"
    onClick={onCancel}
    disabled={submitting}
    className="w-full py-3 rounded-2xl bg-surface border border-border-subtle text-foreground font-medium hover:bg-surface-elevated transition-colors disabled:opacity-50">
    Cancelar y volver al menú
  </button>
  <div className="flex items-center justify-center gap-2 text-xs text-muted-foreground">
    <Lock className="w-3 h-3" />
    Pago seguro procesado por Stripe
  </div>
</form>
);
}

export function EmbeddedCheckout({ pedidoId, sessionId, numeroPedido, hasCocina, hasBebidas, total, onSuccess, onCancel }: Props) {
  const [clientSecret, setClientSecret] = useState<string | null>(null);
  const [loadError, setLoadError] = useState<string | null>(null);
  const [retryCount, setRetryCount] = useState(0);

  useEffect(() => {
    let cancelled = false;
    setClientSecret(null);
    setLoadError(null);

    const timeout = window.setTimeout(() => {
      if (!cancelled) setLoadError('La conexión tarda más de lo esperado. Comprueba tu conexión e inténtalo de nuevo.');
```

```

    }
    if (error || !data?.client_secret) {
      console.error(error);
      setLoadError('No se pudo inicializar el pago. Inténtalo de nuevo.');
```

return;

```

    }
    setClientSecret(data.client_secret);
  })();
  return () => { cancelled = true; clearTimeout(timeout); };
// eslint-disable-next-line react-hooks/exhaustive-deps
}, [pedidoId, sessionId, retryCount]);

return (
  <div className="min-h-screen bg-background px-5 py-8 flex flex-col">
    <div className="max-w-md mx-auto w-full flex flex-col flex-1">
      <button
        onClick={onCancel}
        className="self-start flex items-center gap-2 text-sm text-muted-foreground hover:text-foreground transition-colors mb-6"
      >
        <ArrowLeft className="w-4 h-4" /> Volver
      </button>

      <div className="text-center mb-6">
        <p className="text-muted-foreground text-[10px] uppercase tracking-widest mb-1">Pedido</p>
        <h1 className="font-display font-black text-4xl text-foreground">
          #{String(numeroPedido).padStart(4, '0')}
        </h1>
        <p className="mt-3 text-gold font-display text-2xl font-bold">{total.toFixed(2)} €</p>
      </div>

      {loadError && (
        <div className="flex flex-col gap-3">
          <p className="text-sm text-destructive bg-destructive/10 border border-destructive/30 rounded-xl px-3 py-2 text-center">
            {loadError}
          </p>
          <button
            onClick={() => setRetryCount((c) => c + 1)}
            className="w-full py-3 rounded-2xl bg-primary text-primary-foreground font-semibold"
          >
            Reintentar
          </button>
        </div>
      )}

      {!clientSecret && !loadError && (
        <div className="flex-1 flex items-center justify-center">
          <div className="w-8 h-8 border-2 border-primary border-t-transparent rounded-full animate-spin" />
        </div>
      )}

      {clientSecret && (
        <Elements
          stripe={getStripe()}
          options={{
            clientSecret,
            appearance: {
              theme: 'night',
              variables: {
                colorPrimary: '#E63946',
                colorBackground: '#1a1a1a',
                colorText: '#f5f5f5',
                colorDanger: '#ef4444',
                fontFamily: 'DM Sans, system-ui, sans-serif',
                borderRadius: '12px',
              },
            },
          }}
        >
          <PayForm total={total} pedidoId={pedidoId} sessionId={sessionId} numeroPedido={numeroPedido} hasCocina={hasCocina}
            hasBebidas={hasBebidas} onSuccess={onSuccess} onCancel={onCancel} />
        </Elements>
      )}
    </div>
  </div>
);
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/EuromaniaScreen.tsx */
/* ===== */
import { motion } from 'framer-motion';
import montyEuromaniaWebm from '@assets/monty-euromania.webm';
import montyEuromaniaVideo from '@assets/monty-euromania.mp4';

const COLORS = ['#D72B2B', '#FFD700', '#FF6B35', '#4CAF50', '#2196F3', '#FF69B4', '#FF1493', '#FFA500', '#FFFFFF', '#9C27B0'];

// Genera piezas de confeti con posición y propiedades aleatorias pero fijas
const rand = (min: number, max: number) => min + Math.random() * (max - min);

type Piece = {
  id: number;

```

```

startX: number;
endX: number;
delay: number;
duration: number;
size: number;
color: string;
rotations: number;
isRect: boolean;
wave: number; // 0 = primera ráfaga, 1 = segunda ráfaga
};

const makePieces = (): Piece[] => {
  const pieces: Piece[] = [];
  let id = 0;

  const add = (count: number, xMin: number, xMax: number, driftMin: number, driftMax: number, delayOffset: number, wave: number) => {
    for (let i = 0; i < count; i++) {
      const startX = rand(xMin, xMax);
      pieces.push({
        id: id++,
        startX,
        endX: startX + rand(driftMin, driftMax),
        delay: delayOffset + rand(0, 0.6),
        duration: rand(2.2, 3.8),
        size: rand(7, 14),
        color: COLORS[Math.floor(Math.random() * COLORS.length)],
        rotations: rand(2, 5) * (Math.random() > 0.5 ? 1 : -1),
        isRect: Math.random() > 0.4,
        wave,
      });
    }
  };

  // Ráfaga 1
  add(35, 0, 15, 10, 50, 0.05, 0); // cañón izquierdo
  add(35, 85, 100, -50, -10, 0.05, 0); // cañón derecho
  add(50, 20, 80, -20, 20, 0.1, 0); // lluvia central

  // Ráfaga 2 (1.4 s después)
  add(35, 0, 15, 10, 50, 1.4, 1);
  add(35, 85, 100, -50, -10, 1.4, 1);
  add(50, 20, 80, -20, 20, 1.45, 1);

  return pieces;
};

const PIECES = makePieces();

function Confetti() {
  return (
    <div className="fixed inset-0 pointer-events-none overflow-hidden" style={{ zIndex: 9999 }}>
      {PIECES.map((p) => (
        <motion.div
          key={p.id}
          style={{
            position: 'absolute',
            left: `${p.startX}%`,
            top: -16,
            width: p.isRect ? p.size : p.size * 0.7,
            height: p.isRect ? p.size * 0.5 : p.size * 0.7,
            backgroundColor: p.color,
            borderRadius: p.isRect ? 2 : '50%',
          }}
          initial={{ y: -20, x: 0, rotate: 0, opacity: 1 }}
          animate={{
            y: '105vh',
            x: `${p.endX - p.startX}vw`,
            rotate: 360 * p.rotations,
            opacity: [1, 1, 1, 0.8, 0],
          }}
          transition={{
            delay: p.delay,
            duration: p.duration,
            ease: 'easeIn',
          }}
        />
      ))}
    </div>
  );
}

export function EuromaniaScreen() {
  return (
    <div className="fixed inset-0 flex flex-col items-center justify-center bg-background px-6">
      <Confetti />
      <!-- Video de Monty - WebM con alpha (transparencia real), MP4 como fallback -->
      <motion.video
        autoPlay
        loop
        muted
        playsInline
      />
    </div>
  );
}

```

```

        initial={{ opacity: 0, scale: 0.5, y: 60 }}
        animate={{ opacity: 1, scale: 1, y: 0 }}
        transition={{
          delay: 0.15,
          type: 'spring',
          stiffness: 180,
          damping: 14,
          mass: 1.1,
        }}
        className="w-72 max-w-[78vw] select-none pointer-events-none"
      >
        <source src={montyEuromaniaWebm} type="video/webm" />
        <source src={montyEuromaniaVideo} type="video/mp4" />
      </motion.video>

      { /* Texto */ }
      <motion.div
        initial={{ opacity: 0, y: 18 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ delay: 0.6, duration: 0.45, ease: 'easeOut' }}
        className="mt-2 text-center max-w-xs"
      >
        <p className="font-display font-bold text-foreground text-xl leading-snug">
          ¡Hoy toca fiesta! 🎉 Es día de Euromanía y todos los montaditos
          están a <span className="text-primary">1 €</span>. ¡Ven y aprovecha
          la promoción!
        </p>
        <p className="mt-3 text-muted-foreground text-xs font-body">
          (Promoción válida solo en barra)
        </p>
      </motion.div>
    </div>
  );
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/LocalSelector.tsx */
/* ===== */
import { Loader2, MapPin, Navigation, Search, X, Pin, Check } from 'lucide-react';
import { useEffect, useMemo, useState } from 'react';
import { motion, AnimatePresence } from 'framer-motion';
import { supabase } from '@integrations/supabase/client';
import {
  ActiveLocal,
  formatDistance,
  haversineMeters,
  useActiveLocal,
} from '@lib/active-local';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import logo from '@assets/logo-100montaditos.png';

type LocalRow = {
  id: string;
  nombre: string;
  ciudad: string;
  direccion: string | null;
  lat: number | null;
  lng: number | null;
  activo: boolean;
};

type Step = 'choose' | 'gps' | 'search' | 'confirm';
type Mode = 'gps' | 'search';

interface Props {
  onSelected?: () => void;
  embedded?: boolean; // shown as a "change local" sheet vs initial gate
}

export function LocalSelector({ onSelected, embedded }: Props) {
  const setLocal = useActiveLocal((s) => s.setLocal);
  const [step, setStep] = useState<Step>('choose');
  const [mode, setMode] = useState<Mode>('gps');
  const [coords, setCoords] = useState<{ lat: number; lng: number } | null>(null);
  const [gpsError, setGpsError] = useState<string | null>(null);
  const [locales, setLocales] = useState<LocalRow[]>([]);
  const [loading, setLoading] = useState(false);
  const [query, setQuery] = useState('');
  const [debounced, setDebounced] = useState('');
  const [pendingConfirm, setPendingConfirm] = useState<LocalRow | null>(null);

  // Debounce search input
  useEffect(() => {
    const t = setTimeout(() => setDebounced(query.trim()), 300);
    return () => clearTimeout(t);
  }, [query]);

  // Fetch locales depending on mode
  useEffect(() => {
    if (step !== 'gps' && step !== 'search') return;

```



```

let cancelled = false;
(async () => {
  setLoading(true);
  let q = supabase
    .from('locales')
    .select('id, nombre, ciudad, direccion, lat, lng, activo')
    .eq('activo', true);
  if (step === 'search' && debounced) {
    q = q.or(
      `nombre.ilike.%${debounced}%,ciudad.ilike.%${debounced}%,direccion.ilike.%${debounced}%`
    );
  }
  const { data, error } = await q.limit(50);
  if (!cancelled) {
    if (error) console.error(error);
    setLocales((data as any) ?? []);
    setLoading(false);
  }
})();
return () => {
  cancelled = true;
};
}, [step, debounced]);

// GPS request
const requestGps = () => {
  setStep('gps');
  setMode('gps');
  setGpsError(null);
  if (!navigator.geolocation) {
    setGpsError('Tu dispositivo no soporta geolocalización');
    setStep('search');
    setMode('search');
    return;
  }
  navigator.geolocation.getCurrentPosition(
    (pos) => setCoords({ lat: pos.coords.latitude, lng: pos.coords.longitude }),
    (err) => {
      console.warn('GPS denegado', err);
      setGpsError('Sin acceso a tu ubicación');
      setStep('search');
      setMode('search');
    },
    { enableHighAccuracy: true, timeout: 10000 }
  );
};

// Auto-select if user is within 100m of one local
useEffect(() => {
  if (mode !== 'gps' || !coords || locales.length === 0) return;
  const withDist = locales
    .filter((l) => l.lat !== null && l.lng !== null)
    .map((l) => ({
      l,
      d: haversineMeters(coords.lat, coords.lng, l.lat as number, l.lng as number),
    }));
  withDist.sort((a, b) => a.d - b.d);
  if (withDist[0] && withDist[0].d < 100 && !pendingConfirm) {
    setPendingConfirm(withDist[0].l);
  }
}, [mode, coords, locales, pendingConfirm]);

const sortedLocales = useMemo(() => {
  if (mode === 'gps' && coords) {
    return [...locales]
      .map((l) => ({
        l,
        d:
          l.lat !== null && l.lng !== null
            ? haversineMeters(coords.lat, coords.lng, l.lat, l.lng)
            : Infinity,
      }));
  }
  sortedLocales.sort((a, b) => a.d - b.d);
}, [mode, coords]);
return sortedLocales.map((l) => ({ l, d: Infinity }));
}, [sortedLocales, coords, mode]);

const confirm = (row: LocalRow) => {
  const active: ActiveLocal = {
    id: row.id,
    nombre: row.nombre,
    ciudad: row.ciudad,
    direccion: row.direccion,
  };
  setLocal(active);
  onSelected?.();
};

// ----- UI -----

if (step === 'choose') {
  return (

```

```

<Shell embedded={embedded}>
  <div className="flex flex-col items-center text-center gap-8 max-w-sm mx-auto pt-4">
    <img src={logo} alt="100 Montaditos" className="h-24" />
    <div>
      <h1 className="font-display text-3xl font-black leading-tight">
        ¿En qué <span className="text-primary italic">100 Montaditos</span> estás?
      </h1>
      <p className="text-muted-foreground mt-3 text-sm">
        Selecciona el local para ver la carta y hacer tu pedido.
      </p>
    </div>

    <div className="w-full flex flex-col gap-3">
      <Button
        size="lg"
        className="h-14 text-base font-semibold"
        onClick={requestGps}
      >
        <Navigation className="w-5 h-5 mr-2" />
        Usar mi ubicación
      </Button>
      <Button
        size="lg"
        variant="outline"
        className="h-14 text-base font-semibold"
        onClick={() => {
          setMode('search');
          setStep('search');
        }}
      >
        <Search className="w-5 h-5 mr-2" />
        Buscar mi local
      </Button>
    </div>
  </div>
</Shell>
);
}

return (
  <Shell embedded={embedded}>
    <div className="max-w-lg mx-auto w-full">
      <div className="flex items-center gap-3 mb-4">
        <button
          onClick={() => setStep('choose')}
          className="p-2 rounded-full hover:bg-surface"
          aria-label="Volver"
        >
          <X className="w-5 h-5" />
        </button>
        <h2 className="font-display text-xl font-bold">
          {mode === 'gps' ? 'Locales cercanos' : 'Buscar local'}
        </h2>
      </div>

      {mode === 'search' && (
        <div className="relative mb-4">
          <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" />
          <Input
            autoFocus
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            placeholder="Ciudad, calle o nombre del local..."
            className="pl-10 h-12"
          />
        </div>
      )}

      {mode === 'gps' && !coords && !gpsError && (
        <div className="flex flex-col items-center justify-center py-16 gap-6">
          <div className="relative w-32 h-32">
            <motion.div
              className="absolute inset-0 rounded-full bg-primary/30"
              animate={{ scale: [1, 2], opacity: [0.6, 0] }}
              transition={{ duration: 1.8, repeat: Infinity, ease: 'easeOut' }}
            />
            <motion.div
              className="absolute inset-0 rounded-full bg-primary/30"
              animate={{ scale: [1, 2], opacity: [0.6, 0] }}
              transition={{
                duration: 1.8,
                repeat: Infinity,
                ease: 'easeOut',
                delay: 0.6,
              }}
            />
          <div className="absolute inset-0 m-auto w-16 h-16 rounded-full bg-primary flex items-center justify-center text-primary-foreground">
            <Navigation className="w-7 h-7" />
          </div>
        </div>
        <p className="text-muted-foreground text-sm">Buscando locales cercanos...</p>
      )}
    </div>
  </Shell>
);

```

```

    </div>
  )}

  {loading && (
    <div className="flex items-center justify-center py-10 text-muted-foreground gap-2">
      <Loader2 className="w-4 h-4 animate-spin" /> Cargando...
    </div>
  )}

  <ul className="flex flex-col gap-3">
    {sortedLocales.map(({ l, d }) => {
      const here = d < 100;
      return (
        <li key={l.id}>
          <button
            onClick={() => setPendingConfirm(l)}
            className={`w-full text-left p-4 rounded-xl bg-surface hover:bg-surface-elevated transition-all border-2 ${
              pendingConfirm?.id === l.id
                ? 'border-primary'
                : 'border-transparent'
            }`}
          >
            <div className="flex items-start justify-between gap-3">
              <div className="min-w-0">
                <div className="font-bold truncate">{l.nombre}</div>
                <div className="text-xs text-muted-foreground truncate">
                  {l.direccion} · {l.ciudad}
                </div>
              </div>
              <div className="flex flex-col items-end gap-1 shrink-0">
                {mode === 'gps' && d !== Infinity && (
                  <span className="text-xs font-bold text-primary">
                    {formatDistance(d)}
                  </span>
                )}
                {here && (
                  <span className="text-[10px] font-bold uppercase tracking-wide bg-emerald-500/15 text-emerald-500 px-2 py-0.5 rounded-full">
                    Estás aquí
                  </span>
                )}
              </div>
            </div>
          </button>
        </li>
      );
    })}
    {!loading && sortedLocales.length === 0 && (
      <li className="text-center text-sm text-muted-foreground py-8">
        No se han encontrado locales.
      </li>
    )}
  </ul>
</div>

{/* Confirm modal */}
<AnimatePresence>
  {pendingConfirm && (
    <motion.div
      className="fixed inset-0 z-50 bg-black/60 backdrop-blur-sm flex items-end sm:items-center justify-center p-4"
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      exit={{ opacity: 0 }}
      onClick={() => setPendingConfirm(null)}
    >
      <motion.div
        initial={{ y: 60, opacity: 0 }}
        animate={{ y: 0, opacity: 1 }}
        exit={{ y: 60, opacity: 0 }}
        className="bg-card w-full max-w-sm rounded-2xl p-6 border border-border"
        onClick={(e) => e.stopPropagation()}
      >
        <div className="flex items-center justify-center w-14 h-14 rounded-full bg-primary/15 text-primary mx-auto mb-4">
          <MapPin className="w-7 h-7" />
        </div>
        <p className="text-center text-sm text-muted-foreground mb-1">
          Parece que estás en
        </p>
        <h3 className="font-display text-2xl font-black text-center">
          {pendingConfirm.nombre}
        </h3>
        <p className="text-center text-sm text-muted-foreground mt-1">
          {pendingConfirm.direccion}
        </p>
        <div className="flex flex-col gap-2 mt-6">
          <Button
            size="lg"
            className="h-12"
            onClick={() => confirm(pendingConfirm)}
          >
            <Check className="w-4 h-4 mr-2" />
            Sí, estoy aquí
          </Button>
        </div>
      </motion.div>
    </div>
  )}

```

```

        </Button>
        <Button
          size="lg"
          variant="ghost"
          className="h-12"
          onClick={() => setPendingConfirm(null)}
        >
          Cambiar local
        </Button>
      </div>
    </motion.div>
  </motion.div>
)}
</AnimatePresence>
</Shell>
);
}

function Shell({
  children,
  embedded,
}): {
  children: React.ReactNode;
  embedded?: boolean;
}) {
  return (
    <div
      className={
        embedded
          ? 'p-6'
          : 'min-h-screen bg-background px-4 py-8 flex flex-col'
      }
    >
      {children}
    </div>
  );
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/MenuChatBot.tsx */
/* ===== */
import { useState, useRef, useEffect } from 'react';
import { motion, AnimatePresence } from 'framer-motion';
import { Send, X, Loader2, Plus, Check, ShoppingBag, Minus, Trash2 } from 'lucide-react';
import ReactMarkdown from 'react-markdown';
import { toast } from 'sonner';
import { useQuery } from '@tanstack/react-query';
import { supabase } from '@integrations/supabase/client';
import { useCartStore, useAgeGate } from '@lib/cart-store';
import { useAgotados } from '@hooks/use-agotados';
import { getDrinkImage } from '@lib/drink-image';
import { PRODUCT_IMAGES, MONTADITO_IMAGES_BY_NUMERO } from '@lib/product-images';
import { Monty } from './Monty';
import { getRueda, ruedaComponentes, ruedaAgotada, compLabel, type RuedaMontadito } from '@lib/ruedas';
import { RuedaGourmetDialog } from './RuedaGourmetDialog';
import {
  Dialog,
  DialogContent,
  DialogHeader,
  DialogTitle,
  DialogDescription,
} from '@components/ui/dialog';
import jarraQuijoteImg from '@assets/drinks/jarra-quiote.png';
import jarraSanchoImg from '@assets/drinks/jarra-sancho.png';
import jarraQuijoteLadronVeranoImg from '@assets/drinks/jarra-quiote-ladron-verano.png';
import jarraSanchoLadronVeranoImg from '@assets/drinks/jarra-sancho-ladron-verano.png';
import jarraQuijoteLadronManzanasImg from '@assets/drinks/jarra-quiote-ladron-manzanas.png';
import jarraSanchoLadronManzanasImg from '@assets/drinks/jarra-sancho-ladron-manzanas.png';

type Msg = { role: 'user' | 'assistant'; content: string };

type Product = {
  id: string;
  nombre: string;
  precio: number;
  foto_url: string | null;
  numero?: string | null;
  contiene_alcohol?: boolean;
  seccion?: string | null;
};

type FlyingItem = {
  id: number;
  product: Product;
  fromX: number;
  fromY: number;
  toX: number;
  toY: number;
};

const CHAT_URL = `${import.meta.env.VITE_SUPABASE_URL}/functions/v1/chat-menu`;

```

```

const TYPEWRITER_CPS = 150;
const JARRA_SECTIONS = new Set(['Jarras Heladas', 'Cerveza Premium']);
const SANCHO_EXTRA = 1.0;
// Variantes que dependen de un ingrediente concreto (si falta, se desactiva la opción).
const ALITAS_VARIANT_ING: Record<string, string> = { bbq: 'Salsa BBQ', brava: 'Salsa brava' };
const NACHOS_VARIANT_ING: Record<string, string> = { bacon: 'Bacon ahumado', guacamole: 'Guacamole' };
const GILDA_VARIANT_ING: Record<string, string> = { boqueron: 'Gilda boquerón', anchoa: 'Gilda anchoa' };

const SUGGESTIONS = [
  '¿Qué montaditos de pollo tenéis?',
  'Soy alérgico al huevo, ¿qué puedo pedir?',
  'Recomiéndame algo picante',
  'Opciones vegetarianas',
];

// Normalises a product name for fuzzy matching
function normalizeName(s: string) {
  return s.normalize('NFD').replace(/p{M}/gu, '').toLowerCase().trim();
}

const isJarraProduct = (p: Product) =>
  (!p.seccion && JARRA_SECTIONS.has(p.seccion)) || normalizeName(p.nombre).includes('jarra');

const isGildaProduct = (p: Product) => {
  const n = normalizeName(p.nombre);
  return n === 'gildas' || n === 'gilda';
};

const isNachosProduct = (p: Product) => normalizeName(p.nombre) === 'nachos';

const isAlitasProduct = (p: Product) => normalizeName(p.nombre) === 'alitas de pollo';

function parseMessage(raw: string): { text: string; productNames: string[] } {
  const productNames: string[] = [];
  const text = raw
    .replace(/\[add:([\^\]]+)\]\]/g, (_, name) => {
      productNames.push(name.trim());
      return '';
    })
    .replace(/\n{3,}/g, '\n\n')
    .trim();
  const seen = new Set<string>();
  const unique = productNames.filter((n) => {
    const k = n.toLowerCase();
    if (seen.has(k)) return false;
    seen.add(k);
    return true;
  });
  return { text, productNames: unique };
}

function findProduct(name: string, products: Product[] | undefined): Product | null {
  if (!products) return null;
  const target = name.trim().toLowerCase();

  const numMatch = target.match(/^#?(\d+)(?:s|$)/);
  if (numMatch) {
    const byNum = products.find((p) => p.numero === numMatch[1]);
    if (byNum) return byNum;
  }

  return (
    products.find((p) => p.nombre.toLowerCase() === target) ||
    products.find((p) => p.nombre.toLowerCase().includes(target)) ||
    products.find((p) => target.includes(p.nombre.toLowerCase())) ||
    null
  );
};

function resolveProductImage(product: Product): string | null {
  const rueda = getRueda(product.numero);
  if (rueda?.img) return rueda.img;
  if (product.numero && MONTADITO_IMAGES_BY_NUMERO[product.numero]) {
    return MONTADITO_IMAGES_BY_NUMERO[product.numero];
  }
  return getDrinkImage(product.nombre, product.foto_url);
}

// — Product chip rendered below AI messages —————

interface ProductChipProps {
  product: Product;
  onAdd: (product: Product, originRect: DOMRect) => void;
  agotado?: boolean;
}

function ProductChip({ product, onAdd, agotado = false }: ProductChipProps) {
  const [added, setAdded] = useState(false);
  const ref = useRef<HTMLButtonElement>(null);
  const imageSrc = resolveProductImage(product);

  const handleClick = () => {

```

```

    if (agotado) return; // producto agotado: no se puede añadir desde el chat
    if (!ref.current) return;
    onAdd(product, ref.current.getBoundingClientRect());
    setAdded(true);
    setTimeout(() => setAdded(false), 1200);
  };

  return (
    <motion.button
      ref={ref}
      onClick={handleClick}
      whileTap={agotado ? undefined : { scale: 0.96 }}
      className={`group flex items-center gap-3 w-full p-2 pr-3 rounded-2xl bg-background border transition-colors text-left ${
        agotado ? 'border-destructive/40 opacity-60' : 'border-border hover:border-primary/60 hover:bg-primary/5'
      }`}
    >
      <div className="relative w-12 h-12 rounded-xl overflow-hidden bg-muted shrink-0">
        {imageSrc ? (
          <img
            src={imageSrc}
            alt={product.nombre}
            className="w-full h-full object-contain"
            loading="lazy"
          />
        ) : (
          <div className="w-full h-full flex items-center justify-center text-lg">🍌</div>
        )}
      </div>
      <div className="flex-1 min-w-0">
        <p className="text-xs font-bold text-foreground truncate">
          {product.numero} && (
            <span className="text-muted-foreground font-mono mr-1">#{product.numero}</span>
          )
          {product.nombre}
        </p>
        <p className="text-[11px] text-muted-foreground">{product.precio.toFixed(2)}€</p>
      </div>
      {agotado ? (
        <span className="shrink-0 text-[9px] font-black uppercase tracking-wide text-destructive">
          Agotado
        </span>
      ) : (
        <span
          className={`shrink-0 w-8 h-8 rounded-full flex items-center justify-center transition-colors ${
            added
              ? 'bg-accent text-accent-foreground'
              : 'bg-primary text-primary-foreground group-hover:bg-primary/90'
          }`}
        >
          {added ? <Check className="w-4 h-4" /> : <Plus className="w-4 h-4" />}
        </span>
      )}
    </motion.button>
  );
}

// — Main component —————

export function MenuChatBot() {
  const [open, setOpen] = useState(false);
  const [cartOpen, setCartOpen] = useState(false);
  const [messages, setMessages] = useState<Msg[]>([]);
  const [input, setInput] = useState('');
  const [loading, setLoading] = useState(false);
  const [flying, setFlying] = useState<FlyingItem[]>([]);
  const [cartPulse, setCartPulse] = useState(false);

  // Variant dialog state
  const [pendingVariantProduct, setPendingVariantProduct] = useState<Product | null>(null);
  const [jarraOpen, setJarraOpen] = useState(false);
  const [gildaOpen, setGildaOpen] = useState(false);
  const [nachosOpen, setNachosOpen] = useState(false);
  const [alitasOpen, setAlitasOpen] = useState(false);
  const [ruedaOpen, setRuedaOpen] = useState(false);

  const scrollRef = useRef<HTMLDivElement>(null);
  const cartIconRef = useRef<HTMLButtonElement>(null);
  const flyingIdRef = useRef(0);
  const pendingRectRef = useRef<DOMRect | null>(null);

  const addItem = useCartStore((s) => s.addItem);
  const { isProductoAgotado, isIngredienteAgotado } = useAgotados();

  // Agotado efectivo: marcado/ingrediente, o todas las variantes agotadas (alitas/nachos).
  const productoEfectivoAgotado = (product: Product) => {
    if (isProductoAgotado(product.id)) return true;
    const ruedaDef = getRueda(product.numero);
    if (ruedaDef) return ruedaAgotada(ruedaDef, ruedaMontaditos, isProductoAgotado);
    const n = normalizeName(product.nombre);
    if (n === 'alitas de pollo') return Object.values(ALITAS_VARIANT_ING).every(isIngredienteAgotado);
    if (n === 'nachos') return Object.values(NACHOS_VARIANT_ING).every(isIngredienteAgotado);
    if (n === 'gildas' || n === 'gilda') return Object.values(GILDA_VARIANT_ING).every(isIngredienteAgotado);
  };
}

```

```

    return false;
  });
  const itemCount = useCartStore((s) => s.itemCount);
  const cartItems = useCartStore((s) => s.items);
  const updateQuantity = useCartStore((s) => s.updateQuantity);
  const removeItem = useCartStore((s) => s.removeItem);
  const cartTotal = useCartStore((s) => s.total);
  const requestAlcohol = useAgeGate((s) => s.request);

  const fullTextRef = useRef<string>('');
  const revealedRef = useRef<number>(0);
  const intervalRef = useRef<number | null>(null);
  const abortRef = useRef<AbortController | null>(null);

  const { data: products } = useQuery({
    queryKey: ['all-products-chatbot'],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('menu_productos')
        .select('id, nombre, precio, foto_url, numero, contiene_alcohol, seccion')
        .eq('disponible', true);
      if (error) throw error;
      return data as Product[];
    },
    staleTime: 5 * 60 * 1000,
  });

  // Montaditos (para resolver los componentes de las MontyRuedas).
  const ruedaMontaditos: RuedaMontadito[] = (products ?? []).map((p) => ({ id: p.id, numero: p.numero ?? null, nombre: p.nombre }));

  useEffect(() => {
    scrollRef.current?.scrollTo({ top: scrollRef.current.scrollHeight, behavior: 'smooth' });
  }, [messages, loading]);

  useEffect(() => {
    return () => {
      if (intervalRef.current) window.clearInterval(intervalRef.current);
      abortRef.current?.abort();
    };
  }, []);

  // — Typewriter helpers —————

  const startTypewriter = () => {
    if (intervalRef.current) return;
    const tickMs = 1000 / TYPEWRITER_CPS;
    intervalRef.current = window.setInterval(() => {
      if (revealedRef.current >= fullTextRef.current.length) return;
      revealedRef.current = Math.min(revealedRef.current + 1, fullTextRef.current.length);
      const visible = fullTextRef.current.slice(0, revealedRef.current);
      setMessages((prev) =>
        prev.map((m, i) => (i === prev.length - 1 ? { ...m, content: visible } : m))
      );
    }, tickMs);
  };

  const stopTypewriter = () => {
    if (intervalRef.current) {
      window.clearInterval(intervalRef.current);
      intervalRef.current = null;
    }
  };

  const flushTypewriter = async () => {
    while (revealedRef.current < fullTextRef.current.length) {
      await new Promise((r) => setTimeout(r, 30));
    }
    stopTypewriter();
  };

  // — Flying cart animation —————

  const triggerFlyingAnimation = (imgUrl: string | null, nombre: string, originRect: DOMRect) => {
    const cartRect = cartIconRef.current?.getBoundingClientRect();
    if (!cartRect) return;
    const id = ++flyingIdRef.current;
    setFlying((prev) => [
      ...prev,
      {
        id,
        product: { id: '', nombre, precio: 0, foto_url: imgUrl },
        fromX: originRect.left,
        fromY: originRect.top,
        toX: cartRect.left + cartRect.width / 2 - 20,
        toY: cartRect.top + cartRect.height / 2 - 20,
      },
    ]);
    setTimeout(() => {
      setCartPulse(true);
      setTimeout(() => setCartPulse(false), 400);
    }, 650);
    setTimeout(() => {

```

```

    setFlying((prev) => prev.filter((f) => f.id !== id));
  }, 900);
};

// — Variant add helpers —————

const getJarraImgs = (p: Product) => {
  const n = normalizeName(p.nombre);
  if (n.includes('ladron de verano') || n.includes('tinto de verano'))
    return { q: jarraQuijoteLadronVeranoImg, s: jarraSanchoLadronVeranoImg };
  if (n.includes('ladron de manzana'))
    return { q: jarraQuijoteLadronManzanasImg, s: jarraSanchoLadronManzanasImg };
  return { q: jarraQuijoteImg, s: jarraSanchoImg };
};

const addJarraVariant = (size: 'quijote' | 'sancho') => {
  if (!pendingVariantProduct) return;
  const p = pendingVariantProduct;
  const label = size === 'quijote' ? 'Jarra Quijote' : 'Jarra Sancho';
  const precio = size === 'quijote' ? p.precio : p.precio + SANCHO_EXTRA;
  const { q, s } = getJarraImgs(p);
  const img = size === 'quijote' ? q : s;
  const payload = {
    id: `${p.id}:${size}`,
    productoId: p.id,
    variant: size,
    variantLabel: label,
    nombre: `${p.nombre} · ${label}`,
    precio,
    foto_url: img,
    contiene_alcohol: !!p.contiene_alcohol,
  };
  if (p.contiene_alcohol) {
    requestAlcohol(payload);
  } else {
    addItem(payload);
  }
  pendingRectRef.current && triggerFlyingAnimation(img, payload.nombre, pendingRectRef.current);
  toast.success(`${payload.nombre} añadido al carrito`);
  setJarraOpen(false);
  setPendingVariantProduct(null);
};

const addGildaVariant = (tipo: 'boqueron' | 'anchoa') => {
  if (!pendingVariantProduct) return;
  if (isIngredienteAgotado(GILDA_VARIANT_ING[tipo])) return;
  const p = pendingVariantProduct;
  const label = tipo === 'boqueron' ? 'Boquerón' : 'Anchoa';
  const img = tipo === 'boqueron' ? PRODUCT_IMAGES._gildaBoqueron : PRODUCT_IMAGES._gildaAnchoa;
  const nombre = tipo === 'boqueron' ? 'Gilda de boquerón' : 'Gilda de anchoa';
  const payload = {
    id: `${p.id}:${tipo}`,
    productoId: p.id,
    variant: tipo,
    variantLabel: label,
    nombre,
    precio: p.precio,
    foto_url: img,
    contiene_alcohol: false,
  };
  addItem(payload);
  pendingRectRef.current && triggerFlyingAnimation(img, nombre, pendingRectRef.current);
  toast.success(`${nombre} añadida al carrito`);
  setGildaOpen(false);
  setPendingVariantProduct(null);
};

const addNachosVariant = (salsa: 'bacon' | 'guacamole') => {
  if (!pendingVariantProduct) return;
  if (isIngredienteAgotado(NACHOS_VARIANT_ING[salsa])) return;
  const p = pendingVariantProduct;
  const label = salsa === 'bacon' ? 'Cheddar y bacon ahumado' : 'Cheddar y guacamole';
  const img = salsa === 'bacon' ? PRODUCT_IMAGES._nachosBacon : PRODUCT_IMAGES._nachosGuaca;
  const nombre = `Nachos · ${label}`;
  const payload = {
    id: `${p.id}:${salsa}`,
    productoId: p.id,
    variant: salsa,
    variantLabel: label,
    nombre,
    precio: p.precio,
    foto_url: img,
    contiene_alcohol: false,
  };
  addItem(payload);
  pendingRectRef.current && triggerFlyingAnimation(img, nombre, pendingRectRef.current);
  toast.success('Nachos añadidos al carrito');
  setNachosOpen(false);
  setPendingVariantProduct(null);
};

const addAlitasVariant = (sabor: 'bbq' | 'brava') => {

```



```

if (!pendingVariantProduct) return;
if (isIngredienteAgotado(ALITAS_VARIANT_ING[sabor])) return;
const p = pendingVariantProduct;
const label = sabor === 'bbq' ? 'BBQ' : 'Brava';
const img = sabor === 'bbq' ? PRODUCT_IMAGES._alitasBbq : PRODUCT_IMAGES._alitasBrava;
const nombre = `Alitas de pollo · ${label}`;
const payload = {
  id: `${p.id}::${sabor}`,
  productoId: p.id,
  variant: sabor,
  variantLabel: label,
  nombre,
  precio: p.precio,
  foto_url: img,
  contiene_alcohol: false,
};
addItem(payload);
pendingRectRef.current && triggerFlyingAnimation(img, nombre, pendingRectRef.current);
toast.success(`Alitas ${label} añadidas al carrito`);
setAlitasOpen(false);
setPendingVariantProduct(null);
};

// Añade una MontyRueda (con sus montaditos en `componentes`) con animación.
const addRuedaToCart = (product: Product, cartId: string, componentes: string[], originRect: DOMRect | null) => {
  const resolvedImage = resolveProductImage(product);
  const payload = {
    id: cartId,
    productoId: product.id,
    nombre: product.nombre,
    precio: product.precio,
    foto_url: resolvedImage,
    componentes,
  };
  const cartRect = cartIconRef.current?.getBoundingClientRect();
  if (cartRect && originRect) {
    const id = ++flyingIdRef.current;
    setFlying((prev) => [
      ...prev,
      {
        id,
        product: { ...product, foto_url: resolvedImage },
        fromX: originRect.left,
        fromY: originRect.top,
        toX: cartRect.left + cartRect.width / 2 - 20,
        toY: cartRect.top + cartRect.height / 2 - 20,
      },
    ]);
    setTimeout(() => {
      addItem(payload);
      setCartPulse(true);
      setTimeout(() => setCartPulse(false), 400);
    }, 650);
    setTimeout(() => setFlying((prev) => prev.filter((f) => f.id !== id)), 900);
  } else {
    addItem(payload);
  }
  toast.success(`${product.nombre} añadida al carrito`);
};

// — Main add handler (checks for variants first) —————
const handleAddProduct = (product: Product, originRect: DOMRect) => {
  if (productoEfectivoAgotado(product)) {
    toast.error('Ese producto está agotado ahora mismo');
    return;
  }
  if (isJarraProduct(product)) {
    setPendingVariantProduct(product);
    pendingRectRef.current = originRect;
    setJarraOpen(true);
    return;
  }
  if (isGildaProduct(product)) {
    setPendingVariantProduct(product);
    pendingRectRef.current = originRect;
    setGildaOpen(true);
    return;
  }
  if (isNachosProduct(product)) {
    setPendingVariantProduct(product);
    pendingRectRef.current = originRect;
    setNachosOpen(true);
    return;
  }
  if (isAlitasProduct(product)) {
    setPendingVariantProduct(product);
    pendingRectRef.current = originRect;
    setAlitasOpen(true);
    return;
  }
  const ruedaDef = getRueda(product.numero);

```

```

if (ruedaDef) {
  if (ruedaDef.selectCount) {
    // MontyRueda Gourmets: abrir selector "elige 5".
    setPendingVariantProduct(product);
    pendingRectRef.current = originRect;
    setRuedaOpen(true);
    return;
  }
  // Rueda fija: añadir con sus montaditos como componentes.
  addRuedaToCart(product, product.id, ruedaComponentes(ruedaDef, ruedaMontaditos).map(complabel), originRect);
  return;
}

const resolvedImage = resolveProductImage(product);
const cartRect = cartIconRef.current?.getBoundingClientRect();
if (cartRect) {
  const id = ++flyingIdRef.current;
  setFlying((prev) => [
    ...prev,
    {
      id,
      product: { ...product, foto_url: resolvedImage },
      fromX: originRect.left,
      fromY: originRect.top,
      toX: cartRect.left + cartRect.width / 2 - 20,
      toY: cartRect.top + cartRect.height / 2 - 20,
    },
  ]);
  setTimeout(() => {
    addItem({
      id: product.id,
      productoId: product.id,
      nombre: product.nombre,
      numero: product.numero ?? null,
      precio: product.precio,
      foto_url: resolvedImage,
    });
    setCartPulse(true);
    setTimeout(() => setCartPulse(false), 400);
  }, 650);
  setTimeout(() => {
    setFlying((prev) => prev.filter((f) => f.id !== id));
  }, 900);
} else {
  addItem({
    id: product.id,
    productoId: product.id,
    nombre: product.nombre,
    numero: product.numero ?? null,
    precio: product.precio,
    foto_url: resolvedImage,
  });
}

toast.success(`${product.nombre} añadido al carrito`);
};

// — Send message —————

const send = async (text: string) => {
  const trimmed = text.trim();
  if (!trimmed || loading) return;

  const userMsg: Msg = { role: 'user', content: trimmed };
  const newMessages = [...messages, userMsg];
  setMessages(newMessages);
  setInput('');
  setLoading(true);

  try {
    abortRef.current?.abort();
    abortRef.current = new AbortController();

    const resp = await fetch(CHAT_URL, {
      method: 'POST',
      signal: abortRef.current.signal,
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY}`,
      },
      body: JSON.stringify({ messages: newMessages, session_id: sessionStorage.getItem('montaditos_session') }),
    });

    if (resp.status === 429) {
      toast.error('Demasiadas peticiones, espera un momento.');
```

```

    }
    if (!resp.ok || !resp.body) throw new Error('Error en la respuesta');

    const reader = resp.body.getReader();
    const decoder = new TextDecoder();
    let buffer = '';
    let streamDone = false;

    fullTextRef.current = '';
    revealedRef.current = 0;
    setMessages((prev) => [...prev, { role: 'assistant', content: '' }]);
    startTypewriter();

    while (!streamDone) {
      const { done, value } = await reader.read();
      if (done) break;
      buffer += decoder.decode(value, { stream: true });

      let nl: number;
      while ((nl = buffer.indexOf('\n')) !== -1) {
        let line = buffer.slice(0, nl);
        buffer = buffer.slice(nl + 1);
        if (line.endsWith('\r')) line = line.slice(0, -1);
        if (line.startsWith(':') || line.trim() === '') continue;
        if (!line.startsWith('data: ')) continue;
        const json = line.slice(6).trim();
        if (json === '[DONE]') {
          streamDone = true;
          break;
        }
      }
      try {
        const parsed = JSON.parse(json);
        const content = parsed.choices?.[0]?.delta?.content;
        if (content) fullTextRef.current += content;
      } catch {
        buffer = line + '\n' + buffer;
        break;
      }
    }

    await flushTypewriter();
  } catch (err) {
    if ((err as Error)?.name === 'AbortError') {
      stopTypewriter();
      setMessages((prev) => prev.slice(0, -1));
    } else {
      console.error(err);
      toast.error('Error al conectar con el asistente');
      stopTypewriter();
      setMessages((prev) => prev.slice(0, -1));
    }
  } finally {
    setLoading(false);
  }
};

const isStreamingLast = (idx: number) =>
  idx === messages.length - 1 &&
  messages[idx].role === 'assistant' &&
  revealedRef.current < fullTextRef.current.length;

const jarraImgs = pendingVariantProduct ? getJarraImgs(pendingVariantProduct) : { q: jarraQuijoteImg, s: jarraSanchoImg };

// — Render —————

return (
  <>
    { /* Floating Monty button */ }
    <div className="fixed bottom-5 left-4 z-40 flex items-end gap-2">
      <motion.button
        initial={{ scale: 0, rotate: -20 }}
        animate={{ scale: 1, rotate: 0 }}
        whileTap={{ scale: 0.9 }}
        whileHover={{ scale: 1.05 }}
        onClick={() => setOpen(true)}
        className="relative w-16 h-16 rounded-full bg-gradient-to-br from-primary/20 to-primary/5 shadow-xl flex items-center justify-center border-2 border-primary/30"
        aria-label="Habla con Monty"
      >
        <Monty pose="waving" size={56} animate="float" />
        <span className="absolute -top-1 -right-1 w-3 h-3 rounded-full bg-accent border-2 border-background animate-pulse" />
      </motion.button>

      <motion.div
        initial={{ opacity: 0, x: -10, scale: 0.8 }}
        animate={{ opacity: 1, x: 0, scale: 1 }}
        transition={{ delay: 0.4, type: 'spring' }}
        onClick={() => setOpen(true)}
        className="relative mb-2 cursor-pointer bg-background border border-primary/30 shadow-lg rounded-2xl rounded-bl-sm px-3 py-2 max-w-[170px]"
      >

```

```

    <span className="absolute -left-1.5 bottom-2 w-3 h-3 bg-background border-1 border-b border-primary/30 rotate-45" />
    <p className="text-[11px] font-bold text-foreground leading-tight">
      ¡Hola! Soy <span className="text-primary">Monty</span> 🍌
    </p>
    <p className="text-[10px] text-muted-foreground leading-tight mt-0.5">
      ¡Yo te ayudo a elegir!
    </p>
  </motion.div>
</div>

<AnimatePresence>
  {open && (
    <>
      <motion.div
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        exit={{ opacity: 0 }}
        onClick={() => setOpen(false)}
        className="fixed inset-0 z-40 bg-black/60 backdrop-blur-sm"
      />
      <motion.div
        initial={{ y: '100%' }}
        animate={{ y: 0 }}
        exit={{ y: '100%' }}
        transition={{ type: 'spring', damping: 30, stiffness: 300 }}
        className="fixed inset-x-0 bottom-0 z-50 bg-background rounded-t-3xl shadow-2xl flex flex-col max-h-[85vh] h-[85vh] max-w-lg mx-auto border-t border-border"
      >
        {/* Header */}
        <div className="flex items-center justify-between p-4 border-b border-border">
          <div className="flex items-center gap-2">
            <div className="w-10 h-10 flex items-center justify-center overflow-hidden">
              <Monty pose="waving" size={40} animate="none" />
            </div>
            <div>
              <h3 className="font-display font-bold text-foreground text-sm">
                Monty · Asistente del menú
              </h3>
              <p className="text-[10px] text-muted-foreground">¡Yo te ayudo a elegir!</p>
            </div>
          </div>

          <div className="flex items-center gap-1">
            <motion.button
              ref={cartIconRef}
              onClick={() => setCartOpen((v) => !v)}
              animate={cartPulse ? { scale: [1, 1.25, 1] } : { scale: 1 }}
              transition={{ duration: 0.4 }}
              className={
                { relative w-10 h-10 rounded-full border flex items-center justify-center transition-colors $
                  {
                    cartOpen
                      ? 'bg-primary text-primary-foreground border-primary'
                      : 'bg-gradient-to-br from-primary/20 to-primary/5 border-primary/40 text-primary'
                  }
                }
              }
              aria-label="Carrito"
            >
              <ShoppingBag className="w-4 h-4" />
            </motion.button>
            <AnimatePresence>
              {itemCount() > 0 && (
                <motion.span
                  key={itemCount()}
                  initial={{ scale: 0 }}
                  animate={{ scale: 1 }}
                  exit={{ scale: 0 }}
                  className="absolute -top-1 -right-1 min-w-[18px] h-[18px] px-1 rounded-full bg-accent text-accent-foreground text-[10px] font-bold flex items-center justify-center border border-background"
                >
                  {itemCount()}
                </motion.span>
              )}
            </AnimatePresence>
          </div>
        </div>

        <button
          onClick={() => setOpen(false)}
          className="w-8 h-8 rounded-full hover:bg-muted flex items-center justify-center"
        >
          <X className="w-4 h-4 text-foreground" />
        </button>
      </div>
    </div>

    {/* Mini cart panel */}
    <AnimatePresence initial={false}>
      {cartOpen && (
        <motion.div
          initial={{ height: 0, opacity: 0 }}
          animate={{ height: 'auto', opacity: 1 }}
          exit={{ height: 0, opacity: 0 }}
          transition={{ duration: 0.25 }}
          className="overflow-hidden border-b border-border bg-muted/40"
        >
          <div className="p-3 max-h-[40vh] overflow-y-auto">

```

```

{cartItems.length === 0 ? (
  <div className="text-center py-6">
    <ShoppingBag className="w-8 h-8 mx-auto text-muted-foreground/50 mb-2" />
    <p className="text-xs text-muted-foreground">
      Tu cesta está vacía. ¡Pídemle una recomendación!
    </p>
  </div>
) : (
  <>
    <div className="flex items-center justify-between mb-2 px-1">
      <p className="text-[11px] uppercase tracking-wider font-bold text-muted-foreground">
        Tu cesta
      </p>
      <p className="text-xs font-bold text-primary">
        {cartTotal().toFixed(2)}€
      </p>
    </div>
    <div className="space-y-1.5">
      {cartItems.map((item) => {
        const img = getDrinkImage(item.nombre, item.foto_url);
        return (
          <motion.div
            key={item.id}
            layout
            initial={{ opacity: 0, x: -10 }}
            animate={{ opacity: 1, x: 0 }}
            exit={{ opacity: 0, x: 10 }}
            className="flex items-center gap-2 p-1.5 rounded-xl bg-background border border-border"
          >
            <div className="w-10 h-10 rounded-lg overflow-hidden bg-muted shrink-0 flex items-center justify-
center">
              {img ? (
                <img
                  src={img}
                  alt={item.nombre}
                  className="w-full h-full object-contain"
                  loading="lazy"
                />
              ) : (
                <span className="text-base">🍷</span>
              )}
            </div>
            <div className="flex-1 min-w-0">
              <p className="text-[11px] font-bold text-foreground truncate leading-tight">
                {item.nombre}
              </p>
              <p className="text-[10px] text-muted-foreground">
                {(item.precio * item.cantidad).toFixed(2)}€
              </p>
            </div>
            <div className="flex items-center gap-1 shrink-0">
              <button
                onClick={() => updateQuantity(item.id, item.cantidad - 1)}
                className="w-6 h-6 rounded-full bg-muted hover:bg-muted/70 flex items-center justify-center"
                aria-label="Quitar uno"
              >
                <Minus className="w-3 h-3" />
              </button>
              <span className="text-xs font-bold w-4 text-center">
                {item.cantidad}
              </span>
              <button
                onClick={() => updateQuantity(item.id, item.cantidad + 1)}
                className="w-6 h-6 rounded-full bg-primary text-primary-foreground hover:bg-primary/90 flex
items-center justify-center"
                aria-label="Añadir uno"
              >
                <Plus className="w-3 h-3" />
              </button>
              <button
                onClick={() => removeItem(item.id)}
                className="w-6 h-6 rounded-full text-muted-foreground hover:text-destructive hover:bg-
destructive/10 flex items-center justify-center ml-0.5"
                aria-label="Eliminar"
              >
                <Trash2 className="w-3 h-3" />
              </button>
            </div>
          </motion.div>
        );
      })};
    </div>
  </>
)
</AnimatePresence>

{/* Messages */}
<div ref={scrollRef} className="flex-1 overflow-y-auto p-4 space-y-3">
  {messages.length === 0 && (

```

```

<div className="space-y-3">
  <div className="text-center py-4">
    <div className="flex justify-center mb-2">
      <Monty pose="waving" size={96} animate="float" />
    </div>
    <h4 className="font-display font-bold text-foreground">
      ¡Hola! Soy Monty 🍌
    </h4>
    <p className="text-xs text-muted-foreground mt-1 px-4">
      Pregúntame por ingredientes, alérgenos o píde una recomendación
    </p>
  </div>
  <div className="space-y-2">
    {SUGGESTIONS.map((s) => (
      <button
        key={s}
        onClick={() => send(s)}
        className="w-full text-left px-3 py-2.5 text-xs rounded-xl border border-border hover:border-primary/50
        hover:bg-primary/5 transition-colors text-foreground"
      >
        {s}
      </button>
    ))}
  </div>
</div>
))

{messages.map((m, i) => {
  if (m.role === 'user') {
    return (
      <div key={i} className="flex justify-end">
        <div className="max-w-[85%] rounded-2xl px-3.5 py-2.5 text-sm bg-primary text-primary-foreground rounded-br-
sm">
          {m.content}
        </div>
      </div>
    );
  }
})

const { text, productNames } = parseMessage(m.content);
const stillStreaming = isStreamingLast(i);
const matchedProducts = productNames
  .map((n) => findProduct(n, products))
  .filter((p): p is Product => p !== null);

return (
  <div key={i} className="flex flex-col items-start gap-2 max-w-[90%]">
    {(text || stillStreaming) && (
      <div className="rounded-2xl rounded-bl-sm px-3.5 py-2.5 text-sm bg-muted text-foreground">
        {text ? (
          <div className="prose prose-sm max-w-none prose-p:my-1 prose-ul:my-1 prose-li:my-0 prose-strong:text-
foreground prose-headings:text-foreground text-foreground">
            <ReactMarkdown>{text}</ReactMarkdown>
          </div>
        ) : (
          <Loader2 className="w-4 h-4 animate-spin" />
        )}
      </div>
    )}

    {!stillStreaming && matchedProducts.length > 0 && (
      <div className="flex flex-col gap-1.5 w-full">
        {matchedProducts.map((p) => (
          <ProductChip key={p.id} product={p} onAdd={handleAddProduct} agotado={productoEfectivoAgotado(p)} />
        ))}
      </div>
    )}
  </div>
);
}}
</div>

{/* Input */}
<form
  onSubmit={(e) => {
    e.preventDefault();
    send(input);
  }}
  className="p-3 border-t border-border flex gap-2"
>
  <input
    value={input}
    onChange={(e) => setInput(e.target.value)}
    placeholder="Escribe tu pregunta..."
    disabled={loading}
    className="flex-1 px-4 py-2.5 text-sm rounded-full bg-muted border-0 focus:outline-none focus:ring-2 focus:ring-
primary/30 text-foreground placeholder:text-muted-foreground disabled:opacity-50"
  />
  <button
    type="submit"
    disabled={loading || !input.trim()}
    className="w-10 h-10 rounded-full bg-primary text-primary-foreground flex items-center justify-center

```

```

disabled:opacity-40 shrink-0"
    >
      {loading ? (
        <Loader2 className="w-4 h-4 animate-spin" />
      ) : (
        <Send className="w-4 h-4" />
      )}
    </button>
  </form>
</motion.div>
</>
)}
</AnimatePresence>

{/* Flying items layer */}
<AnimatePresence>
  {flying.map((f) => (
    <motion.div
      key={f.id}
      initial={{ x: f.fromX, y: f.fromY, scale: 1, opacity: 1 }}
      animate={{ x: f.toX, y: f.toY, scale: 0.3, opacity: 0.9 }}
      exit={{ opacity: 0 }}
      transition={{ duration: 0.7, ease: [0.5, 0, 0.75, 0] }}
      className="fixed top-0 left-0 z-[60] w-10 h-10 rounded-full overflow-hidden shadow-2xl ring-2 ring-primary pointer-events-none bg-background"
    >
      {f.product.foto_url ? (
        <img
          src={f.product.foto_url}
          alt=""
          className="w-full h-full object-contain"
          loading="lazy"
        />
      ) : (
        <div className="w-full h-full flex items-center justify-center text-base bg-muted">
          🍷
        </div>
      )}
    </motion.div>
  )]}
</AnimatePresence>

{/* — Jarra variant dialog ————— */}
<Dialog
  open={jarraOpen}
  onOpenChange={(v) => {
    setJarraOpen(v);
    if (!v) setPendingVariantProduct(null);
  }}
>
  <DialogContent className="max-w-sm rounded-xl">
    <DialogHeader>
      <DialogTitle className="font-display text-lg">¿Qué jarra prefieres?</DialogTitle>
      <DialogDescription className="text-sm text-muted-foreground pt-1">
        {pendingVariantProduct?.nombre}
      </DialogDescription>
    </DialogHeader>
    <div className="grid grid-cols-2 gap-3 pt-2">
      <button
        onClick={() => addJarraVariant('quijote')}
        className="flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 border-border hover:border-primary
        hover:bg-primary/5 transition-all active:scale-95"
      >
        <img
          src={jarraImgs.q}
          alt="Jarra Quijote"
          className="h-24 w-auto object-contain"
        />
        <span className="font-display font-bold text-base text-foreground">Quijote</span>
        <span className="text-xs text-muted-foreground">Tamaño estándar</span>
        <span className="text-lg font-black text-gold mt-1">
          {pendingVariantProduct?.precio.toFixed(2)} ?? '-' €
        </span>
      </button>
      <button
        onClick={() => addJarraVariant('sancho')}
        className="flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 border-border hover:border-primary
        hover:bg-primary/5 transition-all active:scale-95"
      >
        <img
          src={jarraImgs.s}
          alt="Jarra Sancho"
          className="h-24 w-auto object-contain"
        />
        <span className="font-display font-bold text-base text-foreground">Sancho</span>
        <span className="text-xs text-muted-foreground">Tamaño grande</span>
        <span className="text-lg font-black text-gold mt-1">
          {pendingVariantProduct
            ? (pendingVariantProduct.precio + SANCHO_EXTRA).toFixed(2)
            : '-'}{ ' ' }
          €
        </span>
      </button>
    </div>
  </DialogContent>

```

```

        </button>
    </div>
</DialogContent>
</Dialog>

{/* — Gilda variant dialog ————— */}
<Dialog
  open={gildaOpen}
  onOpenChange={(v) => {
    setGildaOpen(v);
    if (!v) setPendingVariantProduct(null);
  }}
>
  <DialogContent className="max-w-sm rounded-xl">
    <DialogHeader>
      <DialogTitle className="font-display text-lg">¿Qué gilda prefieres?</DialogTitle>
      <DialogDescription className="text-sm text-muted-foreground pt-1">
        Elige el sabor de tu gilda
      </DialogDescription>
    </DialogHeader>
    <div className="grid grid-cols-2 gap-3 pt-2">
      <button
        onClick={() => addGildaVariant('boqueron')}
        disabled={isIngredienteAgotado(GILDA_VARIANT_ING.boqueron)}
        className={`flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 transition-all
        ${isIngredienteAgotado(GILDA_VARIANT_ING.boqueron) ? 'border-destructive/40 opacity-50' : 'border-border hover:border-primary
        hover:bg-primary/5 active:scale-95'}`}
      >
        <img
          src={PRODUCT_IMAGES._gildaBoqueron}
          alt="Gilda de boquerón"
          className="h-24 w-auto object-contain"
          loading="lazy"
        />
        <span className="font-display font-bold text-base text-foreground">Boquerón</span>
        {isIngredienteAgotado(GILDA_VARIANT_ING.boqueron) ? (
          <span className="text-[11px] font-black uppercase text-destructive mt-1">Agotado</span>
        ) : (
          <>
            <span className="text-xs text-muted-foreground text-center">Suave y fresco</span>
            <span className="text-lg font-black text-gold mt-1">
              {pendingVariantProduct?.precio.toFixed(2) ?? '-'} €
            </span>
          </>
        )}
      </button>
      <button
        onClick={() => addGildaVariant('anchoa')}
        disabled={isIngredienteAgotado(GILDA_VARIANT_ING.anchoa)}
        className={`flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 transition-all
        ${isIngredienteAgotado(GILDA_VARIANT_ING.anchoa) ? 'border-destructive/40 opacity-50' : 'border-border hover:border-primary hover:bg-
        primary/5 active:scale-95'}`}
      >
        <img
          src={PRODUCT_IMAGES._gildaAnchoa}
          alt="Gilda de anchoa"
          className="h-24 w-auto object-contain"
          loading="lazy"
        />
        <span className="font-display font-bold text-base text-foreground">Anchoa</span>
        {isIngredienteAgotado(GILDA_VARIANT_ING.anchoa) ? (
          <span className="text-[11px] font-black uppercase text-destructive mt-1">Agotado</span>
        ) : (
          <>
            <span className="text-xs text-muted-foreground text-center">Intenso y curado</span>
            <span className="text-lg font-black text-gold mt-1">
              {pendingVariantProduct?.precio.toFixed(2) ?? '-'} €
            </span>
          </>
        )}
      </button>
    </div>
  </DialogContent>
</Dialog>

{/* — Nachos variant dialog ————— */}
<Dialog
  open={nachosOpen}
  onOpenChange={(v) => {
    setNachosOpen(v);
    if (!v) setPendingVariantProduct(null);
  }}
>
  <DialogContent className="max-w-sm rounded-xl">
    <DialogHeader>
      <DialogTitle className="font-display text-lg">¿Con qué salsa?</DialogTitle>
      <DialogDescription className="text-sm text-muted-foreground pt-1">
        Elige la salsa para tus nachos
      </DialogDescription>
    </DialogHeader>
    <div className="grid grid-cols-2 gap-3 pt-2">
      <button

```



```

        onClick={() => addNachosVariant('bacon')}
        disabled={isIngredienteAgotado(NACHOS_VARIANT_ING.bacon)}
        className={`flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 transition-all
        ${isIngredienteAgotado(NACHOS_VARIANT_ING.bacon) ? 'border-destructive/40 opacity-50' : 'border-border hover:border-primary hover:bg-primary/5 active:scale-95'}`}
      >
        <img
          src={PRODUCT_IMAGES._nachosBacon}
          alt="Nachos cheddar y bacon"
          className="h-24 w-auto object-contain"
          loading="lazy"
        />
        <span className="font-display font-bold text-sm text-foreground text-center">
          Cheddar y bacon ahumado
        </span>
        <span className="text-lg font-black text-gold mt-1">
          {pendingVariantProduct?.precio.toFixed(2) ?? '-'} €
        </span>
      </button>
      <button
        onClick={() => addNachosVariant('guacamole')}
        disabled={isIngredienteAgotado(NACHOS_VARIANT_ING.guacamole)}
        className={`flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 transition-all
        ${isIngredienteAgotado(NACHOS_VARIANT_ING.guacamole) ? 'border-destructive/40 opacity-50' : 'border-border hover:border-primary hover:bg-primary/5 active:scale-95'}`}
      >
        <img
          src={PRODUCT_IMAGES._nachosGuaca}
          alt="Nachos cheddar y guacamole"
          className="h-24 w-auto object-contain"
          loading="lazy"
        />
        <span className="font-display font-bold text-sm text-foreground text-center">
          Cheddar y guacamole
        </span>
        <span className="text-lg font-black text-gold mt-1">
          {pendingVariantProduct?.precio.toFixed(2) ?? '-'} €
        </span>
      </button>
    </div>
  </DialogContent>
</Dialog>

{/* — Alitas variant dialog ————— */}
<Dialog
  open={alitasOpen}
  onOpenChange={(v) => {
    setAlitasOpen(v);
    if (!v) setPendingVariantProduct(null);
  }}
>
  <DialogContent className="max-w-sm rounded-xl">
    <DialogHeader>
      <DialogTitle className="font-display text-lg">¿Qué sabor?</DialogTitle>
      <DialogDescription className="text-sm text-muted-foreground pt-1">
        Elige el sabor para tus alitas
      </DialogDescription>
    </DialogHeader>
    <div className="grid grid-cols-2 gap-3 pt-2">
      <button
        onClick={() => addAlitasVariant('bbq')}
        disabled={isIngredienteAgotado(ALITAS_VARIANT_ING.bbq)}
        className={`flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 transition-all
        ${isIngredienteAgotado(ALITAS_VARIANT_ING.bbq) ? 'border-destructive/40 opacity-50' : 'border-border hover:border-primary hover:bg-primary/5 active:scale-95'}`}
      >
        <img
          src={PRODUCT_IMAGES._alitasBbq}
          alt="Alitas BBQ"
          className="h-24 w-auto object-contain"
          loading="lazy"
        />
        <span className="font-display font-bold text-base text-foreground">Sabor BBQ</span>
        <span className="text-xs text-muted-foreground text-center">Ahumado y dulce</span>
        <span className="text-lg font-black text-gold mt-1">
          {pendingVariantProduct?.precio.toFixed(2) ?? '-'} €
        </span>
      </button>
      <button
        onClick={() => addAlitasVariant('brava')}
        disabled={isIngredienteAgotado(ALITAS_VARIANT_ING.brava)}
        className={`flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 transition-all
        ${isIngredienteAgotado(ALITAS_VARIANT_ING.brava) ? 'border-destructive/40 opacity-50' : 'border-border hover:border-primary hover:bg-primary/5 active:scale-95'}`}
      >
        <img
          src={PRODUCT_IMAGES._alitasBrava}
          alt="Alitas Brava"
          className="h-24 w-auto object-contain"
          loading="lazy"
        />
        <span className="font-display font-bold text-base text-foreground">Sabor Brava</span>

```

```

        <span className="text-xs text-muted-foreground text-center">Picante e intenso</span>
        <span className="text-lg font-black text-gold mt-1">
            {pendingVariantProduct?.precio.toFixed(2) ?? '-'} €
        </span>
    </button>
</div>
</DialogContent>
</Dialog>

{/* — MontyRueda Gourmets: selector "elige 5" ————— */}
{pendingVariantProduct && getRueda(pendingVariantProduct.numero)?.selectCount && (
    <RuedaGourmetDialog
        open={ruedaOpen}
        onOpenChange={(v) => {
            setRuedaOpen(v);
            if (!v) setPendingVariantProduct(null);
        }}
        nombre={pendingVariantProduct.nombre}
        precio={pendingVariantProduct.precio}
        need={getRueda(pendingVariantProduct.numero)?.selectCount!}
        comps={ruedaComponentes(getRueda(pendingVariantProduct.numero!), ruedaMontaditos)}
        isProductoAgotado={isProductoAgotado}
        onConfirm={(selected) => {
            const p = pendingVariantProduct;
            addRuedaToCart(
                p,
                `${p.id}::${selected.map((m) => m.id).sort().join('-')}`,
                selected.map(complabel),
                pendingRectRef.current,
            );
            setRuedaOpen(false);
            setPendingVariantProduct(null);
        }}
    </>
)}
);
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/Monty.tsx */
/* ===== */
import { motion } from 'framer-motion';
import waving from '@assets/monty-waving.png';
import celebrating from '@assets/monty-celebrating.png';
import thinking from '@assets/monty-thinking.png';
import cooking from '@assets/monty-cooking.png';

const POSES = {
    waving,
    celebrating,
    thinking,
    cooking,
} as const;

export type MontyPose = keyof typeof POSES;

interface MontyProps {
    pose?: MontyPose;
    size?: number;
    className?: string;
    animate?: 'float' | 'bounce' | 'wiggle' | 'none';
    alt?: string;
}

const ANIMATIONS = {
    float: {
        animate: { y: [0, -8, 0] },
        transition: { duration: 2.5, repeat: Infinity, ease: 'easeInOut' as const },
    },
    bounce: {
        animate: { y: [0, -14, 0] },
        transition: { duration: 0.9, repeat: Infinity, ease: 'easeOut' as const },
    },
    wiggle: {
        animate: { rotate: [-3, 3, -3] },
        transition: { duration: 2, repeat: Infinity, ease: 'easeInOut' as const },
    },
    none: { animate: {}, transition: {} },
};

/**
 * Monty — la mascota oficial de la app.
 * Sustituible: cuando subas la versión definitiva, reemplaza los png en src/assets/.
 */
export function Monty({
    pose = 'waving',
    size = 120,
    className = '',
    animate = 'float',
    alt = 'Monty',

```

```

    }: MontyProps) {
      const anim = ANIMATIONS[animate];
      return (
        <motion.img
          src={POSES[pose]}
          alt={alt}
          width={size}
          height={size}
          loading="lazy"
          className={`object-contain drop-shadow-md select-none pointer-events-none ${className}`}
          style={{ width: size, height: size }}
          animate={anim.animate}
          transition={anim.transition}
        />
      );
    }
  }

  /* ===== */
  /* FICHERO: montadito-magic-flow/src/components/client/MontyLoader.tsx */
  /* ===== */
  import { motion } from 'framer-motion';
  import { Monty } from './Monty';

  interface Props {
    message?: string;
    fullscreen?: boolean;
  }

  export function MontyLoader({ message = 'Preparando el menú...', fullscreen = true }: Props) {
    return (
      <div
        className={
          fullscreen
            ? 'fixed inset-0 z-[55] bg-background flex flex-col items-center justify-center px-6'
            : 'flex flex-col items-center justify-center py-12 px-6'
        }
      >
        <motion.div
          animate={{ y: [0, -20, 0] }}
          transition={{ duration: 0.7, repeat: Infinity, ease: 'easeOut' }}
        >
          <Monty pose="waving" size={140} animate="none" />
        </motion.div>

        <motion.div
          initial={{ opacity: 0 }}
          animate={{ opacity: 1 }}
          transition={{ delay: 0.3 }}
          className="mt-6 flex items-center gap-1.5"
        >
          <p className="font-display text-sm text-muted-foreground">{message}</p>
          {[0, 1, 2].map((i) => (
            <motion.span
              key={i}
              className="w-1.5 h-1.5 rounded-full bg-primary"
              animate={{ y: [0, -4, 0], opacity: [0.4, 1, 0.4] }}
              transition={{ duration: 0.9, repeat: Infinity, delay: i * 0.15 }}
            />
          ))}
        </motion.div>
      </div>
    );
  }

  /* ===== */
  /* FICHERO: montadito-magic-flow/src/components/client/MontyRuedaCard.tsx */
  /* ===== */
  import { useMemo, useState } from 'react';
  import { motion, AnimatePresence } from 'framer-motion';
  import { Plus, Minus } from 'lucide-react';
  import { toast } from 'sonner';
  import { useCartStore } from '@lib/cart-store';
  import { getRueda, ruedaComponentes, ruedaAgotada, compLabel, type RuedaMontadito } from '@lib/ruedas';
  import { RuedaGourmetDialog } from './RuedaGourmetDialog';

  interface RuedaProduct {
    id: string;
    nombre: string;
    descripcion: string | null;
    precio: number;
    numero?: string | null;
    foto_url: string | null;
  }

  interface Props {
    product: RuedaProduct;
    montaditos: RuedaMontadito[];
    isProductoAgotado: (id: string) => boolean;
    index: number;
  }

```

```

export function MontyRuedaCard({ product, montaditos, isProductoAgotado, index }: Props) {
  const addItem = useCartStore((s) => s.addItem);
  const updateQuantity = useCartStore((s) => s.updateQuantity);
  const items = useCartStore((s) => s.items);
  const [selectorOpen, setSelectorOpen] = useState(false);

  const def = getRueda(product.numero);
  const comps = useMemo(() => (def ? ruedaComponentes(def, montaditos) : []), [def, montaditos]);

  if (!def) return null;

  const isGourmet = !!def.selectCount;
  const need = def.selectCount ?? 0;
  const agotado = ruedaAgotada(def, montaditos, isProductoAgotado);

  const cantidadEnCarrito = items
    .filter((i) => i.productoId === product.id)
    .reduce((s, i) => s + i.cantidad, 0);

  const addFija = () => {
    addItem({
      id: product.id,
      productoId: product.id,
      nombre: product.nombre,
      precio: product.precio,
      foto_url: product.foto_url,
      componentes: comps.map(complabel),
    });
  };

  const onGourmetConfirm = (selected: RuedaMontadito[]) => {
    addItem({
      id: `${product.id}:${selected.map((m) => m.id).sort().join('-')}`,
      productoId: product.id,
      nombre: product.nombre,
      precio: product.precio,
      foto_url: product.foto_url,
      componentes: selected.map(complabel),
    });
    setSelectorOpen(false);
    toast.success(`${product.nombre} añadida 🍷`);
  };

  const handleAdd = (e: React.MouseEvent) => {
    e.stopPropagation();
    if (agotado) return;
    if (isGourmet) {
      setSelectorOpen(true);
      return;
    }
    addFija();
  };

  return (
    <>
      <motion.div
        initial={{ opacity: 0, y: 10 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ delay: index * 0.02, duration: 0.3 }}
        whileTap={{ scale: 0.98 }}
        className="relative glass-card flex items-center gap-3 p-3 group"
      >
        {def.img ? (
          <img
            src={def.img}
            alt={product.nombre}
            className="w-14 h-14 object-contain shrink-0 drop-shadow-sm"
            loading="lazy"
          />
        ) : (
          <span className="w-12 h-12 rounded-xl bg-accent/15 text-accent flex items-center justify-center text-2xl shrink-0 shadow-sm border border-border border-subtle" aria-hidden>
            🍷
          </span>
        )}

        <div className="flex-1 min-w-0">
          <h3 className="font-display text-sm font-bold text-foreground leading-tight">{product.nombre}</h3>
          {product.descripcion && (
            <p className="text-xs text-muted-foreground mt-0.5 line-clamp-1">{product.descripcion}</p>
          )}
          {isGourmet && comps.length > 0 && (
            <p className="text-[11px] text-muted-foreground/80 mt-0.5 line-clamp-1">
              {comps.map((m) => `#${m.numero}`).join(' · ')}
            </p>
          )}
        </div>

        <div className="flex items-center gap-2 shrink-0">
          <span className="text-sm font-bold text-gold whitespace-nowrap">{product.precio.toFixed(2)} €</span>
          {agotado ? (

```

```

    <span className="text-[10px] font-black uppercase tracking-wide text-destructive border border-destructive/40 rounded-full
px-2.5 py-1">
      Agotado
    </span>
  ) : (
    <AnimatePresence mode="wait" initial={false}>
      {cantidadEnCarrito === 0 || isGourmet ? (
        <motion.button
          key="add"
          type="button"
          initial={{ opacity: 0, scale: 0.6 }}
          animate={{ opacity: 1, scale: 1 }}
          exit={{ opacity: 0, scale: 0.6 }}
          whileTap={{ scale: 0.85 }}
          onClick={handleAdd}
          aria-label={isGourmet ? 'Elegir montaditos de la rueda' : 'Añadir rueda al carrito'}
          className="relative w-8 h-8 rounded-full bg-primary flex items-center justify-center opacity-70 group-hover:opacity-
100 transition-opacity"
        >
          <Plus className="w-4 h-4 text-primary-foreground" />
          {isGourmet && cantidadEnCarrito > 0 && (
            <span className="absolute -top-1 -right-1 min-w-[1.1rem] h-[1.1rem] px-1 rounded-full bg-accent text-accent-
foreground text-[10px] font-bold flex items-center justify-center tabular-nums">
              {cantidadEnCarrito}
            </span>
          )}
        </motion.button>
      ) : (
        <motion.div
          key="stepper"
          initial={{ opacity: 0, scale: 0.8 }}
          animate={{ opacity: 1, scale: 1 }}
          exit={{ opacity: 0, scale: 0.8 }}
          className="flex items-center gap-1 bg-primary/10 border border-primary/40 rounded-full p-0.5"
        >
          <button
            type="button"
            onClick={() => updateQuantity(product.id, cantidadEnCarrito - 1)}
            aria-label="Quitar uno"
            className="w-7 h-7 rounded-full bg-surface-elevated flex items-center justify-center active:scale-90 transition-
transform"
          >
            <Minus className="w-3.5 h-3.5 text-foreground" />
          </button>
          <span className="min-w-[1.25rem] text-center text-sm font-bold text-foreground tabular-nums">
            {cantidadEnCarrito}
          </span>
          <button
            type="button"
            onClick={handleAdd}
            aria-label="Añadir uno más"
            className="w-7 h-7 rounded-full bg-primary flex items-center justify-center active:scale-90 transition-transform"
          >
            <Plus className="w-3.5 h-3.5 text-primary-foreground" />
          </button>
        </motion.div>
      )}
    </AnimatePresence>
  )}
</div>
</motion.div>

{isGourmet && (
  <RuedaGourmetDialog
    open={selectorOpen}
    onOpenChange={setSelectorOpen}
    nombre={product.nombre}
    precio={product.precio}
    need={need}
    comps={comps}
    isProductoAgotado={isProductoAgotado}
    onConfirm={onGourmetConfirm}
  />
)}
</>
);
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/OrderTracking.tsx */
/* ===== */
import { useCallback, useEffect, useRef, useState } from 'react';
import { motion } from 'framer-motion';
import confetti from 'canvas-confetti';
import { toast } from 'sonner';
import { Clock, ChefHat, CheckCircle2, CreditCard, Beer, UtensilsCrossed, AlertTriangle } from 'lucide-react';
import { supabase } from '@integrations/supabase/client';
import { Monty } from './Monty';
import { playBellSound } from '@lib/bell-sound';
import { unlockAudio, startAlarm, stopAlarm, isAudioUnlocked } from '@lib/alarm';
import { usePushSubscription } from '@hooks/use-push-notifications';

```

```

import { AllergenIcon } from './AllergenIcon';
import { AllergenLegalNotice } from './AllergenLegalNotice';

const fireReadyConfetti = () => {
  const defaults = {
    spread: 360,
    ticks: 120,
    gravity: 0.5,
    decay: 0.94,
    startVelocity: 35,
    colors: ['#E63946', '#F4A261', '#E9C46A', '#FFB703', '#06D6A0'],
  };
  const burst = (originY: number, count: number, scalar: number) =>
    confetti({ ...defaults, particleCount: count, scalar, origin: { x: 0.5, y: originY } });
  burst(0.55, 80, 1.3);
  setTimeout(() => burst(0.4, 50, 1), 250);
  setTimeout(() => burst(0.6, 60, 1.5), 500);
  setTimeout(() => burst(0.5, 40, 0.9), 800);
};

interface Props {
  pedidoId: string;
  numeroPedido: number;
  hasCocina: boolean;
  hasBebidas: boolean;
  sessionId?: string;
  onNewOrder: () => void;
  onRetryPayment?: () => void;
}

const STATUS_CONFIG = {
  pendiente_pago: { label: 'Esperando pago', color: 'text-warning', icon: CreditCard, step: 0 },
  pendiente: { label: 'Esperando caja', color: 'text-warning', icon: CreditCard, step: 0 },
  recibido: { label: 'Recibido', color: 'text-warning', icon: Clock, step: 1 },
  preparando: { label: 'En preparación', color: 'text-info', icon: ChefHat, step: 2 },
  listo: { label: 'Listo para recoger', color: 'text-success', icon: CheckCircle2, step: 3 },
  entregado: { label: 'Entregado', color: 'text-success', icon: CheckCircle2, step: 4 },
  cancelado: { label: 'Cancelado', color: 'text-destructive', icon: Clock, step: -1 },
};

type OrderStatus = keyof typeof STATUS_CONFIG;

interface TicketCardProps {
  kind: 'cocina' | 'bebidas';
  status: OrderStatus;
}

function TicketCard({ kind, status }: TicketCardProps) {
  const config = STATUS_CONFIG[status];
  const isReady = status === 'listo';
  const Icon = kind === 'cocina' ? UtensilsCrossed : Beer;
  const title = kind === 'cocina' ? 'Comida · Recoger en barra' : 'Bebida/Aperitivo · Recoger en barra';
  const steps = ['pendiente', 'recibido', 'preparando', 'listo'] as const;

  return (
    <motion.div
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      className={`w-full max-w-sm rounded-3xl border p-5 transition-colors ${
        isReady ? 'border-gold/60 bg-gold/5' : 'border-border-subtle bg-surface'
      }`}
    >
      <div className="flex items-center gap-3 mb-3">
        <div className={`w-10 h-10 rounded-xl flex items-center justify-center ${
          isReady ? 'bg-gold/20 text-gold' : 'bg-surface-elevated text-foreground'
        }`}>
          <Icon className="w-5 h-5" />
        </div>
        <div className="flex-1">
          <p className="text-[10px] uppercase tracking-widest text-muted-foreground">{title}</p>
          <p className={`font-display text-base font-bold ${config.color}`}>{config.label}</p>
        </div>
      </div>

      <div className="flex items-center justify-center w-full">
        {steps.map((step, i) => {
          const stepConfig = STATUS_CONFIG[step];
          const StepIcon = stepConfig.icon;
          const isActive = config.step >= stepConfig.step;
          const isCurrent = config.step === stepConfig.step;
          return (
            <div key={step} className={`flex items-center ${i < steps.length - 1 ? 'flex-1' : ''}`>
              <motion.div
                className={`shrink-0 w-9 h-9 rounded-full flex items-center justify-center border-2 transition-all duration-500 ${
                  isActive
                    ? isCurrent ? 'border-gold bg-gold/20' : 'border-gold/50 bg-gold/10'
                    : 'border-border bg-surface'
                }`}
                animate={isCurrent ? { scale: [1, 1.1, 1] } : {}}
                transition={{ duration: 1.5, repeat: isCurrent ? Infinity : 0 }}
              />

```

```

        <StepIcon className={`w-4 h-4 ${isActive ? 'text-gold' : 'text-muted-foreground'}`} />
      </motion.div>
    {i < steps.length - 1 && (
      <div className="flex-1 h-0.5 mx-1">
        <motion.div
          className="h-full bg-gold rounded-full"
          initial={{ scaleX: 0 }}
          animate={{ scaleX: config.step > stepConfig.step ? 1 : 0 }}
          transition={{ duration: 0.8, ease: 'easeOut' }}
          style={{ transformOrigin: 'left' }}
        />
      </div>
    )}
  </div>
);
}}
</div>

{isReady && (
  <p className="mt-3 text-xs text-center text-gold/90 font-medium">
    {kind === 'cocina'
      ? '¡Acércate a la barra!'
      : '¡Recoge en barra!'}
  </p>
)}
</motion.div>
);
}

export function OrderTracking({ pedidoId, numeroPedido, hasCocina, hasBebidas, sessionId, onNewOrder, onRetryPayment }: Props) {
  const [estadoCocina, setEstadoCocina] = useState<OrderStatus>('pendiente');
  const [estadoBebidas, setEstadoBebidas] = useState<OrderStatus>('pendiente');
  const [estadoGlobal, setEstadoGlobal] = useState<OrderStatus>('pendiente_pago');
  const [retrying, setRetrying] = useState(false);
  const [orderAllergens, setOrderAllergens] = useState<{ codigo: string; nombre: string; icono: string | null }[]>([]);
  // numeroPedido can be 0 if the DB trigger hadn't fired yet when the order was created;
  // refreshOrderStatus will update it once the real value is available.
  const [numeroDisplay, setNumeroDisplay] = useState(numeroPedido);
  const readyFiredCocina = useRef(false);
  const readyFiredBebidas = useRef(false);
  // Aviso sonoro: en iOS hay que desbloquear el audio con un toque del usuario.
  const [audioOn, setAudioOn] = useState(isAudioUnlocked());
  // Modal de alarma a pantalla completa que suena en bucle hasta pulsar OK.
  const [alarmOpen, setAlarmOpen] = useState(false);

  // Notificaciones push (para enterarse con el móvil bloqueado) + aviso iPhone.
  const { subscribe } = usePushSubscription();
  const isIOS = typeof navigator !== 'undefined' && /iphone|ipad|ipod/i.test(navigator.userAgent);
  const [notifPerm, setNotifPerm] = useState<string>(
    typeof Notification !== 'undefined' ? Notification.permission : 'unsupported',
  );
  const activarNotis = async () => {
    await subscribe(sessionId, pedidoId).catch(() => {});
    setNotifPerm(typeof Notification !== 'undefined' ? Notification.permission : 'unsupported');
  };

  // Al desmontar, parar la alarma siempre (evita que siga sonando).
  useEffect(() => () => stopAlarm(), []);

  const activarAviso = useCallback(() => {
    if (unlockAudio()) {
      setAudioOn(true);
      playBellSound(0.4); // pitido corto de confirmación
    }
  }, []);

  const detenerAlarma = useCallback(() => {
    stopAlarm();
    setAlarmOpen(false);
  }, []);

  // Cargar alérgenos del pedido una vez
  useEffect(() => {
    (async () => {
      const { data } = await supabase
        .from('pedido_items')
        .select('producto_id, menu_productos!inner(producto_alergenos(alergenos(codigo, nombre, icono, orden)))')
        .eq('pedido_id', pedidoId)
        .setHeader('x-session-id', sessionId ?? '');
      if (!data) return;
      const seen = new Map<string, any>();
      (data as any[]).forEach((row) => {
        const pas = row?.menu_productos?.producto_alergenos ?? [];
        pas.forEach((pa: any) => {
          const a = pa?.alergenos;
          if (a && !seen.has(a.codigo)) seen.set(a.codigo, a);
        });
      });
      setOrderAllergens(
        Array.from(seen.values()).sort((a, b) => (a.orden ?? 0) - (b.orden ?? 0))
      );
    })();
  });
}

```

```

}, [pedidoId, sessionId]);

const refreshOrderStatus = useCallback(async () => {
  const { data, error } = await supabase
    .from('pedidos')
    .select('estado, estado_cocina, estado_bebidas, numero_pedido')
    .eq('id', pedidoId)
    .setHeader('x-session-id', sessionId ?? '')
    .maybeSingle();

  if (error) return;
  // No rebotar al menú por una lectura nula/transitoria: las eliminaciones
  // reales llegan por la suscripción realtime DELETE. Perder el ticket aquí
  // equivale a perder un pedido ya pagado.
  if (!data) return;

  const d = data as any;
  setEstadoGlobal((d.estado ?? 'pendiente') as OrderStatus);
  setEstadoCocina((d.estado_cocina ?? d.estado) as OrderStatus);
  setEstadoBebidas((d.estado_bebidas ?? d.estado) as OrderStatus);
  // Update the displayed order number if the DB has a real value (trigger may have fired after INSERT)
  if (d.numero_pedido && d.numero_pedido !== numeroDisplay) {
    setNumeroDisplay(d.numero_pedido);
  }
}, [pedidoId, sessionId, onNewOrder, numeroDisplay]);

// Initial fetch – bounce to menu if order missing
useEffect(() => {
  refreshOrderStatus();
  const interval = window.setInterval(refreshOrderStatus, 2500);
  return () => window.clearInterval(interval);
}, [refreshOrderStatus]);

const handleRetryPayment = useCallback(() => {
  if (retrying) return;
  setRetrying(true);
  try {
    onRetryPayment?();
  } finally {
    setRetrying(false);
  }
}, [onRetryPayment, retrying]);

const handleCancelOrder = useCallback(async () => {
  try {
    await supabase.rpc('cancel_own_pending_order', { _pedido_id: pedidoId } as any)
      .setHeader('x-session-id', sessionId ?? '');
    onNewOrder();
  } catch (e) {
    console.error('cancel_own_pending_order', e);
    toast.error('No se pudo cancelar el pedido. Inténtalo de nuevo.');

```



```

    fireReadyConfetti();
    startAlarm();
    setAlarmOpen(true);
    try { navigator.vibrate?.([300, 100, 300, 100, 600]); } catch {}
  }
  if (estadoCocina !== 'listo') readyFiredCocina.current = false;
}, [estadoCocina, hasCocina]);

useEffect(() => {
  if (hasBebidas && estadoBebidas === 'listo' && !readyFiredBebidas.current) {
    readyFiredBebidas.current = true;
    fireReadyConfetti();
    startAlarm();
    setAlarmOpen(true);
    try { navigator.vibrate?.([300, 100, 300, 100, 600]); } catch {}
  }
  if (estadoBebidas !== 'listo') readyFiredBebidas.current = false;
}, [estadoBebidas, hasBebidas]);

const isFinal = (s: OrderStatus) => s === 'entregado' || s === 'cancelado';
const allFinal =
  (!hasCocina || isFinal(estadoCocina)) &&
  (!hasBebidas || isFinal(estadoBebidas));

const anyReady =
  (hasCocina && estadoCocina === 'listo') ||
  (hasBebidas && estadoBebidas === 'listo');

// Pantalla especial: pago pendiente o cancelado por timeout
if (estadoGlobal === 'pendiente_pago' || estadoGlobal === 'cancelado') {
  const isCancelled = estadoGlobal === 'cancelado';
  return (
    <div className="min-h-screen flex flex-col items-center justify-center px-6 py-10 gap-6 text-center">
      <div className={`w-16 h-16 rounded-full flex items-center justify-center ${isCancelled ? 'bg-destructive/15 text-destructive' : 'bg-warning/15 text-warning'}`}>
        <CreditCard className="w-7 h-7" />
      </div>
      <div>
        <p className="text-muted-foreground text-xs uppercase tracking-widest mb-1">Pedido</p>
        <h1 className="font-display font-black leading-none text-foreground text-5xl">
          #{String(numeroDisplay).padStart(4, '0')}
        </h1>
      </div>
      <h2 className="font-display text-2xl font-bold">
        {isCancelled ? 'Pedido cancelado' : 'Pago pendiente'}
      </h2>
      <p className="text-sm text-muted-foreground max-w-xs">
        {isCancelled
          ? 'No se confirmó el pago a tiempo. Puedes hacer un pedido nuevo cuando quieras.'
          : 'Tu pedido se enviará a la cocina en cuanto Stripe confirme el cobro. Si no completaste el pago, vuelve a intentarlo.'}
      </p>
      <div className="flex flex-col gap-3 w-full max-w-xs">
        {!isCancelled && (
          <button
            onClick={handleRetryPayment}
            disabled={retrying}
            className="w-full px-6 py-3 rounded-2xl bg-primary text-primary-foreground font-semibold disabled:opacity-60">
              >
            {retrying ? 'Abriendo...' : 'Reintentar pago'}
          </button>
        )}
        <button
          onClick={handleCancelOrder}
          className="w-full px-6 py-3 rounded-2xl bg-surface border border-border-subtle text-foreground font-medium hover:bg-surface-elevated transition-colors">
          >
          {isCancelled ? 'Volver al menú' : 'Cancelar y volver al menú'}
        </button>
      </div>
    </div>
  );
}

return (
  <div className="min-h-screen flex flex-col items-center px-6 py-10 gap-6 relative overflow-hidden">
    {/* Único número de ticket */}
    <motion.div
      initial={{ scale: 0.5, opacity: 0 }}
      animate={{ scale: 1, opacity: 1 }}
      transition={{ type: 'spring', damping: 15 }}
      className="text-center">
      >
      <p className="text-muted-foreground text-xs uppercase tracking-widest mb-1">
        Tu número de pedido
      </p>
      <h1 className={`font-display font-black leading-none ${anyReady ? 'text-gold' : 'text-foreground'} text-[88px]`} >
        #{String(numeroDisplay).padStart(4, '0')}
      </h1>
    </motion.div>

    <div className="flex flex-col items-center gap-4 w-full">

```

```

    {hasCocina && <TicketCard kind="cocina" status={estadoCocina} />}
    {hasBebidas && <TicketCard kind="bebidas" status={estadoBebidas} />}
  </div>

  {/* Aviso sonoro: en iPhone requiere un toque del usuario para desbloquear el audio */}
  {!allFinal && !audioOn && (
    <button
      onClick={activarAviso}
      className="w-full max-w-sm px-5 py-3 rounded-2xl bg-gold/15 border border-gold/50 text-foreground font-semibold flex items-center justify-center gap-2 active:scale-95 transition-transform"
    >
      📢 Activa el aviso sonoro
    </button>
  )}
  {!allFinal && audioOn && (
    <p className="text-sm text-success font-semibold text-center">📢 Aviso sonoro activado · te avisaremos cuando esté listo</p>
  )}

  {/* iPhone: el interruptor de silencio lateral silencia TODO el audio web */}
  {!allFinal && isIOS && (
    <div className="w-full max-w-sm rounded-2xl border-2 border-amber-500/60 bg-amber-50/70 dark:bg-amber-950/25 px-4 py-3.5 text-center">
      <p className="text-base font-semibold text-amber-700 dark:text-amber-300 leading-snug">
        📞 iPhone: sube el volumen y <b>desactiva el botón de silencio</b> (interruptor lateral). Si está en silencio, no sonará.
      </p>
    </div>
  )}

  {/* Aviso aunque el móvil esté bloqueado (notificación push del sistema) */}
  {!allFinal && notifPerm !== 'granted' && notifPerm !== 'unsupported' && (
    <button
      onClick={activarNotis}
      className="w-full max-w-sm px-5 py-4 rounded-2xl bg-primary/10 border-2 border-primary/40 text-foreground text-base font-bold flex items-center justify-center gap-2 active:scale-95 transition-transform"
    >
      📱 Avisarme aunque tenga el móvil bloqueado
    </button>
  )}
  {!allFinal && notifPerm === 'granted' && (
    <p className="text-sm text-success font-semibold text-center">📱 También te avisaremos con el móvil bloqueado</p>
  )}
  {!allFinal && isIOS && notifPerm !== 'granted' && (
    <p className="text-sm text-muted-foreground text-center max-w-sm leading-snug">
      En iPhone, para que avise con la pantalla apagada: pulsa <b>Compartir ▶ "Añadir a pantalla de inicio"</b> y abre la app desde ahí.
    </p>
  )}

  {orderAllergens.length > 0 && (
    <motion.div
      initial={{ opacity: 0, y: 10 }}
      animate={{ opacity: 1, y: 0 }}
      className="w-full max-w-sm rounded-2xl border border-[#FCD34D]/40 bg-[#FEF3C7]/10 p-4"
    >
      <div className="flex items-center gap-2 mb-2">
        <AlertTriangle className="w-4 h-4 text-[#92400E]" />
        <p className="text-xs font-bold uppercase tracking-wider text-foreground">
          Tu pedido contiene
        </p>
      </div>
      <div className="flex flex-wrap gap-2 mb-2">
        {orderAllergens.map((a) => (
          <div key={a.codigo} className="flex items-center gap-1.5 pr-2 pl-0.5 py-0.5 rounded-full bg-surface">
            <AllergenIcon codigo={a.codigo} nombre={a.nombre} icono={a.icono} size="sm" />
            <span className="text-[11px] font-semibold">{a.nombre}</span>
          </div>
        ))}
      </div>
      <AllergenLegalNotice variant="short" />
    </motion.div>
  )}

  {anyReady && (
    <motion.div
      initial={{ opacity: 0, scale: 0.8 }}
      animate={{ opacity: 1, scale: 1 }}
      className="flex flex-col items-center"
    >
      <Monty pose="celebrating" size={120} animate="bounce" />
    </motion.div>
  )}

  {allFinal && (
    <motion.button
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      transition={{ delay: 0.5 }}
      onClick={onNewOrder}
      className="mt-2 px-8 py-3 rounded-2xl bg-surface border border-border-subtle text-foreground font-medium hover:bg-surface-elevated transition-colors"
    >
      Hacer otro pedido
    </motion.button>
  )}

```

```

    </motion.button>
  })

  {!allFinal && (
    <p className="mt-2 text-xs text-muted-foreground text-center max-w-xs">
      No cierres esta pantalla: es tu comprobante para recoger comida y bebida.
    </p>
  )}

  { /* ALARMA a pantalla completa: suena en bucle hasta que el cliente pulsa OK */
  {alarmOpen && (
    <motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      className="fixed inset-0 z-50 flex flex-col items-center justify-center gap-8 px-8 text-center bg-primary"
    >
      <motion.div
        animate={{ scale: [1, 1.2, 1], rotate: [0, -10, 10, 0] }}
        transition={{ duration: 0.7, repeat: Infinity }}
        className="text-8xl"
        aria-hidden
      >
        📢
      </motion.div>
      <div className="text-primary-foreground">
        <p className="text-sm uppercase tracking-widest opacity-80 mb-2">
          Pedido #{String(numeroDisplay).padStart(4, '0')}
        </p>
        <h2 className="font-display text-4xl font-black leading-tight">
          ¡TU PEDIDO ESTÁ LISTO!
        </h2>
        <p className="mt-3 text-lg font-medium opacity-90">Pasa a recogerlo en la barra</p>
      </div>
      <button
        onClick={detenerAlarma}
        className="w-full max-w-xs px-8 py-5 rounded-3xl bg-white text-primary font-black text-2xl shadow-xl active:scale-95
transition-transform"
      >
        OK, ya voy
      </button>
    </motion.div>
  )}
</div>
);
}

```

```

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/ProductCard.tsx */
/* ===== */
import { useState } from 'react';
import { motion, AnimatePresence } from 'framer-motion';
import { Plus, Minus, AlertCircle } from 'lucide-react';
import { useCartStore, useAgeGate } from '@lib/cart-store';
import { useAllergenFilter } from '@lib/allergen-filter';
import { getDrinkIcon } from '@lib/drink-icon';
import { getDrinkImage } from '@lib/drink-image';
import jarraQuijoteImg from '@assets/drinks/jarra-quiote.png';
import jarraSanchoImg from '@assets/drinks/jarra-sancho.png';
import jarraQuijoteLadronVeranoImg from '@assets/drinks/jarra-quiote-ladron-verano.png';
import jarraSanchoLadronVeranoImg from '@assets/drinks/jarra-sancho-ladron-verano.png';
import jarraQuijoteLadronManzanasImg from '@assets/drinks/jarra-quiote-ladron-manzanas.png';
import jarraSanchoLadronManzanasImg from '@assets/drinks/jarra-sancho-ladron-manzanas.png';
import { PRODUCT_IMAGES, MONTADITO_IMAGES_BY_NUMERO } from '@lib/product-images';
import { AllergenIcon } from './AllergenIcon';
import { AllergenLegalNotice } from './AllergenLegalNotice';
import type { ProductAllergen } from '@hooks/use-menu';
import {
  Dialog,
  DialogContent,
  DialogHeader,
  DialogTitle,
  DialogDescription,
} from '@components/ui/dialog';

interface Product {
  id: string;
  nombre: string;
  descripcion: string | null;
  precio: number;
  foto_url: string | null;
  destacado: boolean;
  nuevo: boolean;
  numero?: string | null;
  tipo_pan?: string | null;
  seccion?: string | null;
  contiene_alcohol?: boolean;
  alergenos?: ProductAllergen[];
}

interface Props {
  product: Product;

```

```

    index: number;
    variant?: 'default' | 'drink';
    hideAllergens?: boolean;
    agotado?: boolean;
    isIngredienteAgotado?: (nombre: string) => boolean;
  }

// Variantes que dependen de un ingrediente concreto: si falta, se desactiva
// esa opción (no el producto entero, salvo que se agoten todas).
const ALITAS_VARIANT_ING: Record<string, string> = { bbq: 'Salsa BBQ', brava: 'Salsa brava' };
const NACHOS_VARIANT_ING: Record<string, string> = { bacon: 'Bacon ahumado', guacamole: 'Guacamole' };
const GILDA_VARIANT_ING: Record<string, string> = { boqueron: 'Gilda boquerón', anchoa: 'Gilda anchoa' };

const BREAD_BADGES: Record<string, string> = {
  '100M': 'bg-muted text-muted-foreground border-border',
  'Chapata': 'bg-[#8B5A2B]/15 text-[#C8956D] border-[#A0522D]/40',
  'Brioche': 'bg-amber-400/15 text-amber-400 border-amber-500/40',
  'Tortilla': 'bg-orange-400/15 text-orange-300 border-orange-400/40',
  'Mini Burger': 'bg-red-500/15 text-red-400 border-red-500/40',
  'Empotrado': 'bg-[#6B8E23]/20 text-[#9CB071] border-[#6B8E23]/50',
};

const JARRA_SECTIONS = new Set(['Jarras Heladas', 'Cerveza Premium']);
const SANCHO_EXTRA = 1.0; // Sancho cuesta 1€ más que Quijote

export function ProductCard({ product, index, variant = 'default', hideAllergens = false, agotado = false, isIngredienteAgotado = () => false }: Props) {
  const addItem = useCartStore((s) => s.addItem);
  const updateQuantity = useCartStore((s) => s.updateQuantity);
  const items = useCartStore((s) => s.items);
  const cantidadEnCarrito = items
    .filter((i) => i.productoId === product.id)
    .reduce((sum, i) => sum + i.cantidad, 0);
  const requestAlcohol = useAgeGate((s) => s.request);
  const excluded = useAllergenFilter((s) => s.excluded);
  const hideMode = useAllergenFilter((s) => s.hideMode);
  const [detailOpen, setDetailOpen] = useState(false);
  const [jarraOpen, setJarraOpen] = useState(false);
  const [gildaOpen, setGildaOpen] = useState(false);
  const [nachosOpen, setNachosOpen] = useState(false);
  const [alitasOpen, setAlitasOpen] = useState(false);

  const num = product.numero ?? null;
  const rest = product.nombre;
  const productImage = variant === 'drink'
    ? getDrinkImage(product.nombre, product.foto_url)
    : ((num ? MONTADITO_IMAGES_BY_NUMERO[num] : undefined) ?? PRODUCT_IMAGES[product.nombre] ?? product.foto_url);
  const drinkIcon = variant === 'drink' && !productImage ? getDrinkIcon(product.nombre) : null;
  const breadBadgeClass = product.tipo_pan ? BREAD_BADGES[product.tipo_pan] ?? 'bg-muted text-muted-foreground border-border' : null;

  const isJarra = !!product.seccion && JARRA_SECTIONS.has(product.seccion);

  const precioQuijote = product.precio;
  const precioSancho = product.precio + SANCHO_EXTRA;
  const productName = product.nombre
    .toLowerCase()
    .normalize('NFD')
    .replace(/[\u0300-\u036f]/g, '');
  const isLadronDeVerano = productName.includes('ladron de verano') || productName.includes('tinto de verano');
  const isLadronDeManzanas = productName.includes('ladron de manzana');
  const isGilda = productName === 'gildas' || productName === 'gilda';
  const isNachos = productName === 'nachos';
  const isAlitas = productName === 'alitas de pollo';

  // Disponibilidad de variantes según ingredientes agotados (alitas/nachos).
  const variantIngMap = isAlitas ? ALITAS_VARIANT_ING : isNachos ? NACHOS_VARIANT_ING : isGilda ? GILDA_VARIANT_ING : null;
  const variantKeys = variantIngMap ? Object.keys(variantIngMap) : [];
  const isVariantAgotada = (key: string) => !!variantIngMap && isIngredienteAgotado(variantIngMap[key]);
  const todasVariantesAgotadas = variantKeys.length > 0 && variantKeys.every((k) => isVariantAgotada(k));
  // Agotado efectivo: marcado/ingrediente común agotado, o TODAS las variantes agotadas.
  const efectivoAgotado = agotado || todasVariantesAgotadas;

  const gildaBoqueronImage = PRODUCT_IMAGES._gildaBoqueron;
  const gildaAnchoaImage = PRODUCT_IMAGES._gildaAnchoa;
  const nachosBaconImage = PRODUCT_IMAGES._nachosBacon;
  const nachosGuacamoleImage = PRODUCT_IMAGES._nachosGuaca;

  const jarraQuijoteOptionImg = isLadronDeVerano
    ? jarraQuijoteLadronVeranoImg
    : isLadronDeManzanas
      ? jarraQuijoteLadronManzanasImg
      : jarraQuijoteImg;
  const jarraSanchoOptionImg = isLadronDeVerano
    ? jarraSanchoLadronVeranoImg
    : isLadronDeManzanas
      ? jarraSanchoLadronManzanasImg
      : jarraSanchoImg;

  const alergenos = hideAllergens ? [] : (product.alergenos ?? []);
  const matchedExcluded = alergenos.filter((a) => excluded.includes(a.codigo));
  const hasExcluded = matchedExcluded.length > 0;

```

```

if (hasExcluded && hideMode === 'hide') return null;

const mappedAlergenos = alergenos.map((a) => ({ codigo: a.codigo, nombre: a.nombre, icono: a.icono }));

const buildVariantPayload = (
  variant: string,
  variantLabel: string,
  nombre: string,
  foto_url: string | undefined | null,
  precio: number = product.precio,
) => ({
  id: `${product.id}::${variant}`,
  productoId: product.id,
  variant,
  variantLabel,
  nombre,
  precio,
  foto_url: foto_url ?? null,
  contiene_alcohol: !!product.contiene_alcohol,
  alergenos: mappedAlergenos,
});

const buildPayload = (jarra?: 'quijote' | 'sancho') => {
  if (jarra) {
    const label = jarra === 'quijote' ? 'Jarra Quijote' : 'Jarra Sancho';
    return buildVariantPayload(jarra, label, `${product.nombre} · ${label}`, productImage ?? product.foto_url, jarra === 'quijote' ?
precioQuijote : precioSancho);
  }
  return {
    id: product.id,
    productoId: product.id,
    nombre: product.nombre,
    numero: num,
    precio: product.precio,
    foto_url: productImage ?? product.foto_url,
    contiene_alcohol: !!product.contiene_alcohol,
    alergenos: mappedAlergenos,
  };
};

const buildGildaPayload = (tipo: 'boqueron' | 'anchoa') =>
  buildVariantPayload(
    tipo,
    tipo === 'boqueron' ? 'Boquerón' : 'Anchoa',
    `Gilda de ${tipo === 'boqueron' ? 'boquerón' : 'anchoa'}`,
    tipo === 'boqueron' ? gildaBoqueronImage : gildaAnchoaImage,
  );

const addJarraVariant = (jarra: 'quijote' | 'sancho') => {
  const payload = buildPayload(jarra);
  if (product.contiene_alcohol) requestAlcohol(payload);
  else addItem(payload);
  setJarraOpen(false);
};

const addGildaVariant = (tipo: 'boqueron' | 'anchoa') => {
  if (isVariantAgotada(tipo)) return;
  const payload = buildGildaPayload(tipo);
  if (product.contiene_alcohol) requestAlcohol(payload);
  else addItem(payload);
  setGildaOpen(false);
};

const buildNachosPayload = (salsa: 'bacon' | 'guacamole') => {
  const label = salsa === 'bacon' ? 'Cheddar y bacon ahumado' : 'Cheddar y guacamole';
  return buildVariantPayload(salsa, label, `Nachos · ${label}`, salsa === 'bacon' ? nachosBaconImage : nachosGuacamoleImage);
};

const addNachosVariant = (salsa: 'bacon' | 'guacamole') => {
  if (isVariantAgotada(salsa)) return;
  const payload = buildNachosPayload(salsa);
  if (product.contiene_alcohol) requestAlcohol(payload);
  else addItem(payload);
  setNachosOpen(false);
};

const alitasBbqImage = PRODUCT_IMAGES._alitasBbq;
const alitasBravaImage = PRODUCT_IMAGES._alitasBrava;

const buildAlitasPayload = (sabor: 'bbq' | 'brava') =>
  buildVariantPayload(
    sabor,
    sabor === 'bbq' ? 'BBQ' : 'Brava',
    `Alitas de pollo · ${sabor === 'bbq' ? 'BBQ' : 'Brava'}`,
    sabor === 'bbq' ? alitasBbqImage : alitasBravaImage,
  );

const addAlitasVariant = (sabor: 'bbq' | 'brava') => {
  if (isVariantAgotada(sabor)) return;
  const payload = buildAlitasPayload(sabor);
  if (product.contiene_alcohol) requestAlcohol(payload);
};

```

```

    else addItem(payload);
    setAlitasOpen(false);
  };

  const handleAdd = (e: React.MouseEvent) => {
    e.stopPropagation();
    if (efectivoAgotado) return; // producto agotado: no se puede añadir
    if (isJarra) {
      setJarraOpen(true);
      return;
    }
    if (isGilda) {
      setGildaOpen(true);
      return;
    }
    if (isNachos) {
      setNachosOpen(true);
      return;
    }
    if (isAlitas) {
      setAlitasOpen(true);
      return;
    }

    const payload = buildPayload();
    if (product.contiene_alcohol) {
      requestAlcohol(payload);
    } else {
      addItem(payload);
    }
  };

  return (
    <>
      <motion.div
        initial={{ opacity: 0, y: 10 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ delay: index * 0.02, duration: 0.3 }}
        whileTap={{ scale: 0.98 }}
        className={`relative glass-card flex items-center gap-3 p-3 cursor-pointer group ${hasExcluded ? 'ring-1 ring-[#FFA012]/50' : ''}`}
        onClick={() => setDetailOpen(true)}
      >
        {productImage && (
          <span className="relative w-12 h-12 rounded-xl bg-card overflow-visible shrink-0 shadow-sm border border-border-subtle">
            <img src={productImage} alt={product.nombre} className="w-full h-full object-contain rounded-xl overflow-hidden"
              loading="lazy" />
            {num && productImage?.startsWith('/assets/img/montaditos/') && (
              <span className="absolute -bottom-1 -right-1 min-w-[1.1rem] h-[1.1rem] px-0.5 rounded-full bg-muted/90 text-muted-foreground text-[9px] font-semibold flex items-center justify-center border border-border leading-none">
                {num}
              </span>
            )}
          </span>
        )}

        {drinkIcon && (
          <span
            className={`w-12 h-12 rounded-xl ${drinkIcon.bg} flex items-center justify-center text-2xl shrink-0 shadow-sm`}
            aria-hidden
          >
            {drinkIcon.emoji}
          </span>
        )}

        {!productImage && !drinkIcon && num && (
          <span className="w-8 h-8 rounded-lg bg-accent/20 text-accent flex items-center justify-center text-xs font-bold shrink-0">
            {num}
          </span>
        )}

        {!num && product.nuevo && (
          <span className="px-1.5 py-0.5 bg-accent text-accent-foreground text-[9px] font-bold uppercase tracking-wider rounded shrink-0">
            Nuevo
          </span>
        )}

        <div className="flex-1 min-w-0">
          <h3 className="font-display text-sm font-bold text-foreground leading-tight truncate">
            {rest}
          </h3>
          {breadBadgeClass && (
            <span className={`inline-block mt-1 px-1.5 py-0.5 text-[9px] font-semibold uppercase tracking-wider rounded border ${breadBadgeClass}`}>
              Pan {product.tipo_pan}
            </span>
          )}
          {product.descripcion && (
            <p className="text-xs text-muted-foreground line-clamp-1 mt-0.5">

```

```

        {product.descripcion}
      </p>
    ))
  </div>

  <div className="flex items-center gap-2 shrink-0">
    <span className="text-sm font-bold text-gold whitespace-nowrap">
      {isJarra
        ? `${precioQuijote.toFixed(2)} / ${precioSancho.toFixed(2)} €`
        : `${product.precio.toFixed(2)} €`}
    </span>
    {efectivoAgotado ? (
      <span className="text-[10px] font-black uppercase tracking-wide text-destructive border border-destructive/40 rounded-full
px-2.5 py-1">
        Agotado
      </span>
    ) : (
      <AnimatePresence mode="wait" initial={false}>
        {cantidadEnCarrito === 0 || isJarra || isGilda || isNachos || isAlitas ? (
          <motion.button
            key="add"
            type="button"
            initial={{ opacity: 0, scale: 0.6 }}
            animate={{ opacity: 1, scale: 1 }}
            exit={{ opacity: 0, scale: 0.6 }}
            whileTap={{ scale: 0.85 }}
            onClick={handleAdd}
            aria-label={isJarra ? 'Elegir tamaño de jarra' : isGilda ? 'Elegir tipo de gilda' : isNachos ? 'Elegir salsa de
nachos' : isAlitas ? 'Elegir sabor de alitas' : 'Añadir al carrito'}
            className="relative w-8 h-8 rounded-full bg-primary flex items-center justify-center opacity-70 group-hover:opacity-
100 transition-opacity"
          >
            <Plus className="w-4 h-4 text-primary-foreground" />
            {(isJarra || isGilda || isNachos || isAlitas) && cantidadEnCarrito > 0 && (
              <span className="absolute -top-1 -right-1 min-w-[1.1rem] h-[1.1rem] px-1 rounded-full bg-accent text-accent-
foreground text-[10px] font-bold flex items-center justify-center tabular-nums">
                {cantidadEnCarrito}
              </span>
            )}
          </motion.button>
        ) : (
          <motion.div
            key="stepper"
            initial={{ opacity: 0, scale: 0.8 }}
            animate={{ opacity: 1, scale: 1 }}
            exit={{ opacity: 0, scale: 0.8 }}
            onClick={(e) => e.stopPropagation()}
            className="flex items-center gap-1 bg-primary/10 border border-primary/40 rounded-full p-0.5"
          >
            <button
              type="button"
              onClick={(e) => {
                e.stopPropagation();
                updateQuantity(product.id, cantidadEnCarrito - 1);
              }}
              aria-label="Quitar uno"
              className="w-7 h-7 rounded-full bg-surface-elevated flex items-center justify-center active:scale-90 transition-
transform"
            >
              <Minus className="w-3.5 h-3.5 text-foreground" />
            </button>
            <span className="min-w-[1.25rem] text-center text-sm font-bold text-foreground tabular-nums">
              {cantidadEnCarrito}
            </span>
            <button
              type="button"
              onClick={handleAdd}
              aria-label="Añadir uno más"
              className="w-7 h-7 rounded-full bg-primary flex items-center justify-center active:scale-90 transition-transform"
            >
              <Plus className="w-3.5 h-3.5 text-primary-foreground" />
            </button>
          </motion.div>
        )}
      </AnimatePresence>
    )}
  </div>

  {efectivoAgotado && (
    <div className="absolute inset-0 rounded-[inherit] bg-background/70 backdrop-blur-[1px] flex items-center justify-center
pointer-events-none">
      <span className="bg-destructive text-destructive-foreground text-[11px] font-black uppercase tracking-wider px-3 py-1.5
rounded-full shadow-lg">
        Producto agotado
      </span>
    </div>
  )}
  {hasExcluded && (

```

```

        <div className="absolute inset-0 rounded-[inherit] bg-[#FFA012]/25 backdrop-blur-[1px] flex items-center justify-center pointer-events-none">
            <span className="bg-[#FFA012] text-white text-[11px] font-bold uppercase tracking-wider px-3 py-1.5 rounded-full flex items-center gap-1.5 shadow-lg">
                <AlertCircle className="w-3.5 h-3.5" />
                Contiene alérgenos seleccionados
            </span>
        </div>
    )}
</motion.div>

<Dialog open={detailOpen} onOpenChange={setDetailOpen}>
    <DialogContent className="max-w-sm rounded-2xl p-0 gap-0 overflow-hidden max-h-[85vh] border-border-subtle">
        {/* Imagen principal – ocupa la parte superior del modal */}
        <div className="relative flex items-center justify-center bg-gradient-to-b from-secondary/60 to-muted/30 pt-6 pb-4 px-6">
            {/* Zona suave circular/beige detrás de la imagen */}
            <div className="absolute w-56 h-56 rounded-full bg-[#F5F0E0]/80 blur-xl" />
            {productImage ? (
                <img
                    src={productImage}
                    alt={product.nombre}
                    className="relative z-10 h-44 sm:h-52 w-auto object-contain drop-shadow-lg"
                    loading="lazy"
                />
            ) : num ? (
                <span className="relative z-10 w-32 h-32 rounded-2xl bg-card flex items-center justify-center text-4xl font-black text-accent shadow-gold">
                    {num}
                </span>
            ) : (
                <span className="relative z-10 w-32 h-32 rounded-2xl bg-card flex items-center justify-center text-4xl shadow-gold">

                </span>
            )}
        </div>

        {/* Contenido textual */}
        <div className="px-5 pb-5 pt-2 max-h-[55vh] overflow-y-auto">
            {breadBadgeClass && (
                <span className={ `inline-block mb-2 px-2 py-0.5 text-[10px] font-semibold uppercase tracking-wider rounded border ${breadBadgeClass}` }>
                    Pan {product.tipo_pan}
                </span>
            )}

            <h2 className="font-display text-2xl sm:text-[1.75rem] font-black text-foreground leading-tight">
                {rest}
            </h2>

            {product.descripcion && (
                <p className="text-sm text-muted-foreground mt-2 leading-relaxed">
                    {product.descripcion}
                </p>
            )}

            {alergenos.length > 0 && (
                <div className="mt-4 pt-4 border-t border-border-subtle">
                    <h4 className="font-display text-sm font-bold mb-3 flex items-center gap-1.5">
                        <AlertCircle className="w-4 h-4 text-muted-foreground" />
                        Alérgenos
                    </h4>
                    <div className="flex flex-wrap gap-3">
                        {alergenos.map((a) => (
                            <AllergenIcon
                                key={a.codigo}
                                codigo={a.codigo}
                                nombre={a.nombre}
                                icono={a.icono}
                                size="md"
                                showLabel
                            />
                        ))}
                    </div>
                    <div className="mt-3">
                        <AllergenLegalNotice variant="short" />
                    </div>
                </div>
            )}
        </div>

        {/* Footer fijo: precio + botón Añadir */}
        <div className="px-5 py-4 border-t border-border-subtle bg-card flex items-center justify-between gap-3">
            <span className="text-xl font-black text-gold whitespace-nowrap">
                {isJarra
                    ? ` ${precioQuijote.toFixed(2)} / ` + ` ${precioSancho.toFixed(2)} €`
                    : ` ${product.precio.toFixed(2)} €` }
            </span>
            <button
                onClick={(e) => {
                    handleAdd(e);
                    setDetailOpen(false);
                }}
            >

```



```

        className="flex items-center gap-2 px-5 py-2.5 rounded-full bg-primary text-primary-foreground text-sm font-bold shadow-
primary active:scale-95 transition-transform"
    >
        <Plus className="w-4 h-4" />
        {isJarra ? 'Elegir tamaño' : isGilda || isNachos || isAlitas ? 'Elegir' : 'Añadir'}
    </button>
</div>
</DialogContent>
</Dialog>

<Dialog open={jarraOpen} onOpenChange={setJarraOpen}>
    <DialogContent className="max-w-sm rounded-xl">
        <DialogHeader>
            <DialogTitle className="font-display text-lg">¿Qué jarra prefieres?</DialogTitle>
            <DialogDescription className="text-sm text-muted-foreground pt-1">
                {product.nombre}
            </DialogDescription>
        </DialogHeader>
        <div className="grid grid-cols-2 gap-3 pt-2">
            <button
                onClick={() => addJarraVariant('quijote')}
                className="flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 border-border hover:border-primary
                hover:bg-primary/5 transition-all active:scale-95"
            >
                <img src={jarraQuijoteOptionImg} alt="Jarra Quijote" className="h-24 w-auto object-contain" />
                <span className="font-display font-bold text-base text-foreground">Quijote</span>
                <span className="text-xs text-muted-foreground">Tamaño estándar</span>
                <span className="text-lg font-black text-gold mt-1">{precioQuijote.toFixed(2)} €</span>
            </button>
            <button
                onClick={() => addJarraVariant('sancho')}
                className="flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 border-border hover:border-primary
                hover:bg-primary/5 transition-all active:scale-95"
            >
                <img src={jarraSanchoOptionImg} alt="Jarra Sancho" className="h-24 w-auto object-contain" />
                <span className="font-display font-bold text-base text-foreground">Sancho</span>
                <span className="text-xs text-muted-foreground">Tamaño grande</span>
                <span className="text-lg font-black text-gold mt-1">{precioSancho.toFixed(2)} €</span>
            </button>
        </div>
    </DialogContent>
</Dialog>

<Dialog open={gildaOpen} onOpenChange={setGildaOpen}>
    <DialogContent className="max-w-sm rounded-xl">
        <DialogHeader>
            <DialogTitle className="font-display text-lg">¿Qué gilda prefieres?</DialogTitle>
            <DialogDescription className="text-sm text-muted-foreground pt-1">
                Elige el sabor de tu gilda
            </DialogDescription>
        </DialogHeader>
        <div className="grid grid-cols-2 gap-3 pt-2">
            <button
                onClick={() => addGildaVariant('boqueron')}
                disabled={isVariantAgotada('boqueron')}
                className={`flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 transition-all
                ${isVariantAgotada('boqueron') ? 'border-destructive/40 opacity-50' : 'border-border hover:border-primary hover:bg-primary/5
                active:scale-95'}`}
            >
                <img src={gildaBoqueronImage} alt="Gilda de boquerón" className="h-24 w-auto object-contain" loading="lazy" />
                <span className="font-display font-bold text-base text-foreground">Boquerón</span>
                {isVariantAgotada('boqueron') ? (
                    <span className="text-[11px] font-black uppercase text-destructive mt-1">Agotado</span>
                ) : (
                    <>
                        <span className="text-xs text-muted-foreground text-center">Suave y fresco</span>
                        <span className="text-lg font-black text-gold mt-1">{product.precio.toFixed(2)} €</span>
                    </>
                )}
            </button>
            <button
                onClick={() => addGildaVariant('anchoa')}
                disabled={isVariantAgotada('anchoa')}
                className={`flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 transition-all
                ${isVariantAgotada('anchoa') ? 'border-destructive/40 opacity-50' : 'border-border hover:border-primary hover:bg-primary/5
                active:scale-95'}`}
            >
                <img src={gildaAnchoaImage} alt="Gilda de anchoa" className="h-24 w-auto object-contain" loading="lazy" />
                <span className="font-display font-bold text-base text-foreground">Anchoa</span>
                {isVariantAgotada('anchoa') ? (
                    <span className="text-[11px] font-black uppercase text-destructive mt-1">Agotado</span>
                ) : (
                    <>
                        <span className="text-xs text-muted-foreground text-center">Intenso y curado</span>
                        <span className="text-lg font-black text-gold mt-1">{product.precio.toFixed(2)} €</span>
                    </>
                )}
            </button>
        </div>
    </DialogContent>
</Dialog>

```

```

<Dialog open={nachosOpen} onOpenChange={setNachosOpen}>
  <DialogContent className="max-w-sm rounded-xl">
    <DialogHeader>
      <DialogTitle className="font-display text-lg">¿Con qué salsa?</DialogTitle>
      <DialogDescription className="text-sm text-muted-foreground pt-1">
        Elige la salsa para tus nachos
      </DialogDescription>
    </DialogHeader>
    <div className="grid grid-cols-2 gap-3 pt-2">
      <button
        onClick={() => addNachosVariant('bacon')}
        disabled={isVariantAgotada('bacon')}
        className={`flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 transition-all ${isVariantAgotada('bacon')
? 'border-destructive/40 opacity-50' : 'border-border hover:border-primary hover:bg-primary/5 active:scale-95'}`}
      >
        <img src={nachosBaconImage} alt="Nachos con cheddar y bacon ahumado" className="h-24 w-auto object-contain"
loading="lazy" />
        <span className="font-display font-bold text-base text-foreground text-center">Cheddar y bacon ahumado</span>
        {isVariantAgotada('bacon')
? <span className="text-[11px] font-black uppercase text-destructive mt-1">Agotado</span>
: <span className="text-lg font-black text-gold mt-1">{product.precio.toFixed(2)} €</span>}
      </button>
      <button
        onClick={() => addNachosVariant('guacamole')}
        disabled={isVariantAgotada('guacamole')}
        className={`flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 transition-all
${isVariantAgotada('guacamole') ? 'border-destructive/40 opacity-50' : 'border-border hover:border-primary hover:bg-primary/5
active:scale-95'}`}
      >
        <img src={nachosGuacamoleImage} alt="Nachos con cheddar y guacamole" className="h-24 w-auto object-contain"
loading="lazy" />
        <span className="font-display font-bold text-base text-foreground text-center">Cheddar y guacamole</span>
        {isVariantAgotada('guacamole')
? <span className="text-[11px] font-black uppercase text-destructive mt-1">Agotado</span>
: <span className="text-lg font-black text-gold mt-1">{product.precio.toFixed(2)} €</span>}
      </button>
    </div>
  </DialogContent>
</Dialog>

<Dialog open={alitasOpen} onOpenChange={setAlitasOpen}>
  <DialogContent className="max-w-sm rounded-xl">
    <DialogHeader>
      <DialogTitle className="font-display text-lg">¿Qué sabor?</DialogTitle>
      <DialogDescription className="text-sm text-muted-foreground pt-1">
        Elige el sabor para tus alitas
      </DialogDescription>
    </DialogHeader>
    <div className="grid grid-cols-2 gap-3 pt-2">
      <button
        onClick={() => addAlitasVariant('bbq')}
        disabled={isVariantAgotada('bbq')}
        className={`flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 transition-all ${isVariantAgotada('bbq') ?
'border-destructive/40 opacity-50' : 'border-border hover:border-primary hover:bg-primary/5 active:scale-95'}`}
      >
        <img src={alitasBbqImage} alt="Alitas BBQ" className="h-24 w-auto object-contain" loading="lazy" />
        <span className="font-display font-bold text-base text-foreground text-center">Sabor BBQ</span>
        <span className="text-xs text-muted-foreground text-center">Ahumado y dulce</span>
        {isVariantAgotada('bbq')
? <span className="text-[11px] font-black uppercase text-destructive mt-1">Agotado</span>
: <span className="text-lg font-black text-gold mt-1">{product.precio.toFixed(2)} €</span>}
      </button>
      <button
        onClick={() => addAlitasVariant('brava')}
        disabled={isVariantAgotada('brava')}
        className={`flex flex-col items-center gap-2 p-4 rounded-2xl bg-card border-2 transition-all ${isVariantAgotada('brava')
? 'border-destructive/40 opacity-50' : 'border-border hover:border-primary hover:bg-primary/5 active:scale-95'}`}
      >
        <img src={alitasBravaImage} alt="Alitas Brava" className="h-24 w-auto object-contain" loading="lazy" />
        <span className="font-display font-bold text-base text-foreground text-center">Sabor Brava</span>
        <span className="text-xs text-muted-foreground text-center">Picante e intenso</span>
        {isVariantAgotada('brava')
? <span className="text-[11px] font-black uppercase text-destructive mt-1">Agotado</span>
: <span className="text-lg font-black text-gold mt-1">{product.precio.toFixed(2)} €</span>}
      </button>
    </div>
  </DialogContent>
</Dialog>
</>

```

```

  );
}

```

```

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/RuedaGourmetDialog.tsx */
/* ===== */
import { useEffect, useState } from 'react';
import { Check } from 'lucide-react';
import { MONTADITO_IMAGES_BY_NUMERO, PRODUCT_IMAGES } from '@lib/product-images';

```

```

import type { RuedaMontadito } from '@lib/ruedas';
import {
  Dialog,
  DialogContent,
  DialogHeader,
  DialogTitle,
  DialogDescription,
} from '@components/ui/dialog';

interface Props {
  open: boolean;
  onOpenChange: (v: boolean) => void;
  nombre: string;
  precio: number;
  need: number;
  comps: RuedaMontadito[];
  isProductoAgotado: (id: string) => boolean;
  onConfirm: (selected: RuedaMontadito[]) => void;
}

/** Selector "elige N montaditos" de la MontyRueda Gourmets (carta y chat de Monty). */
export function RuedaGourmetDialog({ open, onOpenChange, nombre, precio, need, comps, isProductoAgotado, onConfirm }: Props) {
  const [selected, setSelected] = useState<string[]>([]);

  // Reinicia la selección cada vez que se abre.
  useEffect(() => {
    if (open) setSelected([]);
  }, [open]);

  const toggle = (id: string) =>
    setSelected((prev) =>
      prev.includes(id) ? prev.filter((x) => x !== id) : prev.length < need ? [...prev, id] : prev,
    );

  const confirm = () => {
    if (selected.length !== need) return;
    onConfirm(comps.filter((m) => selected.includes(m.id)));
  };

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="max-w-sm rounded-2xl">
        <DialogHeader>
          <DialogTitle className="font-display text-lg">{nombre} · elige {need}</DialogTitle>
          <DialogDescription className="text-sm text-muted-foreground pt-1">
            Seleccionados {selected.length}/{need} · {precio.toFixed(2)} €
          </DialogDescription>
        </DialogHeader>
        <div className="grid grid-cols-1 gap-2 pt-1 max-h-[55vh] overflow-y-auto">
          {comps.map((m) => {
            const ag = isProductoAgotado(m.id);
            const sel = selected.includes(m.id);
            const full = selected.length >= need && !sel;
            const img = (m.numero ? MONTADITO_IMAGES_BY_NUMERO[m.numero] : undefined) ?? PRODUCT_IMAGES[m.nombre];
            return (
              <button
                key={m.id}
                type="button"
                disabled={ag || full}
                onClick={() => toggle(m.id)}
                className={`flex items-center gap-2 p-2.5 rounded-xl border-2 text-left transition-all ${
                  ag
                    ? 'border-destructive/40 opacity-50'
                    : sel
                    ? 'border-primary bg-primary/10'
                    : full
                    ? 'border-border opacity-40'
                    : 'border-border hover:border-primary hover:bg-primary/5 active:scale-[0.98]'
                }`}
              >
                {img ? (
                  <span className="relative w-12 h-12 shrink-0">
                    <img src={img} alt={m.nombre} className="w-full h-full object-contain loading="lazy" />
                    {m.numero && (
                      <span className="absolute bottom-0 right-0 min-w-[1.15rem] h-[1.15rem] px-1 rounded-full bg-muted text-muted-foreground text-[9px] font-bold flex items-center justify-center border border-border leading-none tabular-nums">
                        {m.numero}
                      </span>
                    )}
                  </span>
                ) : (
                  <span className="w-7 h-7 rounded-lg bg-accent/20 text-accent flex items-center justify-center text-xs font-bold shrink-0">
                    {m.numero}
                  </span>
                )}
              </button>
            )
          })}
          <span className="flex-1 text-sm font-medium text-foreground leading-tight">{m.nombre}</span>
          {ag ? (
            <span className="text-[10px] font-black uppercase text-destructive shrink-0">Agotado</span>
          ) : sel ? (
            <span className="w-5 h-5 rounded-full bg-primary flex items-center justify-center shrink-0">
              <Check className="w-3.5 h-3.5 text-primary-foreground" />
            </span>
          ) : (
            <span className="w-5 h-5 rounded-full bg-primary flex items-center justify-center shrink-0">
              <Check className="w-3.5 h-3.5 text-primary-foreground" />
            </span>
          )}
        </div>
      </DialogContent>
    </Dialog>
  );
}

```

```

        </span>
      ) : null}
    </button>
  );
  }
}
</div>
<button
  type="button"
  disabled={selected.length !== need}
  onClick={confirm}
  className="mt-2 w-full py-3 rounded-full bg-primary text-primary-foreground text-sm font-bold shadow-primary active:scale-95
transition-transform disabled:opacity-40 disabled:active:scale-100"
>
  {selected.length === need ? `Añadir rueda · ${precio.toFixed(2)} €` : `Elige ${need - selected.length} más`}
</button>
</DialogContent>
</Dialog>
);
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/SectionTabs.tsx */
/* ===== */
import { motion } from 'framer-motion';

interface Props {
  sections: string[];
  active: string | null;
  onSelect: (section: string | null) => void;
}

const SECTION_COLORS: Record<string, string> = {
  // Montaditos
  'De la casa': 'bg-[#C8102E] text-white',
  'Clásicos': 'bg-[#1E3A5F] text-white',
  'Imprescindibles': 'bg-[#C9A84C] text-[#1a1a1a]',
  'Especiales': 'bg-[#2D6A4F] text-white',
  'MontyCookie': 'bg-[#D4A59A] text-[#1a1a1a]',
  'MontyDinas': 'bg-[#8B5A2B] text-white',
  'MontyPerros': 'bg-[#B8860B] text-white',
  'MontyBurgers': 'bg-[#CD7F32] text-white',
  'MontyPizzas': 'bg-[#8B4513] text-white',
  'MontyGourmet': 'bg-[#4A5568] text-white',
  // Bebidas
  'Clásicas': 'bg-[#C8102E] text-white',
  'Energéticas': 'bg-[#7CFC00] text-[#1a1a1a]',
  'Tardeo Chill': 'bg-[#6B5B95] text-white',
  'Tardeo Premium': 'bg-[#C9A84C] text-[#1a1a1a]',
  'Jarras Heladas': 'bg-[#F4A300] text-[#1a1a1a]',
  'Cerveza Premium': 'bg-[#B8860B] text-white',
  'Cerveza en Botella': 'bg-[#8B5A2B] text-white',
  'Vino': 'bg-[#722F37] text-white',
  'Café e Infusiones': 'bg-[#4A2C2A] text-white',
};

const DEFAULT_COLOR = 'bg-card text-muted-foreground border border-border';

export function SectionTabs({ sections, active, onSelect }: Props) {
  return (
    <div className="flex gap-2 overflow-x-auto pb-2 scrollbar-hide">
      <motion.button
        whileTap={{ scale: 0.95 }}
        onClick={() => onSelect(null)}
        className={`shrink-0 px-4 py-2 rounded-full text-xs font-bold tracking-wide transition-all duration-300 ${
          active === null
            ? 'bg-primary text-primary-foreground shadow-md'
            : 'bg-card text-muted-foreground border border-border'
        }`}
      >
        Todas
      </motion.button>
      {sections.map((sec) => {
        const colorClass = SECTION_COLORS[sec] || DEFAULT_COLOR;
        const isActive = active === sec;
        return (
          <motion.button
            key={sec}
            whileTap={{ scale: 0.95 }}
            onClick={() => onSelect(sec)}
            className={`shrink-0 px-4 py-2 rounded-full text-xs font-bold tracking-wide transition-all duration-300 ${
              isActive ? `${colorClass} shadow-md ring-2 ring-primary/30` : 'bg-card text-muted-foreground border border-border
hover:text-foreground'
            }`}
          >
            {sec}
          </motion.button>
        );
      })}
    </div>
  );
}

```

```

}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/WelcomeScreen.tsx */
/* ===== */
import { useEffect, useRef } from 'react';
import { motion, AnimatePresence } from 'framer-motion';
import confetti from 'canvas-confetti';
import { Monty } from './Monty';

interface Props {
  open: boolean;
  onClose: () => void;
}

const fireConfetti = () => {
  const defaults = {
    spread: 360,
    ticks: 100,
    gravity: 0.6,
    decay: 0.94,
    startVelocity: 30,
    colors: ['#E63946', '#F4A261', '#E9C46A', '#F1FAEE', '#FFB70D'],
  };

  const burst = (originY: number, particleCount: number, scalar: number) =>
    confetti({
      ...defaults,
      particleCount,
      scalar,
      origin: { x: 0.5, y: originY },
    });

  burst(0.6, 50, 1.2);
  setTimeout(() => burst(0.55, 30, 0.9), 200);
  setTimeout(() => burst(0.5, 40, 1.4), 400);
};

export function WelcomeScreen({ open, onClose }: Props) {
  const fired = useRef(false);

  useEffect(() => {
    if (open && !fired.current) {
      fired.current = true;
      // small delay so confetti syncs with Monty's bounce
      setTimeout(fireConfetti, 350);
    }
    if (!open) fired.current = false;
  }, [open]);

  const handleClose = () => {
    fireConfetti();
    setTimeout(onClose, 250);
  };

  return (
    <AnimatePresence>
      {open && (
        <motion.div
          initial={{ opacity: 0 }}
          animate={{ opacity: 1 }}
          exit={{ opacity: 0 }}
          transition={{ duration: 0.3 }}
          className="fixed inset-0 z-[60] flex flex-col items-center justify-center px-6 overflow-hidden"
          style={{
            background:
              'radial-gradient(ellipse at top, hsl(var(--primary) / 0.15), transparent 60%), hsl(var(--background))',
          }}
        >
          {/* Animated background glow */}
          <motion.div
            initial={{ scale: 0, opacity: 0 }}
            animate={{ scale: [0, 1.2, 1], opacity: [0, 0.6, 0.3] }}
            transition={{ duration: 1.2, ease: 'easeOut' }}
            className="absolute w-[500px] h-[500px] rounded-full bg-primary/10 blur-3xl"
          />

          {/* Monty with dramatic entrance: drop + bounce + wiggle */}
          <motion.div
            initial={{ y: -400, opacity: 0, rotate: -25, scale: 0.5 }}
            animate={{
              y: [-400, 30, -20, 10, -8, 0],
              opacity: 1,
              rotate: [-25, 5, -3, 2, -1, 0],
              scale: [0.5, 1.05, 0.95, 1.02, 0.99, 1],
            }}
            transition={{
              duration: 1.6,
              times: [0, 0.45, 0.65, 0.8, 0.92, 1],
              ease: 'easeOut',
            }}
          />
        </motion.div>
      )}
    </AnimatePresence>
  );
}

```

```

        className="relative z-10"
      >
      <motion.div
        animate={{ y: [0, -10, 0] }}
        transition={{
          duration: 1.8,
          repeat: Infinity,
          ease: 'easeInOut',
          delay: 1.6,
        }}
      >
      <Monty pose="waving" size={240} animate="none" />
    </motion.div>
  </motion.div>

  <motion.div
    initial={{ opacity: 0, y: 30 }}
    animate={{ opacity: 1, y: 0 }}
    transition={{ delay: 1.2, duration: 0.5 }}
    className="text-center mt-6 max-w-xs relative z-10"
  >
    <p className="text-xs uppercase tracking-[0.2em] text-muted-foreground mb-2">
      ¡Hola! Soy Monty
    </p>
    <h1 className="font-display text-3xl font-black leading-tight text-foreground">
      Pide tus <span className="text-primary italic">montaditos</span> favoritos sin esperar colas
    </h1>
    <p className="text-sm text-muted-foreground mt-3">
      Yo te ayudo a elegir, tú disfrutas.
    </p>
  </motion.div>

  <motion.button
    initial={{ opacity: 0, y: 30, scale: 0.8 }}
    animate={{ opacity: 1, y: 0, scale: 1 }}
    transition={{ delay: 1.5, type: 'spring', damping: 12 }}
    whileTap={{ scale: 0.94 }}
    whileHover={{ scale: 1.05 }}
    onClick={handleClose}
    className="mt-10 px-10 py-3.5 rounded-full bg-primary text-primary-foreground font-bold shadow-lg shadow-primary/30
relative z-10"
  >
    ¡Vamos allá!
  </motion.button>
</motion.div>
))
</AnimatePresence>
);
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/hooks/use-agotados.ts */
/* ===== */
import { useEffect, useMemo } from 'react';
import { useQuery, useQueryClient } from '@tanstack/react-query';
import { supabase } from '@integrations/supabase/client';
import { useActiveLocal } from '@lib/active-local';

/**
 * Calcula qué productos están AGOTADOS en el local activo:
 * - por INGREDIENTE/PAN agotado (montaditos/ensaladas que lo contienen)
 * - por PRODUCTO marcado agotado tal cual (aperitivos/raciones/bebidas/...)
 *
 * Optimización de velocidad:
 * - El mapa producto→ingrediente apenas cambia → se cachea (10 min).
 * - El estado de "agotados" es diminuto → se refresca cada 3s + realtime.
 * Resultado: instantáneo donde el WebSocket conecta (Android/escritorio) y
 * como mucho 3s donde no (iPhone), en vez de los 20s anteriores.
 */
export function useAgotados() {
  const localId = useActiveLocal((s) => s.local?.id);
  const qc = useQueryClient();

  // Catálogo producto→ingrediente (cambia muy poco)
  const { data: prodIng } = useQuery<{ producto_id: string; ingrediente_id: string }[]>({
    queryKey: ['producto-ingredientes'],
    staleTime: 10 * 60 * 1000,
    queryFn: async () => {
      const { data } = await supabase
        .from('producto_ingredientes' as any)
        .select('producto_id, ingrediente_id');
      return (data ?? []) as any[];
    },
  });

  // Catálogo de ingredientes (id → nombre), para resolver agotados por nombre.
  const { data: catalogo } = useQuery<{ id: string; nombre: string }[]>({
    queryKey: ['ingredientes-catalogo'],
    staleTime: 10 * 60 * 1000,
    queryFn: async () => {
      const { data } = await supabase.from('ingredientes' as any).select('id, nombre');

```

```

    return (data ?? []) as any[];
  },
});

// Estado de agotados del local (pequeño → refresco rápido)
const { data: agot } = useQuery<{ ings: Set<string>; prods: Set<string> }>({
  queryKey: ['local-agotados', localId],
  enabled: !!localId,
  refetchInterval: 3000,
  refetchOnWindowFocus: true,
  queryFn: async () => {
    if (!localId) return { ings: new Set<string>(), prods: new Set<string>() };
    const [ia, pa] = await Promise.all([
      supabase.from('local_ingredientes_agotados' as any).select('ingrediente_id').eq('local_id', localId),
      supabase.from('local_productos_agotados' as any).select('producto_id').eq('local_id', localId),
    ]);
    return {
      ings: new Set<string>(((ia.data ?? []) as any[]).map((r) => r.ingrediente_id)),
      prods: new Set<string>(((pa.data ?? []) as any[]).map((r) => r.producto_id)),
    };
  },
});

// Realtime: si el WebSocket conecta, refresca al instante.
useEffect(() => {
  if (!localId) return;
  let ch: ReturnType<typeof supabase.channel> | null = null;
  const invalidate = () => qc.invalidateQueries({ queryKey: ['local-agotados', localId] });
  try {
    ch = supabase
      .channel(`agotados-${localId}`)
      .on('postgres_changes', { event: '*', schema: 'public', table: 'local_ingredientes_agotados', filter: `local_id=eq.${localId}` }, invalidate)
      .on('postgres_changes', { event: '*', schema: 'public', table: 'local_productos_agotados', filter: `local_id=eq.${localId}` }, invalidate)
      .subscribe();
  } catch {
    /* realtime no disponible: queda el refetchInterval de 3s */
  }
  return () => {
    if (ch) {
      try { supabase.removeChannel(ch); } catch { /* ignore */ }
    }
  };
}, [localId, qc]);

const agotados = useMemo(() => {
  const s = new Set<string>();
  const ings = agot?.ings ?? new Set<string>();
  for (const r of prodIng ?? []) {
    if (ings.has(r.ingrediente_id)) s.add(r.producto_id);
  }
  for (const p of agot?.prods ?? new Set<string>()) s.add(p);
  return s;
}, [prodIng, agot]);

// Nombres de ingredientes agotados (para variantes: alitas/nachos).
const agotadoNombres = useMemo(() => {
  const ids = agot?.ings ?? new Set<string>();
  const set = new Set<string>();
  for (const c of catalogo ?? []) {
    if (ids.has(c.id)) set.add(c.nombre);
  }
  return set;
}, [catalogo, agot]);

return {
  agotados,
  isProductoAgotado: (productoId: string) => agotados.has(productoId),
  isIngredienteAgotado: (nombre: string) => agotadoNombres.has(nombre),
};
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/hooks/use-allergens.ts */
/* ===== */
import { useQuery } from '@tanstack/react-query';
import { supabase } from '@integrations/supabase/client';

export interface Allergen {
  id: string;
  codigo: string;
  nombre: string;
  descripcion: string | null;
  icono: string | null;
  orden: number;
}

export function useAllergens() {
  return useQuery({
    queryKey: ['alergenos'],
  });
}

```

```

    queryFn: async (): Promise<Allergen[]> => {
      const { data, error } = await supabase
        .from('alergenitos' as any)
        .select('*')
        .order('orden');
      if (error) throw error;
      return (data ?? []) as Allergen[];
    },
    staleTime: 5 * 60_000,
  });
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/hooks/use-local-restricciones.ts */
/* ===== */
import { useQuery } from '@tanstack/react-query';
import { supabase } from '@integrations/supabase/client';
import { useActiveLocal } from '@lib/active-local';

export type LocalRestriccion = {
  id: string;
  local_id: string;
  producto_id: string | null;
  seccion: string | null;
  hora_limite: string | null; // 'HH:MM' o null
};

export function useLocalRestricciones() {
  const localId = useActiveLocal((s) => s.local?.id);

  return useQuery<LocalRestriccion[]>({
    queryKey: ['local-restricciones', localId],
    queryFn: async () => {
      if (!localId) return [];
      const { data, error } = await supabase
        .from('local_restricciones' as any)
        .select('id, local_id, producto_id, seccion, hora_limite')
        .eq('local_id', localId);
      if (error) throw error;
      return (data ?? []) as LocalRestriccion[];
    },
    enabled: !!localId,
    staleTime: 10 * 60 * 1000,
  });
}

function isActiveNow(r: LocalRestriccion): boolean {
  if (!r.hora_limite) return true;
  const [h, m] = r.hora_limite.split(':').map(Number);
  const now = new Date();
  return now.getHours() * 60 + now.getMinutes() >= h * 60 + m;
}

export function buildRestriccionFilter(restricciones: LocalRestriccion[]) {
  const productosRestringidos = new Set<string>();
  const seccionesRestringidas = new Set<string>();

  for (const r of restricciones) {
    if (!isActiveNow(r)) continue;
    if (r.producto_id) productosRestringidos.add(r.producto_id);
    if (r.seccion) seccionesRestringidas.add(r.seccion);
  }

  return {
    isProductoAllowed: (id: string) => !productosRestringidos.has(id),
    isSeccionAllowed: (seccion: string) => !seccionesRestringidas.has(seccion),
  };
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/hooks/use-menu.ts */
/* ===== */
import { useQuery } from '@tanstack/react-query';
import { supabase } from '@integrations/supabase/client';

export interface ProductAllergen {
  codigo: string;
  nombre: string;
  icono: string | null;
}

const MENU_STALE_TIME = 5 * 60 * 1000; // 5 min - el menú no cambia con frecuencia

export function useCategories() {
  return useQuery({
    queryKey: ['menu-categories'],
    staleTime: MENU_STALE_TIME,
    queryFn: async () => {
      const { data, error } = await supabase
        .from('menu_categorias')

```



```

        .select('*')
        .order('orden');
    if (error) throw error;
    return data;
  },
});
}

export function useProducts(categoriaId?: string) {
  return useQuery({
    queryKey: ['menu-products', categoriaId],
    staleTime: MENU_STALE_TIME,
    queryFn: async () => {
      let query = supabase
        .from('menu_productos')
        .select('*', producto_alergenos(alergenos(id, codigo, nombre, icono, orden)))
        .eq('disponible', true);
      if (categoriaId) query = query.eq('categoria_id', categoriaId);
      const { data, error } = await query.order('destacado', { ascending: false });
      if (error) throw error;
      return (data ?? []).map((p: any) => {
        const alergenos: ProductAllergen[] = (p.producto_alergenos ?? []).
          .map((pa: any) => pa.alergenos)
          .filter(Boolean)
          .sort((a: any, b: any) => (a.orden ?? 0) - (b.orden ?? 0))
          .map((a: any) => ({ codigo: a.codigo, nombre: a.nombre, icono: a.icono }));
        return { ...p, alergenos };
      });
    },
  });
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/hooks/use-push-notifications.ts */
/* ===== */
import { useCallback } from 'react';
import { supabase } from '@integrations/supabase/client';

const VAPID_PUBLIC_KEY = import.meta.env.VITE_VAPID_PUBLIC_KEY as string | undefined;

function urlBase64ToUint8Array(base64: string): Uint8Array {
  const padding = '='.repeat((4 - (base64.length % 4)) % 4);
  const b64 = (base64 + padding).replace(/-/g, '+').replace(/_/g, '/');
  const raw = atob(b64);
  return Uint8Array.from([...raw].map((c) => c.charCodeAt(0)));
}

export async function registerPushServiceWorker(): Promise<void> {
  if (!('serviceWorker' in navigator)) return;
  try {
    await navigator.serviceWorker.register('/sw.js', { scope: '/' });
  } catch (e) {
    console.error('[SW] registro fallido', e);
  }
}

export function usePushSubscription() {
  const subscribe = useCallback(async (sessionId: string, pedidoId: string): Promise<void> => {
    if (!VAPID_PUBLIC_KEY) return;
    if (!('Notification' in window) || !('PushManager' in window)) return;

    try {
      const permission = await Notification.requestPermission();
      if (permission !== 'granted') return;

      const reg = await navigator.serviceWorker.ready;
      let sub = await reg.pushManager.getSubscription();
      if (!sub) {
        sub = await reg.pushManager.subscribe({
          userVisibleOnly: true,
          applicationServerKey: urlBase64ToUint8Array(VAPID_PUBLIC_KEY),
        });
      }

      await supabase
        .from('push_subscriptions' as any)
        .upsert(
          { session_id: sessionId, pedido_id: pedidoId, subscription: sub.toJSON() },
          { onConflict: 'session_id,pedido_id' }
        );
    } catch (e) {
      console.error('[Push] suscripción fallida', e);
    }
  }, []);

  return { subscribe };
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/hooks/use-toast.ts */

```

```

/* ===== */
import * as React from "react";

import type { ToastActionElement, ToastProps } from "@components/ui/toast";

const TOAST_LIMIT = 1;
const TOAST_REMOVE_DELAY = 1000000;

type ToasterToast = ToastProps & {
  id: string;
  title?: React.ReactNode;
  description?: React.ReactNode;
  action?: ToastActionElement;
};

const actionTypes = {
  ADD_TOAST: "ADD_TOAST",
  UPDATE_TOAST: "UPDATE_TOAST",
  DISMISS_TOAST: "DISMISS_TOAST",
  REMOVE_TOAST: "REMOVE_TOAST",
} as const;

let count = 0;

function genId() {
  count = (count + 1) % Number.MAX_SAFE_INTEGER;
  return count.toString();
}

type ActionType = typeof actionTypes;

type Action =
  | {
    type: ActionType["ADD_TOAST"];
    toast: ToasterToast;
  }
  | {
    type: ActionType["UPDATE_TOAST"];
    toast: Partial<ToasterToast>;
  }
  | {
    type: ActionType["DISMISS_TOAST"];
    toastId?: ToasterToast["id"];
  }
  | {
    type: ActionType["REMOVE_TOAST"];
    toastId?: ToasterToast["id"];
  };

interface State {
  toasts: ToasterToast[];
}

const toastTimeouts = new Map<string, ReturnType<typeof setTimeout>>();

const addToRemoveQueue = (toastId: string) => {
  if (toastTimeouts.has(toastId)) {
    return;
  }

  const timeout = setTimeout(() => {
    toastTimeouts.delete(toastId);
    dispatch({
      type: "REMOVE_TOAST",
      toastId: toastId,
    });
  }, TOAST_REMOVE_DELAY);

  toastTimeouts.set(toastId, timeout);
};

export const reducer = (state: State, action: Action): State => {
  switch (action.type) {
    case "ADD_TOAST":
      return {
        ...state,
        toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
      };

    case "UPDATE_TOAST":
      return {
        ...state,
        toasts: state.toasts.map((t) => (t.id === action.toast.id ? { ...t, ...action.toast } : t)),
      };

    case "DISMISS_TOAST": {
      const { toastId } = action;

      // ! Side effects ! - This could be extracted into a dismissToast() action,
      // but I'll keep it here for simplicity
      if (toastId) {
        addToRemoveQueue(toastId);
      }
    }
  }
}

```

```

    } else {
      state.toasts.forEach((toast) => {
        addToRemoveQueue(toast.id);
      });
    }

    return {
      ...state,
      toasts: state.toasts.map((t) =>
        t.id === toastId || toastId === undefined
          ? {
              ...t,
              open: false,
            }
          : t,
      ),
    };
  }
}

case "REMOVE_TOAST":
  if (action.toastId === undefined) {
    return {
      ...state,
      toasts: [],
    };
  }
  return {
    ...state,
    toasts: state.toasts.filter((t) => t.id !== action.toastId),
  };
}
};

const listeners: Array<(state: State) => void> = [];

let memoryState: State = { toasts: [] };

function dispatch(action: Action) {
  memoryState = reducer(memoryState, action);
  listeners.forEach((listener) => {
    listener(memoryState);
  });
}

type Toast = Omit<ToasterToast, "id">;

function toast({ ...props }: Toast) {
  const id = genId();

  const update = (props: ToasterToast) =>
    dispatch({
      type: "UPDATE_TOAST",
      toast: { ...props, id },
    });
  const dismiss = () => dispatch({ type: "DISMISS_TOAST", toastId: id });

  dispatch({
    type: "ADD_TOAST",
    toast: {
      ...props,
      id,
      open: true,
      onOpenChange: (open) => {
        if (!open) dismiss();
      },
    },
  });
};

return {
  id: id,
  dismiss,
  update,
};
}

function useToast() {
  const [state, setState] = React.useState<State>(memoryState);

  React.useEffect(() => {
    listeners.push(setState);
    return () => {
      const index = listeners.indexOf(setState);
      if (index > -1) {
        listeners.splice(index, 1);
      }
    };
  }, [state]);

  return {
    ...state,
    toast,
    dismiss: (toastId?: string) => dispatch({ type: "DISMISS_TOAST", toastId }),
  };
};

```

```

}

export { useToast, toast };

/* ===== */
/* FICHERO: montadito-magic-flow/src/hooks/use-mobile.tsx */
/* ===== */
import * as React from "react";

const MOBILE_BREAKPOINT = 768;

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState<boolean | undefined>(undefined);

  React.useEffect(() => {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`);
    const onChange = () => {
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    };
    mql.addEventListener("change", onChange);
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    return () => mql.removeEventListener("change", onChange);
  }, []);

  return !!isMobile;
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/active-local.ts */
/* ===== */
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

export type ActiveLocal = {
  id: string;
  nombre: string;
  ciudad: string;
  direccion: string | null;
};

type State = {
  local: ActiveLocal | null;
  setLocal: (l: ActiveLocal | null) => void;
  clear: () => void;
};

export const useActiveLocal = create<State>()(
  persist(
    (set) => ({
      local: null,
      setLocal: (l) => set({ local: l }),
      clear: () => set({ local: null }),
    }),
    { name: 'monty-active-local' }
  )
);

// Haversine distance in meters
export function haversineMeters(
  lat1: number,
  lng1: number,
  lat2: number,
  lng2: number
): number {
  const R = 6371000;
  const toRad = (d: number) => (d * Math.PI) / 180;
  const dLat = toRad(lat2 - lat1);
  const dLng = toRad(lng2 - lng1);
  const a =
    Math.sin(dLat / 2) ** 2 +
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.sin(dLng / 2) ** 2;
  return 2 * R * Math.asin(Math.sqrt(a));
}

export function formatDistance(m: number): string {
  if (m < 1000) return `${Math.round(m)} m`;
  return `${(m / 1000).toFixed(1)} km`;
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/alarm.ts */
/* ===== */
/**
 * Alarma sonora persistente para avisar de "pedido listo".
 *
 * Realidad iOS Safari:
 * - El audio web debe "desbloquearse" con un gesto del usuario (un toque).
 * - Por eso unlockAudio() debe llamarse DENTRO de un handler de click/touch.
 * - Si el interruptor de silencio del iPhone está activado, iOS silencia

```

```

*   TODO el audio web. No hay solución por web (de ahí la alarma visual).
*   - Mientras la pestaña está en primer plano y el audio está desbloqueado,
*     podemos sonar en bucle programáticamente.
*
*   Mantenemos un único AudioContext compartido y vivo (no lo cerramos) para
*   que permanezca desbloqueado entre el gesto inicial y el momento del aviso.
*/

let ctx: AudioContext | null = null;
let loopId: number | null = null;

function getCtx(): AudioContext | null {
  try {
    const AudioCtx =
      window.AudioContext ||
      (window as unknown as { webkitAudioContext: typeof AudioContext }).webkitAudioContext;
    if (!AudioCtx) return null;
    if (!ctx) ctx = new AudioCtx();
    return ctx;
  } catch {
    return null;
  }
}

/** Debe invocarse desde un gesto del usuario (click/touch) para desbloquear iOS. */
export function unlockAudio(): boolean {
  const c = getCtx();
  if (!c) return false;
  try {
    if (c.state === 'suspended') c.resume();
    // Blip casi inaudible para completar el desbloqueo en iOS.
    const o = c.createOscillator();
    const g = c.createGain();
    g.gain.setValueAtTime(0.0001, c.currentTime);
    o.connect(g);
    g.connect(c.destination);
    o.start();
    o.stop(c.currentTime + 0.05);
    return true;
  } catch {
    return false;
  }
}

export function isAudioUnlocked(): boolean {
  return !!ctx && ctx.state === 'running';
}

/** Alarma FUERTE y molesta: pitidos alternando agudo/grave con onda cuadrada
 * (rica en armónicos = se percibe muy alta) a tope con limitador. */
function chime() {
  const c = ctx;
  if (!c) return;
  if (c.state === 'suspended') c.resume();
  const now = c.currentTime;
  const limiter = c.createDynamicsCompressor();
  limiter.threshold.value = -6;
  limiter.ratio.value = 20;
  limiter.attack.value = 0.002;
  limiter.connect(c.destination);
  const dur = 0.14;
  const gap = 0.06;
  for (let i = 0; i < 8; i++) {
    const freq = i % 2 === 0 ? 1320 : 990;
    const t = now + i * (dur + gap);
    const g = c.createGain();
    g.gain.setValueAtTime(0.0001, t);
    g.gain.exponentialRampToValueAtTime(1.0, t + 0.006);
    g.gain.setValueAtTime(1.0, t + dur - 0.02);
    g.gain.exponentialRampToValueAtTime(0.0001, t + dur);
    g.connect(limiter);
    [-6, 0, 6].forEach((det) => {
      const osc = c.createOscillator();
      osc.type = 'square';
      osc.frequency.value = freq;
      osc.detune.value = det;
      osc.connect(g);
      osc.start(t);
      osc.stop(t + dur + 0.02);
    });
  }
}

/** Vibración (solo Android; iOS Safari no soporta la Vibration API). */
function vibrate() {
  try {
    navigator.vibrate?.([500, 250, 500]);
  } catch {
    /* no soportado */
  }
}

```

```

/** Arranca la alarma en bucle (suena y vibra cada ~2s) hasta llamar a stopAlarm(). */
export function startAlarm() {
  // El sonido necesita audio desbloqueado; la vibración es independiente.
  if (ctx) {
    if (ctx.state === 'suspended') ctx.resume();
    chime();
  }
  vibrate();
  if (loopId == null) {
    loopId = window.setInterval(() => {
      if (ctx) chime();
      vibrate();
    }, 2000);
  }
}

export function stopAlarm() {
  if (loopId != null) {
    window.clearInterval(loopId);
    loopId = null;
  }
  try {
    navigator.vibrate?.(0); // cancela cualquier vibración en curso
  } catch {
    /* no soportado */
  }
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/allergen-filter.ts */
/* ===== */
import { create } from 'zustand';
import { persist, createJSONStorage } from 'zustand/middleware';

interface AllergenFilterStore {
  excluded: string[]; // códigos de alérgenos a excluir
  hideMode: 'warn' | 'hide'; // mostrar advertencia o ocultar
  toggle: (codigo: string) => void;
  clear: () => void;
  setHideMode: (mode: 'warn' | 'hide') => void;
}

export const useAllergenFilter = create<AllergenFilterStore>()(
  persist(
    (set) => ({
      excluded: [],
      hideMode: 'warn',
      toggle: (codigo) =>
        set((s) => ({
          excluded: s.excluded.includes(codigo)
            ? s.excluded.filter((c) => c !== codigo)
            : [...s.excluded, codigo],
        })),
      clear: () => set({ excluded: [] }),
      setHideMode: (mode) => set({ hideMode: mode }),
    }),
    {
      name: 'monty-allergen-filter',
      storage: createJSONStorage(() => localStorage),
    }
  )
);

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/bell-sound.ts */
/* ===== */
/**
 * Plays a bell/chime sound using Web Audio API.
 * Synthesized on the fly – no audio file needed.
 *
 * Note: On iOS, if the physical silent switch is ON, no web audio can play.
 * This is an OS-level restriction. On Android and desktop, it will play
 * regardless of media volume settings (uses the system/ringer channel).
 */
export async function playBellSound(volume = 0.6) {
  try {
    const AudioCtx =
      window.AudioContext || (window as unknown as { webkitAudioContext: typeof AudioContext }).webkitAudioContext;
    if (!AudioCtx) return;

    const ctx = new AudioCtx();

    // iOS requires resume after user gesture
    if (ctx.state === 'suspended') {
      await ctx.resume();
    }

    const now = ctx.currentTime;

    // Bell-like sound: two harmonics that decay

```

```

const playTone = (freq: number, startTime: number, duration: number, gain: number) => {
  const osc = ctx.createOscillator();
  const gainNode = ctx.createGain();

  osc.type = 'sine';
  osc.frequency.setValueAtTime(freq, startTime);

  gainNode.gain.setValueAtTime(0, startTime);
  gainNode.gain.linearRampToValueAtTime(gain * volume, startTime + 0.01);
  gainNode.gain.exponentialRampToValueAtTime(0.001, startTime + duration);

  osc.connect(gainNode);
  gainNode.connect(ctx.destination);

  osc.start(startTime);
  osc.stop(startTime + duration);
};

// Two-tone "ding-dong" bell (classic service bell)
// First chime: high bright tone
playTone(1318.51, now, 1.5, 0.4); // E6 fundamental
playTone(2637.02, now, 1.2, 0.15); // E7 harmonic
playTone(3951.07, now, 0.8, 0.08); // B7 sparkle

// Second chime slightly lower (ding-dong effect)
playTone(987.77, now + 0.35, 1.5, 0.4); // B5 fundamental
playTone(1975.53, now + 0.35, 1.2, 0.15); // B6 harmonic
playTone(2959.96, now + 0.35, 0.8, 0.08); // F#7 sparkle

// Auto-close context after sound finishes to free resources
setTimeout(() => {
  ctx.close().catch(() => {});
}, 2500);
} catch (err) {
  console.warn('Could not play bell sound:', err);
}
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/cart-store.ts */
/* ===== */
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

export interface CartItemAllergen {
  codigo: string;
  nombre: string;
  icono?: string | null;
}

export interface CartItem {
  /** Cart key: productId for normal items, `${productId}::${variant}` for variants. */
  id: string;
  /** Real menu_productos.id used when sending the order to the backend. */
  productId: string;
  /** Optional variant key (e.g. 'quijote' | 'sancho'). */
  variant?: string;
  /** Human-readable variant label shown in the cart (e.g. 'Jarra Quijote'). */
  variantLabel?: string;
  nombre: string;
  /** Número del producto en la carta (ej. "13", "B5"). Se muestra en cesta y chat. */
  numero?: string | null;
  precio: number;
  cantidad: number;
  foto_url: string | null;
  contiene_alcohol?: boolean;
  alergenos?: CartItemAllergen[];
  /** Para MontyRuedas: montaditos que la componen (ej. "#86 Hot dog..."). Se envía a cocina en `variante`. */
  componentes?: string[];
}

interface CartStore {
  items: CartItem[];
  localId: string | null;
  mesaId: string | null;
  sessionId: string;
  addItem: (product: Omit<CartItem, 'cantidad'>) => void;
  removeItem: (id: string) => void;
  updateQuantity: (id: string, cantidad: number) => void;
  clearCart: () => void;
  setLocal: (localId: string, mesaId: string) => void;
  total: () => number;
  itemCount: () => number;
  hasAlcohol: () => boolean;
}

const getSessionId = () => {
  let id = sessionStorage.getItem('montaditos_session');
  if (!id) {
    id = crypto.randomUUID();
    sessionStorage.setItem('montaditos_session', id);
  }
}

```

```

    }
    return id;
  };
}

export const useCartStore = create<CartStore>()(
  persist(
    (set, get) => ({
      items: [],
      localId: null,
      mesaId: null,
      sessionId: getSessionId(),
      addItem: (product) => {
        set((state) => {
          const existing = state.items.find((i) => i.id === product.id);
          if (existing) {
            return {
              items: state.items.map((i) =>
                i.id === product.id ? { ...i, cantidad: i.cantidad + 1 } : i
              ),
            };
          }
          return { items: [...state.items, { ...product, cantidad: 1 }] };
        });
      },
      removeItem: (id) => set((s) => ({ items: s.items.filter((i) => i.id !== id) })),
      updateQuantity: (id, cantidad) => {
        if (cantidad <= 0) {
          set((s) => ({ items: s.items.filter((i) => i.id !== id) }));
        } else {
          set((s) => ({
            items: s.items.map((i) => (i.id === id ? { ...i, cantidad } : i)),
          }));
        }
      },
      clearCart: () => set({ items: [] }),
      setLocal: (localId, mesaId) => set({ localId, mesaId }),
      total: () => get().items.reduce((sum, i) => sum + i.precio * i.cantidad, 0),
      itemCount: () => get().items.reduce((sum, i) => sum + i.cantidad, 0),
      hasAlcohol: () => get().items.some((i) => i.contiene_alcohol),
    }),
    {
      name: 'monty-cart',
      // Persistimos solo el contenido del carrito; sessionId se regenera por sesión.
      partialize: (state) => ({
        items: state.items,
        localId: state.localId,
        mesaId: state.mesaId,
      }),
    }
  )
);

// Alcohol age-gate store (sólo se muestra una vez por sesión)
interface AgeGateStore {
  acknowledged: boolean;
  pending: Omit<CartItem, 'cantidad'> | null;
  request: (product: Omit<CartItem, 'cantidad'>) => void;
  confirm: () => void;
  cancel: () => void;
}

const ACK_KEY = 'monty-alcohol-ack';

export const useAgeGate = create<AgeGateStore>((set, get) => ({
  acknowledged: sessionStorage.getItem(ACK_KEY) === '1',
  pending: null,
  request: (product) => {
    if (get().acknowledged) {
      useCartStore.getState().addItem(product);
    } else {
      set({ pending: product });
    }
  },
  confirm: () => {
    sessionStorage.setItem(ACK_KEY, '1');
    const pending = get().pending;
    if (pending) useCartStore.getState().addItem(pending);
    set({ acknowledged: true, pending: null });
  },
  cancel: () => set({ pending: null }),
}));

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/checkout-redirect.ts */
/* ===== */
export const redirectToCheckout = (url: string) => {
  window.open(url, '_top');
  window.setTimeout(() => {
    window.location.assign(url);
  }, 250);
};

```



```

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/drink-icon.ts */
/* ===== */
// Mapeo de bebidas a un icono visual + color de fondo.
// Devuelve null si el nombre no coincide con ninguna bebida conocida.

export type DrinkIcon = { emoji: string; bg: string };

const RULES: Array<{ test: RegExp; icon: DrinkIcon }> = [
  // Cervezas / jarras
  { test: /jarra|cruzcampo|heineken|paulaner|desperados|ladr[oó]n|quijote|sancho|[aá]guila|radler/i,
    icon: { emoji: '🍺', bg: 'bg-amber-500' } },

  // Vinos
  { test: /vino.*tinto|crianza|rioja|ribera/i, icon: { emoji: '🍷', bg: 'bg-red-900' } },
  { test: /vino.*blanco|verdejo/i, icon: { emoji: '🍷', bg: 'bg-yellow-200' } },
  { test: /vino.*rosado|rosado/i, icon: { emoji: '🍷', bg: 'bg-pink-400' } },
  { test: /spritz|petroni/i, icon: { emoji: '🍷', bg: 'bg-orange-400' } },

  // Destilados / combinados
  { test: /whisky|ballantines|jack/i, icon: { emoji: '🍷', bg: 'bg-amber-700' } },
  { test: /vodka|absolut/i, icon: { emoji: '🍷', bg: 'bg-slate-300' } },
  { test: /ginebra|beefeater/i, icon: { emoji: '🍷', bg: 'bg-emerald-400' } },
  { test: /ron|havana|bacardi/i, icon: { emoji: '🍷', bg: 'bg-yellow-700' } },
  { test: /ruavieja|crema/i, icon: { emoji: '🍷', bg: 'bg-amber-200' } },

  // Refrescos
  { test: /coca[\s-]?cola/i, icon: { emoji: '🍷', bg: 'bg-red-700' } },
  { test: /sprite/i, icon: { emoji: '🍷', bg: 'bg-green-500' } },
  { test: /fanta/i, icon: { emoji: '🍷', bg: 'bg-orange-500' } },
  { test: /aquarius/i, icon: { emoji: '🍷', bg: 'bg-cyan-400' } },
  { test: /fuze\s*tea|t[eé]/i, icon: { emoji: '🍷', bg: 'bg-yellow-600' } },
  { test: /appletiser/i, icon: { emoji: '🍷', bg: 'bg-green-600' } },
  { test: /monster/i, icon: { emoji: '🍷', bg: 'bg-lime-500' } },
  { test: /zumo/i, icon: { emoji: '🍷', bg: 'bg-orange-400' } },
  { test: /batido/i, icon: { emoji: '🍷', bg: 'bg-pink-300' } },

  // Calientes
  { test: /caf[eé]/i, icon: { emoji: '☕', bg: 'bg-amber-900' } },
  { test: /infusi[oó]n/i, icon: { emoji: '☕', bg: 'bg-emerald-700' } }
];

const FALLBACK: DrinkIcon = { emoji: '🍷', bg: 'bg-slate-400' };

export function getDrinkIcon(name: string): DrinkIcon {
  for (const r of RULES) if (r.test.test(name)) return r.icon;
  return FALLBACK;
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/drink-image.ts */
/* ===== */
import { PRODUCT_IMAGES } from './product-images';
import absolutSprite from '@assets/drinks/absolut-sprite.png';
import absolutVodka from '@assets/drinks/absolut-vodka.png';
import agua from '@assets/drinks/agua.png';
import appletiser from '@assets/drinks/appletiser.png';
import aquarius from '@assets/drinks/aquarius.png';
import bacardiCocacola from '@assets/drinks/bacardi-cocacola.png';
import ballantines from '@assets/drinks/ballantines.png';
import batido from '@assets/drinks/batido.png';
import beefeater from '@assets/drinks/beefeater.png';
import beefeater00 from '@assets/drinks/beefeater-00.png';
import botellinCruzcampo from '@assets/drinks/botellin-cruzcampo.png';
import cafe from '@assets/drinks/cafe.png';
import cocaCola from '@assets/drinks/coca-cola.png';
import cocaColaZero from '@assets/drinks/coca-cola-zero.png';
import cremaRuavieja from '@assets/drinks/crema-ruavieja.png';
import cruzcampoSinGluten from '@assets/drinks/cruzcampo-sin-gluten.png';
import desperados from '@assets/drinks/desperados.png';
import fantaNaranja from '@assets/drinks/fanta-naranja.png';
import fuzetea from '@assets/drinks/fuzetea.png';
import havanaClub from '@assets/drinks/havana-club.png';
import heineken00 from '@assets/drinks/heineken-00.png';
import infusion from '@assets/drinks/infusion.png';
import jackDanielsCocacola from '@assets/drinks/jack-daniels-cocacola.png';
import monsterBlanco from '@assets/drinks/monster-blanco.png';
import monsterMangoLoco from '@assets/drinks/monster-mango-loco.png';
import monsterVerde from '@assets/drinks/monster-verde.png';
import paulaner from '@assets/drinks/paulaner.png';
import petroni from '@assets/drinks/petroni.png';
import sprite from '@assets/drinks/sprite.png';
import vinoCopa from '@assets/drinks/vino-copa.png';
import vinoBlanco from '@assets/drinks/vino-blanco.png';
import zumoNaranja from '@assets/drinks/zumo-naranja.png';
import jarraCerveza from '@assets/drinks/jarra-cerveza.png';
import jarraTintoVerano from '@assets/drinks/jarra-tinto-verano.png';
import jarraLadronManzanas from '@assets/drinks/jarra-ladron-manzanas.png';

const normalize = (value: string) =>

```

```

value
  .normalize('NFD')
  .replace(/[\u0300-\u036f]/g, '')
  .toLowerCase()
  .trim();

const MATCHERS: Array<{ test: RegExp; src: string }> = [
  { test: /absolut.*sprite|sprite.*absolut/, src: absolutSprite },
  { test: /absolut.*vodka|vodka.*absolut/, src: absolutVodka },
  { test: /agua|mineral/, src: agua },
  { test: /appletiser/, src: appletiser },
  { test: /aquarius/, src: aquarius },
  { test: /bacardi.*(coca|cola|coke)|(coca|cola|coke).*bacardi/, src: bacardiCocacola },
  { test: /ballantine/, src: ballantines },
  { test: /batido/, src: batido },
  { test: /beefeater.*0[.,]?0[0[.,]?0.*beefeater|beefeater.*sin alcohol|sin alcohol.*beefeater/, src: beefeater00 },
  { test: /beefeater/, src: beefeater },
  { test: /cafe|cortado|solo|americano|capuccino|capuchino|descafeinado/, src: cafe },
  { test: /coca[\s-]?cola.*zero|zero.*coca[\s-]?cola|coca[\s-]?cola zero/, src: cocaColaZero },
  { test: /coca[\s-]?cola/, src: cocaCola },
  { test: /crema.*ruavieja|ruavieja.*crema|^ruavieja$/, src: cremaRuavieja },
  { test: /(jack.*daniel|bjack\b).*(coca|cola|coke)|(coca|cola|coke).*(jack.*daniel|bjack\b)/, src: jackDanielsCocacola },
  { test: /havana.*club/, src: havanaClub },
  { test: /monster.*mango.*loco|mango.*loco.*monster/, src: monsterMangoLoco },
  { test: /monster.*(verde|original|green|energy)|^monster$/, src: monsterVerde },
  { test: /paulaner/, src: paulaner },
  { test: /petroni|aperitivo.*naranja|sprit/, src: petroni },
  { test: /\bsprite\b/, src: sprite },
  { test: /vino.*blanco|blanco.*vino|verdejo|sauvignon|albarino|chardonnay/, src: vinoBlanco },
  { test: /tinto de verano|ladron de verano/, src: jarraTintoVerano },
  { test: /ladron.*manzana|manzana.*ladron|cider/, src: jarraLadronManzanas },
  { test: /vino.*tinto|vino.*rosado|ribera|rioja|rosado|copa de vino|^vino$/, src: vinoCopa },
  { test: /zumo|naranja exprimida/, src: zumoNaranja },
  { test: /monster.*blanco|monster.*ultra|ultra.*monster|monster.*zero/, src: monsterBlanco },
  { test: /fanta/, src: fantaNaranja },
  { test: /fuze\s*tea|fuzetea/, src: fuzetea },
  { test: /heineken.*0[.,]?0[0[.,]?0.*heineken|heineken.*sin alcohol|sin alcohol.*heineken/, src: heineken00 },
  { test: /desperados/, src: desperados },
  { test: /cruzcampo.*sin gluten|sin gluten.*cruzcampo/, src: botellinCruzcampo },
  { test: /botellin.*cruzcampo|cruzcampo.*botellin/, src: botellinCruzcampo },
  { test: /infusion|te caliente|manzanilla|poleo|tila/, src: infusion },
  // Catch-all cerveza (excluye ladrón de manzanas y tinto de verano)
  { test: /^(?!.*manzana)(?!.*tinto de verano).*(cerveza|cana|jarra|estrella|mahou|amstel|alhambra|san miguel|cruzcampo|radler|clara|rubia|tostada|negra|\bipa\b|lager|pilsner|weiss|weizen)/, src: jarraCerveza },
];

export function getDrinkImage(name: string, fotoUrl?: string | null) {
  // Check static product image map first (montaditos, aperitivos, raciones)
  if (PRODUCT_IMAGES[name]) return PRODUCT_IMAGES[name];
  const normalized = normalize(name);
  const matched = MATCHERS.find((entry) => entry.test.test(normalized));
  return matched?.src ?? fotoUrl ?? null;
}

```

```

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/product-images.ts */
/* ===== */

import aceitunaasImg    from '@assets/productos/aceitunas.png';
import cheesyImg        from '@assets/productos/cheesy-cheetos-bites.png';
import cucuruchoImg     from '@assets/productos/cucurucho-de-patatas-chip.png';
import gildaAnchoaImg   from '@assets/productos/gilda-anchoa.png';
import gildaBoqueronImg from '@assets/productos/gilda-boqueron.png';
import nachosBaconImg   from '@assets/productos/nachos-cheddar-bacon.png';
import nachosGuacaImg   from '@assets/productos/nachos-cheddar-guacamole.png';
import palomitasPolloImg from '@assets/productos/palomitas-de-pollo.png';
import palomitasGoudaImg from '@assets/productos/palomitas-de-queso-gouda.png';
import patatas4SalsasImg from '@assets/productos/patatas-fritas-4-salsas.png';
import patasBravioliImg from '@assets/productos/patatas-fritas-bravioli.png';
import salchichasImg    from '@assets/productos/salchichas-4-salsas.png';
import alitasBbqImg     from '@assets/productos/alitas-bbq.png';
import alitasBravaImg   from '@assets/productos/alitas-brava.png';
import croquetasJamonImg from '@assets/productos/croquetas-de-jamon.png';
import croquetasMacImg  from '@assets/productos/croquetas-mac-cheese.png';
import patasBravasImg   from '@assets/productos/patatas-bravas.png';
import tirasPolloImg    from '@assets/productos/tiras-de-pollo.png';
import torreznosImg     from '@assets/productos/torreznos.png';
import camperaImg       from '@assets/productos/campera.png';
import texmexImg        from '@assets/productos/texmex.png';

export const PRODUCT_IMAGES: Record<string, string> = {
  'Zumo de naranja':      '/assets/img/drinks/zumo-naranja.png',
  'Zumo de piña':         '/assets/img/drinks/zumo-pina.png',
  'Zumo de melocotón':    '/assets/img/drinks/zumo-melocoton.png',
  'Aceitunas de la abuela': aceitunaasImg,
  'Cheesy Cheetos® Bites': cheesyImg,
  'Cucurucho de patatas chips': cucuruchoImg,
  'Gildas':               gildaBoqueronImg,
  'Nachos':               nachosBaconImg,
  'Palomitas de pollo':   palomitasPolloImg,

```

```

'Palomitas de queso gouda': palomitasGoudaImg,
'Patatas fritas 4 salsas': patatas4SalsasImg,
'Patatas fritas con cheddar y bacon ahumado': '/assets/img/aperitivos/patatas-fritas-cheddar-bacon.png',
'Salchichas 4 salsas': salchichasImg,
'Alitas de pollo': alitasBbqImg,
'Croquetas de Jamón': croquetasJamonImg,
'Croquetas de Mac&Cheese': croquetasMacImg,
'Patatas fritas Bravioli': patasBravioliImg,
'Patatas bravas': patasBravasImg,
'Tiras de pollo': tirasPolloImg,
'Torreznos': torreznosImg,
'Campera': camperaImg,
'TexMex': texmexImg,
_gildaAnchoa: gildaAnchoaImg,
_gildaBoqueron: gildaBoqueronImg,
_alitasBbq: alitasBbqImg,
_alitasBrava: alitasBravaImg,
_nachosBacon: nachosBaconImg,
_nachosGuaca: nachosGuacaImg,

// — Montaditos —
'3 Quesos: queso madurado, queso mozzarella, queso gorgonzola y orégano': '/assets/img/montaditos/montypizza-3-quesos-con-queso-madurado-queso-mozzarella-queso-gorgonzola-y-oregano.png',
'Atún, huevo hilado y mayonesa': '/assets/img/montaditos/montygourmet-de-atun-huevo-hilado-y-mayonesa.png',
'Atún, tomate y mayonesa': '/assets/img/montaditos/montygourmet-de-atun-tomate-y-mayonesa.png',
'BBQ: bacon ahumado, queso mozzarella, cebolla crujiente y salsa BBQ': '/assets/img/montaditos/montypizza-bbq-con-bacon-ahumado-queso-mozzarella-cebolla-crujiente-y-salsa-bbq.png',
'Bacon ahumado y queso madurado': '/assets/img/montaditos/bacon-ahumado-y-queso-madurado.png',
'Bacon ahumado, cebolla crujiente y salsa 100M': '/assets/img/montaditos/bacon-ahumado-cebolla-crujiente-y-salsa-naranja.png',
'Bacon ahumado, patatas paja y mayonesa': '/assets/img/montaditos/bacon-ahumado-patatas-paja-y-mayonesa.png',
'Bacon ahumado, tomate y mayonesa': '/assets/img/montaditos/bacon-ahumado-tomate-y-mayonesa.png',
'Bacon ahumado, tomate y queso gorgonzola': '/assets/img/montaditos/bacon-ahumado-tomate-y-queso-gorgonzola.png',
'Bacon ahumado, tomate y queso madurado': '/assets/img/montaditos/bacon-ahumado-tomate-y-queso-madurado.png',
'Burger, bacon ahumado y salsa cheddar': '/assets/img/montaditos/montyburger-de-burger-bacon-ahumado-y-salsa-cheddar.png',
'Burger, guacamole y bacon ahumado': '/assets/img/montaditos/montyburger-de-burger-guacamole-y-bacon-ahumado.png',
'Burger, queso madurado y mojo picón': '/assets/img/montaditos/montyburger-de-burger-queso-madurado-y-mojo-picon.png',
'Burger, queso madurado y pepperoni': '/assets/img/montaditos/montyburger-de-burger-queso-madurado-y-pepperoni.png',
'Burger, queso madurado, tomate y mayonesa': '/assets/img/montaditos/montyburger-de-burger-queso-madurado-tomate-y-mayonesa.png',
'Calamarcitos y guacamole': '/assets/img/montaditos/calamarcitos-y-guacamole.png',
'Calamarcitos y mayonesa': '/assets/img/montaditos/calamarcitos-y-mayonesa.png',
'Calamarcitos y salsa 100M': '/assets/img/montaditos/calamarcitos-y-salsa-naranja.png',
'Calamarcitos y salsa alioli': '/assets/img/montaditos/calamarcitos-y-salsa-alioli.png',
'Calamarcitos, salsa brava y mayonesa': '/assets/img/montaditos/calamarcitos-salsa-brava-y-mayonesa.png',
'Calamarcitos, tomate y mayonesa': '/assets/img/montaditos/calamarcitos-tomate-y-mayonesa.png',
'Carrillera al vino tinto': '/assets/img/montaditos/carrillera-al-vino-tinto.png',
'Carrillera al vino tinto y bacon ahumado': '/assets/img/montaditos/carrillera-al-vino-tinto-y-bacon-ahumado.png',
'Carrillera al vino tinto y cebolla crujiente': '/assets/img/montaditos/carrillera-al-vino-tinto-y-cebolla-crujiente.png',
'Carrillera al vino tinto y patatas paja': '/assets/img/montaditos/carrillera-al-vino-tinto-y-patatas-paja.png',
'Carrillera al vino tinto y salsa alioli': '/assets/img/montaditos/carrillera-al-vino-tinto-y-salsa-alioli.png',
'Carrillera al vino tinto y tomate': '/assets/img/montaditos/carrillera-al-vino-tinto-y-tomate.png',
'Chorizo parrillero y guacamole': '/assets/img/montaditos/chorizo-parrillero-y-guacamole.png',
'Chorizo parrillero y queso gorgonzola': '/assets/img/montaditos/chorizo-parrillero-y-queso-gorgonzola.png',
'Chorizo parrillero y salsa BBQ': '/assets/img/montaditos/chorizo-parrillero-y-salsa-bbq.png',
'Chorizo parrillero y salsa brava': '/assets/img/montaditos/chorizo-parrillero-y-salsa-brava.png',
'Hot dog, cebolla crujiente y mojo picón': '/assets/img/montaditos/montyperro-de-hot-dog-cebolla-crujiente-y-mojo-picon.png',
'Hot dog, cebolla crujiente y salsa 100M': '/assets/img/montaditos/montyperro-de-hot-dog-cebolla-crujiente-y-salsa-100.png',
'Hot dog, guacamole y salsa cheddar': '/assets/img/montaditos/montyperro-de-hot-dog-guacamole-y-salsa-cheddar.png',
'Hot dog, ketchup y mayonesa': '/assets/img/montaditos/montyperro-de-hot-dog-ketchup-y-mayonesa.png',
'Hot dog, patatas paja y salsa alioli': '/assets/img/montaditos/montyperro-de-hot-dog-patatas-paja-y-salsa-alioli.png',
'Jamón Gran Reserva y aceite de oliva': '/assets/img/montaditos/jamon-gran-reserva-y-aceite-de-oliva.png',
'Jamón Gran Reserva y mantequilla': '/assets/img/montaditos/jamon-gran-reserva-y-mantequilla.png',
'Jamón Gran Reserva y tomate': '/assets/img/montaditos/jamon-gran-reserva-y-tomate.png',
'Jamón Gran Reserva y tortilla de patatas': '/assets/img/montaditos/jamon-gran-reserva-y-tortilla-de-patatas.png',
'Jamón Gran Reserva, tomate y patatas paja': '/assets/img/montaditos/jamon-gran-reserva-tomate-y-patatas-paja.png',
'Jamón cocido, queso madurado y mantequilla': '/assets/img/montaditos/montygourmet-de-jamon-cocido-queso-madurado-y-mantequilla.png',
'Jamón cocido, queso madurado, tomate y mayonesa': '/assets/img/montaditos/montygourmet-de-jamon-cocido-queso-madurado-tomate-y-mayonesa.png',
'Lomo al ajillo y mojo picón': '/assets/img/montaditos/lomo-al-ajillo-y-mojo-picon.png',
'Lomo al ajillo y queso gorgonzola': '/assets/img/montaditos/lomo-al-ajillo-y-queso-gorgonzola.png',
'Lomo al ajillo y queso madurado': '/assets/img/montaditos/lomo-al-ajillo-y-queso-madurado.png',
'Lomo al ajillo y salsa 100M': '/assets/img/montaditos/lomo-al-ajillo-y-salsa-100-m.png',
'Lomo al ajillo, bacon ahumado y salsa alioli': '/assets/img/montaditos/lomo-al-ajillo-bacon-ahumado-y-salsa-alioli.png',
'Lomo al ajillo, tomate y mayonesa': '/assets/img/montaditos/lomo-al-ajillo-tomate-y-mayonesa.png',
'Lomo al ajillo, tomate y patatas paja': '/assets/img/montaditos/lomo-al-ajillo-tomate-y-patatas-paja.png',
'Montycookie chocolate y sirope de chocolate': '/assets/img/montaditos/montycookie-chocolate-y-sirope-de-chocolate.png',
'Montycookie chocolate y sirope de pistacho': '/assets/img/montaditos/montycookie-chocolate-y-sirope-de-pistacho.png',
'Montycookie doble chocolate y sirope de caramelo toffee': '/assets/img/montaditos/montycookie-doble-chocolate-y-sirope-de-caramelo-toffee.png',
'Pepperoni: pepperoni, queso mozzarella, salsa pizza y orégano': '/assets/img/montaditos/montypizza-pepperoni-con-pepperoni-queso-mozzarella-salsa-pizza-y-oregano.png',
'Piadina de jamón Gran Reserva, queso mozzarella y orégano': '/assets/img/montaditos/piadina-de-jamon-gran-reserva-queso-mozzarella-y-oregano.png',
'Piadina de jamón cocido y queso mozzarella': '/assets/img/montaditos/piadina-de-jamon-cocido-y-queso-mozzarella.png',
'Piadina de jamón cocido, queso madurado y tomate': '/assets/img/montaditos/piadina-de-jamon-cocido-queso-madurado-y-tomate.png',
'Piadina de pepperoni y queso mozzarella': '/assets/img/montaditos/piadina-de-pepperoni-y-queso-mozzarella.png',
'Piadina de pollo, tomate, queso mozzarella y orégano': '/assets/img/montaditos/piadina-de-pollo-tomate-queso-mozzarella-y-oregano.png',
'Pintxo donostiarrá y atún': '/assets/img/montaditos/montygourmet-de-pintxo-donostiarrá-y-atun.png',
'Pintxo donostiarrá y huevo hilado': '/assets/img/montaditos/montygourmet-de-pintxo-donostiarrá-y-huevo-hilado.png',
'Pollo y queso madurado': '/assets/img/montaditos/pollo-y-queso-madurado.png',
'Pollo y salsa alioli': '/assets/img/montaditos/pollo-y-salsa-alioli.png',

```

```

'Pollo, bacon ahumado y mayonesa': '/assets/img/montaditos/pollo-bacon-ahumado-y-mayonesa.png',
'Pollo, patatas paja y salsa BBQ': '/assets/img/montaditos/pollo-patatas-paja-y-salsa-bbq.png',
'Pollo, patatas paja y salsa de mostaza y miel': '/assets/img/montaditos/pollo-patatas-paja-y-salsa-de-mostaza-y-miel.png',
'Pollo, tomate y mojo picón': '/assets/img/montaditos/pollo-tomate-y-mojo-picon.png',
'Pollo: pollo kebab, queso mozzarella, salsa pizza y orégano': '/assets/img/montaditos/montypizza-de-pollo-kebab-queso-mozzarella-salsa-pizza-y-oregano.png',
'Pollo kebab y salsa BBQ': '/assets/img/montaditos/pollo-kebab-y-salsa-bbq.png',
'Pollo kebab y salsa cheddar': '/assets/img/montaditos/pollo-kebab-y-salsa-cheddar.png',
'Pollo kebab y tomate': '/assets/img/montaditos/pollo-kebab-y-tomate.png',
'Pollo kebab, bacon ahumado y mayonesa': '/assets/img/montaditos/pollo-kebab-bacon-ahumado-y-mayonesa.png',
'Pollo kebab, patatas paja y salsa BBQ': '/assets/img/montaditos/pollo-kebab-patatas-paja-y-salsa-bbq.png',
'Pollo kebab, tomate y salsa 100M': '/assets/img/montaditos/pollo-kebab-tomate-y-salsa-naranja.png',
'Pulled Pork: pulled pork BBQ, queso mozzarella, cebolla crujiente y salsa BBQ': '/assets/img/montaditos/montypizza-pulled-pork-con-pulled-pork-bbq-queso-mozzarella-cebolla-crujiente-y-salsa-bbq.png',
'Pulled pork BBQ': '/assets/img/montaditos/pulled-pork-bbq.png',
'Pulled pork BBQ y bacon ahumado': '/assets/img/montaditos/pulled-pork-bbq-y-bacon-ahumado.png',
'Pulled pork BBQ y cebolla crujiente': '/assets/img/montaditos/pulled-pork-bbq-y-cebolla-crujiente.png',
'Pulled pork BBQ y patatas paja': '/assets/img/montaditos/pulled-pork-bbq-y-patatas-paja.png',
'Pulled pork BBQ y salsa brava': '/assets/img/montaditos/pulled-pork-bbq-y-salsa-brava.png',
'Pulled pork BBQ y salsa cheddar': '/assets/img/montaditos/pulled-pork-bbq-y-salsa-cheddar.png',
'Salmón ahumado y guacamole': '/assets/img/montaditos/salmon-ahumado-y-guacamole.png',
'Salmón ahumado y huevo hilado': '/assets/img/montaditos/montygourmet-de-salmon-ahumado-y-huevo-hilado.png',
'Salmón ahumado y pintxo donostiarrá': '/assets/img/montaditos/montygourmet-de-salmon-ahumado-y-pintxo-donostiarrá.png',
'Salmón ahumado y queso gorgonzola': '/assets/img/montaditos/salmon-ahumado-y-queso-gorgonzola.png',
'Salmón ahumado y salsa de mostaza y miel': '/assets/img/montaditos/salmon-ahumado-y-salsa-de-mostaza-y-miel.png',
'Salmón ahumado y tomate': '/assets/img/montaditos/salmon-ahumado-y-tomate.png',
'Tortilla de patatas y mojo picón': '/assets/img/montaditos/tortilla-de-patatas-y-mojo-picon.png',
'Tortilla de patatas y queso madurado': '/assets/img/montaditos/tortilla-de-patatas-y-queso-madurado.png',
'Tortilla de patatas y tomate': '/assets/img/montaditos/tortilla-de-patatas-y-tomate.png',
'Tortilla de patatas, bacon ahumado y salsa alioli': '/assets/img/montaditos/tortilla-de-patatas-bacon-ahumado-y-salsa-alioli.png',
'Tortilla de patatas, cebolla crujiente y salsa BBQ': '/assets/img/montaditos/tortilla-de-patatas-cebolla-crujiente-y-salsa-bbq.png',
'Tortilla de patatas, patatas paja y salsa 100M': '/assets/img/montaditos/tortilla-de-patatas-patatas-paja-y-salsa-naranja.png',
'Tortilla de patatas, tomate y mayonesa': '/assets/img/montaditos/tortilla-de-patatas-tomate-y-mayonesa.png',
'Torreznos y mayonesa': '/assets/img/montaditos/torreznos-y-mayonesa.png',
'Torreznos y salsa 100M': '/assets/img/montaditos/torreznos-y-salsa-naranja.png',
'Torreznos y salsa alioli': '/assets/img/montaditos/torreznos-y-salsa-alioli.png',
'Torreznos y salsa brava': '/assets/img/montaditos/torreznos-y-salsa-brava.png',
};

// Productos que comparten nombre con otros pero tienen pan diferente (MontyGourmet).
// Este mapa sobreescribe PRODUCT_IMAGES cuando el número coincide.
export const MONTADITO_IMAGES_BY_NUMERO: Record<string, string> = {
  '91': '/assets/img/montaditos/montygourmet-de-tortilla-de-patatas-tomate-y-mayonesa.png',
  '100': '/assets/img/montaditos/montygourmet-de-jamon-gran-reserva-y-mantequilla.png',
};

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/ruedas.ts */
/* ===== */
// Configuración de las MontyRuedas.
// Cada rueda = los montaditos de un tipo (por su número de carta). El criterio
// viene del pan mapeado en la BD de ingredientes (piadina/perrito/burger/pizza/
// gourmet) + las cookies como producto completo. Si cambia la carta, ajustar aquí.

export interface RuedaDef {
  numero: string; // número del producto rueda en la carta (RU1..RU6)
  tipo: string; // etiqueta del tipo
  montaditoNumbers: string[]; // números de los montaditos que la forman (o el pool a elegir)
  selectCount?: number; // si está definido, el cliente elige N de montaditoNumbers
  img?: string; // imagen (sin fondo) de la rueda; si falta, se usa el emoji 🍷
}

export const RUEDAS: Record<string, RuedaDef> = {
  RU1: { numero: 'RU1', tipo: 'MontyDinas', montaditoNumbers: ['71', '72', '73', '74', '75'], img: '/assets/img/ruedas/dinas.png' },
  RU2: { numero: 'RU2', tipo: 'MontyPerros', montaditoNumbers: ['76', '77', '78', '79', '80'], img: '/assets/img/ruedas/perros.png' },
  RU3: { numero: 'RU3', tipo: 'MontyBurguers', montaditoNumbers: ['81', '82', '83', '84', '85'], img: '/assets/img/ruedas/burguers.png' },
  RU4: { numero: 'RU4', tipo: 'MontyPizzas', montaditoNumbers: ['86', '87', '88', '89', '90'], img: '/assets/img/ruedas/pizzas.png' },
  RU5: { numero: 'RU5', tipo: 'MontyGourmets', montaditoNumbers: ['91', '92', '93', '94', '95', '96', '97', '98', '99', '100'], selectCount: 5, img: '/assets/img/ruedas/gourmets.png' },
};

export const getRueda = (numero?: string | null): RuedaDef | null =>
  numero && RUEDAS[numero] ? RUEDAS[numero] : null;

export const isRueda = (numero?: string | null): boolean => !!numero && numero in RUEDAS;

// — Helpers compartidos (carta y chat de Monty) —
export interface RuedaMontadito {
  id: string;
  numero: string | null;
  nombre: string;
}

/** Montaditos que componen una rueda (resueltos desde la lista del menú, en orden). */
export const ruedaComponentes = (def: RuedaDef, montaditos: RuedaMontadito[]): RuedaMontadito[] => {
  const byNum = new Map<string, RuedaMontadito>();

```

```

    for (const m of montaditos) if (m.numero) byNum.set(m.numero, m);
    return def.montaditoNumbers.map((n) => byNum.get(n)).filter(Boolean) as RuedaMontadito[];
};

/** Etiqueta de un montadito para cocina/cesta (ej. "#86 Hot dog..."). */
export const complabel = (m: RuedaMontadito) => `#${m.numero} ${m.nombre}`;

/**
 * ¿Está agotada la rueda?
 * - Fija: si falta cualquiera de sus montaditos.
 * - Gourmet (selectCount): si quedan menos disponibles que los que hay que elegir.
 */
export const ruedaAgotada = (
  def: RuedaDef,
  montaditos: RuedaMontadito[],
  isProductoAgotado: (id: string) => boolean,
): boolean => {
  const comps = ruedaComponentes(def, montaditos);
  if (def.selectCount) return comps.filter((m) => !isProductoAgotado(m.id)).length < def.selectCount;
  return comps.some((m) => isProductoAgotado(m.id));
};

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/section-images.ts */
/* ===== */
import montaditosImg from '@assets/secciones/montaditos.png';
import montyahorroImg from '@assets/secciones/montyahorro.png';
import bebidasImg from '@assets/secciones/bebidas.png';
import aperitivosImg from '@assets/secciones/aperitivos.png';
import racionesImg from '@assets/secciones/raciones.png';
import ensaladasImg from '@assets/secciones/ensaladas.png';

export const SECTION_IMAGES: Record<string, string> = {
  Montaditos: montaditosImg,
  MontyRuedas: montyahorroImg,
  Bebidas: bebidasImg,
  Aperitivos: aperitivosImg,
  Raciones: racionesImg,
  Ensaladas: ensaladasImg,
};

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/stripe.ts */
/* ===== */
import { loadStripe, Stripe } from '@stripe/stripe-js';

const PUBLISHABLE_KEY = import.meta.env.VITE_PAYMENTS_CLIENT_TOKEN as string | undefined;

let stripePromise: Promise<Stripe | null> | null = null;

export const getStripe = () => {
  if (!PUBLISHABLE_KEY) {
    console.error('VITE_PAYMENTS_CLIENT_TOKEN no está configurado');
    return Promise.resolve(null);
  }
  if (!stripePromise) stripePromise = loadStripe(PUBLISHABLE_KEY);
  return stripePromise;
};

/* ===== */
/* FICHERO: montadito-magic-flow/src/lib/Utils.ts */
/* ===== */
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/main.tsx */
/* ===== */
import { createRoot } from "react-dom/client";
import App from "./App.tsx";
import "./index.css";
import { supabase } from "@integrations/supabase/client";
import { ErrorBoundary } from "./components/ErrorBoundary";

// Inyecta el session_id del cliente como header en todas las llamadas a PostgREST
// para que las RLS puedan validar que el cliente solo accede a SUS pedidos.
try {
  let sid = sessionStorage.getItem("montaditos_session");
  if (!sid) {
    sid = crypto.randomUUID();
    sessionStorage.setItem("montaditos_session", sid);
  }
  // @ts-expect-error: rest.headers es interno pero estable en supabase-js v2
  supabase.rest.headers["x-session-id"] = sid;

```

```

} catch (e) {
  console.warn("No se pudo inyectar x-session-id", e);
}

createRoot(document.getElementById("root")!).render(
  <ErrorBoundary>
    <App />
  </ErrorBoundary>
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/functions/chat-menu/index.ts */
/* ===== */
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.45.0";

const corsHeaders = {
  "Access-Control-Allow-Origin": "https://pedidos100montaditos.es",
  "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type",
};

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") {
    return new Response(null, { headers: corsHeaders });
  }

  try {
    const { messages, session_id } = await req.json();
    const OPENAI_API_KEY = Deno.env.get("OPENAI_API_KEY");
    if (!OPENAI_API_KEY) throw new Error("OPENAI_API_KEY no configurada");

    // Cargar menú con categorías
    const supabase = createClient(
      Deno.env.get("SUPABASE_URL")!,
      Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
    );

    // SEC-004: rate limit por sesión (20/5min) e IP (60/5min). Fail-open si la
    // función aún no existe (r.data == null); bloquea solo cuando devuelve false.
    const ip = (req.headers.get("x-forwarded-for") ?? "unknown").split(",")[0].trim();
    const rl = await Promise.all([
      supabase.rpc("check_rate_limit", { p_key: `chat:sess:${session_id} ?? ip`, p_limit: 20, p_window_secs: 300 }),
      supabase.rpc("check_rate_limit", { p_key: `chat:ip:${ip}`, p_limit: 60, p_window_secs: 300 }),
    ]);
    if (rl.some((r) => r.data === false)) {
      return new Response(
        JSON.stringify({ error: "Has enviado demasiados mensajes. Espera un momento." }),
        { status: 429, headers: { ...corsHeaders, "Content-Type": "application/json" } }
      );
    }

    const [{ data: cats }, { data: prods }] = await Promise.all([
      supabase.from("menu_categorias").select("id, nombre").order("orden"),
      supabase
        .from("menu_productos")
        .select("nombre, descripcion, precio, categoria_id, numero")
        .eq("disponible", true),
    ]);

    const catMap = new Map((cats ?? []).map((c: any) => [c.id, c.nombre]));
    const menuText = (prods ?? []).map(
      (p: any) =>
        `- [${catMap.get(p.categoria_id) ?? "Otro"}] ${p.numero ? `#${p.numero}` : ""} ${p.nombre} - ${p.precio} €\n`
    ).join("\n");

    const systemPrompt = `Eres el asistente virtual de "100 Montaditos". Ayudas a los clientes a elegir productos del menú según sus gustos, alergias o necesidades.`;

    REGLAS:
    - Responde SIEMPRE en español, de forma breve, amable y directa.
    - Recomienda SOLO productos del menú listado abajo. Nunca inventes productos.
    - Cuando recomiendes, lista 2-5 opciones con su nombre exacto y precio.
    - Los montaditos tienen un número identificador del #1 al #100. Cuando un cliente pregunte por un número (ej: "el 3", "el número 15", "ponme el 42"), identifica el montadito por su número en el menú y recomiéndalo indicando su nombre completo.
    - Si el cliente menciona una alergia o ingrediente que no quiere, analiza descripciones y nombres para excluir productos que probablemente lo contengan. Si no estás seguro, avísale de que confirme con el personal.
    - El menú incluye MONTADITOS, APERITIVOS, RACIONES, ENSALADAS, MONTYRUEDAS y BEBIDAS. ANTES de decir que algo NO está, busca a fondo en TODA la lista de abajo (incluida la sección [Aperitivos]). Por ejemplo, las "Gildas" y las "Aceitunas de la abuela" SÍ están (son aperitivos). Solo di que no existe si de verdad no aparece en la lista.
    - MONTYRUEDAS: son bandejas de 5 montaditos del mismo tipo (Dinas, Perros, Burguers, Pizzas) por 8€ (2€ menos que comprándolos sueltos). La MontyRueda Gourmets cuesta 8€ y deja al cliente ELEGIR 5 montaditos gourmet de los 10 disponibles (al añadirla se abre un selector en la app). Si te piden una rueda o cuál sale más a cuenta, recomiéndala con su etiqueta [[add:NOMBRE EXACTO]] (ej. [[add:MontyRueda Pizzas]]).
    - Si pregunta por algo que no está en la carta, dilo claramente.
    - Mantén el texto BREVE (1-3 frases). NO escribas listas de productos en el texto, los productos se mostrarán como botones aparte.
    - IMPORTANTE: Para cada producto que recomiendes, añade al FINAL del mensaje (después del texto) una línea por producto con esta etiqueta exacta: [[add:NOMBRE_EXACTO_DEL_PRODUCTO]] usando el nombre TAL CUAL aparece en el menú (sin negritas, sin precio, SIN el número #). Ejemplo de respuesta completa:

```

"¡El #3 es el Pulled pork BBQ! 🍖"
[[add:Pulled pork BBQ]]"
- NO incluyas el número ni el precio en la etiqueta. Solo el nombre exacto del producto.

MENÚ DISPONIBLE:

```
`${menuText}`;

const response = await fetch(
  "https://api.openai.com/v1/chat/completions",
  {
    method: "POST",
    headers: {
      Authorization: `Bearer ${OPENAI_API_KEY}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      model: "gpt-4o-mini",
      messages: [{ role: "system", content: systemPrompt }, ...messages],
      temperature: 0.3,
      stream: true,
    }),
  }
);

if (!response.ok) {
  if (response.status === 429) {
    return new Response(
      JSON.stringify({ error: "Demasiadas peticiones, espera un momento." }),
      {
        status: 429,
        headers: { ...corsHeaders, "Content-Type": "application/json" },
      }
    );
  }
  if (response.status === 402) {
    return new Response(
      JSON.stringify({
        error: "Créditos AI agotados. Contacta al administrador.",
      }),
      {
        status: 402,
        headers: { ...corsHeaders, "Content-Type": "application/json" },
      }
    );
  }
  const t = await response.text();
  console.error("AI gateway error:", response.status, t);
  return new Response(JSON.stringify({ error: "AI gateway error" }), {
    status: 500,
    headers: { ...corsHeaders, "Content-Type": "application/json" },
  });
}

return new Response(response.body, {
  headers: { ...corsHeaders, "Content-Type": "text/event-stream" },
});
} catch (e) {
  console.error("chat-menu error:", e);
  return new Response(
    JSON.stringify({
      error: e instanceof Error ? e.message : "Unknown error",
    }),
    {
      status: 500,
      headers: { ...corsHeaders, "Content-Type": "application/json" },
    }
  );
}
});

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/functions/confirm-payment/index.ts */
/* ===== */
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.45.0";
import { createStripeClient } from "../_shared/stripe.ts";

const corsHeaders = {
  "Access-Control-Allow-Origin": "https://pedidos100montaditos.es",
  "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type, x-session-id",
};

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return new Response("ok", { headers: corsHeaders });

  try {
    const { pedido_id, session_id, payment_intent_id } = await req.json();
    if (!pedido_id || !session_id || !payment_intent_id) {
      return new Response(JSON.stringify({ error: "missing params" }), {
        status: 400, headers: { ...corsHeaders, "Content-Type": "application/json" },
      });
    }
  }
}
```

```

const supabase = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
);

const { data: pedido, error: pErr } = await supabase
  .from("pedidos")
  .select("id, session_id, estado, stripe_payment_id")
  .eq("id", pedido_id)
  .maybeSingle();
if (pErr || !pedido) throw new Error("pedido no encontrado");
if (pedido.session_id !== session_id) throw new Error("session_id no coincide");

// Idempotente: si ya está confirmado, devolvemos ok
if (pedido.estado !== "pendiente_pago") {
  return new Response(JSON.stringify({ ok: true, already: true }), {
    headers: { ...corsHeaders, "Content-Type": "application/json" },
  });
}

const env = Deno.env.get("STRIPE_LIVE_API_KEY") ? "live" : "sandbox";
const stripe = createStripeClient(env);

const intent = await stripe.paymentIntents.retrieve(payment_intent_id);
const paid = intent.status === "succeeded" || intent.status === "processing";
if (!paid) {
  return new Response(JSON.stringify({ ok: false, status: intent.status }), {
    status: 400, headers: { ...corsHeaders, "Content-Type": "application/json" },
  });
}
if (intent.metadata?.pedido_id && intent.metadata.pedido_id !== pedido_id) {
  throw new Error("pedido_id no coincide con el PaymentIntent");
}

const hasCocina = !(await supabase.from("pedido_items").select("id").eq("pedido_id", pedido_id).eq("destino",
"cocina").limit(1)).data?.length;
const hasBebidas = !(await supabase.from("pedido_items").select("id").eq("pedido_id", pedido_id).eq("destino",
"bebidas").limit(1)).data?.length;

const { error: upErr } = await supabase
  .from("pedidos")
  .update({
    estado: "pendiente",
    estado_cocina: hasCocina ? "pendiente" : null,
    estado_bebidas: hasBebidas ? "pendiente" : null,
    stripe_payment_id: intent.id,
  })
  .eq("id", pedido_id)
  .eq("estado", "pendiente_pago");
if (upErr) throw upErr;

return new Response(JSON.stringify({ ok: true }), {
  headers: { ...corsHeaders, "Content-Type": "application/json" },
});
} catch (e) {
  console.error(e);
  const knownErrors = ["pedido no encontrado", "session_id no coincide", "pedido no está pendiente de pago"];
  const msg = knownErrors.includes((e as Error).message) ? (e as Error).message : "Error interno del servidor";
  return new Response(JSON.stringify({ error: msg }), {
    status: 500, headers: { ...corsHeaders, "Content-Type": "application/json" },
  });
}
});

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/functions/create-checkout/index.ts */
/* ===== */
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.45.0";
import { createStripeClient } from "../_shared/stripe.ts";

const corsHeaders = {
  "Access-Control-Allow-Origin": "https://pedidos100montaditos.es",
  "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type, x-session-id",
};

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return new Response("ok", { headers: corsHeaders });

  try {
    const { pedido_id, session_id, return_url } = await req.json();
    if (!pedido_id || !session_id) {
      return new Response(JSON.stringify({ error: "missing params" }), {
        status: 400, headers: { ...corsHeaders, "Content-Type": "application/json" },
      });
    }

    const supabase = createClient(
      Deno.env.get("SUPABASE_URL")!,
      Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
    );

```



```

const { data: pedido, error: pErr } = await supabase
  .from("pedidos")
  .select("id, total, session_id, numero_pedido, estado")
  .eq("id", pedido_id)
  .maybeSingle();
if (pErr || !pedido) throw new Error("pedido no encontrado");
if (pedido.session_id !== session_id) throw new Error("session_id no coincide");

const { data: items, error: iErr } = await supabase
  .from("pedido_items")
  .select("cantidad, precio_unitario, menu_productos(nombre, foto_url)")
  .eq("pedido_id", pedido_id);
if (iErr || !items?.length) throw new Error("sin items");

const env = Deno.env.get("STRIPE_LIVE_API_KEY") ? "live" : "sandbox";
const stripe = createStripeClient(env);

const base = (return_url || req.headers.get("origin") || "").replace(/\/$/, "");

const session = await stripe.checkout.sessions.create({
  mode: "payment",
  success_url: `${base}?pago=ok&pedido=${pedido_id}`,
  cancel_url: `${base}?pago=cancel&pedido=${pedido_id}`,
  client_reference_id: pedido_id,
  metadata: { pedido_id, session_id },
  payment_intent_data: { description: `Pedido #${pedido.numero_pedido}` },
  line_items: items.map((it: any) => ({
    quantity: it.cantidad,
    price_data: {
      currency: "eur",
      unit_amount: Math.round(Number(it.precio_unitario) * 100),
      product_data: {
        name: it.menu_productos?.nombre ?? "Producto",
        ...(it.menu_productos?.foto_url ? { images: [it.menu_productos.foto_url] } : {}),
      },
    },
  })),
});

return new Response(JSON.stringify({ url: session.url, id: session.id }), {
  headers: { ...corsHeaders, "Content-Type": "application/json" },
});
} catch (e) {
  console.error(e);
  const knownErrors = ["pedido no encontrado", "session_id no coincide", "pedido no está pendiente de pago"];
  const msg = knownErrors.includes((e as Error).message) ? (e as Error).message : "Error interno del servidor";
  return new Response(JSON.stringify({ error: msg }), {
    status: 500, headers: { ...corsHeaders, "Content-Type": "application/json" },
  });
}
});

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/functions/create-payment-intent/index.ts */
/* ===== */
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.45.0";
import { createStripeClient } from "../_shared/stripe.ts";

const corsHeaders = {
  "Access-Control-Allow-Origin": "https://pedidos100montaditos.es",
  "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type, x-session-id",
};

const MAX_PI_ATTEMPTS = 5;

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return new Response("ok", { headers: corsHeaders });

  try {
    const { pedido_id, session_id } = await req.json();
    if (!pedido_id || !session_id) {
      return new Response(JSON.stringify({ error: "missing params" }), {
        status: 400, headers: { ...corsHeaders, "Content-Type": "application/json" },
      });
    }

    const supabase = createClient(
      Deno.env.get("SUPABASE_URL")!,
      Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
    );

    const { data: pedido, error: pErr } = await supabase
      .from("pedidos")
      .select("id, total, session_id, numero_pedido, estado, stripe_payment_id, pi_attempts")
      .eq("id", pedido_id)
      .maybeSingle();
    if (pErr || !pedido) throw new Error("pedido no encontrado");
    if (pedido.session_id !== session_id) throw new Error("session_id no coincide");
    if (pedido.estado !== "pendiente_pago") throw new Error("pedido no está pendiente de pago");
  }
});

```

```

// Recalcular total desde los precios reales de DB, nunca confiar en el total del cliente
const { data: items, error: iErr } = await supabase
  .from("pedido_items")
  .select("cantidad, precio_unitario")
  .eq("pedido_id", pedido_id);
if (iErr || !items?.length) throw new Error("items del pedido no encontrados");
const totalRecalculado = items.reduce((sum, i) => sum + i.cantidad * Number(i.precio_unitario), 0);
if (totalRecalculado <= 0) throw new Error("importe inválido");

await supabase.from("pedidos").update({ total: totalRecalculado }).eq("id", pedido_id);

const env = Deno.env.get("STRIPE_LIVE_API_KEY") ? "live" : "sandbox";
const stripe = createStripeClient(env);
const amount = Math.round(totalRecalculado * 100);

let intent;
if (pedido.stripe_payment_id?.startsWith("pi_")) {
  try {
    intent = await stripe.paymentIntents.retrieve(pedido.stripe_payment_id);
    if (intent.status === "succeeded") {
      // Pago ya realizado - confirmar el pedido inline (el webhook puede haberse retrasado)
      const hasCocina = !(await supabase.from("pedido_items").select("id").eq("pedido_id", pedido_id).eq("destino",
        "cocina").limit(1)).data?.length;
      const hasBebidas = !(await supabase.from("pedido_items").select("id").eq("pedido_id", pedido_id).eq("destino",
        "bebidas").limit(1)).data?.length;
      await supabase.from("pedidos").update({
        estado: "pendiente",
        estado_cocina: hasCocina ? "pendiente" : null,
        estado_bebidas: hasBebidas ? "pendiente" : null,
      }).eq("id", pedido_id).eq("estado", "pendiente_pago");
      return new Response(JSON.stringify({ already_paid: true }), {
        headers: { ...corsHeaders, "Content-Type": "application/json" },
      });
    }
    if (intent.status === "canceled") {
      intent = null;
    } else if (intent.amount !== amount) {
      intent = await stripe.paymentIntents.update(intent.id, { amount });
    }
  } catch { intent = null; }
}

if (!intent) {
  if ((pedido.pi_attempts ?? 0) >= MAX_PI_ATTEMPTS) {
    throw new Error("demasiados intentos de pago para este pedido");
  }
  intent = await stripe.paymentIntents.create({
    amount,
    currency: "eur",
    automatic_payment_methods: { enabled: true },
    description: `Pedido #${pedido.numero_pedido}`,
    metadata: { pedido_id, session_id },
  });
  await supabase.from("pedidos").update({
    stripe_payment_id: intent.id,
    pi_attempts: (pedido.pi_attempts ?? 0) + 1,
  }).eq("id", pedido_id);
}

return new Response(JSON.stringify({
  client_secret: intent.client_secret,
  payment_intent_id: intent.id,
}), {
  headers: { ...corsHeaders, "Content-Type": "application/json" },
});
} catch (e) {
  console.error(e);
  const knownErrors = [
    "pedido no encontrado", "session_id no coincide", "pedido no está pendiente de pago",
    "items del pedido no encontrados", "importe inválido", "demasiados intentos de pago para este pedido",
  ];
  const msg = knownErrors.includes((e as Error).message) ? (e as Error).message : "Error interno del servidor";
  return new Response(JSON.stringify({ error: msg }), {
    status: 500, headers: { ...corsHeaders, "Content-Type": "application/json" },
  });
}
});

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/functions/notify-order-ready/index.ts */
/* ===== */
import webpush from "npm:web-push@3.6.7";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.45.0";

const VAPID_PUBLIC_KEY = Deno.env.get("VAPID_PUBLIC_KEY")!;
const VAPID_PRIVATE_KEY = Deno.env.get("VAPID_PRIVATE_KEY")!;
const PUSH_NOTIFY_SECRET = Deno.env.get("PUSH_NOTIFY_SECRET") ?? "";

webpush.setVapidDetails("mailto:info@hat3x.com", VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY);

Deno.serve(async (req) => {

```

```

if (req.method === "OPTIONS") {
  return new Response("ok", { headers: { "Access-Control-Allow-Origin": "*" } });
}

const secret = req.headers.get("x-push-notify-secret");
if (!secret || secret !== PUSH_NOTIFY_SECRET) {
  return new Response("unauthorized", { status: 401 });
}

const { pedido_id, session_id, numero_pedido } = await req.json();
if (!pedido_id || !session_id) {
  return new Response("missing params", { status: 400 });
}

const supabase = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!,
);

const { data: subs } = await supabase
  .from("push_subscriptions")
  .select("subscription")
  .eq("session_id", session_id)
  .eq("pedido_id", pedido_id);

if (!subs?.length) {
  return new Response(JSON.stringify({ sent: 0 }), {
    headers: { "Content-Type": "application/json" },
  });
}

const numStr = String(numero_pedido ?? "").padStart(4, "0");
const payload = JSON.stringify({
  title: "¡Tu pedido está listo! 🍷",
  body: `Pedido #${numStr} - Puedes pasar a recogerlo ya.`,
  numero_pedido,
});

const results = await Promise.allSettled(
  subs.map((row) => webpush.sendNotification(row.subscription, payload)),
);

// Eliminar suscripciones expiradas que el push service rechaza (410 = gone)
const expiredEndpoints: string[] = [];
results.forEach((r, i) => {
  if (r.status === "rejected") {
    const status = (r.reason as any)?.statusCode;
    if (status === 410 || status === 404) {
      expiredEndpoints.push((subs[i].subscription as any).endpoint);
    }
  }
});
if (expiredEndpoints.length) {
  for (const endpoint of expiredEndpoints) {
    await supabase
      .from("push_subscriptions")
      .delete()
      .eq("subscription->endpoint" as any, endpoint);
  }
}

const sent = results.filter((r) => r.status === "fulfilled").length;
return new Response(JSON.stringify({ sent, total: subs.length }), {
  headers: { "Content-Type": "application/json" },
});
});

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/functions/payments-webhook/index.ts */
/* ===== */
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.45.0";

const corsHeaders = {
  "Access-Control-Allow-Origin": "https://pedidos100montaditos.es",
  "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type, stripe-signature",
};

async function verifyStripeSignature(payload: string, sigHeader: string, secret: string): Promise<boolean> {
  try {
    const parts = Object.fromEntries(sigHeader.split(",").map((p) => p.split("=")));
    const t = parts["t"];
    const v1 = parts["v1"];
    if (!t || !v1) return false;
    const key = await crypto.subtle.importKey(
      "raw", new TextEncoder().encode(secret),
      { name: "HMAC", hash: "SHA-256" }, false, ["sign"]
    );
    const sig = await crypto.subtle.sign("HMAC", key, new TextEncoder().encode(`${t}.${payload}`));
    const hex = Array.from(new Uint8Array(sig)).map((b) => b.toString(16).padStart(2, "0")).join("");
    return hex === v1;
  } catch {

```

```

    return false;
  }
}

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return new Response("ok", { headers: corsHeaders });

  const liveSecret = Deno.env.get("PAYMENTS_WEBHOOK_SECRET");
  const sandboxSecret = Deno.env.get("PAYMENTS_SANDBOX_WEBHOOK_SECRET");
  const secret = liveSecret || sandboxSecret;

  if (!secret) {
    console.error("Webhook secret no configurado");
    return new Response("webhook not configured", { status: 500 });
  }

  const sig = req.headers.get("stripe-signature");
  if (!sig) return new Response("missing stripe-signature", { status: 400 });

  const payload = await req.text();
  const ok = await verifyStripeSignature(payload, sig, secret);
  if (!ok) {
    console.warn("Firma no válida");
    return new Response("invalid signature", { status: 400, headers: corsHeaders });
  }

  let event: any;
  try { event = JSON.parse(payload); } catch {
    return new Response("bad json", { status: 400, headers: corsHeaders });
  }

  const supabase = createClient(
    Deno.env.get("SUPABASE_URL")!,
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
  );

  const type: string = event.type || "";
  const obj = event.data?.object || {};

  const pedidoId =
    obj.metadata?.pedido_id ||
    obj.client_reference_id ||
    event.data?.metadata?.pedido_id ||
    null;

  const paymentId =
    (type.startsWith("payment_intent") ? obj.id : obj.payment_intent) ||
    obj.id ||
    null;

  // Registrar y deduplicar por event.id (Stripe puede reintentar el mismo evento)
  const { error: logErr } = await supabase.from("webhook_logs").insert({
    event_id: event.id,
    event_type: type,
    pedido_id: pedidoId || null,
    payload: event,
  });
  if (logErr?.code === "23505") {
    // Evento ya procesado – responder 200 para que Stripe no siga reintentando
    return new Response(JSON.stringify({ received: true, duplicate: true }), {
      headers: { ...corsHeaders, "Content-Type": "application/json" },
    });
  }
  if (logErr) console.error("webhook_logs insert error", logErr);

  const isPaid =
    type === "checkout.session.completed" ||
    type === "checkout.session.async_payment_succeeded" ||
    type === "payment_intent.succeeded" ||
    type === "transaction.completed";

  if (isPaid && pedidoId) {
    const hasCocina = !(await supabase.from("pedido_items").select("id").eq("pedido_id", pedidoId).eq("destino",
    "cocina").limit(1)).data?.length;
    const hasBebidas = !(await supabase.from("pedido_items").select("id").eq("pedido_id", pedidoId).eq("destino",
    "bebidas").limit(1)).data?.length;

    const { error } = await supabase
      .from("pedidos")
      .update({
        estado: "pendiente",
        estado_cocina: hasCocina ? "pendiente" : null,
        estado_bebidas: hasBebidas ? "pendiente" : null,
        stripe_payment_id: paymentId,
      })
      .eq("id", pedidoId)
      .eq("estado", "pendiente_pago");
    if (error) console.error("update pedido error", error);
  }

  return new Response(JSON.stringify({ received: true }), {
    headers: { ...corsHeaders, "Content-Type": "application/json" },
  });
});

```

```

    });
  });

/* ===== */
/* FICHERO: 100M Dashboard/src/pages/Agotados.tsx */
/* ===== */
import { useState } from "react";
import { useQuery } from "@tanstack/react-query";
import { Search, Ban, CheckCircle2 } from "lucide-react";
import { toast } from "sonner";
import { supabase } from "@lib/supabase";
import { StaffHeader } from "@components/StaffHeader";
import { useStaffLocal } from "@lib/staff-local";
import { MONTADITO_IMAGES_BY_NUMERO } from "@lib/product-images";
import { getDrinkImage } from "@lib/drink-image";
import { INGREDIENTE_IMAGES } from "@lib/ingrediente-images";

interface Ingrediente {
  id: string;
  nombre: string;
  tipo: "ingrediente" | "pan";
}

interface ProdSuelto {
  id: string;
  numero: string | null;
  nombre: string;
  seccion: string; // Aperitivos | Raciones | Bebidas | Cookies
}

// Categorías de menu_productos que se marcan a nivel de producto entero.
const CATS_PRODUCTO = ["Aperitivos", "Raciones", "Bebidas"];
// Las cookies son montaditos #68-70 pero se gestionan como producto completo.
const COOKIE_NUMEROS = new Set(["68", "69", "70"]);

const norm = (s: string) => (s || "").toLowerCase();
const normNoAcc = (s: string) => norm(s).normalize("NFD").replace(/[\u0300-\u0304]/g, "");
// Salsas: detectadas por nombre (no hay tipo 'salsa' en BD). Empieza por "salsa" o
// está en la lista de salsas sin esa palabra. Guacamole NO es salsa (es ingrediente).
const SALSA_EXTRA = new Set(["mojo picon", "mayonesa", "ketchup", "mostaza", "mostaza y miel", "cheddar"]);
const isSalsa = (nombre: string) => {
  const n = normNoAcc(nombre);
  return n.startsWith("salsa") || SALSA_EXTRA.has(n);
};

// Las gildas se gestionan por variante (boquerón/anchoa) como "productos" en Aperitivos.
const GILDA_VARIANT_IMG: Record<string, string> = {
  "Gilda boquerón": "/assets/img/aperitivos/gilda-boqueron.png",
  "Gilda anchoa": "/assets/img/aperitivos/gilda-anchoa.png",
};
const isGildaVariant = (nombre: string) => nombre in GILDA_VARIANT_IMG;

// Secciones por INGREDIENTE (marca local_ingredientes_agotados)
const ING_SECTIONS: { key: string; emoji: string; title: string; filter: (i: Ingrediente) => boolean }[] = [
  { key: "pan", emoji: "🍞", title: "Panes", filter: (i) => i.tipo === "pan" },
  { key: "salsa", emoji: "🥗", title: "Salsas", filter: (i) => i.tipo === "ingrediente" && isSalsa(i.nombre) },
  { key: "ing", emoji: "🥬", title: "Ingredientes", filter: (i) => i.tipo === "ingrediente" && !isSalsa(i.nombre) && !isGildaVariant(i.nombre) },
];

// Secciones por PRODUCTO (marca local_productos_agotados)
const PROD_SECTIONS: { key: string; emoji: string; title: string }[] = [
  { key: "Aperitivos", emoji: "🍷", title: "Aperitivos" },
  { key: "Raciones", emoji: "🍷", title: "Raciones" },
  { key: "Bebidas", emoji: "🍷", title: "Bebidas" },
  { key: "Cookies", emoji: "🍪", title: "Cookies" },
];

export default function Agotados() {
  const localId = useStaffLocal((s) => s.local?.id ?? null);
  const [q, setQ] = useState("");

  const ingsQ = useQuery({
    queryKey: ["ag-ingredientes"],
    queryFn: async () => {
      const { data } = await supabase.from("ingredientes").select("id, nombre, tipo").order("nombre");
      return (data ?? []) as Ingrediente[];
    },
  });

  const prodsQ = useQuery({
    queryKey: ["ag-productos-sueltos"],
    queryFn: async () => {
      const { data } = await supabase
        .from("menu_productos")
        .select("id, numero, nombre, disponible, menu_categorias(nombre)")
        .eq("disponible", true);
      const out: ProdSuelto[] = [];
      for (const p of (data ?? []) as any[]) {
        const cat = p.menu_categorias?.nombre;
        if (CATS_PRODUCTO.includes(cat)) {
          // La "Gildas" se gestiona por variante (boquerón/anchoa), no como producto único.
        }
      }
    },
  });

```

```

        if (cat === "Aperitivos" && (p.nombre ?? "").toLowerCase() === "gildas") continue;
        out.push({ id: p.id, numero: p.numero, nombre: p.nombre, seccion: cat });
    } else if (cat === "Montaditos" && COOKIE_NUMEROS.has(String(p.numero))) {
        out.push({ id: p.id, numero: p.numero, nombre: p.nombre, seccion: "Cookies" });
    }
}
return out;
},
});

const agIngQ = useQuery({
    queryKey: ["ag-local-ings", localId],
    enabled: !!localId,
    refetchInterval: 15000,
    queryFn: async () => {
        const { data } = await supabase.from("local_ingredientes_agotados").select("ingrediente_id").eq("local_id", localId);
        return new Set<string>(((data ?? []) as any[]).map((r) => r.ingrediente_id));
    },
});

const agProdQ = useQuery({
    queryKey: ["ag-local-prods", localId],
    enabled: !!localId,
    refetchInterval: 15000,
    queryFn: async () => {
        const { data } = await supabase.from("local_productos_agotados").select("producto_id").eq("local_id", localId);
        return new Set<string>(((data ?? []) as any[]).map((r) => r.producto_id));
    },
});

const agIng = agIngQ.data ?? new Set<string>();
const agProd = agProdQ.data ?? new Set<string>();

const toggleIng = async (id: string) => {
    if (!localId) return;
    const isAg = agIng.has(id);
    const { error } = isAg
        ? await supabase.from("local_ingredientes_agotados").delete().eq("local_id", localId).eq("ingrediente_id", id)
        : await supabase.from("local_ingredientes_agotados").insert({ local_id: localId, ingrediente_id: id });
    if (error) toast.error("Error: " + error.message);
    else agIngQ.refetch();
};

const toggleProd = async (id: string) => {
    if (!localId) return;
    const isAg = agProd.has(id);
    const { error } = isAg
        ? await supabase.from("local_productos_agotados").delete().eq("local_id", localId).eq("producto_id", id)
        : await supabase.from("local_productos_agotados").insert({ local_id: localId, producto_id: id });
    if (error) toast.error("Error: " + error.message);
    else agProdQ.refetch();
};

const match = (s: string) => !q || norm(s).includes(norm(q));
const totalAgotados = agIng.size + agProd.size;

const Chip = ({ activo, label, img, onClick }: { activo: boolean; label: string; img?: string | null; onClick: () => void }) => (
    <button
        onClick={onClick}
        className={`flex items-center gap-2 rounded-xl border px-2.5 py-2 text-left text-sm transition-colors ${
            activo
                ? "border-destructive/60 bg-destructive/15 text-destructive"
                : "border-border bg-card hover:bg-muted"
        }`}
    >
        {img ? (
            <img src={img} alt="" className="w-9 h-9 object-contain shrink-0 loading="lazy" />
        ) : (
            <span className="w-9 h-9 shrink-0 rounded-lg bg-muted/40" aria-hidden />
        )}
        <span className={`flex-1 min-w-0 truncate ${activo ? "font-semibold line-through" : "font-medium"}`}>{label}</span>
        {activo ? <Ban className="h-4 w-4 shrink-0" /> : <CheckCircle2 className="h-4 w-4 shrink-0 text-muted-foreground/40" />}
    </button>
);

// Imagen de un producto (cookies por número, aperitivos/raciones/bebidas por nombre).
const productImg = (p: ProdSuelto): string | null =>
    (p.numero ? MONTADITO_IMAGES_BY_NUMERO[p.numero] : undefined) ?? getDrinkImage(p.nombre) ?? null;

const SectionHeader = ({ emoji, title }: { emoji: string; title: string }) => (
    <h2 className="mb-2 flex items-center gap-2 text-sm font-bold uppercase tracking-wider text-muted-foreground">
        <span aria-hidden className="text-base">{emoji}</span> {title}
    </h2>
);

const ings = ingsQ.data ?? [];
const prods = prodsQ.data ?? [];

return (
    <div className="min-h-screen bg-background">
        <StaffHeader title="Agotados" subtitle="Marca lo que se ha acabado; desaparece de la app del cliente" />
        <main className="mx-auto max-w-[1100px] space-y-6 p-6">

```

```

    {!localId && (
      <div className="rounded-md border border-amber-500/50 bg-amber-50/40 p-4 text-sm">
        Selecciona un local para gestionar los agotados.
      </div>
    )}

    <div className="sticky top-[73px] z-20 -mx-6 bg-background/90 px-6 py-2 backdrop-blur">
      <div className="relative">
        <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-muted-foreground" />
        <input
          value={q}
          onChange={(e) => setQ(e.target.value)}
          placeholder="Buscar pan, salsa, ingrediente o producto..."
          className="w-full rounded-xl border border-border bg-card py-3 pl-10 pr-4 text-base outline-none focus:border-primary"
        />
      </div>
      {totalAgotados > 0 && <p className="mt-2 text-xs text-destructive">{totalAgotados} agotado(s) ahora mismo</p>}
    </div>

    /* Secciones por ingrediente */
    {ING_SECTIONS.map((sec) => {
      const list = ings.filter((i) => sec.filter(i) && match(i.nombre));
      if (list.length === 0) return null;
      return (
        <section key={sec.key}>
          <SectionHeader emoji={sec.emoji} title={sec.title} />
          <div className="grid grid-cols-2 gap-2 sm:grid-cols-3 md:grid-cols-4">
            {list.map((i) => (
              <Chip key={i.id} activo={agIng.has(i.id)} label={i.nombre} img={INGREDIENTE_IMAGES[i.nombre] ?? null} onClick={() =>
toggleIng(i.id)} />
            ))}
          </div>
        </section>
      );
    })}

    /* Secciones por producto */
    {PROD_SECTIONS.map((sec) => {
      const list = prods.filter((p) => p.seccion === sec.key && (match(p.nombre) || match(p.numero ?? "")));
      // En Aperitivos, las gildas se gestionan como 2 variantes (boquerón/anchoa).
      const gildaVars =
        sec.key === "Aperitivos"
          ? ings.filter((i) => isGildaVariant(i.nombre) && match(i.nombre))
          : [];
      if (list.length === 0 && gildaVars.length === 0) return null;
      return (
        <section key={sec.key}>
          <SectionHeader emoji={sec.emoji} title={sec.title} />
          <div className="grid grid-cols-2 gap-2 sm:grid-cols-3 md:grid-cols-4">
            {list.map((p) => (
              <Chip
                key={p.id}
                activo={agProd.has(p.id)}
                label={` ${p.numero ? p.numero + " " : ""} ${p.nombre} `}
                img={productImg(p)}
                onClick={() => toggleProd(p.id)}
              />
            ))}
            {gildaVars.map((i) => (
              <Chip
                key={i.id}
                activo={agIng.has(i.id)}
                label={i.nombre}
                img={GILDA_VARIANT_IMG[i.nombre]}
                onClick={() => toggleIng(i.id)}
              />
            ))}
          </div>
        </section>
      );
    })}

    {(ingsQ.isLoading || prodsQ.isLoading) && (
      <p className="text-center text-sm text-muted-foreground">Cargando...</p>
    )}
  </main>
</div>
);
}

```

```

/* ===== */
/* FICHERO: 100M Dashboard/src/pages/Caja.tsx */
/* ===== */
import { useEffect, useMemo, useRef, useState } from "react";
import { supabase, type EstadoPedido } from "@lib/supabase";
import { useStaffLocal } from "@lib/staff-local";
import { playBeep, unlockAudio } from "@lib/beep";
import { StaffHeader } from "@components/StaffHeader";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Input } from "@components/ui/input";
import { Button } from "@components/ui/button";

```

```

import { Badge } from "@components/ui/badge";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@components/ui/select";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@components/ui/table";
import { Dialog, DialogContent, DialogHeader, DialogTitle } from "@components/ui/dialog";
import { toast } from "sonner";
import { Search, ShoppingBag, Euro, Receipt, Clock, GlassWater, ChefHat, Check } from "lucide-react";
import { format } from "date-fns";

type TipoPedido = "cocina" | "bebidas" | "mixto";

interface Pedido {
  id: string;
  numero_pedido: number;
  tipo: TipoPedido;
  estado: EstadoPedido | null;
  estado_cocina: EstadoPedido | null;
  estado_bebidas: EstadoPedido | null;
  total: number;
  notas: string | null;
  created_at: string;
}

interface ItemLite {
  id: string;
  pedido_id: string;
  cantidad: number;
  precio_unitario: number;
  destino: "cocina" | "bebidas";
  variante: string | null;
  producto: { nombre: string; numero: string | null } | null;
}

/** Nº de carta + nombre + variante (ej. "#13 Pulled pork BBQ · Jarra Sancho"). */
const itemLabel = (it: ItemLite): string => {
  const num = it.producto?.numero ? `#${it.producto.numero}` : "";
  const v = it.variante ? it.variante.replace(/\n/g, " · ") : "";
  return num + (it.producto?.nombre ?? "-") + (v ? ` · ${v}` : "");
};

const ESTADO_VARIANT: Record<EstadoPedido, { label: string; cls: string }> = {
  pendiente: { label: "Pendiente", cls: "bg-amber-500 text-white" },
  recibido: { label: "Recibido", cls: "bg-[hsl(var(--status-recibido))] text-white" },
  preparando: { label: "Preparando", cls: "bg-[hsl(var(--status-preparando))] text-white" },
  listo: { label: "Listo", cls: "bg-[hsl(var(--status-listo))] text-white" },
  entregado: { label: "Entregado", cls: "bg-[hsl(var(--status-entregado))] text-white" },
  cancelado: { label: "Cancelado", cls: "bg-muted text-muted-foreground" },
};

// Inicio del día de HOY en Europe/Madrid (instante UTC), independiente de la
// zona horaria configurada en la tablet – así los pedidos del día no "desaparecen".
const startOfDayIso = () => {
  const now = new Date();
  const ymd = new Intl.DateTimeFormat('sv-SE', { timeZone: 'Europe/Madrid' }).format(now); // YYYY-MM-DD
  for (const off of ['+02:00', '+01:00']) {
    const cand = new Date(`${ymd}T00:00:00${off}`);
    const back = new Intl.DateTimeFormat('sv-SE', {
      timeZone: 'Europe/Madrid', year: 'numeric', month: '2-digit', day: '2-digit',
      hour: '2-digit', minute: '2-digit', hour12: false,
    }).format(cand);
    if (back === `${ymd} 00:00`) return cand.toISOString();
  }
  return new Date(`${ymd}T00:00:00+01:00`).toISOString();
};

const isActive = (e: EstadoPedido | null) =>
  !e && e !== "pendiente" && e !== "entregado" && e !== "cancelado";

// Fuente de verdad: el campo `destino` guardado en pedido_items.
// Fallback defensivo: si llegara nulo, asume cocina.
const effectiveDestino = (it: ItemLite): "cocina" | "bebidas" =>
  it.destino === "bebidas" ? "bebidas" : "cocina";

const Caja = () => {
  const localId = useStaffLocal((s) => s.local?.id ?? null);
  const [pedidos, setPedidos] = useState<Pedido[]>([]);
  const [items, setItems] = useState<Record<string, ItemLite[]>>({});
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState("");
  const [estadoFilter, setEstadoFilter] = useState<string>("all");
  const [detailId, setDetailId] = useState<string | null>(null);

  const loadPedidos = async () => {
    if (!localId) {
      setPedidos([]);
      setItems({});
      setLoading(false);
    }
  }

```



```

    return;
  }
  const { data, error } = await supabase
    .from("pedidos")
    .select(
      "id, numero_pedido, tipo, estado, estado_cocina, estado_bebidas, total, notas, created_at",
    )
    .eq("local_id", localId)
    .gte("created_at", startOfDayIso())
    .order("created_at", { ascending: false });
  if (error) {
    toast.error("Error cargando pedidos", { description: error.message });
    setLoading(false);
    return;
  }
  const list = (data ?? []) as Pedido[];
  setPedidos(list);

  if (list.length > 0) {
    const ids = list.map((p) => p.id);
    const { data: its } = await supabase
      .from("pedido_items")
      .select(
        "id, pedido_id, cantidad, precio_unitario, destino, variante, producto:menu_productos(nombre, numero)",
      )
      .in("pedido_id", ids);
    const grouped: Record<string, ItemLite[]> = {};
    (its ?? []).forEach((it) => {
      const item = it as unknown as ItemLite;
      if (!grouped[item.pedido_id]) grouped[item.pedido_id] = [];
      grouped[item.pedido_id].push(item);
    });
    setItems(grouped);
  } else {
    setItems({});
  }
  setLoading(false);
};

useEffect(() => {
  if (!localId) return;
  loadPedidos();

  // Respaldo por polling: si el WebSocket Realtime no entrega eventos (RLS,
  // red del APK, etc.), refrescamos cada 5s para que los pedidos nuevos
  // aparezcan solos sin tener que salir y volver a entrar.
  const poll = setInterval(loadPedidos, 5000);

  let ch: ReturnType<typeof supabase.channel> | null = null;
  try {
    ch = supabase
      .channel(`caja-pedidos-${localId}`)
      .on(
        "postgres_changes",
        {
          event: "*",
          schema: "public",
          table: "pedidos",
          filter: `local_id=eq.${localId}`,
        },
        () => loadPedidos(),
      )
      .on(
        "postgres_changes",
        { event: "*", schema: "public", table: "pedido_items" },
        () => loadPedidos(),
      )
      .subscribe();
  } catch (e) {
    console.warn("[realtime] no disponible en caja, se usa polling", e);
  }
  return () => {
    clearInterval(poll);
    if (ch) {
      try { supabase.removeChannel(ch); } catch { /* ignore */ }
    }
  };
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [localId]);

// Sección Bebidas: pedidos con estado_bebidas activo
const bebidaActivos = useMemo(
  () => pedidos.filter((p) => isActive(p.estado_bebidas)),
  [pedidos],
);

// Sección Cocina (vista en Caja): pedidos con cocina LISTO para entregar
const cocinaParaEntregar = useMemo(
  () => pedidos.filter((p) => p.estado_cocina === "listo"),
  [pedidos],
);

const stats = useMemo(() => {

```

```

const total = pedidos.length;
const ingresos = pedidos.reduce((s, p) => s + Number(p.total), 0);
const ticket = total ? ingresos / total : 0;
const pendientes = pedidos.filter(
  (p) => isActive(p.estado_cocina) || isActive(p.estado_bebidas),
).length;
const listos = pedidos.filter(
  (p) => p.estado_cocina === "listo" || p.estado_bebidas === "listo",
).length;
const entregados = pedidos.filter(
  (p) =>
    (p.estado_cocina === null || p.estado_cocina === "entregado") &&
    (p.estado_bebidas === null || p.estado_bebidas === "entregado"),
).length;
return { total, ingresos, ticket, pendientes, listos, entregados };
}, [pedidos]);

const filtered = useMemo(() => {
  return pedidos.filter((p) => {
    if (estadoFilter !== "all") {
      if (p.estado_cocina !== estadoFilter && p.estado_bebidas !== estadoFilter) return false;
    }
    if (search && !String(p.numero_pedido).includes(search.trim())) return false;
    return true;
  });
}, [pedidos, search, estadoFilter]);

const porTicar = useMemo(
  () =>
    pedidos.filter(
      (p) => p.estado_cocina === "pendiente" || p.estado_bebidas === "pendiente",
    ),
  [pedidos],
);

// 📢 Pitido al llegar un pedido NUEVO a "Por ticar". No suena en la carga
// inicial (solo cuando aparece un id que no estaba antes).
const seenRef = useRef<Set<string>>(new Set());
const beepInitRef = useRef(false);
useEffect(() => {
  if (loading) return;
  const ids = porTicar.map((p) => p.id);
  if (!beepInitRef.current) {
    seenRef.current = new Set(ids);
    beepInitRef.current = true;
    return;
  }
  const hayNuevos = ids.some((id) => !seenRef.current.has(id));
  seenRef.current = new Set(ids);
  if (hayNuevos) playBeep();
}, [porTicar, loading]);

// Desbloquea el audio del WebView de Android al primer toque del usuario.
useEffect(() => {
  const unlock = () => unlockAudio();
  window.addEventListener("pointerdown", unlock, { once: true });
  return () => window.removeEventListener("pointerdown", unlock);
}, []);

const ticarPedido = async (p: Pedido) => {
  const patch: Record<string, string> = { updated_at: new Date().toISOString() };
  if (p.estado_cocina === "pendiente") patch.estado_cocina = "recibido";
  if (p.estado_bebidas === "pendiente") patch.estado_bebidas = "recibido";
  const { error } = await supabase.from("pedidos").update(patch).eq("id", p.id);
  if (error) toast.error("Error", { description: error.message });
  else toast.success(`Pedido #${p.numero_pedido} ticado y enviado`);
};

const ticarSeccion = async (p: Pedido, seccion: "cocina" | "bebidas") => {
  const col = seccion === "cocina" ? "estado_cocina" : "estado_bebidas";
  const { error } = await supabase
    .from("pedidos")
    .update({ [col]: "recibido", updated_at: new Date().toISOString() })
    .eq("id", p.id);
  if (error) toast.error("Error", { description: error.message });
  else toast.success(`${seccion} de #${p.numero_pedido} enviado`);
};

const updateSeccion = async (
  id: string,
  seccion: "cocina" | "bebidas",
  estado: EstadoPedido,
) => {
  const col = seccion === "cocina" ? "estado_cocina" : "estado_bebidas";
  const { error } = await supabase
    .from("pedidos")
    .update({ [col]: estado, updated_at: new Date().toISOString() })
    .eq("id", id);
  if (error) toast.error("Error", { description: error.message });
  else toast.success(`${seccion} → ${estado}`);
};

```

```

const detailPedido = pedidos.find((p) => p.id === detailId) ?? null;
const detailItems = detailId ? items[detailId] ?? [] : [];

return (
  <div className="min-h-screen bg-background">
    <StaffHeader title="Caja · Dashboard del día" subtitle={format(new Date(), "EEEE d 'de' MMMM yyyy")} />
    <main className="mx-auto max-w-[1800px] space-y-6 p-6">
      <div className="grid grid-cols-2 gap-4 md:grid-cols-4">
        <StatCard icon={<ShoppingBag />} label="Pedidos hoy" value={stats.total.toString()} />
        <StatCard icon={<Euro />} label="Ingresos" value={`$${stats.ingresos.toFixed(2)} €`} />
        <StatCard icon={<Receipt />} label="Ticket medio" value={`$${stats.ticket.toFixed(2)} €`} />
        <StatCard
          icon={<Clock />}
          label="Activos / Listos / Entreg."
          value={`$${stats.pendientes} · $${stats.listos} · $${stats.entregados}`}
        />
      </div>

      {/* POR TICAR: pedidos del cliente que aún no han pasado por caja */}
      <Card className="border-amber-500/50 bg-amber-50/40 dark:bg-amber-950/10">
        <CardHeader>
          <CardTitle className="flex items-center gap-2 text-lg">
            <Receipt className="h-5 w-5 text-amber-600" />
            Por ticar ({porTicar.length})
          </CardTitle>
        </CardHeader>
        <CardContent className="space-y-3">
          {porTicar.length === 0 && (
            <div className="rounded-md border border-dashed border-amber-500 p-6 text-center text-sm text-muted-foreground">
              Sin pedidos pendientes de ticar
            </div>
          )}
          {porTicar.map((p) => {
            const its = items[p.id] ?? [];
            const cocinaIts = its.filter((it) => effectiveDestino(it) === "cocina");
            const bebidaIts = its.filter((it) => effectiveDestino(it) === "bebidas");
            const cocinaPend = p.estado_cocina === "pendiente";
            const bebidaPend = p.estado_bebidas === "pendiente";
            return (
              <Card key={`t-${p.id}`} className="border-1-4 border-1-amber-500 p-3">
                <div className="mb-2 flex items-start justify-between">
                  <div>
                    <div className="flex items-center gap-2">
                      <span className="text-2xl font-bold">#{p.numero_pedido}</span>
                      <Badge className="bg-amber-500 text-white">Por ticar</Badge>
                    </div>
                    <div className="text-xs text-muted-foreground">
                      {format(new Date(p.created_at), "HH:mm")}
                    </div>
                  </div>
                  <div className="text-sm font-semibold">{Number(p.total).toFixed(2)} €</div>
                </div>
                <div className="mb-3 grid gap-2 sm:grid-cols-2">
                  {cocinaIts.length > 0 && (
                    <div>
                      <div className="mb-1 flex items-center gap-1 text-[10px] font-semibold uppercase tracking-wider text-orange-700">
                        <ChefHat className="h-3 w-3" /> Cocina
                        {cocinaPend && <Badge className="m-1 bg-amber-500 text-white">Por ticar</Badge>}
                      </div>
                      <ul className="text-sm">
                        {cocinaIts.map((it) => (
                          <li key={it.id}>
                            <span className="font-bold">{it.cantidad}</span> {itemLabel(it)}
                          </li>
                        ))}
                      </ul>
                    </div>
                  )}
                  {bebidaIts.length > 0 && (
                    <div>
                      <div className="mb-1 flex items-center gap-1 text-[10px] font-semibold uppercase tracking-wider text-sky-700">
                        <GlassWater className="h-3 w-3" /> Bebidas/Aperitivos
                        {bebidaPend && <Badge className="m-1 bg-amber-500 text-white">Por ticar</Badge>}
                      </div>
                      <ul className="text-sm">
                        {bebidaIts.map((it) => (
                          <li key={it.id}>
                            <span className="font-bold">{it.cantidad}</span> {itemLabel(it)}
                          </li>
                        ))}
                      </ul>
                    </div>
                  )}
                </div>
                {p.notas && (
                  <div className="mb-2 rounded-md bg-muted p-2 text-xs italic">🗨️ {p.notas}</div>
                )}
                <div className="flex flex-wrap gap-2">
                  {cocinaPend && bebidaPend && (
                    <Button size="sm" onClick={() => ticarPedido(p)}>

```

```

        <Check className="h-4 w-4" /> Ticar todo y enviar
    </Button>
  )}
  {cocinaPend && (
    <Button
      size="sm"
      variant={cocinaPend && bebidaPend ? "outline" : "default"}
      onClick={() => ticarSeccion(p, "cocina")}
    >
      <ChefHat className="h-4 w-4" /> Ticar y enviar a cocina
    </Button>
  )}
  {bebidaPend && (
    <Button
      size="sm"
      variant={cocinaPend && bebidaPend ? "outline" : "default"}
      onClick={() => ticarSeccion(p, "bebidas")}
    >
      <GlassWater className="h-4 w-4" /> Ticar y enviar a bebidas
    </Button>
  )}
  <Button size="sm" variant="ghost" onClick={() => setDetailId(p.id)}>
    Ver
  </Button>
</div>
</Card>
);
  )}
</CardContent>
</Card>

<div className="grid grid-cols-1 gap-6 lg:grid-cols-2">
  { /* BEBIDAS */ }
  <Card className="border-sky-500/40 bg-sky-50/30 dark:bg-sky-950/10">
    <CardHeader>
      <CardTitle className="flex items-center gap-2 text-lg">
        <GlassWater className="h-5 w-5 text-sky-600" />
        Bebidas/Aperitivos ({bebidaActivos.length})
      </CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
      {bebidaActivos.length === 0 && (
        <div className="rounded-md border border-dashed border-border p-6 text-center text-sm text-muted-foreground">
          Sin tickets de bebida
        </div>
      )}
      {bebidaActivos.map((p) => (
        <SeccionTicket
          key={`b-${p.id}`}
          pedido={p}
          items={items[p.id] ?? []}.filter((it) => effectiveDestino(it) === "bebidas")}
          seccion="bebidas"
          accent="sky"
          onAction={(estado) => updateSeccion(p.id, "bebidas", estado)}
          onView={() => setDetailId(p.id)}
        />
      ))}
    </CardContent>
  </Card>

  { /* COCINA: solo para entregar desde caja */ }
  <Card className="border-orange-500/40 bg-orange-50/30 dark:bg-orange-950/10">
    <CardHeader>
      <CardTitle className="flex items-center gap-2 text-lg">
        <ChefHat className="h-5 w-5 text-orange-600" />
        Cocina · listos para entregar ({cocinaParaEntregar.length})
      </CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
      {cocinaParaEntregar.length === 0 && (
        <div className="rounded-md border border-dashed border-border p-6 text-center text-sm text-muted-foreground">
          Sin tickets listos
        </div>
      )}
      {cocinaParaEntregar.map((p) => (
        <SeccionTicket
          key={`c-${p.id}`}
          pedido={p}
          items={items[p.id] ?? []}.filter((it) => effectiveDestino(it) === "cocina")}
          seccion="cocina"
          accent="orange"
          onAction={(estado) => updateSeccion(p.id, "cocina", estado)}
          onView={() => setDetailId(p.id)}
        />
      ))}
    </CardContent>
  </Card>
</div>

{ /* Filtros */ }
<Card>
  <CardContent className="flex flex-col gap-3 p-4 md:flex-row md:items-center">

```

```

<div className="relative flex-1">
  <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-muted-foreground" />
  <Input
    placeholder="Buscar por número de pedido.."
    value={search}
    onChange={(e) => setSearch(e.target.value)}
    className="p1-9"
  />
</div>
<Select value={estadoFilter} onChange={setEstadoFilter}>
  <SelectTrigger className="md:w-56">
    <SelectValue placeholder="Estado" />
  </SelectTrigger>
  <SelectContent>
    <SelectItem value="all">Todos los estados</SelectItem>
    <SelectItem value="pendiente">Pendiente</SelectItem>
    <SelectItem value="recibido">Recibido</SelectItem>
    <SelectItem value="preparando">Preparando</SelectItem>
    <SelectItem value="listo">Listo</SelectItem>
    <SelectItem value="entregado">Entregado</SelectItem>
  </SelectContent>
</Select>
</CardContent>
</Card>

{/* Tabla del día */}
<Card>
  <CardHeader>
    <CardTitle className="text-lg">Pedidos del día ({filtered.length})</CardTitle>
  </CardHeader>
  <CardContent>
    {loading ? (
      <div className="py-12 text-center text-muted-foreground">Cargando...</div>
    ) : filtered.length === 0 ? (
      <div className="py-12 text-center text-muted-foreground">Sin pedidos para mostrar</div>
    ) : (
      <Table>
        <TableHeader>
          <TableRow>
            <TableHead>Nº</TableHead>
            <TableHead>Tipo</TableHead>
            <TableHead>Hora</TableHead>
            <TableHead>Productos</TableHead>
            <TableHead>Cocina</TableHead>
            <TableHead>Bebidas/Aperitivos</TableHead>
            <TableHead className="text-right">Total</TableHead>
            <TableHead className="text-right">Acciones</TableHead>
          </TableRow>
        </TableHeader>
        <TableBody>
          {filtered.map((p) => {
            const its = items[p.id] ?? [];
            const cocinaIts = its.filter((it) => effectiveDestino(it) === "cocina");
            const bebidaIts = its.filter((it) => effectiveDestino(it) === "bebidas");
            return (
              <TableRow key={p.id}>
                <TableCell className="font-bold align-top">#{p.numero_pedido}</TableCell>
                <TableCell className="align-top">
                  <TipoBadge tipo={p.tipo} />
                </TableCell>
                <TableCell className="align-top">{format(new Date(p.created_at), "HH:mm")}</TableCell>
                <TableCell className="align-top text-sm">
                  {its.length === 0 ? (
                    <span className="text-xs text-muted-foreground">--</span>
                  ) : (
                    <div className="space-y-1">
                      {cocinaIts.length > 0 && (
                        <div>
                          <div className="flex items-center gap-1 text-[10px] font-semibold uppercase tracking-wider text-orange-700">
                            <ChefHat className="h-3 w-3" /> Cocina
                          </div>
                          <ul>
                            {cocinaIts.map((it) => (
                              <li key={it.id}>
                                <span className="font-bold">{it.cantidad}</span> {itemLabel(it)}
                              </li>
                            ))}
                          </ul>
                        </div>
                      )}
                    </div>
                  )}
                </TableCell>
                <div>
                  <div className="flex items-center gap-1 text-[10px] font-semibold uppercase tracking-wider text-sky-700">
                    <GlassWater className="h-3 w-3" /> Bebidas/Aperitivos
                  </div>
                  <ul>
                    {bebidaIts.map((it) => (
                      <li key={it.id}>
                        <span className="font-bold">{it.cantidad}</span> {itemLabel(it)}
                      </li>
                    ))}
                  </ul>
                </div>
              </TableRow>
            )}
          )}
        </TableBody>
      </Table>
    )}
  </CardContent>
</Card>

```



```

};

const TipoBadge = ({ tipo }: { tipo: TipoPedido }) => {
  if (tipo === "bebidas")
    return (
      <Badge variant="outline" className="border-sky-500 text-sky-700">
        <GlassWater className="mr-1 h-3 w-3" /> Bebidas/Aperitivos
      </Badge>
    );
  if (tipo === "cocina")
    return (
      <Badge variant="outline" className="border-orange-500 text-orange-700">
        <ChefHat className="mr-1 h-3 w-3" /> Cocina
      </Badge>
    );
  return (
    <Badge variant="outline" className="border-purple-500 text-purple-700">
      Mixto
    </Badge>
  );
};

const SeccionTicket = ({
  pedido,
  items,
  seccion,
  accent,
  onAction,
  onView,
}: {
  pedido: Pedido;
  items: ItemLite[];
  seccion: "cocina" | "bebidas";
  accent: "sky" | "orange";
  onAction: (estado: EstadoPedido) => void;
  onView: () => void;
}) => {
  const borderCls = accent === "sky" ? "border-l-sky-500" : "border-l-orange-500";
  const estado = seccion === "cocina" ? pedido.estado_cocina : pedido.estado_bebidas;
  const v = estado ? ESTADO_VARIANT[estado] : null;

  const renderActions = () => {
    if (seccion === "bebidas") {
      if (estado === "pendiente" || estado === "recibido")
        return (
          <Button size="sm" className="flex-1" onClick={() => onAction("preparando")}>
            Comenzar
          </Button>
        );
      if (estado === "preparando")
        return (
          <Button size="sm" className="flex-1" onClick={() => onAction("listo")}>
            <Check className="h-4 w-4" /> Listo
          </Button>
        );
      if (estado === "listo")
        return (
          <Button size="sm" className="flex-1" onClick={() => onAction("entregado")}>
            Entregar
          </Button>
        );
    } else {
      // cocina: caja sólo entrega
      if (estado === "listo")
        return (
          <Button size="sm" className="flex-1" onClick={() => onAction("entregado")}>
            Entregar
          </Button>
        );
    }
    return null;
  };

  return (
    <Card className={`border-l-4 ${borderCls} p-3`} >
      <div className="mb-2 flex items-start justify-between">
        <div>
          <div className="text-2xl font-bold">#{pedido.numero_pedido}</div>
          <div className="text-xs text-muted-foreground">
            {format(new Date(pedido.created_at), "HH:mm")}
            {pedido.tipo === "mixto" && (
              <Badge variant="outline" className="ml-2 border-purple-500 text-purple-700">
                Mixto
              </Badge>
            )}
          </div>
        </div>
        <div className="text-right">
          {v && <Badge className={v.cls}>{v.label}</Badge>}
          <div className="mt-1 text-sm font-semibold">{Number(pedido.total).toFixed(2)} €</div>
        </div>
      </div>
    </Card>
  );
};

```

```

<ul className="mb-2 space-y-0.5 text-sm">
  {items.map((it) => (
    <li key={it.id}>
      <span className="font-bold">{it.cantidad}</span> {itemLabel(it)}
    </li>
  ))}
  {items.length === 0 && (
    <li className="text-xs italic text-muted-foreground">Sin items en esta sección</li>
  )}
</ul>
{pedido.notas && (
  <div className="mb-2 rounded-md bg-muted p-2 text-xs italic"> 🗨 {pedido.notas}</div>
)}
<div className="flex gap-2">
  <Button size="sm" variant="outline" onClick={onView}>
    Ver
  </Button>
  {renderActions()}
</div>
</Card>
);
};

const StatCard = ({ icon, label, value }: { icon: React.ReactNode; label: string; value: string }) => (
  <Card>
    <CardContent className="flex items-center gap-4 p-5">
      <div className="flex h-12 w-12 items-center justify-center rounded-xl bg-primary/10 text-primary [&svg]:h-5 [&svg]:w-5">
        {icon}
      </div>
      <div>
        <div className="text-xs uppercase tracking-wider text-muted-foreground">{label}</div>
        <div className="text-xl font-bold">{value}</div>
      </div>
    </CardContent>
  </Card>
);

export default Caja;

/* ===== */
/* FICHERO: 100M Dashboard/src/pages/Cocina.tsx */
/* ===== */
import { useEffect, useRef, useState } from "react";
import { supabase, type EstadoPedido } from "@lib/supabase";
import { useStaffLocal } from "@lib/staff-local";
import { playBeep, unlockAudio } from "@lib/beep";
import { StaffHeader } from "@components/StaffHeader";
import { Button } from "@components/ui/button";
import { Card } from "@components/ui/card";
import { toast } from "sonner";
import { ChefHat, Clock, ArrowRight, ArrowLeft, Check } from "lucide-react";
import { formatDistanceToNowStrict } from "date-fns";
import { es } from "date-fns/locale";

interface Pedido {
  id: string;
  numero_pedido: number;
  estado_cocina: EstadoPedido;
  total: number;
  notas: string | null;
  created_at: string;
}

interface PedidoItem {
  id: string;
  pedido_id: string;
  cantidad: number;
  destino: "cocina" | "bebidas";
  variante: string | null;
  producto: { nombre: string } | null;
}

/** Nombre del item + variante (ej. "Nachos · Cheddar y bacon"). */
const itemLabel = (it: PedidoItem): string =>
  (it.producto?.nombre ?? "-") + (it.variante ? ` · ${it.variante}` : "");

const COLUMNS: { key: EstadoPedido; title: string; next?: EstadoPedido; prev?: EstadoPedido; nextLabel?: string; prevLabel?: string }
[] = [
  { key: "recibido", title: "Recibido", next: "preparando", nextLabel: "Comenzar" },
  { key: "preparando", title: "Preparando", next: "listo", prev: "recibido", nextLabel: "Marcar listo", prevLabel: "Volver" },
  { key: "listo", title: "Listo", next: "entregado", prev: "preparando", nextLabel: "Entregar", prevLabel: "Volver" },
];

const minutesSince = (iso: string) => Math.floor((Date.now() - new Date(iso).getTime()) / 60000);

const borderForAge = (mins: number) => {
  if (mins < 5) return "border-l-[hsl(var(--timer-fresh))]";
  if (mins < 10) return "border-l-[hsl(var(--timer-warning))]";
  return "border-l-[hsl(var(--timer-late))]";
};

```



```

const Cocina = () => {
  const localId = useStaffLocal((s) => s.local?.id ?? null);
  const [pedidos, setPedidos] = useState<Pedido[]>([]);
  const [items, setItems] = useState<Record<string, PedidoItem[]>>({});
  const [loading, setLoading] = useState(true);
  const [, setTick] = useState(0);

  useEffect(() => {
    const id = setInterval(() => setTick((t) => t + 1), 30000);
    return () => clearInterval(id);
  }, []);

  const loadAll = async () => {
    if (!localId) {
      setPedidos([]);
      setItems({});
      setLoading(false);
      return;
    }
    const { data: peds, error } = await supabase
      .from("pedidos")
      .select("id, numero_pedido, estado_cocina, total, notas, created_at")
      .eq("local_id", localId)
      .not("estado_cocina", "is", null)
      .in("estado_cocina", ["recibido", "preparando", "listo"])
      .order("created_at", { ascending: true });

    if (error) {
      toast.error("Error cargando pedidos", { description: error.message });
      setLoading(false);
      return;
    }
    const list = (peds ?? []) as Pedido[];
    setPedidos(list);

    if (list.length > 0) {
      const ids = list.map((p) => p.id);
      const { data: its } = await supabase
        .from("pedido_items")
        .select("id, pedido_id, cantidad, destino, variante, producto:menu_productos(nombre)")
        .in("pedido_id", ids)
        .eq("destino", "cocina");
      const grouped: Record<string, PedidoItem[]> = {};
      (its ?? []).forEach((it) => {
        const item = it as unknown as PedidoItem;
        if (!grouped[item.pedido_id]) grouped[item.pedido_id] = [];
        grouped[item.pedido_id].push(item);
      });
      setItems(grouped);
    } else {
      setItems({});
    }
    setLoading(false);
  };

  useEffect(() => {
    if (!localId) return;
    loadAll();

    // Respaldo por polling: si el WebSocket Realtime no entrega eventos (RLS,
    // red del APK, etc.), refrescamos cada 5s para que los pedidos nuevos
    // aparezcan solos sin tener que salir y volver a entrar.
    const poll = setInterval(loadAll, 5000);

    let channel: ReturnType<typeof supabase.channel> | null = null;
    try {
      channel = supabase
        .channel(`kds-pedidos-${localId}`)
        .on(
          "postgres_changes",
          {
            event: "*",
            schema: "public",
            table: "pedidos",
            filter: `local_id=eq.${localId}`,
          },
          () => loadAll(),
        )
        .on(
          "postgres_changes",
          { event: "*", schema: "public", table: "pedido_items" },
          () => loadAll(),
        )
        .subscribe();
    } catch (e) {
      console.warn("[realtime] no disponible en cocina, se usa polling", e);
    }
    return () => {
      clearInterval(poll);
      if (channel) {
        try { supabase.removeChannel(channel); } catch { /* ignore */ }
      }
    };
  }, []);

```

```

    });
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [localId]);

  // 🚨 Alarma al llegar un pedido NUEVO a cocina (id que no estaba antes).
  // No suena en la carga inicial.
  const seenRef = useRef<Set<string>>(new Set());
  const beepInitRef = useRef(false);
  useEffect(() => {
    if (loading) return;
    const ids = pedidos.map((p) => p.id);
    if (!beepInitRef.current) {
      seenRef.current = new Set(ids);
      beepInitRef.current = true;
      return;
    }
    if (ids.some((id) => !seenRef.current.has(id))) playBeep();
    seenRef.current = new Set(ids);
  }, [pedidos, loading]);

  // Desbloquea el audio del WebView de Android al primer toque del usuario.
  useEffect(() => {
    const unlock = () => unlockAudio();
    window.addEventListener("pointerdown", unlock, { once: true });
    return () => window.removeEventListener("pointerdown", unlock);
  }, []);

  const updateEstado = async (id: string, estado_cocina: EstadoPedido) => {
    const { error } = await supabase
      .from("pedidos")
      .update({ estado_cocina, updated_at: new Date().toISOString() })
      .eq("id", id);
    if (error) {
      toast.error("No se pudo actualizar", { description: error.message });
    } else {
      toast.success(`Cocina → ${estado_cocina}`);
    }
  };

  return (
    <div className="min-h-screen bg-background">
      <StaffHeader title="Cocina · KDS" subtitle="Pedidos en tiempo real" />
      <main className="mx-auto max-w-[1800px] p-6">
        {loading ? (
          <div className="text-center text-muted-foreground py-20">Cargando pedidos...</div>
        ) : (
          <div className="grid grid-cols-1 gap-6 md:grid-cols-3">
            {COLUMNS.map((col) => {
              const colPedidos = pedidos.filter((p) => p.estado_cocina === col.key);
              return (
                <section key={col.key} className="flex flex-col">
                  <div className="mb-4 flex items-center justify-between">
                    <h2 className="text-xl font-bold">{col.title}</h2>
                    <span className="rounded-full bg-muted px-3 py-1 text-sm font-medium">{colPedidos.length}</span>
                  </div>
                  <div className="space-y-4">
                    {colPedidos.length === 0 && (
                      <div className="rounded-lg border border-dashed border-border p-8 text-center text-sm text-muted-foreground">
                        Sin pedidos
                      </div>
                    )}
                    {colPedidos.map((p) => {
                      const mins = minutesSince(p.created_at);
                      const its = items[p.id] ?? [];
                      return (
                        <Card key={p.id} className={`border-1-4 p-5 ${borderForAge(mins)} `>
                          <div className="mb-3 flex items-start justify-between">
                            <div>
                              <div className="text-3xl font-bold">#{p.numero_pedido}</div>
                              <div className="flex items-center gap-1 text-sm text-muted-foreground">
                                <Clock className="h-3.5 w-3.5" />
                                {formatDistanceToNowStrict(new Date(p.created_at), { locale: es })}
                              </div>
                            </div>
                            <div className="text-right text-sm font-semibold text-muted-foreground">
                              {Number(p.total).toFixed(2)} €
                            </div>
                          </div>
                          <ul className="mb-4 space-y-1.5 text-base">
                            {its.map((it) => {
                              const comps = it.variante?.includes("\n") ? it.variante.split("\n") : null;
                              return (
                                <li key={it.id}>
                                  <div className="flex items-baseline gap-2">
                                    <span className="font-bold text-primary">{it.cantidad}</span>
                                    <span>{comps ? (it.producto?.nombre ?? "-") : itemLabel(it)}</span>
                                  </div>
                                  {comps && (
                                    <ul className="ml-7 mt-0.5 space-y-0.5 text-sm text-muted-foreground">
                                      {comps.map((c, ci) => (
                                        <li key={ci}>• {c}</li>
                                      ))}
                                    </ul>
                                  )}
                                </li>
                              )
                            )}
                          </ul>
                        </Card>
                      )
                    )}
                  </div>
                </section>
              )
            })}
          </div>
        )}
      </main>
    </div>
  );

```

```

        </ul>
      )}
    </li>
  );
  }}}
  {its.length === 0 && (
    <li className="text-sm italic text-muted-foreground">Sin items de cocina</li>
  )}
</ul>
{p.notas && (
  <div className="mb-3 rounded-md bg-muted p-2 text-sm italic">
    🍴 {p.notas}
  </div>
)}
<div className="flex gap-2">
  {col.prev && (
    <Button variant="outline" size="sm" onClick={() => updateEstado(p.id, col.prev!)}>
      <ArrowLeft className="h-4 w-4" />
      {col.prevLabel}
    </Button>
  )}
  {col.next && (
    <Button size="sm" className="flex-1" onClick={() => updateEstado(p.id, col.next!)}>
      {col.next === "listo" ? <Check className="h-4 w-4" /> : <ArrowRight className="h-4 w-4" />}
      {col.nextLabel}
    </Button>
  )}
</div>
</Card>
  );
  }}}
</div>
</section>
  );
  }}}
</div>
)}
{pedidos.length === 0 && !loading && (
  <div className="mt-12 flex flex-col items-center justify-center text-muted-foreground">
    <ChefHat className="mb-3 h-12 w-12 opacity-40" />
    <p>No hay pedidos activos</p>
  </div>
)}
</main>
</div>
);
};

```

export default Cocina;

```

/* ===== */
/* FICHERO: 100M Dashboard/src/pages/Historico.tsx */
/* ===== */
import { useEffect, useMemo, useState } from "react";
import { supabase, type EstadoPedido } from "@lib/supabase";
import { StaffHeader } from "@components/StaffHeader";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Badge } from "@components/ui/badge";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@components/ui/select";
import {
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from "@components/ui/table";
import {
  Dialog,
  DialogContent,
  DialogHeader,
  DialogTitle,
} from "@components/ui/dialog";
import { toast } from "sonner";
import {
  Calendar as CalendarIcon,
  Euro,
  Receipt,
  ShoppingBag,
  Download,
  Search,
  ArrowUpDown,
  ChevronLeft,
}

```

```

    ChevronRight,
  } from "lucide-react";
import { format } from "date-fns";
import { es } from "date-fns/locale";
import {
  ResponsiveContainer,
  BarChart,
  Bar,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip,
  Legend,
} from "recharts";

type Periodo = "dia" | "mes" | "anio";
type SortKey = "numero_pedido" | "created_at" | "total" | "estado";
type SortDir = "asc" | "desc";

interface Pedido {
  id: string;
  numero_pedido: number;
  estado: EstadoPedido;
  estado_cocina: EstadoPedido | null;
  estado_bebidas: EstadoPedido | null;
  tipo: "cocina" | "bebidas" | "mixto";
  total: number;
  notas: string | null;
  created_at: string;
}

interface Agregado {
  hora: string;
  dia: string;
  mes: string;
  anio: string;
  estado: EstadoPedido;
  tickets: number;
  ingresos: number;
}

interface DetailItem {
  id: string;
  cantidad: number;
  precio_unitario: number;
  destino: "cocina" | "bebidas";
  variante: string | null;
  producto: { nombre: string } | null;
}

/** Nombre del item + variante (ej. "Jarra Cruzcampo · Jarra Sancho"). */
const itemLabel = (it: DetailItem): string => {
  const v = it.variante ? it.variante.replace(/\n/g, " · ") : "";
  return (it.producto?.nombre ?? "-") + (v ? ` · ${v}` : "");
};

const ESTADO_VARIANT: Record<EstadoPedido, { label: string; cls: string }> = {
  pendiente: { label: "Pendiente", cls: "bg-amber-500 text-white" },
  recibido: { label: "Recibido", cls: "bg-[hsl(var(--status-recibido))] text-white" },
  preparando: { label: "Preparando", cls: "bg-[hsl(var(--status-preparando))] text-white" },
  listo: { label: "Listo", cls: "bg-[hsl(var(--status-listo))] text-white" },
  entregado: { label: "Entregado", cls: "bg-[hsl(var(--status-entregado))] text-white" },
  cancelado: { label: "Cancelado", cls: "bg-muted text-muted-foreground" },
};

const ESTADO_FALLBACK = { label: "-", cls: "bg-muted text-muted-foreground" };
const variantFor = (e: EstadoPedido | null | undefined) =>
  (e && ESTADO_VARIANT[e]) || ESTADO_FALLBACK;

const PAGE_SIZES = [10, 25, 50, 100];
const pad = (n: number) => String(n).padStart(2, "0");

const rangeFor = (periodo: Periodo, fecha: Date) => {
  const start = new Date(fecha);
  const end = new Date(fecha);
  if (periodo === "dia") {
    start.setHours(0, 0, 0, 0);
    end.setHours(23, 59, 59, 999);
  } else if (periodo === "mes") {
    start.setDate(1);
    start.setHours(0, 0, 0, 0);
    end.setMonth(end.getMonth() + 1, 0);
    end.setHours(23, 59, 59, 999);
  } else {
    start.setMonth(0, 1);
    start.setHours(0, 0, 0, 0);
    end.setMonth(11, 31);
    end.setHours(23, 59, 59, 999);
  }
  return { start: start.toISOString(), end: end.toISOString() };
};

const todayInputValue = () => {

```

```

const d = new Date();
return `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}`;
};

const Historico = () => {
  const [periodo, setPeriodo] = useState<Periodo>("dia");
  const [fechaStr, setFechaStr] = useState(todayInputValue());
  const [estadoFilter, setEstadoFilter] = useState<string>("entregado");
  const [search, setSearch] = useState("");

  // Agregaciones (desde la vista)
  const [agregados, setAgregados] = useState<Agregado[]>([]);
  const [aggLoading, setAggLoading] = useState(true);

  // Tabla paginada
  const [pedidos, setPedidos] = useState<Pedido[]>([]);
  const [tableLoading, setTableLoading] = useState(true);
  const [totalRows, setTotalRows] = useState(0);
  const [page, setPage] = useState(0);
  const [pageSize, setPageSize] = useState(25);
  const [sortKey, setSortKey] = useState<SortKey>("created_at");
  const [sortDir, setSortDir] = useState<SortDir>("desc");

  const [detailId, setDetailId] = useState<string | null>(null);
  const [detailItems, setDetailItems] = useState<DetailItem[]>([]);
  const [detailLoading, setDetailLoading] = useState(false);

  const fecha = useMemo(() => {
    const [y, m, d] = fechaStr.split("-").map(Number);
    return new Date(y, (m ?? 1) - 1, d ?? 1);
  }, [fechaStr]);

  const range = useMemo(() => rangeFor(periodo, fecha), [periodo, fecha]);

  // Cargar agregaciones desde la vista
  const loadAgregados = async () => {
    setAggLoading(true);
    let query = supabase
      .from("v_pedidos_agregados")
      .select("hora, dia, mes, anio, estado, tickets, ingresos")
      .gte("hora", range.start)
      .lte("hora", range.end);
    if (estadoFilter !== "all") query = query.eq("estado", estadoFilter);
    const { data, error } = await query;
    if (error) {
      toast.error("Error cargando agregados", { description: error.message });
      setAgregados([]);
    } else {
      setAgregados((data ?? []) as Agregado[]);
    }
    setAggLoading(false);
  };

  // Cargar tabla paginada
  const loadTable = async () => {
    setTableLoading(true);
    let q = supabase
      .from("pedidos")
      .select(
        "id, numero_pedido, estado, estado_cocina, estado_bebidas, tipo, total, notas, created_at",
        { count: "exact" },
      )
      .gte("created_at", range.start)
      .lte("created_at", range.end);
    if (estadoFilter !== "all") {
      q = q.or(
        `estado.eq.${estadoFilter},estado_cocina.eq.${estadoFilter},estado_bebidas.eq.${estadoFilter}`,
      );
    }
    if (search.trim()) {
      const n = Number(search.trim());
      if (!Number.isNaN(n)) q = q.eq("numero_pedido", n);
    }
    q = q.order(sortKey, { ascending: sortDir === "asc" });
    const from = page * pageSize;
    const to = from + pageSize - 1;
    const { data, error, count } = await q.range(from, to);
    if (error) {
      toast.error("Error cargando tickets", { description: error.message });
    } else {
      setPedidos((data ?? []) as Pedido[]);
      setTotalRows(count ?? 0);
    }
    setTableLoading(false);
  };

  useEffect(() => {
    loadAgregados();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [periodo, fechaStr, estadoFilter]);

  useEffect(() => {

```

```

    loadTable();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [periodo, fechaStr, estadoFilter, search, page, pageSize, sortKey, sortDir]);

// Reset paginación al cambiar filtros
useEffect(() => {
  setPage(0);
}, [periodo, fechaStr, estadoFilter, search, pageSize]);

const stats = useMemo(() => {
  const total = agregados.reduce((s, a) => s + a.tickets, 0);
  const ingresos = agregados.reduce((s, a) => s + Number(a.ingresos), 0);
  const ticket = total ? ingresos / total : 0;
  return { total, ingresos, ticket };
}, [agregados]);

// Series para gráfico y tabla agregada
const grupos = useMemo(() => {
  const map = new Map<
    string,
    { sortKey: string; label: string; tickets: number; ingresos: number }
  >();
  agregados.forEach((a) => {
    let bucketIso = "";
    if (periodo === "dia") bucketIso = a.hora;
    else if (periodo === "mes") bucketIso = a.dia;
    else bucketIso = a.mes;
    const d = new Date(bucketIso);
    let label = "";
    let key = "";
    if (periodo === "dia") {
      key = pad(d.getHours());
      label = `${pad(d.getHours())}h`;
    } else if (periodo === "mes") {
      key = pad(d.getDate());
      label = pad(d.getDate());
    } else {
      key = pad(d.getMonth() + 1);
      label = format(d, "MMM", { locale: es });
    }
    const cur = map.get(key) ?? { sortKey: key, label, tickets: 0, ingresos: 0 };
    cur.tickets += a.tickets;
    cur.ingresos += Number(a.ingresos);
    map.set(key, cur);
  });
  return Array.from(map.values()).sort((a, b) => a.sortKey.localeCompare(b.sortKey));
}, [agregados, periodo]);

const periodoLabel = useMemo(() => {
  if (periodo === "dia") return format(fecha, "EEEE d 'de' MMMM yyyy", { locale: es });
  if (periodo === "mes") return format(fecha, "MMMM yyyy", { locale: es });
  return format(fecha, "yyyy");
}, [periodo, fecha]);

const openDetail = async (id: string) => {
  setDetailId(id);
  setDetailLoading(true);
  const { data } = await supabase
    .from("pedido_items")
    .select("id, cantidad, precio_unitario, destino, variante, producto:menu_productos(nombre)")
    .eq("pedido_id", id);
  setDetailItems((data ?? []) as unknown as DetailItem[]);
  setDetailLoading(false);
};

const exportCsv = async () => {
  // Exporta TODO el rango actual (no solo la página)
  let q = supabase
    .from("pedidos")
    .select("numero_pedido, estado, total, notas, created_at")
    .gte("created_at", range.start)
    .lte("created_at", range.end);
  if (estadoFilter !== "all") q = q.eq("estado", estadoFilter);
  q = q.order(sortKey, { ascending: sortDir === "asc" });
  const { data, error } = await q;
  if (error) {
    toast.error("Error exportando", { description: error.message });
    return;
  }
  const rows = (data ?? []).map((p) => {
    const d = new Date(p.created_at as string);
    return [
      p.numero_pedido,
      format(d, "yyyy-MM-dd"),
      format(d, "HH:mm"),
      p.estado,
      Number(p.total).toFixed(2),
      ((p.notas as string | null) ?? "").replace(/\r\n;/g, " "),
    ].join(",");
  });
  const csv = ["numero", "fecha", "hora", "estado", "total", "notas"].join(",") + ", " + rows.join("\n");
  const blob = new Blob([csv], { type: "text/csv;charset=utf-8;" });

```

```

const url = URL.createObjectURL(blob);
const a = document.createElement("a");
a.href = url;
a.download = `historico-${periodo}-${fechaStr}.csv`;
a.click();
URL.revokeObjectURL(url);
};

const toggleSort = (key: SortKey) => {
  if (sortKey === key) setSortDir(sortDir === "asc" ? "desc" : "asc");
  else {
    setSortKey(key);
    setSortDir("desc");
  }
};

const detailPedido = pedidos.find((p) => p.id === detailId) ?? null;
const totalPages = Math.max(1, Math.ceil(totalRows / pageSize));

return (
  <div className="min-h-screen bg-background">
    <StaffHeader title="Histórico de tickets" subtitle={periodoLabel} />
    <main className="mx-auto max-w-[1800px] space-y-6 p-6">
      {/* Controles */}
      <Card>
        <CardContent className="flex flex-col gap-3 p-4 md:flex-row md:items-end">
          <div className="flex flex-col gap-1">
            <label className="text-xs uppercase tracking-wider text-muted-foreground">Periodo</label>
            <Select value={periodo} onChange={(v) => setPeriodo(v as Periodo)}>
              <SelectTrigger className="md:w-40"><SelectValue /></SelectTrigger>
              <SelectContent>
                <SelectItem value="dia">Día</SelectItem>
                <SelectItem value="mes">Mes</SelectItem>
                <SelectItem value="anio">Año</SelectItem>
              </SelectContent>
            </Select>
          </div>
          <div className="flex flex-col gap-1">
            <label className="text-xs uppercase tracking-wider text-muted-foreground">
              {periodo === "dia" ? "Día" : periodo === "mes" ? "Mes" : "Año"}
            </label>
            <Input type="date" value={fechaStr} onChange={(e) => setFechaStr(e.target.value)} className="md:w-52" />
          </div>
          <div className="flex flex-col gap-1">
            <label className="text-xs uppercase tracking-wider text-muted-foreground">Estado</label>
            <Select value={estadoFilter} onChange={setEstadoFilter}>
              <SelectTrigger className="md:w-44"><SelectValue /></SelectTrigger>
              <SelectContent>
                <SelectItem value="all">Todos</SelectItem>
                <SelectItem value="entregado">Entregado</SelectItem>
                <SelectItem value="listo">Listo</SelectItem>
                <SelectItem value="preparando">Preparando</SelectItem>
                <SelectItem value="recibido">Recibido</SelectItem>
                <SelectItem value="pendiente">Pendiente</SelectItem>
              </SelectContent>
            </Select>
          </div>
          <div className="relative flex-1">
            <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-muted-foreground" />
            <Input placeholder="Buscar nº pedido..." value={search} onChange={(e) => setSearch(e.target.value)} className="pl-9" />
          </div>
          <Button variant="outline" onClick={exportCsv}>
            <Download className="h-4 w-4" />Exportar CSV
          </Button>
        </CardContent>
      </Card>

      {/* Stats */}
      <div className="grid grid-cols-2 gap-4 md:grid-cols-3">
        <StatCard icon={ShoppingBag} label="Tickets" value={stats.total.toString()} />
        <StatCard icon={Euro} label="Ingresos" value={` ${stats.ingresos.toFixed(2)} €`} />
        <StatCard icon={Receipt} label="Ticket medio" value={` ${stats.ticket.toFixed(2)} €`} />
      </div>

      {/* Gráfico */}
      <Card>
        <CardHeader>
          <CardTitle className="flex items-center gap-2 text-lg">
            <CalendarIcon className="h-5 w-5" />
            {periodo === "dia" ? "Por hora" : periodo === "mes" ? "Por día" : "Por mes"}
          </CardTitle>
        </CardHeader>
        <CardContent>
          {aggLoading ? (
            <div className="py-12 text-center text-muted-foreground">Cargando...</div>
          ) : grupos.length === 0 ? (
            <div className="py-12 text-center text-muted-foreground">Sin datos</div>
          ) : (
            <div className="h-72 w-full">
              <ResponsiveContainer width="100%" height="100%">
                <BarChart data={grupos} margin={{ top: 8, right: 16, bottom: 8, left: 0 }}>
                  <CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" />

```

```

<XAxis dataKey="label" tick={{ fill: "hsl(var(--muted-foreground))", fontSize: 12 }} />
<YAxis yAxisId="left" tick={{ fill: "hsl(var(--muted-foreground))", fontSize: 12 }} />
<YAxis yAxisId="right" orientation="right" tick={{ fill: "hsl(var(--muted-foreground))", fontSize: 12 }} />
<Tooltip
  contentStyle={{
    background: "hsl(var(--card))",
    border: "1px solid hsl(var(--border))",
    borderRadius: 8,
    fontSize: 12,
  }}
  formatter={(v: number, name) =>
    name === "Ingresos" ? [`$${v.toFixed(2)} €`, name] : [v, name]
  }
/>
<Legend wrapperStyle={{ fontSize: 12 }} />
<Bar yAxisId="left" dataKey="tickets" name="Tickets" fill="hsl(var(--primary))" radius={[4, 4, 0, 0]} />
<Bar yAxisId="right" dataKey="ingresos" name="Ingresos" fill="hsl(var(--status-listo))" radius={[4, 4, 0, 0]} />
</BarChart>
</ResponsiveContainer>
</div>
)}
</CardContent>
</Card>

{/* Tabla paginada */}
<Card>
  <CardHeader className="flex flex-row items-center justify-between">
    <CardTitle className="text-lg">Tickets ({totalRows})</CardTitle>
    <div className="flex items-center gap-2 text-sm">
      <span className="text-muted-foreground">Por página</span>
      <Select value={String(pageSize)} onChange={(v) => setPageSize(Number(v))}>
        <SelectTrigger className="w-20"><SelectValue /></SelectTrigger>
        <SelectContent>
          {PAGE_SIZES.map((s) => (
            <SelectItem key={s} value={String(s)}>{s}</SelectItem>
          ))}
        </SelectContent>
      </Select>
    </div>
  </CardHeader>
  <CardContent>
    {tableLoading ? (
      <div className="py-12 text-center text-muted-foreground">Cargando...</div>
    ) : pedidos.length === 0 ? (
      <div className="py-12 text-center text-muted-foreground">Sin tickets</div>
    ) : (
      <>
        <Table>
          <TableHeader>
            <TableRow>
              <SortableHead onClick={() => toggleSort("numero_pedido")} active={sortKey === "numero_pedido"} dir=
{sortDir}>Nº</SortableHead>
              <SortableHead onClick={() => toggleSort("created_at")} active={sortKey === "created_at"} dir={sortDir}>Fecha /
Hora</SortableHead>
              <TableHead>Cocina</TableHead>
              <TableHead>Bebidas</TableHead>
              <SortableHead onClick={() => toggleSort("total")} active={sortKey === "total"} dir={sortDir} className="text-
right">Total</SortableHead>
              <TableHead className="text-right">Acciones</TableHead>
            </TableRow>
          </TableHeader>
          <TableBody>
            {pedidos.map((p) => {
              const d = new Date(p.created_at);
              return (
                <TableRow key={p.id}>
                  <TableCell className="font-bold">#{p.numero_pedido}</TableCell>
                  <TableCell>{format(d, "dd/MM/yyyy HH:mm")}</TableCell>
                  <TableCell>
                    {p.estado_cocina ? (
                      <Badge className={variantFor(p.estado_cocina).cls}>
                        {variantFor(p.estado_cocina).label}
                      </Badge>
                    ) : (
                      <span className="text-xs text-muted-foreground">—</span>
                    )}
                  </TableCell>
                  <TableCell>
                    {p.estado_bebidas ? (
                      <Badge className={variantFor(p.estado_bebidas).cls}>
                        {variantFor(p.estado_bebidas).label}
                      </Badge>
                    ) : (
                      <span className="text-xs text-muted-foreground">—</span>
                    )}
                  </TableCell>
                  <TableCell className="text-right font-semibold">{Number(p.total).toFixed(2)} €</TableCell>
                  <TableCell className="text-right">
                    <Button size="sm" variant="outline" onClick={() => openDetail(p.id)}>Ver</Button>
                  </TableCell>
                </TableRow>
              );
            })}
          </TableBody>
        </Table>
      </>
    )}
  </CardContent>
</Card>

```



```

    ))}
    </TableBody>
  </Table>

  { /* Paginación */ }
  <div className="mt-4 flex items-center justify-between text-sm">
    <div className="text-muted-foreground">
      Página {page + 1} de {totalPages} · {totalRows} tickets
    </div>
    <div className="flex gap-2">
      <Button variant="outline" size="sm" disabled={page === 0} onClick={() => setPage((p) => Math.max(0, p - 1))}>>
        <ChevronLeft className="h-4 w-4" /> Anterior
      </Button>
      <Button variant="outline" size="sm" disabled={page + 1 >= totalPages} onClick={() => setPage((p) => p + 1)}>>
        Siguiente<ChevronRight className="h-4 w-4" />
      </Button>
    </div>
  </div>
</>
})}
</CardContent>
</Card>
</main>

<Dialog open={!detailId} onOpenChange={(o) => !o && setDetailId(null)}>
  <DialogContent>
    <DialogHeader>
      <DialogTitle>
        Pedido #{detailPedido?.numero_pedido} .{" "}
        {detailPedido && format(new Date(detailPedido.created_at), "dd/MM/yyyy HH:mm")}
      </DialogTitle>
    </DialogHeader>
    {detailLoading ? (
      <div className="py-6 text-center text-muted-foreground">Cargando...</div>
    ) : (
      <div className="space-y-3">
        {detailPedido && (
          <div className="flex flex-wrap gap-2 text-xs">
            {detailPedido.estado_cocina && (
              <Badge className={variantFor(detailPedido.estado_cocina).cls}>
                Cocina: {variantFor(detailPedido.estado_cocina).label}
              </Badge>
            )}
            {detailPedido.estado_bebidas && (
              <Badge className={variantFor(detailPedido.estado_bebidas).cls}>
                Bebidas: {variantFor(detailPedido.estado_bebidas).label}
              </Badge>
            )}
          </div>
        )}
      </div>
    )}
    {[["cocina", "bebidas"] as const].map((dest) => {
      const grupo = detallItems.filter((it) => it.destino === dest);
      if (grupo.length === 0) return null;
      return (
        <div key={dest}>
          <div className="mb-1 text-xs font-semibold uppercase tracking-wider text-muted-foreground">
            {dest}
          </div>
          <ul className="divide-y divide-border">
            {grupo.map((it) => (
              <li key={it.id} className="flex items-center justify-between py-2">
                <div>
                  <span className="font-bold">{it.cantidad}</span>{ " "}
                  <span>{itemLabel(it)}</span>
                </div>
                <div className="text-sm text-muted-foreground">
                  {(it.cantidad * Number(it.precio_unitario)).toFixed(2)} €
                </div>
              </li>
            )})}
          </ul>
        </div>
      );
    })}
  </DialogContent>
</Dialog>
</div>
);
};

const SortableHead = ({
  children,
  onClick,

```

```

    active,
    dir,
    className,
  }: {
    children: React.ReactNode;
    onClick: () => void;
    active: boolean;
    dir: SortDir;
    className?: string;
  }) => (
    <TableHead className={className}>
      <button
        type="button"
        onClick={onClick}
        className={`inline-flex items-center gap-1 hover:text-foreground ${active ? "text-foreground" : "text-muted-foreground"}`}
      >
        {children}
        <ArrowUpDown className="h-3 w-3" />
        {active && <span className="text-xs">{dir === "asc" ? "↑" : "↓"}</span>}
      </button>
    </TableHead>
  );

```

```

const StatCard = ({ icon, label, value }: { icon: React.ReactNode; label: string; value: string }) => (
  <Card>
    <CardContent className="flex items-center gap-4 p-5">
      <div className="flex h-12 w-12 items-center justify-center rounded-xl bg-primary/10 text-primary [&svg]:h-5 [&svg]:w-5">
        {icon}
      </div>
      <div>
        <div className="text-xs uppercase tracking-wider text-muted-foreground">{label}</div>
        <div className="text-xl font-bold">{value}</div>
      </div>
    </CardContent>
  </Card>
);

```

```
export default Historico;
```

```

/* ===== */
/* FICHERO: 100M Dashboard/src/pages/Index.tsx */
/* ===== */
import { useEffect } from "react";
import { useNavigate } from "react-router-dom";
import { Card, CardContent } from "@components/ui/card";
import { StaffHeader } from "@components/StaffHeader";
import { useAuth } from "@hooks/useAuth";
import { ChefHat, Receipt } from "lucide-react";

```

```

const Index = () => {
  const navigate = useNavigate();
  const { role, loading } = useAuth();

  useEffect(() => {
    if (loading) return;
    if (role === "cocina") navigate("/cocina", { replace: true });
    else if (role === "caja") navigate("/caja", { replace: true });
    // admin se queda aquí
  }, [role, loading, navigate]);

  if (loading || (role && role !== "admin")) {
    return (
      <div className="flex min-h-screen items-center justify-center bg-background text-muted-foreground">
        Cargando...
      </div>
    );
  }

  return (
    <div className="min-h-screen bg-background">
      <StaffHeader title="Panel admin" subtitle="Selecciona una vista" />
      <main className="mx-auto max-w-4xl p-6">
        <div className="grid grid-cols-1 gap-6 md:grid-cols-2">
          <button onClick={() => navigate("/cocina")} className="text-left">
            <Card className="transition-all hover:scale-[1.02] hover:shadow-lg">
              <CardContent className="flex flex-col items-start gap-4 p-8">
                <div className="flex h-14 w-14 items-center justify-center rounded-2xl bg-primary text-primary-foreground">
                  <ChefHat className="h-7 w-7" />
                </div>
                <div>
                  <h2 className="text-2xl font-bold">Cocina</h2>
                  <p className="text-muted-foreground">KDS estilo tablero · pedidos en tiempo real</p>
                </div>
              </CardContent>
            </Card>
          </button>
          <button onClick={() => navigate("/caja")} className="text-left">
            <Card className="transition-all hover:scale-[1.02] hover:shadow-lg">
              <CardContent className="flex flex-col items-start gap-4 p-8">
                <div className="flex h-14 w-14 items-center justify-center rounded-2xl bg-primary text-primary-foreground">
                  <Receipt className="h-7 w-7" />
                </div>
              </CardContent>
            </Card>
          </button>
        </div>
      </main>
    </div>
  );

```

```

        </div>
        <div>
            <h2 className="text-2xl font-bold">Caja</h2>
            <p className="text-muted-foreground">Dashboard del día · stats y pedidos</p>
        </div>
    </CardContent>
  </Card>
</button>
</div>
</main>
</div>
);
};

export default Index;

/* ===== */
/* FICHERO: 100M Dashboard/src/pages/Login.tsx */
/* ===== */
import { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@/components/ui/card";
import { useAuth } from "@/hooks/useAuth";
import { supabase } from "@/lib/supabase";
import { useStaffLocal } from "@/lib/staff-local";
import { toast } from "sonner";
import { ChefHat } from "lucide-react";

// Acceso del personal con un solo botón: no tienen que recordar contraseña.
const ADMIN_EMAIL = "admin@test.com";
const ADMIN_PASSWORD = "Admin100M!264c6a3a";

const Login = () => {
  const { signIn, user, role, loading } = useAuth();
  const navigate = useNavigate();
  const [submitting, setSubmitting] = useState(false);

  useEffect(() => {
    if (loading) return;
    if (!user || !role) return;
    (async () => {
      const { data } = await supabase
        .from("user_roles")
        .select("role, local_id, locales(id, nombre, ciudad)")
        .eq("user_id", user.id);
      const isAdmin = (data ?? []).some((r) => r.role === "admin");
      const fixed = (data ?? []).find(
        (r) => r.role !== "admin" && r.local_id && r.locales,
      );
      const setLocal = useStaffLocal.getState().setLocal;
      const currentLocal = useStaffLocal.getState().local;

      if (fixed) {
        const raw = fixed.locales as unknown;
        const loc = Array.isArray(raw)
          ? raw[0]
          : (raw as { id: string; nombre: string; ciudad: string | null });
        if (loc) setLocal({ id: loc.id, nombre: loc.nombre, ciudad: loc.ciudad });
        navigate(role === "cocina" ? "/cocina" : "/caja", { replace: true });
        return;
      }

      if (role === "admin" || isAdmin) {
        if (!currentLocal) navigate("/seleccionar-local", { replace: true });
        else navigate("/", { replace: true });
        return;
      }

      if (!currentLocal) navigate("/seleccionar-local", { replace: true });
      else navigate(role === "cocina" ? "/cocina" : "/caja", { replace: true });
    })();
  }, [user, role, loading, navigate]);

  const handleAdmin = async () => {
    setSubmitting(true);
    const { error } = await signIn(ADMIN_EMAIL, ADMIN_PASSWORD);
    setSubmitting(false);
    if (error) {
      toast.error("No se pudo iniciar sesión", { description: error.message });
    } else {
      toast.success("Sesión iniciada");
    }
  };

  return (
    <div className="flex min-h-screen items-center justify-center bg-gradient-to-br from-background to-muted px-4">
      <Card className="w-full max-w-md shadow-xl">
        <CardHeader className="space-y-3 text-center">
          <div className="mx-auto flex h-14 w-14 items-center justify-center rounded-2xl bg-primary text-primary-foreground">
            <ChefHat className="h-7 w-7" />
          </div>
        </CardHeader>
      </Card>
    </div>
  );
};

```

```

        </div>
        <CardTitle className="text-3xl">Staff</CardTitle>
        <CardDescription>Pulsa para entrar al panel</CardDescription>
      </CardHeader>
      <CardContent>
        <Button onClick={handleAdmin} className="w-full" size="lg" disabled={submitting}>
          {submitting ? "Entrando..." : "Admin"}
        </Button>
      </CardContent>
    </Card>
  </div>
);
};

export default Login;

/* ===== */
/* FICHERO: 100M Dashboard/src/pages/NotFound.tsx */
/* ===== */
import { useLocation } from "react-router-dom";
import { useEffect } from "react";

const NotFound = () => {
  const location = useLocation();

  useEffect(() => {
    console.error("404 Error: User attempted to access non-existent route:", location.pathname);
  }, [location.pathname]);

  return (
    <div className="flex min-h-screen items-center justify-center bg-muted">
      <div className="text-center">
        <h1 className="mb-4 text-4xl font-bold">404</h1>
        <p className="mb-4 text-xl text-muted-foreground">Oops! Page not found</p>
        <a href="/" className="text-primary underline hover:text-primary/90">
          Return to Home
        </a>
      </div>
    </div>
  );
};

export default NotFound;

/* ===== */
/* FICHERO: 100M Dashboard/src/pages/StaffLocalPicker.tsx */
/* ===== */
import { useEffect, useMemo, useState } from "react";
import { useNavigate } from "react-router-dom";
import { supabase } from "@lib/supabase";
import { useStaffLocal, type StaffLocal } from "@lib/staff-local";
import { useAuth } from "@hooks/useAuth";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Input } from "@components/ui/input";
import { Button } from "@components/ui/button";
import { MapPin, Search } from "lucide-react";
import { toast } from "sonner";

interface LocalRow {
  id: string;
  nombre: string;
  ciudad: string | null;
  direccion: string | null;
}

const StaffLocalPicker = () => {
  const navigate = useNavigate();
  const { user, role, loading } = useAuth();
  const setLocal = useStaffLocal((s) => s.setLocal);
  const [q, setQ] = useState("");
  const [debounced, setDebounced] = useState("");
  const [rows, setRows] = useState<LocalRow[]>([]);
  const [selected, setSelected] = useState<LocalRow | null>(null);
  const [busy, setBusy] = useState(false);

  useEffect(() => {
    if (!loading && !user) navigate("/login", { replace: true });
  }, [user, loading, navigate]);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(q.trim()), 300);
    return () => clearTimeout(id);
  }, [q]);

  useEffect(() => {
    let cancel = false;
    const run = async () => {
      let query = supabase
        .from("locales")
        .select("id, nombre, ciudad, direccion")
    }
  }

```

```

        .eq("activo", true)
        .limit(50);
    if (debounced) {
        const like = `%${debounced}%`;
        query = query.or(
            `nombre.ilike.${like},ciudad.ilike.${like},direccion.ilike.${like}`,
        );
    }
    const { data, error } = await query;
    if (cancel) return;
    if (error) {
        toast.error("No se pudieron cargar los locales", { description: error.message });
        return;
    }
    setRows((data ?? []) as LocalRow[]);
};
run();
return () => {
    cancel = true;
};
}, [debounced]);

const confirm = () => {
    if (!selected) return;
    setBusy(true);
    const local: StaffLocal = {
        id: selected.id,
        nombre: selected.nombre,
        ciudad: selected.ciudad,
    };
    setLocal(local);
    toast.success(`Local activo: ${local.nombre}`);
    if (role === "cocina") navigate("/cocina", { replace: true });
    else navigate("/caja", { replace: true });
};

const empty = useMemo(() => rows.length === 0, [rows]);

return (
    <div className="min-h-screen bg-gradient-to-br from-background to-muted px-4 py-10">
        <div className="mx-auto max-w-3xl space-y-6">
            <Card>
                <CardHeader>
                    <CardTitle className="flex items-center gap-2">
                        <MapPin className="h-5 w-5 text-primary" />
                        Selecciona el local
                    </CardTitle>
                    <p className="text-sm text-muted-foreground">
                        Necesitas un local activo para operar en Caja o Cocina.
                    </p>
                </CardHeader>
                <CardContent className="space-y-4">
                    <div className="relative">
                        <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-muted-foreground" />
                        <Input
                            placeholder="Buscar por nombre, ciudad o dirección..."
                            value={q}
                            onChange={(e) => setQ(e.target.value)}
                            className="pl-9"
                            autoFocus
                        />
                    </div>
                    <div className="space-y-2">
                        {empty && (
                            <div className="rounded-md border border-dashed p-6 text-center text-sm text-muted-foreground">
                                Sin resultados
                            </div>
                        )}
                        {rows.map((r) => {
                            const active = selected?.id === r.id;
                            return (
                                <button
                                    key={r.id}
                                    type="button"
                                    onClick={() => setSelected(r)}
                                    className={`w-full rounded-lg border p-4 text-left transition-colors ${
                                        active
                                            ? "border-primary bg-primary/5"
                                            : "border-border hover:border-primary/40"
                                    }`}
                                >
                                    <div className="font-bold">{r.nombre}</div>
                                    <div className="text-sm text-muted-foreground">
                                        {[r.direccion, r.ciudad].filter(Boolean).join(" · ") || "-"}
                                    </div>
                                </button>
                            );
                        })}
                    </div>
                </CardContent>
            </Card>
            <div className="flex justify-end">
                <Button onClick={confirm} disabled={!selected || busy}>
                    Confirmar local
                </Button>
            </div>
        </div>
    </div>

```

```

        </Button>
      </div>
    </CardContent>
  </Card>
</div>
</div>
);
};

export default StaffLocalPicker;

/* ===== */
/* FICHERO: 100M Dashboard/src/components/NavLink.tsx */
/* ===== */
import { NavLink as RouterNavLink, NavLinkProps } from "react-router-dom";
import { forwardRef } from "react";
import { cn } from "@lib/utils";

interface NavLinkCompatProps extends Omit<NavLinkProps, "className"> {
  className?: string;
  activeClassName?: string;
  pendingClassName?: string;
}

const NavLink = forwardRef<HTMLAnchorElement, NavLinkCompatProps>(
  ({ className, activeClassName, pendingClassName, to, ...props }, ref) => {
    return (
      <RouterNavLink
        ref={ref}
        to={to}
        className={({ isActive, isPending }) =>
          cn(className, isActive && activeClassName, isPending && pendingClassName)
        }
        {...props}
      />
    );
  },
);

NavLink.displayName = "NavLink";

export { NavLink };

/* ===== */
/* FICHERO: 100M Dashboard/src/components/ProtectedRoute.tsx */
/* ===== */
import { ReactNode } from "react";
import { Navigate } from "react-router-dom";
import { useAuth } from "@hooks/useAuth";
import type { AppRole } from "@lib/supabase";

interface Props {
  children: ReactNode;
  requiredRole?: AppRole | AppRole[];
}

const roleHome = (role: AppRole | null): string => {
  if (role === "caja") return "/caja";
  if (role === "cocina") return "/cocina";
  if (role === "admin") return "/";
  return "/login";
};

export const ProtectedRoute = ({ children, requiredRole }: Props) => {
  const { user, role, loading } = useAuth();

  if (loading) {
    return (
      <div className="flex min-h-screen items-center justify-center bg-background">
        <div className="text-muted-foreground">Cargando...</div>
      </div>
    );
  }

  if (!user) return <Navigate to="/login" replace />;

  if (requiredRole) {
    const allowed = Array.isArray(requiredRole) ? requiredRole : [requiredRole];
    // Admin tiene acceso a todo
    if (role !== "admin" && (!role || !allowed.includes(role))) {
      return <Navigate to={roleHome(role)} replace />;
    }
  }

  return <>{children}</>;
};

/* ===== */
/* FICHERO: 100M Dashboard/src/components/StaffGuard.tsx */

```

```

/* ===== */
import { ReactNode, useEffect, useState } from "react";
import { Navigate } from "react-router-dom";
import { useAuth } from "@hooks/useAuth";
import { useStaffLocal } from "@lib/staff-local";
import { supabase, type AppRole } from "@lib/supabase";

interface Props {
  children: ReactNode;
  role: "caja" | "cocina";
}

export const StaffGuard = ({ children, role }: Props) => {
  const { user, hasRole, loading } = useAuth();
  const local = useStaffLocal((s) => s.local);
  const setLocal = useStaffLocal((s) => s.setLocal);
  const [resolving, setResolving] = useState(true);

  // Si no hay local en store, intenta resolver desde user_roles (staff con local fijo).
  useEffect(() => {
    let cancel = false;
    const run = async () => {
      if (!user || !local) {
        setResolving(false);
        return;
      }
      const { data } = await supabase
        .from("user_roles")
        .select("role, local_id, locales(id, nombre, ciudad)")
        .eq("user_id", user.id);
      if (cancel) return;
      const fixed = (data ?? []).find(
        (r: { role: AppRole; local_id: string | null; locales: unknown }) =>
          r.role !== "admin" && r.local_id && r.locales,
      );
      if (fixed) {
        const raw = fixed.locales as unknown;
        const loc = Array.isArray(raw) ? raw[0] : (raw as { id: string; nombre: string; ciudad: string | null });
        if (loc) setLocal({ id: loc.id, nombre: loc.nombre, ciudad: loc.ciudad });
      }
      setResolving(false);
    };
    run();
    return () => {
      cancel = true;
    };
  }, [user, local, setLocal]);

  if (loading || resolving) {
    return (
      <div className="flex min-h-screen items-center justify-center bg-background">
        <div className="text-muted-foreground">Cargando...</div>
      </div>
    );
  }

  if (!user) return <Navigate to="/login" replace />;
  if (!hasRole(role) && !hasRole("admin")) return <Navigate to="/login" replace />;
  if (!useStaffLocal.getState().local) return <Navigate to="/seleccionar-local" replace />;

  return <>{children}</>;
};

/* ===== */
/* FICHERO: 100M Dashboard/src/components/StaffHeader.tsx */
/* ===== */
import { Logout, MapPin, ChevronDown } from "lucide-react";
import { Button } from "@components/ui/button";
import { useAuth } from "@hooks/useAuth";
import { useLocation, useNavigate } from "react-router-dom";
import { useStaffLocal } from "@lib/staff-local";

interface Props {
  title: string;
  subtitle?: string;
}

export const StaffHeader = ({ title, subtitle }: Props) => {
  const { user, role, signOut } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();
  const isHistorico = location.pathname.startsWith("/historico");
  const isAgotados = location.pathname.startsWith("/agotados");
  const local = useStaffLocal((s) => s.local);
  const isAdmin = role === "admin";

  const handleSignOut = async () => {
    await signOut();
    useStaffLocal.getState().clear();
    navigate("/login", { replace: true });
  };

```

```

const changeLocal = () => {
  if (!isAdmin) return;
  navigate("/seleccionar-local");
};

return (
  <header className="sticky top-0 z-40 border-b border-border bg-card/80 backdrop-blur">
    <div className="mx-auto flex max-w-[1800px] items-center justify-between px-6 py-4">
      <div className="flex items-center gap-4">
        <div>
          <h1 className="text-2xl font-bold tracking-tight">{title}</h1>
          {subtitle && <p className="text-sm text-muted-foreground">{subtitle}</p>}
        </div>
        {local && (
          <button
            type="button"
            onClick={changeLocal}
            disabled={!isAdmin}
            className={`flex items-center gap-2 rounded-full border border-border bg-muted/50 px-3 py-1.5 text-sm ${
              isAdmin ? "cursor-pointer hover:bg-muted" : "cursor-default"
            }`}
          >
            <MapPin className="h-4 w-4 text-primary" />
            <span className="font-medium">{local.nombre}</span>
            {local.ciudad && (
              <span className="text-muted-foreground">· {local.ciudad}</span>
            )}
            {isAdmin && <ChevronDown className="h-3.5 w-3.5 text-muted-foreground" />}
          </button>
        )}
      </div>
      <div className="flex items-center gap-4">
        <div className="text-right">
          <div className="text-sm font-medium">{user?.email}</div>
          <div className="text-xs uppercase tracking-wider text-muted-foreground">{role}</div>
        </div>
        {(role === "caja" || role === "cocina" || role === "admin") && (
          <Button
            variant="outline"
            size="sm"
            onClick={() =>
              isAdmin
                ? navigate(role === "cocina" ? "/cocina" : role === "caja" ? "/caja" : "/")
                : navigate("/agotados")
            }
          >
            {isAdmin ? "Volver" : "Agotados"}
          </Button>
        )}
        {(role === "caja" || role === "admin") && (
          <Button
            variant="outline"
            size="sm"
            onClick={() => navigate(isHistorico ? "/caja" : "/historico")}
          >
            {isHistorico ? "Caja" : "Histórico"}
          </Button>
        )}
        {role === "admin" && (
          <Button variant="outline" size="sm" onClick={() => navigate("/")}>
            Panel
          </Button>
        )}
        <Button variant="ghost" size="sm" onClick={handleSignOut}>
          <LogOut className="h-4 w-4" />
          Salir
        </Button>
      </div>
    </div>
  </header>
);
};

```

```

/* ===== */
/* FICHERO: 100M Dashboard/src/lib/beep.ts */
/* ===== */
// Alarma de "pedido nuevo".
// - En la APK (Capacitor) suena de forma NATIVA (fichero alarm.wav, fuerte y
//   fiable, sin las limitaciones del WebView).
// - En navegador (web) cae a Web Audio.
import { Capacitor } from "@capacitor/core";
import { NativeAudio } from "@capacitor-community/native-audio";

```

```

const ASSET_ID = "alarm-pedido";
let preloadStarted = false;
let nativeReady = false;

```

```

async function ensureNativePreload(): Promise<boolean> {
  if (preloadStarted) return nativeReady;
  preloadStarted = true;

```



```

try {
  await NativeAudio.preload({
    assetId: ASSET_ID,
    assetPath: "public/alarm.wav", // Capacitor copia /public a assets/public
    audioChannelNum: 1,
    isUrl: false,
    volume: 1.0,
  });
  nativeReady = true;
} catch {
  nativeReady = false;
}
return nativeReady;
}

// — Web Audio (respaldo para navegador) —————
let ctx: AudioContext | null = null;
function getCtx(): AudioContext | null {
  if (typeof window === "undefined") return null;
  if (!ctx) {
    const AC = window.AudioContext || (window as unknown as { webkitAudioContext?: typeof AudioContext }).webkitAudioContext;
    if (!AC) return null;
    ctx = new AC();
  }
  return ctx;
}

function playWebBeep(): void {
  const c = getCtx();
  if (!c) return;
  if (c.state === "suspended") c.resume().catch(() => {});
  const limiter = c.createDynamicsCompressor();
  limiter.threshold.value = -6; limiter.ratio.value = 20; limiter.attack.value = 0.002;
  limiter.connect(c.destination);
  const now = c.currentTime;
  const dur = 0.13, gap = 0.07;
  for (let i = 0; i < 12; i++) {
    const freq = i % 2 === 0 ? 1320 : 990;
    const t = now + i * (dur + gap);
    const g = c.createGain();
    g.gain.setValueAtTime(0.0001, t);
    g.gain.exponentialRampToValueAtTime(1.0, t + 0.006);
    g.gain.setValueAtTime(1.0, t + dur - 0.02);
    g.gain.exponentialRampToValueAtTime(0.0001, t + dur);
    g.connect(limiter);
    const osc = c.createOscillator();
    osc.type = "square"; osc.frequency.value = freq;
    osc.connect(g); osc.start(t); osc.stop(t + dur + 0.02);
  }
}

/** Prepara el audio tras un gesto del usuario (necesario en Android). */
export function unlockAudio(): void {
  if (Capacitor.isNativePlatform()) { ensureNativePreload(); return; }
  const c = getCtx();
  if (c && c.state === "suspended") c.resume().catch(() => {});
}

/** Reproduce la alarma (nativa en APK, Web Audio en navegador). */
export function playBeep(): void {
  if (Capacitor.isNativePlatform()) {
    (async () => {
      const ok = await ensureNativePreload();
      if (ok) {
        try {
          await NativeAudio.stop({ assetId: ASSET_ID }).catch(() => {});
          await NativeAudio.play({ assetId: ASSET_ID });
          return;
        } catch {
          /* cae a web audio */
        }
      }
    })();
    playWebBeep();
  }
  return;
}
playWebBeep();
}

/* ===== */
/* FICHERO: 100M Dashboard/src/lib/drink-image.ts */
/* ===== */
import { PRODUCT_IMAGES } from './product-images';
import absolutSprite from '@assets/drinks/absolut-sprite.png';
import absolutVodka from '@assets/drinks/absolut-vodka.png';
import agua from '@assets/drinks/agua.png';
import appletiser from '@assets/drinks/appletiser.png';
import aquarius from '@assets/drinks/aquarius.png';
import bacardiCocacola from '@assets/drinks/bacardi-cocacola.png';
import ballantines from '@assets/drinks/ballantines.png';
import batido from '@assets/drinks/batido.png';

```

```

import beefeater from '@assets/drinks/beefeater.png';
import beefeater00 from '@assets/drinks/beefeater-00.png';
import botellinCruzcampo from '@assets/drinks/botellin-cruzcampo.png';
import cafe from '@assets/drinks/cafe.png';
import cocaCola from '@assets/drinks/coca-cola.png';
import cocaColaZero from '@assets/drinks/coca-cola-zero.png';
import cremaRuavieja from '@assets/drinks/crema-ruavieja.png';
import cruzcampoSinGluten from '@assets/drinks/cruzcampo-sin-gluten.png';
import desperados from '@assets/drinks/desperados.png';
import fantaNaranja from '@assets/drinks/fanta-naranja.png';
import fuzetea from '@assets/drinks/fuzetea.png';
import havanaClub from '@assets/drinks/havana-club.png';
import heineken00 from '@assets/drinks/heineken-00.png';
import infusion from '@assets/drinks/infusion.png';
import jackDanielsCocacola from '@assets/drinks/jack-daniels-cocacola.png';
import monsterBlanco from '@assets/drinks/monster-blanco.png';
import monsterMangoLoco from '@assets/drinks/monster-mango-loco.png';
import monsterVerde from '@assets/drinks/monster-verde.png';
import paulaner from '@assets/drinks/paulaner.png';
import petroni from '@assets/drinks/petroni.png';
import sprite from '@assets/drinks/sprite.png';
import vinoCopa from '@assets/drinks/vino-copa.png';
import vinoBlanco from '@assets/drinks/vino-blanco.png';
import zumoNaranja from '@assets/drinks/zumo-naranja.png';
import jarraCerveza from '@assets/drinks/jarra-cerveza.png';
import jarraTintoVerano from '@assets/drinks/jarra-tinto-verano.png';
import jarraLadronManzanas from '@assets/drinks/jarra-ladron-manzanas.png';

const normalize = (value: string) =>
  value
    .normalize('NFD')
    .replace(/[\u0300-\u036f]/g, '')
    .toLowerCase()
    .trim();

const MATCHERS: Array<{ test: RegExp; src: string }> = [
  { test: /absolut.*sprite|sprite.*absolut/, src: absolutSprite },
  { test: /absolut.*vodka|vodka.*absolut/, src: absolutVodka },
  { test: /agua|mineral/, src: agua },
  { test: /appletiser/, src: appletiser },
  { test: /aquarius/, src: aquarius },
  { test: /bacardi.*(coca|cola|coke)|(coca|cola|coke).*bacardi/, src: bacardiCocacola },
  { test: /ballantine/, src: ballantines },
  { test: /batido/, src: batido },
  { test: /beefeater.*0[.]]?0[0[.]]?0.*beefeater|beefeater.*sin alcohol|sin alcohol.*beefeater/, src: beefeater00 },
  { test: /beefeater/, src: beefeater },
  { test: /cafe|cortado|solo|americano|capuccino|capuchino|descafeinado/, src: cafe },
  { test: /coca[\s-]?cola.*zero|zero.*coca[\s-]?cola|coca[\s-]?cola zero/, src: cocaColaZero },
  { test: /coca[\s-]?cola/, src: cocaCola },
  { test: /crema.*ruavieja|ruavieja.*crema|^ruavieja$/, src: cremaRuavieja },
  { test: /(jack.*daniel|bjack\b).*(coca|cola|coke)|(coca|cola|coke).*(jack.*daniel|bjack\b)/, src: jackDanielsCocacola },
  { test: /havana.*club/, src: havanaClub },
  { test: /monster.*mango.*loco|mango.*loco.*monster/, src: monsterMangoLoco },
  { test: /monster.*(verde|original|green|energy)|^monster$/, src: monsterVerde },
  { test: /paulaner/, src: paulaner },
  { test: /petroni|aperitivo.*naranja|sprit/, src: petroni },
  { test: /\bsprite\b/, src: sprite },
  { test: /vino.*blanco|blanco.*vino|verdejo|sauvignon|albarino|chardonnay/, src: vinoBlanco },
  { test: /tinto de verano|ladron de verano/, src: jarraTintoVerano },
  { test: /ladron.*manzana|manzana.*ladron|cider/, src: jarraLadronManzanas },
  { test: /vino.*tinto|vino.*rosado|ribera|rioja|rosado|copa de vino|^vino$/, src: vinoCopa },
  { test: /zumo|naranja exprimida/, src: zumoNaranja },
  { test: /monster.*blanco|monster.*ultra|ultra.*monster|monster.*zero/, src: monsterBlanco },
  { test: /fanta/, src: fantaNaranja },
  { test: /fuze\s*tea|fuzetea/, src: fuzetea },
  { test: /heineken.*0[.]]?0[0[.]]?0.*heineken|heineken.*sin alcohol|sin alcohol.*heineken/, src: heineken00 },
  { test: /desperados/, src: desperados },
  { test: /cruzcampo.*sin gluten|sin gluten.*cruzcampo/, src: botellinCruzcampo },
  { test: /botellin.*cruzcampo|cruzcampo.*botellin/, src: botellinCruzcampo },
  { test: /infusion|te caliente|manzanilla|poleo|tila/, src: infusion },
  // Catch-all cerveza (excluye ladrón de manzanas y tinto de verano)
  { test: /^(?!.*manzana)(?!.*tinto de verano).*(cerveza|cana|jarra|estrella|mahou|amstel|alhambra|san miguel|cruzcampo|radler|clara|rubia|tostada|negra|bipa|b|lager|pilsner|weiss|weizen)/, src: jarraCerveza },
];

export function getDrinkImage(name: string, fotoUrl?: string | null) {
  // Check static product image map first (montaditos, aperitivos, raciones)
  if (PRODUCT_IMAGES[name]) return PRODUCT_IMAGES[name];
  const normalized = normalize(name);
  const matched = MATCHERS.find((entry) => entry.test.test(normalized));
  return matched?.src ?? fotoUrl ?? null;
}

/* ===== */
/* FICHERO: 100M Dashboard/src/lib/ingrediente-images.ts */
/* ===== */
// Imágenes de ingredientes para Agotados (generado, fondo transparente u2net).
export const INGREDIENTE_IMAGES: Record<string, string> = {
  "Aceite de oliva": "/assets/img/ingredientes/aceite-de-oliva.png",

```

```

"Aceitunas": "/assets/img/ingredientes/aceitunas.png",
"Atún": "/assets/img/ingredientes/atun.png",
"Bacon ahumado": "/assets/img/ingredientes/bacon-ahumado.png",
"Calamar": "/assets/img/ingredientes/calamar.png",
"Carne de burger": "/assets/img/ingredientes/carne-de-burger.png",
"Carrillera": "/assets/img/ingredientes/carrillera.png",
"Cebolla crujiente": "/assets/img/ingredientes/cebolla-crujiente.png",
"Cheddar": "/assets/img/ingredientes/cheddar.png",
"Chorizo": "/assets/img/ingredientes/chorizo.png",
"Guacamole": "/assets/img/ingredientes/guacamole.png",
"Huevo hilado": "/assets/img/ingredientes/huevo-hilado.png",
"Jamón Gran Reserva": "/assets/img/ingredientes/jamon-gran-reserva.png",
"Jamón cocido": "/assets/img/ingredientes/jamon-cocido.png",
"Kétchup": "/assets/img/ingredientes/ketchup.png",
"Lechuga": "/assets/img/ingredientes/lechuga.png",
"Lomo al ajillo": "/assets/img/ingredientes/lomo-al-ajillo.png",
"Mantequilla": "/assets/img/ingredientes/mantequilla.png",
"Mayonesa": "/assets/img/ingredientes/mayonesa.png",
"Maíz": "/assets/img/ingredientes/maiz.png",
"Mojo picón": "/assets/img/ingredientes/mojo-picon.png",
"Mostaza": "/assets/img/ingredientes/mostaza.png",
"Mostaza y miel": "/assets/img/ingredientes/mostaza-y-miel.png",
"Mozzarella / Mozzamix": "/assets/img/ingredientes/mozzarella.png",
"Nachos TexMex": "/assets/img/ingredientes/nachos-texmex.png",
"Orégano": "/assets/img/ingredientes/oregano.png",
"Pan burger": "/assets/img/ingredientes/pan-burger.png",
"Pan clásico 100M": "/assets/img/ingredientes/pan-clasico-100m.png",
"Pan gourmet": "/assets/img/ingredientes/pan-gourmet.png",
"Pan perrito": "/assets/img/ingredientes/pan-perrito.png",
"Pan piadina": "/assets/img/ingredientes/pan-piadina.png",
"Patatas paja": "/assets/img/ingredientes/patatas-paja.png",
"Pepperoni": "/assets/img/ingredientes/pepperoni.png",
"Picatostes": "/assets/img/ingredientes/picatostes.png",
"Pintxo donostiarra": "/assets/img/ingredientes/pintxo-donostiarra.png",
"Pollo": "/assets/img/ingredientes/pollo.png",
"Pollo Kebab": "/assets/img/ingredientes/pollo-kebab.png",
"Pulled pork": "/assets/img/ingredientes/pulled-pork.png",
"Queso gorgonzola": "/assets/img/ingredientes/queso-gorgonzola.png",
"Queso madurado": "/assets/img/ingredientes/queso-madurado.png",
"Salchicha / Hot dog": "/assets/img/ingredientes/salchicha.png",
"Salmón ahumado": "/assets/img/ingredientes/salmon-ahumado.png",
"Salsa 100M": "/assets/img/ingredientes/salsa-100m.png",
"Salsa BBQ": "/assets/img/ingredientes/salsa-bbq.png",
"Salsa alioli": "/assets/img/ingredientes/salsa-alioli.png",
"Salsa brava": "/assets/img/ingredientes/salsa-brava.png",
"Salsa pomodoro / pizza": "/assets/img/ingredientes/salsa-pomodoro.png",
"Tomate": "/assets/img/ingredientes/tomate.png",
"Torreznos": "/assets/img/ingredientes/torreznos.png",
"Tortilla de patatas": "/assets/img/ingredientes/tortilla-de-patatas.png",
};

/* ===== */
/* FICHERO: 100M Dashboard/src/lib/product-images.ts */
/* ===== */
import aceitunaasImg from '@assets/productos/aceitunas.png';
import cheesyImg from '@assets/productos/cheesy-cheetos-bites.png';
import cucuruchoImg from '@assets/productos/cucurucho-de-patatas-chip.png';
import gildaAnchoaImg from '@assets/productos/gilda-anchoa.png';
import gildaBoqueronImg from '@assets/productos/gilda-boqueron.png';
import nachosBaconImg from '@assets/productos/nachos-cheddar-bacon.png';
import nachosGuacaImg from '@assets/productos/nachos-cheddar-guacamole.png';
import palomitasPolloImg from '@assets/productos/palomitas-de-pollo.png';
import palomitasGoudaImg from '@assets/productos/palomitas-de-queso-gouda.png';
import patatas4SalsasImg from '@assets/productos/patatas-fritas-4-salsas.png';
import patasBravioliImg from '@assets/productos/patatas-fritas-bravioli.png';
import salchichasImg from '@assets/productos/salchichas-4-salsas.png';
import alitasBbqImg from '@assets/productos/alitas-bbq.png';
import alitasBravaImg from '@assets/productos/alitas-brava.png';
import croquetasJamonImg from '@assets/productos/croquetas-de-jamon.png';
import croquetasMacImg from '@assets/productos/croquetas-mac-cheese.png';
import patasBravasImg from '@assets/productos/patatas-bravas.png';
import tirasPolloImg from '@assets/productos/tiras-de-pollo.png';
import torreznosImg from '@assets/productos/torreznos.png';
import camperaImg from '@assets/productos/campera.png';
import texmexImg from '@assets/productos/texmex.png';

export const PRODUCT_IMAGES: Record<string, string> = {
  'Aceitunas de la abuela': aceitunaasImg,
  'Cheesy Cheetos® Bites': cheesyImg,
  'Cucurucho de patatas chips': cucuruchoImg,
  'Gildas': gildaBoqueronImg,
  'Nachos': nachosBaconImg,
  'Palomitas de pollo': palomitasPolloImg,
  'Palomitas de queso gouda': palomitasGoudaImg,
  'Patatas fritas 4 salsas': patatas4SalsasImg,
  'Patatas fritas con cheddar y bacon ahumado': '/assets/img/aperitivos/patatas-fritas-cheddar-bacon.png',
  'Salchichas 4 salsas': salchichasImg,
  'Alitas de pollo': alitasBbqImg,
  'Croquetas de Jamón': croquetasJamonImg,
  'Croquetas de Mac&Cheese': croquetasMacImg,
  'Patatas fritas Bravioli': patasBravioliImg,

```

```

'Patatas bravas': patasBravasImg,
'Tiras de pollo': tirasPolloImg,
'Torreznos': torreznosImg,
'Campera': camperaImg,
'TexMex': texmexImg,
_gildaAnchoa: gildaAnchoaImg,
_gildaBoqueron: gildaBoqueronImg,
_alitasBbq: alitasBbqImg,
_alitasBrava: alitasBravaImg,
_nachosBacon: nachosBaconImg,
_nachosGuaca: nachosGuacaImg,

// — Montaditos —
'3 Quesos: queso madurado, queso mozzarella, queso gorgonzola y orégano': '/assets/img/montaditos/montypizza-3-quesos-con-queso-madurado-queso-mozzarella-queso-gorgonzola-y-oregano.png',
'Atún, huevo hilado y mayonesa': '/assets/img/montaditos/montygourmet-de-atun-huevo-hilado-y-mayonesa.png',
'Atún, tomate y mayonesa': '/assets/img/montaditos/montygourmet-de-atun-tomate-y-mayonesa.png',
'BBQ: bacon ahumado, queso mozzarella, cebolla crujiente y salsa BBQ': '/assets/img/montaditos/montypizza-bbq-con-bacon-ahumado-queso-mozzarella-cebolla-crujiente-y-salsa-bbq.png',
'Bacon ahumado y queso madurado': '/assets/img/montaditos/bacon-ahumado-y-queso-madurado.png',
'Bacon ahumado, cebolla crujiente y salsa 100M': '/assets/img/montaditos/bacon-ahumado-cebolla-crujiente-y-salsa-naranja.png',
'Bacon ahumado, patatas paja y mayonesa': '/assets/img/montaditos/bacon-ahumado-patatas-paja-y-mayonesa.png',
'Bacon ahumado, tomate y mayonesa': '/assets/img/montaditos/bacon-ahumado-tomate-y-mayonesa.png',
'Bacon ahumado, tomate y queso gorgonzola': '/assets/img/montaditos/bacon-ahumado-tomate-y-queso-gorgonzola.png',
'Bacon ahumado, tomate y queso madurado': '/assets/img/montaditos/bacon-ahumado-tomate-y-queso-madurado.png',
'Burger, bacon ahumado y salsa cheddar': '/assets/img/montaditos/montyburger-de-burger-bacon-ahumado-y-salsa-cheddar.png',
'Burger, guacamole y bacon ahumado': '/assets/img/montaditos/montyburger-de-burger-guacamole-y-bacon-ahumado.png',
'Burger, queso madurado y mojo picón': '/assets/img/montaditos/montyburger-de-burger-queso-madurado-y-mojito-picon.png',
'Burger, queso madurado y pepperoni': '/assets/img/montaditos/montyburger-de-burger-queso-madurado-y-pepperoni.png',
'Burger, queso madurado, tomate y mayonesa': '/assets/img/montaditos/montyburger-de-burger-queso-madurado-tomate-y-mayonesa.png',
'Calamarcitos y guacamole': '/assets/img/montaditos/calamarcitos-y-guacamole.png',
'Calamarcitos y mayonesa': '/assets/img/montaditos/calamarcitos-y-mayonesa.png',
'Calamarcitos y salsa 100M': '/assets/img/montaditos/calamarcitos-y-salsa-naranja.png',
'Calamarcitos y salsa alioli': '/assets/img/montaditos/calamarcitos-y-salsa-alioli.png',
'Calamarcitos, salsa brava y mayonesa': '/assets/img/montaditos/calamarcitos-salsa-brava-y-mayonesa.png',
'Calamarcitos, tomate y mayonesa': '/assets/img/montaditos/calamarcitos-tomate-y-mayonesa.png',
'Carrillera al vino tinto': '/assets/img/montaditos/carrillera-al-vino-tinto.png',
'Carrillera al vino tinto y bacon ahumado': '/assets/img/montaditos/carrillera-al-vino-tinto-y-bacon-ahumado.png',
'Carrillera al vino tinto y cebolla crujiente': '/assets/img/montaditos/carrillera-al-vino-tinto-y-cebolla-crujiente.png',
'Carrillera al vino tinto y patatas paja': '/assets/img/montaditos/carrillera-al-vino-tinto-y-patatas-paja.png',
'Carrillera al vino tinto y salsa alioli': '/assets/img/montaditos/carrillera-al-vino-tinto-y-salsa-alioli.png',
'Carrillera al vino tinto y tomate': '/assets/img/montaditos/carrillera-al-vino-tinto-y-tomate.png',
'Chorizo parrillero y guacamole': '/assets/img/montaditos/chorizo-parrillero-y-guacamole.png',
'Chorizo parrillero y queso gorgonzola': '/assets/img/montaditos/chorizo-parrillero-y-queso-gorgonzola.png',
'Chorizo parrillero y salsa BBQ': '/assets/img/montaditos/chorizo-parrillero-y-salsa-bbq.png',
'Chorizo parrillero y salsa brava': '/assets/img/montaditos/chorizo-parrillero-y-salsa-brava.png',
'Hot dog, cebolla crujiente y mojo picón': '/assets/img/montaditos/montyperro-de-hot-dog-cebolla-crujiente-y-mojito-picon.png',
'Hot dog, cebolla crujiente y salsa 100M': '/assets/img/montaditos/montyperro-de-hot-dog-cebolla-crujiente-y-salsa-100.png',
'Hot dog, guacamole y salsa cheddar': '/assets/img/montaditos/montyperro-de-hot-dog-guacamole-y-salsa-cheddar.png',
'Hot dog, ketchup y mayonesa': '/assets/img/montaditos/montyperro-de-hot-dog-ketchup-y-mayonesa.png',
'Hot dog, patatas paja y salsa alioli': '/assets/img/montaditos/montyperro-de-hot-dog-patatas-paja-y-salsa-alioli.png',
'Jamón Gran Reserva y aceite de oliva': '/assets/img/montaditos/jamon-gran-reserva-y-aceite-de-oliva.png',
'Jamón Gran Reserva y mantequilla': '/assets/img/montaditos/jamon-gran-reserva-y-mantequilla.png',
'Jamón Gran Reserva y tomate': '/assets/img/montaditos/jamon-gran-reserva-y-tomate.png',
'Jamón Gran Reserva y tortilla de patatas': '/assets/img/montaditos/jamon-gran-reserva-y-tortilla-de-patatas.png',
'Jamón Gran Reserva, tomate y patatas paja': '/assets/img/montaditos/jamon-gran-reserva-tomate-y-patatas-paja.png',
'Jamón cocido, queso madurado y mantequilla': '/assets/img/montaditos/montygourmet-de-jamon-cocido-queso-madurado-y-mantequilla.png',
'Jamón cocido, queso madurado, tomate y mayonesa': '/assets/img/montaditos/montygourmet-de-jamon-cocido-queso-madurado-tomate-y-mayonesa.png',
'Lomo al ajillo y mojo picón': '/assets/img/montaditos/lomo-al-ajillo-y-mojito-picon.png',
'Lomo al ajillo y queso gorgonzola': '/assets/img/montaditos/lomo-al-ajillo-y-queso-gorgonzola.png',
'Lomo al ajillo y queso madurado': '/assets/img/montaditos/lomo-al-ajillo-y-queso-madurado.png',
'Lomo al ajillo y salsa 100M': '/assets/img/montaditos/lomo-al-ajillo-y-salsa-100-m.png',
'Lomo al ajillo, bacon ahumado y salsa alioli': '/assets/img/montaditos/lomo-al-ajillo-bacon-ahumado-y-salsa-alioli.png',
'Lomo al ajillo, tomate y mayonesa': '/assets/img/montaditos/lomo-al-ajillo-tomate-y-mayonesa.png',
'Lomo al ajillo, tomate y patatas paja': '/assets/img/montaditos/lomo-al-ajillo-tomate-y-patatas-paja.png',
'Montycookie chocolate y sirope de chocolate': '/assets/img/montaditos/montycookie-chocolate-y-sirope-de-chocolate.png',
'Montycookie chocolate y sirope de pistacho': '/assets/img/montaditos/montycookie-chocolate-y-sirope-de-pistacho.png',
'Montycookie doble chocolate y sirope de caramelo toffee': '/assets/img/montaditos/montycookie-doble-chocolate-y-sirope-de-caramelo-toffee.png',
'Pepperoni: pepperoni, queso mozzarella, salsa pizza y orégano': '/assets/img/montaditos/montypizza-pepperoni-con-pepperoni-queso-mozzarella-salsa-pizza-y-oregano.png',
'Piadina de jamón Gran Reserva, queso mozzarella y orégano': '/assets/img/montaditos/piadina-de-jamon-gran-reserva-queso-mozzarella-y-oregano.png',
'Piadina de jamón cocido y queso mozzarella': '/assets/img/montaditos/piadina-de-jamon-cocido-y-queso-mozzarella.png',
'Piadina de jamón cocido, queso madurado y tomate': '/assets/img/montaditos/piadina-de-jamon-cocido-queso-madurado-y-tomate.png',
'Piadina de pepperoni y queso mozzarella': '/assets/img/montaditos/piadina-de-pepperoni-y-queso-mozzarella.png',
'Piadina de pollo, tomate, queso mozzarella y orégano': '/assets/img/montaditos/piadina-de-pollo-tomate-queso-mozzarella-y-oregano.png',
'Pintxo donostiarrá y atún': '/assets/img/montaditos/montygourmet-de-pintxo-donostiarrá-y-atun.png',
'Pintxo donostiarrá y huevo hilado': '/assets/img/montaditos/montygourmet-de-pintxo-donostiarrá-y-huevo-hilado.png',
'Pollo y queso madurado': '/assets/img/montaditos/pollo-y-queso-madurado.png',
'Pollo y salsa alioli': '/assets/img/montaditos/pollo-y-salsa-alioli.png',
'Pollo, bacon ahumado y mayonesa': '/assets/img/montaditos/pollo-bacon-ahumado-y-mayonesa.png',
'Pollo, patatas paja y salsa BBQ': '/assets/img/montaditos/pollo-patatas-paja-y-salsa-bbq.png',
'Pollo, patatas paja y salsa de mostaza y miel': '/assets/img/montaditos/pollo-patatas-paja-y-salsa-de-mostaza-y-miel.png',
'Pollo, tomate y mojo picón': '/assets/img/montaditos/pollo-tomate-y-mojito-picon.png',
'Pollo: pollo kebab, queso mozzarella, salsa pizza y orégano': '/assets/img/montaditos/montypizza-de-pollo-kebab-queso-mozzarella-salsa-pizza-y-oregano.png',
'Pollo kebab y salsa BBQ': '/assets/img/montaditos/pollo-kebab-y-salsa-bbq.png',
'Pollo kebab y salsa cheddar': '/assets/img/montaditos/pollo-kebab-y-salsa-cheddar.png',

```

```

'Pollo kebab y tomate': '/assets/img/montaditos/pollo-kebab-y-tomate.png',
'Pollo kebab, bacon ahumado y mayonesa': '/assets/img/montaditos/pollo-kebab-bacon-ahumado-y-mayonesa.png',
'Pollo kebab, patatas paja y salsa BBQ': '/assets/img/montaditos/pollo-kebab-patatas-paja-y-salsa-bbq.png',
'Pollo kebab, tomate y salsa 100M': '/assets/img/montaditos/pollo-kebab-tomate-y-salsa-naranja.png',
'Pulled Pork: pulled pork BBQ, queso mozzarella, cebolla crujiente y salsa BBQ': '/assets/img/montaditos/montypizza-pulled-pork-con-
pulled-pork-bbq-queso-mozzarella-cebolla-crujiente-y-salsa-bbq.png',
'Pulled pork BBQ': '/assets/img/montaditos/pulled-pork-bbq.png',
'Pulled pork BBQ y bacon ahumado': '/assets/img/montaditos/pulled-pork-bbq-y-bacon-ahumado.png',
'Pulled pork BBQ y cebolla crujiente': '/assets/img/montaditos/pulled-pork-bbq-y-cebolla-crujiente.png',
'Pulled pork BBQ y patatas paja': '/assets/img/montaditos/pulled-pork-bbq-y-patatas-paja.png',
'Pulled pork BBQ y salsa brava': '/assets/img/montaditos/pulled-pork-bbq-y-salsa-brava.png',
'Pulled pork BBQ y salsa cheddar': '/assets/img/montaditos/pulled-pork-bbq-y-salsa-cheddar.png',
'Salmón ahumado y guacamole': '/assets/img/montaditos/salmon-ahumado-y-guacamole.png',
'Salmón ahumado y huevo hilado': '/assets/img/montaditos/montygourmet-de-salmon-ahumado-y-huevo-hilado.png',
'Salmón ahumado y pintxo donostiarra': '/assets/img/montaditos/montygourmet-de-salmon-ahumado-y-pintxo-donostiarra.png',
'Salmón ahumado y queso gorgonzola': '/assets/img/montaditos/salmon-ahumado-y-queso-gorgonzola.png',
'Salmón ahumado y salsa de mostaza y miel': '/assets/img/montaditos/salmon-ahumado-y-salsa-de-mostaza-y-miel.png',
'Salmón ahumado y tomate': '/assets/img/montaditos/salmon-ahumado-y-tomate.png',
'Tortilla de patatas y mojo picón': '/assets/img/montaditos/tortilla-de-patatas-y-mojo-picon.png',
'Tortilla de patatas y queso madurado': '/assets/img/montaditos/tortilla-de-patatas-y-queso-madurado.png',
'Tortilla de patatas y tomate': '/assets/img/montaditos/tortilla-de-patatas-y-tomate.png',
'Tortilla de patatas, bacon ahumado y salsa alioli': '/assets/img/montaditos/tortilla-de-patatas-bacon-ahumado-y-salsa-alioli.png',
'Tortilla de patatas, cebolla crujiente y salsa BBQ': '/assets/img/montaditos/tortilla-de-patatas-cebolla-crujiente-y-salsa-
bbq.png',
'Tortilla de patatas, patatas paja y salsa 100M': '/assets/img/montaditos/tortilla-de-patatas-patatas-paja-y-salsa-naranja.png',
'Tortilla de patatas, tomate y mayonesa': '/assets/img/montaditos/tortilla-de-patatas-tomate-y-mayonesa.png',
'Torreznos y mayonesa': '/assets/img/montaditos/torreznos-y-mayonesa.png',
'Torreznos y salsa 100M': '/assets/img/montaditos/torreznos-y-salsa-naranja.png',
'Torreznos y salsa alioli': '/assets/img/montaditos/torreznos-y-salsa-alioli.png',
'Torreznos y salsa brava': '/assets/img/montaditos/torreznos-y-salsa-brava.png',
};

// Productos que comparten nombre con otros pero tienen pan diferente (MontyGourmet).
// Este mapa sobrescribe PRODUCT_IMAGES cuando el numero coincide.
export const MONTADITO_IMAGES_BY_NUMERO: Record<string, string> = {
  '91': '/assets/img/montaditos/montygourmet-de-tortilla-de-patatas-tomate-y-mayonesa.png',
  '100': '/assets/img/montaditos/montygourmet-de-jamon-gran-reserva-y-mantequilla.png',
};

/* ===== */
/* FICHERO: 100M Dashboard/src/lib/staff-local.ts */
/* ===== */
import { create } from "zustand";
import { persist } from "zustand/middleware";

export type StaffLocal = {
  id: string;
  nombre: string;
  ciudad: string | null;
};

type State = {
  local: StaffLocal | null;
  setLocal: (l: StaffLocal | null) => void;
  clear: () => void;
};

export const useStaffLocal = create<State>()((
  persist(
    (set) => ({
      local: null,
      setLocal: (l) => set({ local: l }),
      clear: () => set({ local: null }),
    }),
    { name: "staff-active-local" },
  ),
));

/* ===== */
/* FICHERO: 100M Dashboard/src/lib/supabase.ts */
/* ===== */
import { createClient } from "@supabase/supabase-js";

const SUPABASE_URL = import.meta.env.VITE_SUPABASE_URL as string;
const SUPABASE_ANON_KEY = import.meta.env.VITE_SUPABASE_ANON_KEY as string;

export const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY, {
  auth: {
    persistSession: true,
    autoRefreshToken: true,
    storage: localStorage,
  },
});

export type AppRole = "cocina" | "caja" | "admin";
export type EstadoPedido =
  | "pendiente"
  | "recibido"
  | "preparando"
  | "listo"

```

```

    | "entregado"
    | "cancelado";
export type TipoPedido = "cocina" | "bebidas" | "mixto";
export type DestinoItem = "cocina" | "bebidas";

/* ===== */
/* FICHERO: 100M Dashboard/src/lib/Utils.ts */
/* ===== */
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
    return twMerge(clsx(inputs));
}

/* ===== */
/* FICHERO: 100M Dashboard/src/hooks/use-toast.ts */
/* ===== */
import * as React from "react";

import type { ToastActionElement, ToastProps } from "@components/ui/toast";

const TOAST_LIMIT = 1;
const TOAST_REMOVE_DELAY = 1000000;

type ToasterToast = ToastProps & {
    id: string;
    title?: React.ReactNode;
    description?: React.ReactNode;
    action?: ToastActionElement;
};

const actionTypes = {
    ADD_TOAST: "ADD_TOAST",
    UPDATE_TOAST: "UPDATE_TOAST",
    DISMISS_TOAST: "DISMISS_TOAST",
    REMOVE_TOAST: "REMOVE_TOAST",
} as const;

let count = 0;

function genId() {
    count = (count + 1) % Number.MAX_SAFE_INTEGER;
    return count.toString();
}

type ActionType = typeof actionTypes;

type Action =
    | {
        type: ActionType["ADD_TOAST"];
        toast: ToasterToast;
    }
    | {
        type: ActionType["UPDATE_TOAST"];
        toast: Partial<ToasterToast>;
    }
    | {
        type: ActionType["DISMISS_TOAST"];
        toastId?: ToasterToast["id"];
    }
    | {
        type: ActionType["REMOVE_TOAST"];
        toastId?: ToasterToast["id"];
    };

interface State {
    toasts: ToasterToast[];
}

const toastTimeouts = new Map<string, ReturnType<typeof setTimeout>>();

const addToRemoveQueue = (toastId: string) => {
    if (toastTimeouts.has(toastId)) {
        return;
    }

    const timeout = setTimeout(() => {
        toastTimeouts.delete(toastId);
        dispatch({
            type: "REMOVE_TOAST",
            toastId: toastId,
        });
    }, TOAST_REMOVE_DELAY);

    toastTimeouts.set(toastId, timeout);
};

export const reducer = (state: State, action: Action): State => {
    switch (action.type) {

```

```

case "ADD_TOAST":
  return {
    ...state,
    toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
  };

case "UPDATE_TOAST":
  return {
    ...state,
    toasts: state.toasts.map((t) => (t.id === action.toast.id ? { ...t, ...action.toast } : t)),
  };

case "DISMISS_TOAST": {
  const { toastId } = action;

  // ! Side effects ! - This could be extracted into a dismissToast() action,
  // but I'll keep it here for simplicity
  if (toastId) {
    addToRemoveQueue(toastId);
  } else {
    state.toasts.forEach((toast) => {
      addToRemoveQueue(toast.id);
    });
  }

  return {
    ...state,
    toasts: state.toasts.map((t) =>
      t.id === toastId || toastId === undefined
        ? {
            ...t,
            open: false,
          }
        : t,
    ),
  };
}

case "REMOVE_TOAST":
  if (action.toastId === undefined) {
    return {
      ...state,
      toasts: [],
    };
  }
  return {
    ...state,
    toasts: state.toasts.filter((t) => t.id !== action.toastId),
  };
}
};

const listeners: Array<(state: State) => void> = [];

let memoryState: State = { toasts: [] };

function dispatch(action: Action) {
  memoryState = reducer(memoryState, action);
  listeners.forEach((listener) => {
    listener(memoryState);
  });
}

type Toast = Omit<ToasterToast, "id">;

function toast({ ...props }: Toast) {
  const id = genId();

  const update = (props: ToasterToast) =>
    dispatch({
      type: "UPDATE_TOAST",
      toast: { ...props, id },
    });
  const dismiss = () => dispatch({ type: "DISMISS_TOAST", toastId: id });

  dispatch({
    type: "ADD_TOAST",
    toast: {
      ...props,
      id,
      open: true,
      onOpenChange: (open) => {
        if (!open) dismiss();
      },
    },
  },
  );
};

return {
  id: id,
  dismiss,
  update,
};
}

```

```

function useToast() {
  const [state, setState] = React.useState<State>(memoryState);

  React.useEffect(() => {
    listeners.push(setState);
    return () => {
      const index = listeners.indexOf(setState);
      if (index > -1) {
        listeners.splice(index, 1);
      }
    };
  }, [state]);

  return {
    ...state,
    toast,
    dismiss: (toastId?: string) => dispatch({ type: "DISMISS_TOAST", toastId }),
  };
}

export { useToast, toast };

/* ===== */
/* FICHERO: 100M Dashboard/src/hooks/use-mobile.tsx */
/* ===== */
import * as React from "react";

const MOBILE_BREAKPOINT = 768;

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState<boolean | undefined>(undefined);

  React.useEffect(() => {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`);
    const onChange = () => {
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    };
    mql.addEventListener("change", onChange);
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    return () => mql.removeEventListener("change", onChange);
  }, []);

  return !!isMobile;
}

/* ===== */
/* FICHERO: 100M Dashboard/src/hooks/useAuth.tsx */
/* ===== */
import { createContext, useCallback, useContext, useEffect, useState, ReactNode } from "react";
import type { Session, User } from "@supabase/supabase-js";
import { supabase, type AppRole } from "@lib/supabase";

interface AuthContextValue {
  user: User | null;
  session: Session | null;
  role: AppRole | null;
  roles: AppRole[];
  loading: boolean;
  hasRole: (r: AppRole) => boolean;
  signIn: (email: string, password: string) => Promise<{ error: Error | null }>;
  signOut: () => Promise<void>;
}

const AuthContext = createContext<AuthContextValue | undefined>(undefined);

export const AuthProvider = ({ children }: { children: ReactNode }) => {
  const [user, setUser] = useState<User | null>(null);
  const [session, setSession] = useState<Session | null>(null);
  const [role, setRole] = useState<AppRole | null>(null);
  const [roles, setRoles] = useState<AppRole[]>([]);
  const [loading, setLoading] = useState(true);

  const loadRole = async (userId: string) => {
    const { data, error } = await supabase
      .from("user_roles")
      .select("role")
      .eq("user_id", userId)
      .order("role", { ascending: true });

    if (error || !data || data.length === 0) {
      setRole(null);
      setRoles([]);
      return;
    }

    const all = data.map((r) => r.role as AppRole);
    setRoles(all);
    if (all.includes("admin")) setRole("admin");
    else if (all.includes("caja")) setRole("caja");
    else if (all.includes("cocina")) setRole("cocina");
  };

```



```

    else setRole(all[0]);
  });

useEffect(() => {
  const { data: sub } = supabase.auth.onAuthStateChange((_event, newSession) => {
    setSession(newSession);
    setUser(newSession?.user ?? null);
    if (newSession?.user) {
      setTimeout(() => loadRole(newSession.user.id), 0);
    } else {
      setRole(null);
      setRoles([]);
    }
  });

  supabase.auth.getSession().then(({ data: { session: existing } }) => {
    setSession(existing);
    setUser(existing?.user ?? null);
    if (existing?.user) {
      loadRole(existing.user.id).finally(() => setLoading(false));
    } else {
      setLoading(false);
    }
  });

  return () => sub.subscription.unsubscribe();
}, []);

const signIn = async (email: string, password: string) => {
  const { error } = await supabase.auth.signInWithPassword({ email, password });
  return { error };
};

const signOut = async () => {
  await supabase.auth.signOut();
  setRole(null);
  setRoles([]);
};

const hasRole = useCallback(
  (r: AppRole) => roles.includes(r) || role === r,
  [roles, role],
);

return (
  <AuthContext.Provider value={{ user, session, role, roles, loading, hasRole, signIn, signOut }}>
    {children}
  </AuthContext.Provider>
);
};

// eslint-disable-next-line react-refresh/only-export-components
export const useAuth = () => {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within AuthProvider");
  return ctx;
};

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260410094300_71194f58-1d7c-41db-a8d5-ca49017dbd34.sql */
/* ===== */

-- Create enum for order status
CREATE TYPE public.order_status AS ENUM ('recibido', 'preparando', 'listo', 'entregado', 'cancelado');

-- Create enum for staff roles
CREATE TYPE public.app_role AS ENUM ('admin', 'caja', 'cocina');

-- Locales table
CREATE TABLE public.locales (
  id UUID NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
  nombre TEXT NOT NULL,
  ciudad TEXT NOT NULL,
  direccion TEXT,
  slug TEXT NOT NULL UNIQUE,
  activo BOOLEAN NOT NULL DEFAULT true,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Mesas table
CREATE TABLE public.mesas (
  id UUID NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
  local_id UUID NOT NULL REFERENCES public.locales(id) ON DELETE CASCADE,
  numero_mesa INTEGER NOT NULL,
  activa BOOLEAN NOT NULL DEFAULT true,
  UNIQUE(local_id, numero_mesa)
);

-- Menu categories
CREATE TABLE public.menu_categorias (

```

```

id UUID NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
nombre TEXT NOT NULL,
orden INTEGER NOT NULL DEFAULT 0,
imagen_url TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Menu products
CREATE TABLE public.menu_productos (
id UUID NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
categoria_id UUID NOT NULL REFERENCES public.menu_categorias(id) ON DELETE CASCADE,
local_id UUID REFERENCES public.locales(id) ON DELETE CASCADE,
nombre TEXT NOT NULL,
descripcion TEXT,
precio NUMERIC(10,2) NOT NULL,
foto_url TEXT,
disponible BOOLEAN NOT NULL DEFAULT true,
destacado BOOLEAN NOT NULL DEFAULT false,
nuevo BOOLEAN NOT NULL DEFAULT false,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Pedidos table
CREATE TABLE public.pedidos (
id UUID NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
local_id UUID NOT NULL REFERENCES public.locales(id),
mesa_id UUID REFERENCES public.mesas(id),
numero_pedido INTEGER NOT NULL,
estado public.order_status NOT NULL DEFAULT 'recibido',
total NUMERIC(10,2) NOT NULL DEFAULT 0,
stripe_payment_id TEXT,
session_id TEXT NOT NULL,
notas TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Create index for fast lookup by local + number
CREATE INDEX idx_pedidos_local_numero ON public.pedidos(local_id, numero_pedido);
CREATE INDEX idx_pedidos_session ON public.pedidos(session_id);

-- Pedido items
CREATE TABLE public.pedido_items (
id UUID NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
pedido_id UUID NOT NULL REFERENCES public.pedidos(id) ON DELETE CASCADE,
producto_id UUID NOT NULL REFERENCES public.menu_productos(id),
cantidad INTEGER NOT NULL DEFAULT 1,
precio_unitario NUMERIC(10,2) NOT NULL
);

-- User roles table
CREATE TABLE public.user_roles (
id UUID NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
role public.app_role NOT NULL,
local_id UUID REFERENCES public.locales(id) ON DELETE CASCADE,
UNIQUE(user_id, role, local_id)
);

-- Enable RLS on all tables
ALTER TABLE public.locales ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.mesas ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.menu_categorias ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.menu_productos ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.pedidos ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.pedido_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.user_roles ENABLE ROW LEVEL SECURITY;

-- Security definer function to check roles
CREATE OR REPLACE FUNCTION public.has_role(_user_id UUID, _role public.app_role)
RETURNS BOOLEAN
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path = public
AS $$
SELECT EXISTS (
SELECT 1 FROM public.user_roles
WHERE user_id = _user_id AND role = _role
)
$$;

CREATE OR REPLACE FUNCTION public.has_role_for_local(_user_id UUID, _role public.app_role, _local_id UUID)
RETURNS BOOLEAN
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path = public
AS $$
SELECT EXISTS (
SELECT 1 FROM public.user_roles

```

```

    WHERE user_id = _user_id AND (role = _role OR role = 'admin') AND (local_id = _local_id OR role = 'admin')
)
$$;

-- Locales: public read
CREATE POLICY "Anyone can view active locales" ON public.locales FOR SELECT USING (activo = true);
CREATE POLICY "Admins can manage locales" ON public.locales FOR ALL USING (public.has_role(auth.uid(), 'admin'));

-- Mesas: public read
CREATE POLICY "Anyone can view active mesas" ON public.mesas FOR SELECT USING (activa = true);
CREATE POLICY "Admins can manage mesas" ON public.mesas FOR ALL USING (public.has_role(auth.uid(), 'admin'));

-- Categories: public read
CREATE POLICY "Anyone can view categories" ON public.menu_categorias FOR SELECT USING (true);
CREATE POLICY "Admins can manage categories" ON public.menu_categorias FOR ALL USING (public.has_role(auth.uid(), 'admin'));

-- Products: public read available
CREATE POLICY "Anyone can view available products" ON public.menu_productos FOR SELECT USING (disponible = true);
CREATE POLICY "Admins can manage products" ON public.menu_productos FOR ALL USING (public.has_role(auth.uid(), 'admin'));

-- Pedidos: session-based for clients, role-based for staff
CREATE POLICY "Clients can create pedidos" ON public.pedidos FOR INSERT WITH CHECK (true);
CREATE POLICY "Clients can view own pedidos" ON public.pedidos FOR SELECT USING (true);
CREATE POLICY "Staff can update pedidos" ON public.pedidos FOR UPDATE USING (
    public.has_role_for_local(auth.uid(), 'caja', local_id)
    OR public.has_role_for_local(auth.uid(), 'cocina', local_id)
);

-- Pedido items
CREATE POLICY "Anyone can create pedido items" ON public.pedido_items FOR INSERT WITH CHECK (true);
CREATE POLICY "Anyone can view pedido items" ON public.pedido_items FOR SELECT USING (true);

-- User roles: admin only
CREATE POLICY "Admins can manage user roles" ON public.user_roles FOR ALL USING (public.has_role(auth.uid(), 'admin'));
CREATE POLICY "Users can view own roles" ON public.user_roles FOR SELECT USING (auth.uid() = user_id);

-- Updated_at trigger function
CREATE OR REPLACE FUNCTION public.update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SET search_path = public;

CREATE TRIGGER update_locales_updated_at BEFORE UPDATE ON public.locales FOR EACH ROW EXECUTE FUNCTION
public.update_updated_at_column();
CREATE TRIGGER update_productos_updated_at BEFORE UPDATE ON public.menu_productos FOR EACH ROW EXECUTE FUNCTION
public.update_updated_at_column();
CREATE TRIGGER update_pedidos_updated_at BEFORE UPDATE ON public.pedidos FOR EACH ROW EXECUTE FUNCTION
public.update_updated_at_column();

-- Enable realtime on pedidos
ALTER PUBLICATION supabase_realtime ADD TABLE public.pedidos;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260410094313_b40ec4c8-91ee-43b6-86f1-00506ec52f75.sql */
/* ===== */

-- Drop overly permissive policies
DROP POLICY "Clients can create pedidos" ON public.pedidos;
DROP POLICY "Anyone can create pedido items" ON public.pedido_items;

-- Require session_id for creating orders
CREATE POLICY "Clients can create pedidos with session" ON public.pedidos
FOR INSERT WITH CHECK (session_id IS NOT NULL AND session_id != '');

-- Items can only be added to existing pedidos
CREATE POLICY "Items can be added to pedidos" ON public.pedido_items
FOR INSERT WITH CHECK (
    EXISTS (SELECT 1 FROM public.pedidos WHERE id = pedido_id)
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260423213725_6f7b769a-05e9-4467-83e9-47570bb888cc.sql */
/* ===== */
CREATE OR REPLACE FUNCTION public.assign_numero_pedido_diario()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
    next_num integer;
BEGIN
    SELECT COALESCE(MAX(numero_pedido), 0) + 1
    INTO next_num
    FROM public.pedidos
    WHERE local_id = NEW.local_id
    AND (created_at AT TIME ZONE 'Europe/Madrid')::date = (now() AT TIME ZONE 'Europe/Madrid')::date;

```

```

NEW.numero_pedido := next_num;
RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS set_numero_pedido_diario ON public.pedidos;
CREATE TRIGGER set_numero_pedido_diario
BEFORE INSERT ON public.pedidos
FOR EACH ROW
EXECUTE FUNCTION public.assign_numero_pedido_diario();

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260505154543_780ba33f-2971-46c0-893a-8140e555591d.sql */
/* ===== */
DELETE FROM public.pedido_items; DELETE FROM public.pedidos;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260505161042_2e4be789-ea2a-4296-8e03-f86d4c140e1a.sql */
/* ===== */
ALTER TABLE public.pedidos ADD COLUMN IF NOT EXISTS tipo text NOT NULL DEFAULT 'completo';

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260505162449_d2b99ee6-5c39-498a-8b7a-57f5102ac567.sql */
/* ===== */
-- Añadir estados independientes para cocina y bebidas en un mismo pedido
ALTER TABLE public.pedidos
ADD COLUMN IF NOT EXISTS estado_cocina public.order_status,
ADD COLUMN IF NOT EXISTS estado_bebidas public.order_status;

-- Marcar el destino de cada item del pedido
ALTER TABLE public.pedido_items
ADD COLUMN IF NOT EXISTS destino text NOT NULL DEFAULT 'cocina'
CHECK (destino IN ('cocina','bebidas'));

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260512142004_c7e5509a-b193-440f-b43f-19a4feb46efe.sql */
/* ===== */
CREATE OR REPLACE FUNCTION public.resolve_pedido_item_destino(_producto_id uuid)
RETURNS text
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path TO 'public'
AS $$
SELECT CASE
    WHEN mc.nombre = 'Bebidas' THEN 'bebidas'
    ELSE 'cocina'
END
FROM public.menu_productos mp
LEFT JOIN public.menu_categorias mc ON mc.id = mp.categoria_id
WHERE mp.id = _producto_id
$$;

CREATE OR REPLACE FUNCTION public.set_pedido_item_destino()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $$
BEGIN
    NEW.destino := COALESCE(public.resolve_pedido_item_destino(NEW.producto_id), 'cocina');
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS set_pedido_item_destino_before_write ON public.pedido_items;
CREATE TRIGGER set_pedido_item_destino_before_write
BEFORE INSERT OR UPDATE OF producto_id ON public.pedido_items
FOR EACH ROW
EXECUTE FUNCTION public.set_pedido_item_destino();

CREATE OR REPLACE FUNCTION public.sync_pedido_sections(_pedido_id uuid)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $$
DECLARE
    has_cocina boolean;
    has_bebidas boolean;
BEGIN
    SELECT
        COALESCE(bool_or(destino = 'cocina'), false),
        COALESCE(bool_or(destino = 'bebidas'), false)
    INTO has_cocina, has_bebidas
    FROM public.pedido_items
    WHERE pedido_id = _pedido_id;

    UPDATE public.pedidos
    SET
        tipo = CASE

```

```

        WHEN has_cocina AND has_bebidas THEN 'mixto'
        WHEN has_bebidas THEN 'bebidas'
        ELSE 'cocina'
    END,
    estado_cocina = CASE
        WHEN has_cocina THEN COALESCE(estado_cocina, 'recibido'::public.order_status)
        ELSE NULL
    END,
    estado_bebidas = CASE
        WHEN has_bebidas THEN COALESCE(estado_bebidas, 'recibido'::public.order_status)
        ELSE NULL
    END,
    updated_at = now()
WHERE id = _pedido_id;
END;
$$;

CREATE OR REPLACE FUNCTION public.sync_pedido_sections_from_items()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        PERFORM public.sync_pedido_sections(OLD.pedido_id);
        RETURN OLD;
    END IF;

    PERFORM public.sync_pedido_sections(NEW.pedido_id);

    IF TG_OP = 'UPDATE' AND OLD.pedido_id IS DISTINCT FROM NEW.pedido_id THEN
        PERFORM public.sync_pedido_sections(OLD.pedido_id);
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS sync_pedido_sections_after_items_write ON public.pedido_items;
CREATE TRIGGER sync_pedido_sections_after_items_write
AFTER INSERT OR UPDATE OF pedido_id, producto_id, destino OR DELETE ON public.pedido_items
FOR EACH ROW
EXECUTE FUNCTION public.sync_pedido_sections_from_items();

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260512142021_62717ba3-d2a3-4839-880a-7ad82614095b.sql */
/* ===== */
REVOKE ALL ON FUNCTION public.resolve_pedido_item_destino(uuid) FROM PUBLIC, anon, authenticated;
REVOKE ALL ON FUNCTION public.set_pedido_item_destino() FROM PUBLIC, anon, authenticated;
REVOKE ALL ON FUNCTION public.sync_pedido_sections(uuid) FROM PUBLIC, anon, authenticated;
REVOKE ALL ON FUNCTION public.sync_pedido_sections_from_items() FROM PUBLIC, anon, authenticated;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260514141018_54e36bb1-bbbe-4928-bb29-cf6972bca4f1.sql */
/* ===== */
-- Añadir campos geo, contacto y horarios al local
ALTER TABLE public.locales
ADD COLUMN IF NOT EXISTS lat numeric,
ADD COLUMN IF NOT EXISTS lng numeric,
ADD COLUMN IF NOT EXISTS telefono text,
ADD COLUMN IF NOT EXISTS sitio_web text,
ADD COLUMN IF NOT EXISTS horarios jsonb;

-- Insertar local de Móstoles si no existe (por slug único)
INSERT INTO public.locales (nombre, ciudad, direccion, slug, lat, lng, telefono, sitio_web, horarios, activo)
SELECT
'Móstoles - Av. de la Constitución',
'Móstoles',
'Av. de la Constitución, 23, 28931 Móstoles, Madrid',
'mostoles-constitucion',
40.3223,
-3.8652,
'913519001',
'https://spain.100montaditos.com',
'{
  "lunes": {"open": "10:00", "close": "23:00"},
  "martes": {"open": "10:00", "close": "23:00"},
  "miercoles": {"open": "10:00", "close": "23:00"},
  "jueves": {"open": "10:00", "close": "23:00"},
  "viernes": {"open": "10:00", "close": "24:00"},
  "sabado": {"open": "11:30", "close": "24:00"},
  "domingo": {"open": "11:30", "close": "23:00"}
}::jsonb,
true
WHERE NOT EXISTS (SELECT 1 FROM public.locales WHERE slug = 'mostoles-constitucion');

-- Asegurar que los locales existentes (Gran Vía seed) tengan también lat/lng aproximadas si están vacías
UPDATE public.locales
SET lat = 40.4200, lng = -3.7025
WHERE slug = 'madrid-gran-via' AND (lat IS NULL OR lng IS NULL);

```

```

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260514141551_b4f1c034-e72a-4a7c-a82f-f6cc3ae702f6.sql */
/* ===== */
-- 1) REVOKE EXECUTE en funciones SECURITY DEFINER de uso interno
REVOKE EXECUTE ON FUNCTION public.resolve_pedido_item_destino(uuid) FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.set_pedido_item_destino() FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.sync_pedido_sections(uuid) FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.sync_pedido_sections_from_items() FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.assign_numero_pedido_diario() FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.update_updated_at_column() FROM PUBLIC, anon, authenticated;

-- 2) pedidos: SELECT por rol/local
DROP POLICY IF EXISTS "Clients can view own pedidos" ON public.pedidos;
DROP POLICY IF EXISTS "Anon can view pedidos for tracking" ON public.pedidos;
DROP POLICY IF EXISTS "Staff can view pedidos of their local" ON public.pedidos;

CREATE POLICY "Anon can view pedidos for tracking"
ON public.pedidos FOR SELECT TO anon
USING (true);

CREATE POLICY "Staff can view pedidos of their local"
ON public.pedidos FOR SELECT TO authenticated
USING (
    public.has_role(auth.uid(), 'admin':public.app_role)
    OR EXISTS (
        SELECT 1 FROM public.user_roles ur
        WHERE ur.user_id = auth.uid()
        AND ur.role IN ('caja':public.app_role, 'cocina':public.app_role)
        AND (ur.local_id IS NULL OR ur.local_id = pedidos.local_id)
    )
);

-- 3) pedido_items: SELECT/INSERT acotados por rol/local del pedido padre
DROP POLICY IF EXISTS "Anyone can view pedido items" ON public.pedido_items;
DROP POLICY IF EXISTS "Items can be added to pedidos" ON public.pedido_items;
DROP POLICY IF EXISTS "Anon can view pedido items" ON public.pedido_items;
DROP POLICY IF EXISTS "Staff can view pedido items of their local" ON public.pedido_items;
DROP POLICY IF EXISTS "Anon can add items to existing pedidos" ON public.pedido_items;
DROP POLICY IF EXISTS "Staff can add items to pedidos of their local" ON public.pedido_items;

CREATE POLICY "Anon can view pedido items"
ON public.pedido_items FOR SELECT TO anon
USING (true);

CREATE POLICY "Staff can view pedido items of their local"
ON public.pedido_items FOR SELECT TO authenticated
USING (
    public.has_role(auth.uid(), 'admin':public.app_role)
    OR EXISTS (
        SELECT 1
        FROM public.pedidos p
        JOIN public.user_roles ur ON ur.user_id = auth.uid()
        WHERE p.id = pedido_items.pedido_id
        AND ur.role IN ('caja':public.app_role, 'cocina':public.app_role)
        AND (ur.local_id IS NULL OR ur.local_id = p.local_id)
    )
);

CREATE POLICY "Anon can add items to existing pedidos"
ON public.pedido_items FOR INSERT TO anon
WITH CHECK (EXISTS (SELECT 1 FROM public.pedidos p WHERE p.id = pedido_items.pedido_id));

CREATE POLICY "Staff can add items to pedidos of their local"
ON public.pedido_items FOR INSERT TO authenticated
WITH CHECK (
    public.has_role(auth.uid(), 'admin':public.app_role)
    OR EXISTS (
        SELECT 1
        FROM public.pedidos p
        JOIN public.user_roles ur ON ur.user_id = auth.uid()
        WHERE p.id = pedido_items.pedido_id
        AND ur.role IN ('caja':public.app_role, 'cocina':public.app_role)
        AND (ur.local_id IS NULL OR ur.local_id = p.local_id)
    )
);

-- 4) Realtime: añadir pedido_items si no estaba (pedidos ya lo está)
DO $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM pg_publication_tables
        WHERE pubname = 'supabase_realtime'
        AND schemaname = 'public'
        AND tablename = 'pedido_items'
    ) THEN
        EXECUTE 'ALTER PUBLICATION supabase_realtime ADD TABLE public.pedido_items';
    END IF;
END $$;

ALTER TABLE public.pedidos REPLICATION IDENTITY FULL;
ALTER TABLE public.pedido_items REPLICATION IDENTITY FULL;

```

```

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260514141752_5ad23951-c370-4898-bf20-988a8b4ac1ee.sql */
/* ===== */
-- Helper: extrae el session_id del header x-session-id (devuelve '' si no existe)
CREATE OR REPLACE FUNCTION public.current_session_id()
RETURNS text
LANGUAGE sql
STABLE
AS $$
    SELECT COALESCE(
        NULLIF(
            current_setting('request.headers', true)::json -> 'x-session-id',
            ''
        ),
        ''
    );
$$;

-- pedidos: anon solo ve los suyos
DROP POLICY IF EXISTS "Anon can view pedidos for tracking" ON public.pedidos;
CREATE POLICY "Anon can view own pedidos by session"
ON public.pedidos FOR SELECT TO anon
USING (
    session_id IS NOT NULL
    AND session_id <> ''
    AND session_id = public.current_session_id()
);

-- pedidos: anon solo crea pedidos cuya session_id coincide con su header
DROP POLICY IF EXISTS "Clients can create pedidos with session" ON public.pedidos;
CREATE POLICY "Anon can create pedidos with own session"
ON public.pedidos FOR INSERT TO anon
WITH CHECK (
    session_id IS NOT NULL
    AND session_id <> ''
    AND session_id = public.current_session_id()
);

-- Mantener INSERT abierto a authenticated también (staff podría crear)
CREATE POLICY "Authenticated can create pedidos"
ON public.pedidos FOR INSERT TO authenticated
WITH CHECK (session_id IS NOT NULL AND session_id <> '');

-- pedido_items: anon solo ve los items de sus pedidos
DROP POLICY IF EXISTS "Anon can view pedido items" ON public.pedido_items;
CREATE POLICY "Anon can view own pedido items by session"
ON public.pedido_items FOR SELECT TO anon
USING (
    EXISTS (
        SELECT 1 FROM public.pedidos p
        WHERE p.id = pedido_items.pedido_id
        AND p.session_id = public.current_session_id()
        AND p.session_id <> ''
    )
);

-- pedido_items: anon solo añade items a pedidos de su sesión
DROP POLICY IF EXISTS "Anon can add items to existing pedidos" ON public.pedido_items;
CREATE POLICY "Anon can add items to own pedidos"
ON public.pedido_items FOR INSERT TO anon
WITH CHECK (
    EXISTS (
        SELECT 1 FROM public.pedidos p
        WHERE p.id = pedido_items.pedido_id
        AND p.session_id = public.current_session_id()
        AND p.session_id <> ''
    )
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260514141804_2a9e2680-7665-4f11-a22e-a248026570cf.sql */
/* ===== */
CREATE OR REPLACE FUNCTION public.current_session_id()
RETURNS text
LANGUAGE sql
STABLE
SET search_path = public
AS $$
    SELECT COALESCE(
        NULLIF(
            current_setting('request.headers', true)::json -> 'x-session-id',
            ''
        ),
        ''
    );
$$;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260514145859_ad97e50f-8a05-4e5d-b676-926111a353cc.sql */
/* ===== */

DROP POLICY IF EXISTS "Anon can create pedidos with own session" ON public.pedidos;
CREATE POLICY "Anon can create pedidos with own session"

```

```

ON public.pedidos
FOR INSERT
TO anon
WITH CHECK (session_id IS NOT NULL AND session_id <> '');

DROP POLICY IF EXISTS "Anon can add items to own pedidos" ON public.pedido_items;
CREATE POLICY "Anon can add items to own pedidos"
ON public.pedido_items
FOR INSERT
TO anon
WITH CHECK (
  EXISTS (
    SELECT 1 FROM public.pedidos p
    WHERE p.id = pedido_items.pedido_id
    AND p.session_id IS NOT NULL
    AND p.session_id <> ''
  )
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260522105451_ebb712b9-6ea2-4f7a-9ca0-889b72fb4029.sql */
/* ===== */
DELETE FROM public.pedido_items; DELETE FROM public.pedidos;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260522113046_06360563-1977-43b5-a698-2d5c5c000db1.sql */
/* ===== */
ALTER TYPE public.app_role ADD VALUE IF NOT EXISTS 'franchisee';

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260522113105_a763bc1f-95b8-484d-a17f-74f1a99f05c6.sql */
/* ===== */

CREATE TABLE public.franchisees (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  nombre text NOT NULL,
  email text NOT NULL UNIQUE,
  stripe_account_id text UNIQUE,
  stripe_onboarding_completed boolean NOT NULL DEFAULT false,
  application_fee_percent numeric NOT NULL DEFAULT 0,
  activo boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

ALTER TABLE public.franchisees ENABLE ROW LEVEL SECURITY;

CREATE TRIGGER trg_franchisees_updated_at
BEFORE UPDATE ON public.franchisees
FOR EACH ROW EXECUTE FUNCTION public.update_updated_at_column();

CREATE OR REPLACE FUNCTION public.get_user_franchisee_id(user_id uuid)
RETURNS uuid
LANGUAGE sql
STABLE SECURITY DEFINER
SET search_path = public
AS $$
  SELECT ur.local_id::uuid
  FROM public.user_roles ur
  WHERE ur.user_id = _user_id AND ur.role = 'franchisee'
  LIMIT 1
$$;

REVOKE EXECUTE ON FUNCTION public.get_user_franchisee_id(uuid) FROM PUBLIC, anon;

CREATE POLICY "Admins manage franchisees"
ON public.franchisees FOR ALL
USING (public.has_role(auth.uid(), 'admin'))
WITH CHECK (public.has_role(auth.uid(), 'admin'));

CREATE POLICY "Franchisee views own record"
ON public.franchisees FOR SELECT
USING (id = public.get_user_franchisee_id(auth.uid()));

ALTER TABLE public.locales
ADD COLUMN franchisee_id uuid REFERENCES public.franchisees(id) ON DELETE RESTRICT;

INSERT INTO public.franchisees (nombre, email, application_fee_percent)
VALUES ('Móstoles', 'mostoles@hat3x.com', 0);

UPDATE public.locales
SET franchisee_id = (SELECT id FROM public.franchisees WHERE email = 'mostoles@hat3x.com');

ALTER TABLE public.locales
ALTER COLUMN franchisee_id SET NOT NULL;

CREATE POLICY "Franchisee views own locales pedidos"
ON public.pedidos FOR SELECT
USING (
  EXISTS (
    SELECT 1 FROM public.locales l

```



```

        WHERE l.id = pedidos.local_id
        AND l.franchisee_id = public.get_user_franchisee_id(auth.uid())
    )
);

CREATE POLICY "Franchisee views own pedido items"
ON public.pedido_items FOR SELECT
USING (
    EXISTS (
        SELECT 1 FROM public.pedidos p
        JOIN public.locales l ON l.id = p.local_id
        WHERE p.id = pedido_items.pedido_id
        AND l.franchisee_id = public.get_user_franchisee_id(auth.uid())
    )
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260522123916_1f8a7fba-2c2a-4ad8-8c27-86f7938a1269.sql */
/* ===== */
CREATE POLICY "Franchisee views own locales"
ON public.locales
FOR SELECT
TO authenticated
USING (franchisee_id = public.get_user_franchisee_id(auth.uid()));

CREATE POLICY "Franchisee updates own locales"
ON public.locales
FOR UPDATE
TO authenticated
USING (franchisee_id = public.get_user_franchisee_id(auth.uid()))
WITH CHECK (franchisee_id = public.get_user_franchisee_id(auth.uid()));

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260522132618_c598bcd8-798a-464e-8fe2-1dad5dd855fc.sql */
/* ===== */

-- Enable required extensions
CREATE EXTENSION IF NOT EXISTS pg_cron;

-- Function to auto-cancel stale pedidos
CREATE OR REPLACE FUNCTION public.auto_cancel_stale_pedidos()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $$
BEGIN
    UPDATE public.pedidos
    SET
        estado = 'cancelado'::public.order_status,
        estado_cocina = CASE WHEN estado_cocina IS NOT NULL THEN 'cancelado'::public.order_status ELSE NULL END,
        estado_bebidas = CASE WHEN estado_bebidas IS NOT NULL THEN 'cancelado'::public.order_status ELSE NULL END,
        updated_at = now(),
        notas = COALESCE(notas, '') || CASE WHEN COALESCE(notas, '') = '' THEN ' ' ELSE ' | ' END || '[Auto-cancelado por inactividad >
3h]'
    WHERE estado IN ('pendiente', 'recibido', 'preparando', 'listo')
    AND created_at < now() - interval '3 hours';
END;
$$;

-- Unschedule any existing job with same name to keep it idempotent
DO $$
BEGIN
    PERFORM cron.unschedule('auto-cancel-stale-pedidos');
EXCEPTION WHEN OTHERS THEN
    NULL;
END $$;

-- Schedule every 5 minutes
SELECT cron.schedule(
    'auto-cancel-stale-pedidos',
    '*/*5 * * * *',
    $$ SELECT public.auto_cancel_stale_pedidos(); $$
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260523221022_17b17a83-0832-4579-9e6e-cfd11f811d13.sql */
/* ===== */
GRANT EXECUTE ON FUNCTION public.get_user_franchisee_id(uuid) TO anon, authenticated;
GRANT EXECUTE ON FUNCTION public.has_role(uuid, public.app_role) TO anon, authenticated;
GRANT EXECUTE ON FUNCTION public.has_role_for_local(uuid, public.app_role, uuid) TO anon, authenticated;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260529191501_ed7d388a-4b68-4825-9f3b-8d9339c25a3d.sql */
/* ===== */
ALTER TYPE public.order_status ADD VALUE IF NOT EXISTS 'pendiente';
ALTER TYPE public.order_status ADD VALUE IF NOT EXISTS 'pendiente_pago' BEFORE 'pendiente';

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260529200556_36d2bc05-93a0-4a74-9293-80f02042142b.sql */

```

```

/* ===== */
ALTER TABLE public.menu_productos ADD COLUMN IF NOT EXISTS contiene_alcohol boolean NOT NULL DEFAULT false;
ALTER TABLE public.pedidos ADD COLUMN IF NOT EXISTS edad_verificada_cliente boolean NOT NULL DEFAULT false;

UPDATE public.menu_productos mp
SET contiene_alcohol = true
FROM public.menu_categorias mc
WHERE mp.categoria_id = mc.id
AND mc.nombre = 'Bebidas'
AND (
    mp.nombre ILIKE '%cerveza%' OR
    mp.nombre ILIKE '%mahou%' OR
    mp.nombre ILIKE '%estrella%' OR
    mp.nombre ILIKE '%alhambra%' OR
    mp.nombre ILIKE '%heineken%' OR
    mp.nombre ILIKE '%vino%' OR
    mp.nombre ILIKE '%tinto%' OR
    mp.nombre ILIKE '%blanco%' OR
    mp.nombre ILIKE '%rosado%' OR
    mp.nombre ILIKE '%rioja%' OR
    mp.nombre ILIKE '%ribera%' OR
    mp.nombre ILIKE '%verdejo%' OR
    mp.nombre ILIKE '%albariño%' OR
    mp.nombre ILIKE '%sangría%' OR
    mp.nombre ILIKE '%sangria%' OR
    mp.nombre ILIKE '%tinto de verano%' OR
    mp.nombre ILIKE '%calimochos%' OR
    mp.nombre ILIKE '%clara%' OR
    mp.nombre ILIKE '%cava%' OR
    mp.nombre ILIKE '%champ%' OR
    mp.nombre ILIKE '%sidra%' OR
    mp.nombre ILIKE '%vermut%' OR
    mp.nombre ILIKE '%vermouth%' OR
    mp.nombre ILIKE '%whisky%' OR
    mp.nombre ILIKE '%whiskey%' OR
    mp.nombre ILIKE '%ron %' OR mp.nombre ILIKE 'ron' OR mp.nombre ILIKE 'ron-%' OR
    mp.nombre ILIKE '%ginebra%' OR
    mp.nombre ILIKE '%gin tonic%' OR
    mp.nombre ILIKE '%vodka%' OR
    mp.nombre ILIKE '%tequila%' OR
    mp.nombre ILIKE '%licor%' OR
    mp.nombre ILIKE '%orujo%' OR
    mp.nombre ILIKE '%pacharán%' OR
    mp.nombre ILIKE '%pacharan%'
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260529202407_7cd66c4e-18ab-4a37-b587-3c379d443150.sql */
/* ===== */

CREATE OR REPLACE FUNCTION public.auto_cancel_stale_pending_payment()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $$
BEGIN
    UPDATE public.pedidos
    SET
        estado = 'cancelado':public.order_status,
        estado_cocina = CASE WHEN estado_cocina IS NOT NULL THEN 'cancelado':public.order_status ELSE NULL END,
        estado_bebidas = CASE WHEN estado_bebidas IS NOT NULL THEN 'cancelado':public.order_status ELSE NULL END,
        updated_at = now(),
        notas = COALESCE(notas, '') || CASE WHEN COALESCE(notas, '') = '' THEN '' ELSE ' | ' END || '[Auto-cancelado: pago no confirmado
en 30 min]'
    WHERE estado = 'pendiente_pago'
    AND created_at < now() - interval '30 minutes';
END;
$$;

CREATE EXTENSION IF NOT EXISTS pg_cron;
CREATE EXTENSION IF NOT EXISTS pg_net;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260529203630_8435aa6d-9614-454c-9260-5049becf98d6.sql */
/* ===== */
-- Tighten staff/franchisee SELECT policies on pedidos and pedido_items
-- so that orders in 'pendiente_pago' never leak into Caja/Cocina apps.

-- pedidos: staff SELECT
DROP POLICY IF EXISTS "Staff can view pedidos of their local" ON public.pedidos;
CREATE POLICY "Staff can view pedidos of their local"
ON public.pedidos
FOR SELECT
TO authenticated
USING (
    estado <> 'pendiente_pago':public.order_status
    AND (
        public.has_role(auth.uid(), 'admin':public.app_role)
        OR EXISTS (
            SELECT 1 FROM public.user_roles ur

```

```

WHERE ur.user_id = auth.uid()
AND ur.role = ANY (ARRAY['caja':public.app_role, 'cocina':public.app_role])
AND (ur.local_id IS NULL OR ur.local_id = pedidos.local_id)
)
)
);

-- pedidos: franchisee SELECT
DROP POLICY IF EXISTS "Franchisee views own locales pedidos" ON public.pedidos;
CREATE POLICY "Franchisee views own locales pedidos"
ON public.pedidos
FOR SELECT
USING (
    estado <> 'pendiente_pago':public.order_status
    AND EXISTS (
        SELECT 1 FROM public.locales l
        WHERE l.id = pedidos.local_id
        AND l.franchisee_id = public.get_user_franchisee_id(auth.uid())
    )
);

-- pedido_items: staff SELECT
DROP POLICY IF EXISTS "Staff can view pedido items of their local" ON public.pedido_items;
CREATE POLICY "Staff can view pedido items of their local"
ON public.pedido_items
FOR SELECT
TO authenticated
USING (
    public.has_role(auth.uid(), 'admin':public.app_role)
    OR EXISTS (
        SELECT 1
        FROM public.pedidos p
        JOIN public.user_roles ur ON ur.user_id = auth.uid()
        WHERE p.id = pedido_items.pedido_id
        AND p.estado <> 'pendiente_pago':public.order_status
        AND ur.role = ANY (ARRAY['caja':public.app_role, 'cocina':public.app_role])
        AND (ur.local_id IS NULL OR ur.local_id = p.local_id)
    )
);

-- pedido_items: franchisee SELECT
DROP POLICY IF EXISTS "Franchisee views own pedido items" ON public.pedido_items;
CREATE POLICY "Franchisee views own pedido items"
ON public.pedido_items
FOR SELECT
USING (
    EXISTS (
        SELECT 1
        FROM public.pedidos p
        JOIN public.locales l ON l.id = p.local_id
        WHERE p.id = pedido_items.pedido_id
        AND p.estado <> 'pendiente_pago':public.order_status
        AND l.franchisee_id = public.get_user_franchisee_id(auth.uid())
    )
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260529204321_7bbbc7b1-1b58-4c01-a917-42195f0c126d.sql */
/* ===== */
CREATE OR REPLACE FUNCTION public.cancel_own_pending_order(_pedido_id uuid)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
    _session text;
BEGIN
    _session := public.current_session_id();
    IF _session IS NULL OR _session = '' THEN
        RAISE EXCEPTION 'Missing session';
    END IF;

    UPDATE public.pedidos
    SET
        estado = 'cancelado':public.order_status,
        estado_cocina = CASE WHEN estado_cocina IS NOT NULL THEN 'cancelado':public.order_status ELSE NULL END,
        estado_bebidas = CASE WHEN estado_bebidas IS NOT NULL THEN 'cancelado':public.order_status ELSE NULL END,
        updated_at = now(),
        notas = COALESCE(notas, '') || CASE WHEN COALESCE(notas, '') = '' THEN '' ELSE ' | ' END || '[Cancelado por el cliente antes del
pago]'
    WHERE id = _pedido_id
    AND session_id = _session
    AND estado = 'pendiente_pago':public.order_status;
END;
$$;

GRANT EXECUTE ON FUNCTION public.cancel_own_pending_order(uuid) TO anon, authenticated;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260529204631_429762d1-ca6f-4d20-a8cd-201677f039f1.sql */
/* ===== */

```

```

DROP POLICY IF EXISTS "Staff can view pedidos of their local" ON public.pedidos;
DROP POLICY IF EXISTS "Franchisee views own locales pedidos" ON public.pedidos;
DROP POLICY IF EXISTS "Staff can view pedido items of their local" ON public.pedido_items;
DROP POLICY IF EXISTS "Franchisee views own pedido items" ON public.pedido_items;

CREATE POLICY "Staff can view paid active pedidos of their local"
ON public.pedidos
FOR SELECT
TO authenticated
USING (
    estado NOT IN ('pendiente_pago'::public.order_status, 'cancelado'::public.order_status)
    AND (
        public.has_role(auth.uid(), 'admin'::public.app_role)
        OR EXISTS (
            SELECT 1
            FROM public.user_roles ur
            WHERE ur.user_id = auth.uid()
            AND ur.role = ANY (ARRAY['caja'::public.app_role, 'cocina'::public.app_role])
            AND (ur.local_id IS NULL OR ur.local_id = pedidos.local_id)
        )
    )
);

CREATE POLICY "Franchisee views own paid active pedidos"
ON public.pedidos
FOR SELECT
TO authenticated
USING (
    estado NOT IN ('pendiente_pago'::public.order_status, 'cancelado'::public.order_status)
    AND EXISTS (
        SELECT 1
        FROM public.locales l
        WHERE l.id = pedidos.local_id
        AND l.franchisee_id = public.get_user_franchisee_id(auth.uid())
    )
);

CREATE POLICY "Staff can view paid active pedido items of their local"
ON public.pedido_items
FOR SELECT
TO authenticated
USING (
    public.has_role(auth.uid(), 'admin'::public.app_role)
    OR EXISTS (
        SELECT 1
        FROM public.pedidos p
        JOIN public.user_roles ur ON ur.user_id = auth.uid()
        WHERE p.id = pedido_items.pedido_id
        AND p.estado NOT IN ('pendiente_pago'::public.order_status, 'cancelado'::public.order_status)
        AND ur.role = ANY (ARRAY['caja'::public.app_role, 'cocina'::public.app_role])
        AND (ur.local_id IS NULL OR ur.local_id = p.local_id)
    )
);

CREATE POLICY "Franchisee views own paid active pedido items"
ON public.pedido_items
FOR SELECT
TO authenticated
USING (
    EXISTS (
        SELECT 1
        FROM public.pedidos p
        JOIN public.locales l ON l.id = p.local_id
        WHERE p.id = pedido_items.pedido_id
        AND p.estado NOT IN ('pendiente_pago'::public.order_status, 'cancelado'::public.order_status)
        AND l.franchisee_id = public.get_user_franchisee_id(auth.uid())
    )
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260530005018_2366d543-5a86-416d-8bad-d79dfb11a88c.sql */
/* ===== */

-- Marcar bebidas alcohólicas
UPDATE public.menu_productos
SET contiene_alcohol = true
WHERE (
    nombre ILIKE '%cerveza%'
    OR nombre ILIKE '%cruzcampo%'
    OR nombre ILIKE '%heineken%'
    OR nombre ILIKE '%paulaner%'
    OR nombre ILIKE '%desperados%'
    OR nombre ILIKE '%el águila%'
    OR nombre ILIKE '%el aguila%'
    OR nombre ILIKE '%radler%'
    OR nombre ILIKE '%jarra%'
    OR nombre ILIKE '%vino%'
    OR nombre ILIKE '%ribera%'
    OR nombre ILIKE '%rioja%'
    OR nombre ILIKE '%verdejo%'
    OR nombre ILIKE '%rosado%'
    OR nombre ILIKE '%tinto%'

```

```

OR nombre ILIKE '%blanco%'
OR nombre ILIKE '%ron %'
OR nombre ILIKE '%havana%'
OR nombre ILIKE '%bacardi%'
OR nombre ILIKE '%vodka%'
OR nombre ILIKE '%absolut%'
OR nombre ILIKE '%whisky%'
OR nombre ILIKE '%ballantines%'
OR nombre ILIKE '%jack %'
OR nombre ILIKE '%ginebra%'
OR nombre ILIKE '%beefeater%'
OR nombre ILIKE '%spritz%'
OR nombre ILIKE '%petroni%'
OR nombre ILIKE '%ladrón%'
OR nombre ILIKE '%ladron%'
OR nombre ILIKE '%sancho%'
OR nombre ILIKE '%quijote%'
OR nombre ILIKE '%sidra%'
OR nombre ILIKE '%manzana%'
OR nombre ILIKE '%crema ruavieja%'
OR nombre ILIKE '%ruavieja%'
);

-- Desmarcar versiones sin alcohol (0.0)
UPDATE public.menu_productos
SET contiene_alcohol = false
WHERE nombre ILIKE '%0,0%'
      OR nombre ILIKE '%0.0%'
      OR nombre ILIKE '%sin alcohol%';

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260530041004_8e85f36d-dee4-4c71-996c-60fa6f4033ca.sql */
/* ===== */

-- Maestra de alérgenos (14 obligatorios UE 1169/2011)
CREATE TABLE public.alergenos (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  codigo TEXT NOT NULL UNIQUE,
  nombre TEXT NOT NULL,
  descripcion TEXT,
  icono TEXT,
  orden INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

GRANT SELECT ON public.alergenos TO anon;
GRANT SELECT ON public.alergenos TO authenticated;
GRANT ALL ON public.alergenos TO service_role;

ALTER TABLE public.alergenos ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Anyone can view alergenos"
ON public.alergenos FOR SELECT
USING (true);

CREATE POLICY "Admins manage alergenos"
ON public.alergenos FOR ALL
USING (public.has_role(auth.uid(), 'admin':app_role))
WITH CHECK (public.has_role(auth.uid(), 'admin':app_role));

-- Relación productos <-> alérgenos
CREATE TABLE public.producto_alergenos (
  producto_id UUID NOT NULL,
  alergeno_id UUID NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (producto_id, alergeno_id)
);

CREATE INDEX idx_producto_alergenos_alergeno ON public.producto_alergenos(alergeno_id);

GRANT SELECT ON public.producto_alergenos TO anon;
GRANT SELECT ON public.producto_alergenos TO authenticated;
GRANT ALL ON public.producto_alergenos TO service_role;

ALTER TABLE public.producto_alergenos ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Anyone can view producto_alergenos"
ON public.producto_alergenos FOR SELECT
USING (true);

CREATE POLICY "Admins manage producto_alergenos"
ON public.producto_alergenos FOR ALL
USING (public.has_role(auth.uid(), 'admin':app_role))
WITH CHECK (public.has_role(auth.uid(), 'admin':app_role));

-- Seed 14 alérgenos oficiales
INSERT INTO public.alergenos (codigo, nombre, descripcion, icono, orden) VALUES
('gluten', 'Gluten', 'Cereales que contienen gluten: trigo, centeno, cebada, avena', '🌾', 1),
('crustaceos', 'Crustáceos', 'Cangrejos, langostas, gambas, langostinos', '🦀', 2),
('huevos', 'Huevos', 'Huevos y productos a base de huevo', '🥚', 3),
('pescado', 'Pescado', 'Pescado y productos a base de pescado', '🐟', 4),
('cacahuetes', 'Cacahuetes', 'Cacahuetes y productos a base de cacahuetes', '🥜', 5),

```

```

('soja',      'Soja',      'Soja y productos a base de soja', '🌱', 6),
('lacteos',   'Lácteos',   'Leche y sus derivados incluida la lactosa', '🥛', 7),
('frutas_cascara', 'Frutos de cáscara', 'Almendras, avellanas, nueces, anacardos, pistachos', '🌰', 8),
('apio',      'Apio',      'Apio y productos derivados', '🌿', 9),
('mostaza',   'Mostaza',   'Mostaza y productos derivados', '🌱', 10),
('sesamo',    'Sésamo',    'Semillas de sésamo y productos a base de sésamo', '🌱', 11),
('sulfitos',  'Sulfitos',  'Dióxido de azufre y sulfitos en concentraciones superiores a 10mg/kg', '🍷', 12),
('altramuces', 'Altramuces', 'Altramuces y productos a base de altramuces', '🌱', 13),
('moluscos',  'Moluscos',  'Moluscos y productos a base de moluscos', '🐚', 14);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260530042858_ba94e6eb-700c-46fb-81a1-ecffcdb3ac84.sql */
/* ===== */
ALTER TABLE public.producto_alergenos
  ADD CONSTRAINT producto_alergenos_producto_id_fkey FOREIGN KEY (producto_id) REFERENCES public.menu_productos(id) ON DELETE CASCADE,
  ADD CONSTRAINT producto_alergenos_alergeno_id_fkey FOREIGN KEY (alergeno_id) REFERENCES public.alergenos(id) ON DELETE CASCADE,
  ADD CONSTRAINT producto_alergenos_unique UNIQUE (producto_id, alergeno_id);
NOTIFY pgrst, 'reload schema';

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260530043949_d81594b7-cce0-43b9-a350-ccaa3d917207.sql */
/* ===== */

ALTER TABLE public.menu_productos
  ADD COLUMN IF NOT EXISTS numero TEXT,
  ADD COLUMN IF NOT EXISTS tipo_pan TEXT;

ALTER TABLE public.menu_categorias
  ADD COLUMN IF NOT EXISTS logo_url TEXT;

CREATE INDEX IF NOT EXISTS idx_menu_productos_numero ON public.menu_productos(numero);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260530044858_ef3de65f-646e-42ca-bcd1-3e51fe49d983.sql */
/* ===== */

ALTER TABLE public.menu_productos
  ADD COLUMN IF NOT EXISTS seccion TEXT;
CREATE INDEX IF NOT EXISTS idx_menu_productos_seccion ON public.menu_productos(seccion);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260601143831_bd5a7d88-599e-4818-83ea-f045fbbf0ed2.sql */
/* ===== */

-- Reemplazar las ensaladas con las dos oficiales de 100 Montaditos
DELETE FROM public.menu_productos
WHERE categoria_id = (SELECT id FROM public.menu_categorias WHERE nombre ILIKE 'ensaladas');

INSERT INTO public.menu_productos (categoria_id, nombre, descripcion, precio, disponible, destacado, nuevo)
SELECT id, 'Campera', 'Lechuga, pollo, tomate, queso madurado, aceitunas, picatostes y salsa alioli', 6.50, true, false, false
FROM public.menu_categorias WHERE nombre ILIKE 'ensaladas';

INSERT INTO public.menu_productos (categoria_id, nombre, descripcion, precio, disponible, destacado, nuevo)
SELECT id, 'TexMex', 'Lechuga, pollo kebab, tomate, maiz, aceitunas, nachos sabor TexMex y salsa mostaza y miel', 6.50, true, true,
true
FROM public.menu_categorias WHERE nombre ILIKE 'ensaladas';

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260609184612_c043e72b-632d-40a7-9e74-b537e9e985e7.sql */
/* ===== */

-- 1) Fix anon INSERT policies to require session match
DROP POLICY IF EXISTS "Anon can create pedidos with own session" ON public.pedidos;
CREATE POLICY "Anon can create pedidos with own session"
ON public.pedidos
FOR INSERT
TO anon
WITH CHECK (
  session_id IS NOT NULL
  AND session_id <> ''
  AND session_id = public.current_session_id()
);

DROP POLICY IF EXISTS "Anon can add items to own pedidos" ON public.pedido_items;
CREATE POLICY "Anon can add items to own pedidos"
ON public.pedido_items
FOR INSERT
TO anon
WITH CHECK (
  EXISTS (
    SELECT 1 FROM public.pedidos p
    WHERE p.id = pedido_items.pedido_id
    AND p.session_id IS NOT NULL
    AND p.session_id <> ''
    AND p.session_id = public.current_session_id()
  )
);

-- 2) Fix get_user_franchisee_id: it must return franchisees.id, not user_roles.local_id.

```

```

-- Resolve via locales.franchisee_id.
CREATE OR REPLACE FUNCTION public.get_user_franchisee_id(_user_id uuid)
RETURNS uuid
LANGUAGE sql
STABLE SECURITY DEFINER
SET search_path TO 'public'
AS $function$
    SELECT l.franchisee_id
    FROM public.user_roles ur
    JOIN public.locales l ON l.id = ur.local_id
    WHERE ur.user_id = _user_id AND ur.role = 'franchisee'
    LIMIT 1
$function$;

-- 3) Lock down SECURITY DEFINER functions: revoke broad EXECUTE.
-- Keep cancel_own_pending_order callable by anon/authenticated (RPC used by clients).
REVOKE EXECUTE ON FUNCTION public.has_role(uuid, public.app_role) FROM PUBLIC, anon;
REVOKE EXECUTE ON FUNCTION public.has_role_for_local(uuid, public.app_role, uuid) FROM PUBLIC, anon;
REVOKE EXECUTE ON FUNCTION public.get_user_franchisee_id(uuid) FROM PUBLIC, anon;
REVOKE EXECUTE ON FUNCTION public.resolve_pedido_item_destino(uuid) FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.set_pedido_item_destino() FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.sync_pedido_sections(uuid) FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.sync_pedido_sections_from_items() FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.auto_cancel_stale_pedidos() FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.auto_cancel_stale_pending_payment() FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.assign_numero_pedido_diario() FROM PUBLIC, anon, authenticated;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260609184627_90be342e-cc76-475a-8756-91049dcfbb38.sql */
/* ===== */

-- These functions are referenced in RLS policies and must be executable
-- by the roles the policies apply to, otherwise policy checks fail.
GRANT EXECUTE ON FUNCTION public.has_role(uuid, public.app_role) TO authenticated, anon;
GRANT EXECUTE ON FUNCTION public.has_role_for_local(uuid, public.app_role, uuid) TO authenticated;
GRANT EXECUTE ON FUNCTION public.get_user_franchisee_id(uuid) TO authenticated;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260610000000_fix_enum_and_columns.sql */
/* ===== */
-- Fix: ensure all order_status enum values exist
ALTER TYPE public.order_status ADD VALUE IF NOT EXISTS 'pendiente';
ALTER TYPE public.order_status ADD VALUE IF NOT EXISTS 'pendiente_pago';

-- Fix: ensure all pedidos columns exist
ALTER TABLE public.pedidos
    ADD COLUMN IF NOT EXISTS tipo TEXT NOT NULL DEFAULT 'cocina',
    ADD COLUMN IF NOT EXISTS estado_cocina public.order_status,
    ADD COLUMN IF NOT EXISTS estado_bebidas public.order_status,
    ADD COLUMN IF NOT EXISTS edad_verificada_cliente BOOLEAN NOT NULL DEFAULT false;

-- Fix: ensure pedido_items.destino column exists
ALTER TABLE public.pedido_items
    ADD COLUMN IF NOT EXISTS destino TEXT NOT NULL DEFAULT 'cocina';

-- Fix: ensure menu_productos optional columns exist
ALTER TABLE public.menu_productos
    ADD COLUMN IF NOT EXISTS numero TEXT,
    ADD COLUMN IF NOT EXISTS tipo_pan TEXT,
    ADD COLUMN IF NOT EXISTS seccion TEXT;

-- Fix: ensure menu_categorias.logo_url column exists
ALTER TABLE public.menu_categorias
    ADD COLUMN IF NOT EXISTS logo_url TEXT;

-- Ensure franchisees table exists (created in a later migration)
CREATE TABLE IF NOT EXISTS public.franchisees (
    id UUID NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
    nombre TEXT NOT NULL,
    email TEXT,
    telefono TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
ALTER TABLE IF EXISTS public.franchisees ENABLE ROW LEVEL SECURITY;

-- Ensure locales has franchisee_id and GPS columns
ALTER TABLE public.locales
    ADD COLUMN IF NOT EXISTS franchisee_id UUID REFERENCES public.franchisees(id) ON DELETE SET NULL,
    ADD COLUMN IF NOT EXISTS lat NUMERIC,
    ADD COLUMN IF NOT EXISTS lng NUMERIC;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260610100000_auto_cancel_stale_orders.sql */
/* ===== */
-- Cancelación automática de pedidos expirados
-- Pedidos en pendiente_pago > 15 min se cancelan solos

CREATE EXTENSION IF NOT EXISTS pg_cron;

```

```

CREATE OR REPLACE FUNCTION public.cancelar_pedidos_expirados()
RETURNS integer
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
    filas_afectadas integer;
BEGIN
    UPDATE public.pedidos
    SET estado = 'cancelado'
    WHERE estado = 'pendiente_pago'
    AND created_at < NOW() - INTERVAL '15 minutes';

    GET DIAGNOSTICS filas_afectadas = ROW_COUNT;
    RETURN filas_afectadas;
END;
$$;

-- Solo ejecutable por service_role, no por anon/authenticated
REVOKE ALL ON FUNCTION public.cancelar_pedidos_expirados() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.cancelar_pedidos_expirados() TO service_role;

-- Programar cada 5 minutos
SELECT cron.schedule(
    'cancelar-pedidos-expirados',
    '* / 5 * * * *',
    'SELECT public.cancelar_pedidos_expirados()'
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260610110000_rate_limit_and_webhook_logs.sql */
/* ===== */
-- Contador de intentos de creación de PaymentIntent por pedido (rate limiting)
ALTER TABLE pedidos ADD COLUMN IF NOT EXISTS pi_attempts INT NOT NULL DEFAULT 0;

-- Tabla de log de webhooks Stripe para auditoría y deduplicación
CREATE TABLE IF NOT EXISTS webhook_logs (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    event_id    TEXT          NOT NULL UNIQUE,
    event_type  TEXT          NOT NULL,
    pedido_id   UUID          REFERENCES pedidos(id) ON DELETE SET NULL,
    payload     JSONB         NOT NULL,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS webhook_logs_pedido_id_idx ON webhook_logs(pedido_id);
CREATE INDEX IF NOT EXISTS webhook_logs_event_type_idx ON webhook_logs(event_type);
CREATE INDEX IF NOT EXISTS webhook_logs_created_at_idx ON webhook_logs(created_at DESC);

ALTER TABLE webhook_logs ENABLE ROW LEVEL SECURITY;
-- El service_role (Edge Functions) bypassa RLS – no se necesitan políticas públicas

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260610120000_push_notifications.sql */
/* ===== */
-- Tabla de suscripciones Web Push por pedido
CREATE TABLE IF NOT EXISTS push_subscriptions (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id  TEXT          NOT NULL,
    pedido_id   UUID          REFERENCES pedidos(id) ON DELETE CASCADE,
    subscription JSONB         NOT NULL,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    UNIQUE (session_id, pedido_id)
);

CREATE INDEX IF NOT EXISTS push_sub_session_idx ON push_subscriptions(session_id);
CREATE INDEX IF NOT EXISTS push_sub_pedido_idx ON push_subscriptions(pedido_id);

ALTER TABLE push_subscriptions ENABLE ROW LEVEL SECURITY;

-- Permitir inserción anónima (la suscripción contiene un endpoint público del navegador)
CREATE POLICY "allow_insert_push_sub" ON push_subscriptions
FOR INSERT WITH CHECK (true);

-- — Trigger: notificar cuando un pedido pasa a 'listo' —————
--
-- PREREQUISITO (ejecutar UNA VEZ en el SQL Editor de Supabase):
--
--     ALTER DATABASE postgres SET "app.settings.push_notify_secret" = 'TU_SECRETO_AQUI';
--
-- El mismo valor debe añadirse como secret de la Edge Function:
--     npx supabase secrets set PUSH_NOTIFY_SECRET=TU_SECRETO_AQUI
--     npx supabase secrets set VAPID_PUBLIC_KEY=BBUwbhLXyo0sHB5G2v5E1h10ix1SRw1fPvFngRo8K7413pbaiaQsc1ZGNYdS2Xu2aJdX_OiiUaCz35FDNxsphk
--     npx supabase secrets set VAPID_PRIVATE_KEY=kengIIKDnLJF5ja3zuJYKnVc-3ZNXbplw4_r39KyaQ
--
CREATE OR REPLACE FUNCTION notify_order_listo()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$

```



```

DECLARE
    v_secret TEXT;
    v_fired BOOLEAN := FALSE;
BEGIN
    v_secret := current_setting('app.settings.push_notify_secret', TRUE);
    IF v_secret IS NULL OR v_secret = '' THEN RETURN NEW; END IF;

    IF NEW.estado_cocina = 'listo' AND OLD.estado_cocina IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;
    IF NEW.estado_bebidas = 'listo' AND OLD.estado_bebidas IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;
    IF NEW.estado = 'listo' AND OLD.estado IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;

    IF v_fired THEN
        PERFORM net.http_post(
            url := 'https://znmhqnmmtkaillwfscu.supabase.co/functions/v1/notify-order-ready',
            headers := jsonb_build_object(
                'Content-Type', 'application/json',
                'x-push-notify-secret', v_secret
            ),
            body := jsonb_build_object(
                'pedido_id', NEW.id,
                'session_id', NEW.session_id,
                'numero_pedido', NEW.numero_pedido
            )::text
        );
    END IF;
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS on_pedido_listo ON pedidos;
CREATE TRIGGER on_pedido_listo
AFTER UPDATE ON pedidos
FOR EACH ROW EXECUTE FUNCTION notify_order_listo();

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260610130000_push_notify_secret.sql */
/* ===== */
-- Reemplaza la función del trigger con el secreto incrustado directamente
-- (SECURITY DEFINER garantiza que solo el propietario de la función puede ejecutarla)
CREATE OR REPLACE FUNCTION notify_order_listo()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_secret TEXT; -- se lee de public.app_secrets (nunca hardcodeado en el repo)
    v_fired BOOLEAN := FALSE;
BEGIN
    SELECT value INTO v_secret FROM public.app_secrets WHERE key = 'push_notify_secret';

    IF NEW.estado_cocina = 'listo' AND OLD.estado_cocina IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;
    IF NEW.estado_bebidas = 'listo' AND OLD.estado_bebidas IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;
    IF NEW.estado = 'listo' AND OLD.estado IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;

    IF v_fired THEN
        PERFORM net.http_post(
            url := 'https://znmhqnmmtkaillwfscu.supabase.co/functions/v1/notify-order-ready',
            headers := jsonb_build_object(
                'Content-Type', 'application/json',
                'x-push-notify-secret', v_secret
            ),
            body := jsonb_build_object(
                'pedido_id', NEW.id,
                'session_id', NEW.session_id,
                'numero_pedido', NEW.numero_pedido
            )::text
        );
    END IF;
    RETURN NEW;
END;
$$;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260612000000_alergenos_menu_activo.sql */
/* ===== */
-- MIGRACIÓN: Alérgenos para todos los productos activos (excepto MontyAhorro)
-- Fecha: 2026-06-12
-- Reglamento UE 1169/2011 – 14 alérgenos de declaración obligatoria
--
-- ⚠️ AVISO LEGAL: Esta información ha sido inferida a partir de los ingredientes
-- visibles en el nombre y descripción de cada producto, y validada contra el
-- histórico de alérgenos del menú anterior (mismo establecimiento).
-- DEBE SER REVISADA Y VALIDADA por el establecimiento antes de publicarse
-- en producción. Los alérgenos de trazas no están incluidos.
--
-- Seguro de re-ejecutar: ON CONFLICT DO NOTHING previene duplicados.
-- =====

-- =====
-- 1. MONTADITOS – menú activo numerado 01-100
-- Estrategia: SELECT por numero (evita hardcodear 100 UUIDs).
```

```

-- Todos contienen GLUTEN (base de pan de trigo).
-- categoria_id = f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9
-- =====
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
AND p.disponible = true
AND (p.numero, a.codigo) IN (

-- — DE LA CASA (01-10) —————
-- 01 Jamón Gran Reserva y aceite de oliva
('01','gluten'),
-- 02 Tortilla de patatas y tomate
('02','gluten'), ('02','huevos'),
-- 03 Pulled pork BBQ
('03','gluten'),
-- 04 Pollo y salsa alioli [alioli → huevos]
('04','gluten'), ('04','huevos'),
-- 05 Carrillera al vino tinto [vino → sulfitos]
('05','gluten'), ('05','sulfitos'),
-- 06 Calamarcitos y mayonesa [calamar → moluscos | mayo → huevos]
('06','gluten'), ('06','huevos'), ('06','moluscos'),
-- 07 Pollo kebab y salsa BBQ [kebab → soja+apio | BBQ → apio]
('07','gluten'), ('07','soja'), ('07','apio'),
-- 08 Bacon ahumado y queso madurado [queso → lácteos]
('08','gluten'), ('08','lacteos'),
-- 09 Torreznos y salsa brava
('09','gluten'),
-- 10 Lomo al ajillo y salsa 100M
('10','gluten'),

-- — CLÁSICOS (11-26) —————
-- 11 Tortilla de patatas y queso madurado
('11','gluten'), ('11','huevos'), ('11','lacteos'),
-- 12 Tortilla de patatas, bacon ahumado y salsa alioli
('12','gluten'), ('12','huevos'),
-- 13 Tortilla de patatas, tomate y mayonesa
('13','gluten'), ('13','huevos'),
-- 14 Tortilla de patatas y mojo picón
('14','gluten'), ('14','huevos'),
-- 15 Tortilla de patatas, patatas paja y salsa 100M
('15','gluten'), ('15','huevos'),
-- 16 Tortilla de patatas, cebolla crujiente y salsa BBQ [BBQ → apio]
('16','gluten'), ('16','huevos'), ('16','apio'),
-- 17 Pollo y queso madurado
('17','gluten'), ('17','lacteos'),
-- 18 Pollo, tomate y mojo picón
('18','gluten'),
-- 19 Pollo, patatas paja y salsa de mostaza y miel
('19','gluten'), ('19','mostaza'),
-- 20 Pollo, bacon ahumado y mayonesa
('20','gluten'), ('20','huevos'),
-- 21 Pollo, patatas paja y salsa BBQ [BBQ → apio]
('21','gluten'), ('21','apio'),
-- 22 Pollo kebab y tomate
('22','gluten'), ('22','soja'), ('22','apio'),
-- 23 Pollo kebab y salsa cheddar
('23','gluten'), ('23','soja'), ('23','apio'), ('23','lacteos'),
-- 24 Pollo kebab, tomate y salsa 100M
('24','gluten'), ('24','soja'), ('24','apio'),
-- 25 Pollo kebab, patatas paja y salsa BBQ
('25','gluten'), ('25','soja'), ('25','apio'),
-- 26 Pollo kebab, bacon ahumado y mayonesa
('26','gluten'), ('26','soja'), ('26','apio'), ('26','huevos'),

-- — IMPRESCINDIBLES (27-47) —————
-- 27 Pulled pork BBQ y salsa cheddar
('27','gluten'), ('27','lacteos'),
-- 28 Pulled pork BBQ y bacon ahumado
('28','gluten'),
-- 29 Pulled pork BBQ y salsa brava
('29','gluten'),
-- 30 Pulled pork BBQ y patatas paja
('30','gluten'),
-- 31 Pulled pork BBQ y cebolla crujiente
('31','gluten'),
-- 32 Lomo al ajillo y queso madurado
('32','gluten'), ('32','lacteos'),
-- 33 Lomo al ajillo y queso gorgonzola
('33','gluten'), ('33','lacteos'),
-- 34 Lomo al ajillo y mojo picón
('34','gluten'),
-- 35 Lomo al ajillo, tomate y patatas paja
('35','gluten'),
-- 36 Lomo al ajillo, tomate y mayonesa
('36','gluten'), ('36','huevos'),
-- 37 Lomo al ajillo, bacon ahumado y salsa alioli
('37','gluten'), ('37','huevos'),
-- 38 Calamarcitos y salsa alioli
('38','gluten'), ('38','huevos'), ('38','moluscos'),

```

```

-- 39 Calamarcitos y salsa 100M
('39','gluten'), ('39','moluscos'),
-- 40 Calamarcitos y guacamole
('40','gluten'), ('40','moluscos'),
-- 41 Calamarcitos, salsa brava y mayonesa
('41','gluten'), ('41','huevos'), ('41','moluscos'),
-- 42 Calamarcitos, tomate y mayonesa
('42','gluten'), ('42','huevos'), ('42','moluscos'),
-- 43 Bacon ahumado, tomate y mayonesa
('43','gluten'), ('43','huevos'),
-- 44 Bacon ahumado, cebolla crujiente y salsa 100M
('44','gluten'),
-- 45 Bacon ahumado, tomate y queso gorgonzola
('45','gluten'), ('45','lacteos'),
-- 46 Bacon ahumado, patatas paja y mayonesa
('46','gluten'), ('46','huevos'),
-- 47 Bacon ahumado, tomate y queso madurado
('47','gluten'), ('47','lacteos'),

-- — ESPECIALES (48-67) —————
-- 48 Jamón Gran Reserva y mantequilla [mantequilla → lácteos]
('48','gluten'), ('48','lacteos'),
-- 49 Jamón Gran Reserva y tomate
('49','gluten'),
-- 50 Jamón Gran Reserva, tomate y patatas paja
('50','gluten'),
-- 51 Jamón Gran Reserva y tortilla de patatas
('51','gluten'), ('51','huevos'),
-- 52 Carrillera al vino tinto y salsa alioli
('52','gluten'), ('52','huevos'), ('52','sulfitos'),
-- 53 Carrillera al vino tinto y patatas paja
('53','gluten'), ('53','sulfitos'),
-- 54 Carrillera al vino tinto y tomate
('54','gluten'), ('54','sulfitos'),
-- 55 Carrillera al vino tinto y cebolla crujiente
('55','gluten'), ('55','sulfitos'),
-- 56 Carrillera al vino tinto y bacon ahumado
('56','gluten'), ('56','sulfitos'),
-- 57 Torreznos y mayonesa
('57','gluten'), ('57','huevos'),
-- 58 Torreznos y salsa alioli
('58','gluten'), ('58','huevos'),
-- 59 Torreznos y salsa 100M
('59','gluten'),
-- 60 Salmón ahumado y queso gorgonzola
('60','gluten'), ('60','pescado'), ('60','lacteos'),
-- 61 Salmón ahumado y tomate
('61','gluten'), ('61','pescado'),
-- 62 Salmón ahumado y salsa de mostaza y miel
('62','gluten'), ('62','pescado'), ('62','mostaza'),
-- 63 Salmón ahumado y guacamole
('63','gluten'), ('63','pescado'),
-- 64 Chorizo parrillero y salsa brava
('64','gluten'),
-- 65 Chorizo parrillero y queso gorgonzola
('65','gluten'), ('65','lacteos'),
-- 66 Chorizo parrillero y salsa BBQ [BBQ → apio]
('66','gluten'), ('66','apio'),
-- 67 Chorizo parrillero y guacamole
('67','gluten'),

-- — MONTYCOOKIE (68-70) – galleta con chocolate —————
-- 68 MontyCookie doble chocolate y sirope de caramelo toffee
('68','gluten'), ('68','huevos'), ('68','lacteos'), ('68','sesamo'),
-- 69 MontyCookie chocolate y sirope de pistacho [pistacho → frutas_cascara]
('69','gluten'), ('69','huevos'), ('69','lacteos'), ('69','sesamo'), ('69','frutas_cascara'),
-- 70 MontyCookie chocolate y sirope de chocolate
('70','gluten'), ('70','huevos'), ('70','lacteos'), ('70','sesamo'),

-- — MONTYDINAS (71-75) – piadina (pan plano de trigo) —————
-- 71 Piadina de jamón cocido y queso mozzarella
('71','gluten'), ('71','lacteos'),
-- 72 Piadina de pepperoni y queso mozzarella
('72','gluten'), ('72','lacteos'),
-- 73 Piadina de pollo, tomate, queso mozzarella y orégano
('73','gluten'), ('73','lacteos'),
-- 74 Piadina de jamón Gran Reserva, queso mozzarella y orégano
('74','gluten'), ('74','lacteos'),
-- 75 Piadina de jamón cocido, queso madurado y tomate
('75','gluten'), ('75','lacteos'),

-- — MONTYPERROS (76-80) – pan brioche con sésamo —————
-- 76 Hot dog, ketchup y mayonesa [salchicha → soja | pan → sésamo]
('76','gluten'), ('76','soja'), ('76','sesamo'), ('76','huevos'),
-- 77 Hot dog, cebolla crujiente y mojo picón
('77','gluten'), ('77','soja'), ('77','sesamo'),
-- 78 Hot dog, guacamole y salsa cheddar
('78','gluten'), ('78','soja'), ('78','sesamo'), ('78','lacteos'),
-- 79 Hot dog, patatas paja y salsa alioli
('79','gluten'), ('79','soja'), ('79','sesamo'), ('79','huevos'),
-- 80 Hot dog, cebolla crujiente y salsa 100M
('80','gluten'), ('80','soja'), ('80','sesamo'),

```

```

-- — MONTYBURGERS (81-85) – pan brioche con sésamo —————
-- 81 Burger, queso madurado, tomate y mayonesa
('81','gluten'), ('81','huevos'), ('81','lacteos'), ('81','soja'), ('81','sesamo'),
-- 82 Burger, queso madurado y mojo picón
('82','gluten'), ('82','lacteos'), ('82','soja'), ('82','sesamo'),
-- 83 Burger, guacamole y bacon ahumado
('83','gluten'), ('83','soja'), ('83','sesamo'),
-- 84 Burger, bacon ahumado y salsa cheddar
('84','gluten'), ('84','lacteos'), ('84','soja'), ('84','sesamo'),
-- 85 Burger, queso madurado y pepperoni
('85','gluten'), ('85','lacteos'), ('85','soja'), ('85','sesamo'),

-- — MONTYPIZZAS (86-90) – base de pan tipo empotraíto —————
-- 86 BBQ: bacon ahumado, queso mozzarella, cebolla crujiente y salsa BBQ
('86','gluten'), ('86','lacteos'), ('86','soja'), ('86','apio'),
-- 87 Pollo: pollo kebab, queso mozzarella, salsa pizza y orégano
('87','gluten'), ('87','lacteos'), ('87','soja'), ('87','apio'),
-- 88 3 Quesos: queso madurado, queso mozzarella, queso gorgonzola y orégano
('88','gluten'), ('88','lacteos'),
-- 89 Pulled Pork: pulled pork BBQ, queso mozzarella, cebolla crujiente y salsa BBQ
('89','gluten'), ('89','lacteos'), ('89','apio'),
-- 90 Pepperoni: pepperoni, queso mozzarella, salsa pizza y orégano
('90','gluten'), ('90','lacteos'), ('90','soja'),

-- — MONTYGOURMET (91-100) – pan especial tipo empotraíto —————
-- 91 Tortilla de patatas, tomate y mayonesa
('91','gluten'), ('91','huevos'),
-- 92 Salmón ahumado y huevo hilado [huevo hilado → huevos]
('92','gluten'), ('92','pescado'), ('92','huevos'),
-- 93 Salmón ahumado y pintxo donostiarra [anchoa → pescado | pintxo → huevos]
('93','gluten'), ('93','pescado'), ('93','huevos'),
-- 94 Pintxo donostiarra y atún [boquerón/anchoa+atún → pescado]
('94','gluten'), ('94','pescado'), ('94','huevos'),
-- 95 Pintxo donostiarra y huevo hilado
('95','gluten'), ('95','pescado'), ('95','huevos'),
-- 96 Jamón cocido, queso madurado y mantequilla
('96','gluten'), ('96','lacteos'),
-- 97 Jamón cocido, queso madurado, tomate y mayonesa
('97','gluten'), ('97','huevos'), ('97','lacteos'),
-- 98 Atún, tomate y mayonesa
('98','gluten'), ('98','pescado'), ('98','huevos'),
-- 99 Atún, huevo hilado y mayonesa
('99','gluten'), ('99','pescado'), ('99','huevos'),
-- 100 Jamón Gran Reserva y mantequilla
('100','gluten'), ('100','lacteos')
)
ON CONFLICT DO NOTHING;

-- =====
-- 2. BEBIDAS – con alérgenos declarables
-- Nota: Los destilados (gin, vodka, ron, whisky) están exentos de declarar
-- gluten por Regl. UE 1169/2011 Anexo II, nota 8 (destilación elimina proteínas).
-- Las cervezas SÍ contienen gluten (proceso no destilado).
-- Los vinos contienen sulfitos (>10 mg/L, umbral de declaración).
-- categoria_id = 2a9b31b6-8752-4bbd-8950-3219458f7fa9
-- =====
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN alergenos a
WHERE p.categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND p.disponible = true
AND (p.numero, a.codigo) IN (
-- B19 Petroni Spritz [espumoso base vino → sulfitos]
('B19','sulfitos'),
-- B20 Crema Ruavieja [crema de licor → lácteos] – ya en BD, ON CONFLICT seguro
('B20','lacteos'),
-- B21 Jarra Cruzcampo Especial [cerveza malta cebada → gluten]
('B21','gluten'),
-- B22 Jarra Ladrón de Verano [base vino → sulfitos]
('B22','sulfitos'),
-- B23 Jarra Cruzcampo Radler [cerveza → gluten]
('B23','gluten'),
-- B24 Jarra Ladrón de Manzanas [sidra/vino manzana → sulfitos]
('B24','sulfitos'),
-- B25 Jarra Heineken [cerveza → gluten]
('B25','gluten'),
-- B26 Jarra El Águila Dorada [cerveza → gluten]
('B26','gluten'),
-- B28 Heineken 0.0 [sin alcohol pero malta cebada → gluten]
('B28','gluten'),
-- B29 Desperados [cerveza con tequila → gluten]
('B29','gluten'),
-- B30 Paulaner [cerveza alemana → gluten]
('B30','gluten'),
-- B31 Buenos Días Ribera de Duero [vino → sulfitos]
('B31','sulfitos'),
-- B32 Cune Rosado Navarra [vino → sulfitos]
('B32','sulfitos'),
-- B33 Cune Blanco Verdejo [vino → sulfitos]
('B33','sulfitos')
)
ON CONFLICT DO NOTHING;

```

```

('B33','sulfitos'),
-- B34 Cune Tinto Crianza Rioja [vino → sulfitos]
('B34','sulfitos'),
-- B36 Café con extra de crema Ruavieja [crema → lácteos] – ya en BD
('B36','lacteos'),
-- B39 Batido [lácteos] – ya en BD
('B39','lacteos')
)
ON CONFLICT DO NOTHING;

-- =====
-- 3. APERITIVOS – INSERT directo con UUIDs completos
-- categoria_id = 32447a0c-6911-4c3f-98d8-88a58d3e15ca
-- =====
INSERT INTO producto_alergenos (producto_id, alergeno_id) VALUES
-- Aceitunas de la abuela [aceitunas conservadas → sulfitos]
('f700cf0b-d6a8-4178-9dff-41f34099f0d8', '0c29a237-d755-4bc6-9bac-3853dae16710'),

-- Cheesy Cheetos® Bites [maíz + queso → gluten (cross-contact) + lácteos]
('0aa86602-8494-437b-9e60-5f95d71d8900', 'ff6a5259-3965-4ced-972a-6fcc1eb4ce63'), -- gluten
('0aa86602-8494-437b-9e60-5f95d71d8900', 'dc3a0d9d-0901-4dba-9d9b-910cbee7b454'), -- lacteos

-- Palomitas de pollo [masa frita → gluten | rebozado → huevos]
('fb40b377-e42f-4d3d-b347-4dc1cf8ddcd', 'ff6a5259-3965-4ced-972a-6fcc1eb4ce63'), -- gluten
('fb40b377-e42f-4d3d-b347-4dc1cf8ddcd', 'd91f784a-ca67-42be-b9eb-3f7983570b43'), -- huevos

-- Palomitas de queso gouda [masa frita → gluten | queso → lácteos | rebozado → huevos]
('17dc39bf-d4ba-4728-91eb-f7c4cba31442', 'ff6a5259-3965-4ced-972a-6fcc1eb4ce63'), -- gluten
('17dc39bf-d4ba-4728-91eb-f7c4cba31442', 'dc3a0d9d-0901-4dba-9d9b-910cbee7b454'), -- lacteos
('17dc39bf-d4ba-4728-91eb-f7c4cba31442', 'd91f784a-ca67-42be-b9eb-3f7983570b43'), -- huevos

-- Patatas fritas con cheddar y bacon ahumado [salsa cheddar → lácteos | bacon → sulfitos]
('3ca1c5df-1578-419d-920d-25ba5012e8b6', 'dc3a0d9d-0901-4dba-9d9b-910cbee7b454'), -- lacteos
('3ca1c5df-1578-419d-920d-25ba5012e8b6', '0c29a237-d755-4bc6-9bac-3853dae16710'), -- sulfitos

-- Salchichas 4 salsas [salchicha Frankfurt → gluten + soja | alioli → huevos]
('a83faca2-9849-4c16-a1ea-f682fd51b923', 'ff6a5259-3965-4ced-972a-6fcc1eb4ce63'), -- gluten
('a83faca2-9849-4c16-a1ea-f682fd51b923', '01e23771-5359-4c2c-a975-979554db43b3'), -- soja
('a83faca2-9849-4c16-a1ea-f682fd51b923', 'd91f784a-ca67-42be-b9eb-3f7983570b43'), -- huevos

-- Patatas fritas 4 salsas [alioli → huevos]
('9ab4f327-1535-4103-b292-2068d18d3d50', 'd91f784a-ca67-42be-b9eb-3f7983570b43'), -- huevos

-- Gildas [boquerón o anchoa → pescado]
('a1923e24-7fe9-4228-9301-fac8e37c9c7', '165d4deb-ebf0-4b4e-855b-883552908913'), -- pescado

-- Nachos [salsa cheddar → lácteos]
('c13184e9-096a-4032-af6a-b9feacb56904', 'dc3a0d9d-0901-4dba-9d9b-910cbee7b454') -- lacteos

-- Cucurucho de patatas chips → sin alérgenos declarables (patata + aceite)
ON CONFLICT DO NOTHING;

-- =====
-- 4. RACIONES – INSERT directo con UUIDs completos
-- categoria_id = 1b1a043f-3ff3-4cdf-ad9f-3a2bab122ccf
-- =====
INSERT INTO producto_alergenos (producto_id, alergeno_id) VALUES
-- Alitas de pollo [rebozado → gluten + huevos]
('2c747f14-7fad-440b-8947-7c913fbfc644', 'ff6a5259-3965-4ced-972a-6fcc1eb4ce63'), -- gluten
('2c747f14-7fad-440b-8947-7c913fbfc644', 'd91f784a-ca67-42be-b9eb-3f7983570b43'), -- huevos

-- Croquetas de Jamón [bechamel → gluten + lácteos + huevos]
('3795cecd-4939-49fa-99a5-46b3acf1cb42', 'ff6a5259-3965-4ced-972a-6fcc1eb4ce63'), -- gluten
('3795cecd-4939-49fa-99a5-46b3acf1cb42', 'dc3a0d9d-0901-4dba-9d9b-910cbee7b454'), -- lacteos
('3795cecd-4939-49fa-99a5-46b3acf1cb42', 'd91f784a-ca67-42be-b9eb-3f7983570b43'), -- huevos

-- Patatas fritas Bravioli [alioli → huevos]
('52f15185-4156-4227-868e-12f054e6406b', 'd91f784a-ca67-42be-b9eb-3f7983570b43'), -- huevos

-- Croquetas de Mac&Cheese [pasta + bechamel → gluten + lácteos + huevos]
('f105cb80-8834-4b88-b8bd-a13e0fcbab07', 'ff6a5259-3965-4ced-972a-6fcc1eb4ce63'), -- gluten
('f105cb80-8834-4b88-b8bd-a13e0fcbab07', 'dc3a0d9d-0901-4dba-9d9b-910cbee7b454'), -- lacteos
('f105cb80-8834-4b88-b8bd-a13e0fcbab07', 'd91f784a-ca67-42be-b9eb-3f7983570b43'), -- huevos

-- Tiras de pollo [rebozado → gluten + huevos]
('a40bd701-89cc-4457-9a6b-f179eeb6ad20', 'ff6a5259-3965-4ced-972a-6fcc1eb4ce63'), -- gluten
('a40bd701-89cc-4457-9a6b-f179eeb6ad20', 'd91f784a-ca67-42be-b9eb-3f7983570b43') -- huevos

-- Torreznos (crujientes) → sin alérgenos declarables (piel de cerdo + aceite)
ON CONFLICT DO NOTHING;

-- =====
-- 5. ENSALADAS – INSERT directo con UUIDs completos
-- categoria_id = 4170d84e-abbd-4310-a387-f6171017828a
-- =====
INSERT INTO producto_alergenos (producto_id, alergeno_id) VALUES
-- TexMex: pollo kebab, tomate, maíz, aceitunas, nachos TexMex, mostaza y miel
-- [kebab → soja + apio | nachos TexMex (trigo) → gluten | mostaza y miel → mostaza]
('bd45575b-709f-4f36-9e97-00c313cdb71f', 'ff6a5259-3965-4ced-972a-6fcc1eb4ce63'), -- gluten

```

```

('bd45575b-709f-4f36-9e97-00c313cdb71f', '01e23771-5359-4c2c-a975-979554db43b3'), -- soja
('bd45575b-709f-4f36-9e97-00c313cdb71f', '29d3d99d-1c0d-471e-8cbb-b6c079888bf2'), -- apio
('bd45575b-709f-4f36-9e97-00c313cdb71f', '5508d416-4387-4c25-ad27-59c1c962463a'), -- mostaza

-- Campera: pollo, tomate, queso madurado, aceitunas, picatostes, alioli
-- [picatostes → gluten | queso → lácteos | alioli → huevos | aceitunas → sulfitos]
('3d61ba4a-c009-4d0c-bd74-aa20ea5720e4', 'ff6a5259-3965-4ced-972a-6fcc1eb4ce63'), -- gluten
('3d61ba4a-c009-4d0c-bd74-aa20ea5720e4', 'dc3a0d9d-0901-4dba-9d9b-910cbee7b454'), -- lacteos
('3d61ba4a-c009-4d0c-bd74-aa20ea5720e4', 'd91f784a-ca67-42be-b9eb-3f7983570b43'), -- huevos
('3d61ba4a-c009-4d0c-bd74-aa20ea5720e4', '0c29a237-d755-4bc6-9bac-3853dae16710') -- sulfitos
ON CONFLICT DO NOTHING;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260613000000_fix_trigger_exception_handler.sql */
/* ===== */
-- Fix: el trigger notify_order_listo no tiene EXCEPTION handler.
-- Si net.http_post lanza cualquier excepción (permisos, timeout síncrono,
-- versión incompatible de pg_net, etc.), toda la transacción se cancelaba
-- y el UPDATE de estado_cocina nunca se confirmaba.
-- Ahora la notificación push es best-effort: si falla, se ignora.

CREATE OR REPLACE FUNCTION notify_order_listo()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_secret TEXT; -- se lee de public.app_secrets (nunca hardcodeado en el repo)
    v_fired BOOLEAN := FALSE;
BEGIN
    SELECT value INTO v_secret FROM public.app_secrets WHERE key = 'push_notify_secret';

    IF NEW.estado_cocina = 'listo' AND OLD.estado_cocina IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;
    IF NEW.estado_bebidas = 'listo' AND OLD.estado_bebidas IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;
    IF NEW.estado = 'listo' AND OLD.estado IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;

    IF v_fired THEN
        BEGIN
            PERFORM net.http_post(
                url := 'https://znmhqnmmtkaillwfscu.supabase.co/functions/v1/notify-order-ready',
                headers := jsonb_build_object(
                    'Content-Type', 'application/json',
                    'x-push-notify-secret', v_secret
                ),
                body := jsonb_build_object(
                    'pedido_id', NEW.id,
                    'session_id', NEW.session_id,
                    'numero_pedido', NEW.numero_pedido
                )::text
            );
        EXCEPTION WHEN OTHERS THEN
            -- La notificación push es best-effort; el UPDATE de pedido no debe fallar por esto.
            NULL;
        END;
    END IF;

    RETURN NEW;
END;
$$;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260613010000_actualizacion_precios.sql */
/* ===== */
-- ACTUALIZACIÓN DE PRECIOS – Junio 2026
-- =====
-- Categorías:
-- Montaditos : f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9
-- Bebidas : 2a9b31b6-8752-4bbd-8950-3219458f7fa9
-- Aperitivos : 32447a0c-6911-4c3f-98d8-88a58d3e15ca
-- Raciones : 1b1a043f-3ff3-4cdf-ad9f-3a2bab122ccf
-- Ensaladas : 4170d84e-abbd-4310-a387-f6171017828a
-- =====

-- — 1. MONTADITOS —————

-- De la Casa (01-10): 1.10 €
UPDATE public.menu_productos SET precio = 1.10
WHERE categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
AND numero ~ '^d+$' AND numero::integer BETWEEN 1 AND 10;

-- Clásicos (11-26): 1.40 €
UPDATE public.menu_productos SET precio = 1.40
WHERE categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
AND numero ~ '^d+$' AND numero::integer BETWEEN 11 AND 26;

-- Imprescindibles (27-47): 1.60 €
UPDATE public.menu_productos SET precio = 1.60
WHERE categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
AND numero ~ '^d+$' AND numero::integer BETWEEN 27 AND 47;

-- Especiales (48-67): 1.80 €

```

```

UPDATE public.menu_productos SET precio = 1.80
WHERE categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
AND numero ~ '^d+$' AND numero::integer BETWEEN 48 AND 67;

-- MontyCookie (68-70): 1.80 €
UPDATE public.menu_productos SET precio = 1.80
WHERE categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
AND numero ~ '^d+$' AND numero::integer BETWEEN 68 AND 70;

-- MontyDinas (71-75): 2.00 €
UPDATE public.menu_productos SET precio = 2.00
WHERE categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
AND numero ~ '^d+$' AND numero::integer BETWEEN 71 AND 75;

-- MontyPerros (76-80): 2.00 €
UPDATE public.menu_productos SET precio = 2.00
WHERE categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
AND numero ~ '^d+$' AND numero::integer BETWEEN 76 AND 80;

-- MontyBurgers (81-85): 2.00 €
UPDATE public.menu_productos SET precio = 2.00
WHERE categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
AND numero ~ '^d+$' AND numero::integer BETWEEN 81 AND 85;

-- MontyPizzas (86-90): 2.00 €
UPDATE public.menu_productos SET precio = 2.00
WHERE categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
AND numero ~ '^d+$' AND numero::integer BETWEEN 86 AND 90;

-- MontyGourmets (91-100): 2.00 €
UPDATE public.menu_productos SET precio = 2.00
WHERE categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
AND numero ~ '^d+$' AND numero::integer BETWEEN 91 AND 100;

-- — 2. ENSALADAS —————

UPDATE public.menu_productos SET precio = 3.50
WHERE categoria_id = '4170d84e-abbd-4310-a387-f6171017828a';

-- — 3. RACIONES —————

UPDATE public.menu_productos SET precio = 7.50
WHERE categoria_id = '1b1a043f-3ff3-4cdf-ad9f-3a2bab122ccf';

-- — 4. APERITIVOS —————
-- UUIDs de la migración 20260612000000_alergenosen_menu_activo.sql.

-- Aceitunas: 1.00 €
UPDATE public.menu_productos SET precio = 1.00
WHERE id = 'f700cf0b-d6a8-4178-9dff-41f34099f0d8';

-- Gildas: 1.50 €
UPDATE public.menu_productos SET precio = 1.50
WHERE id = 'a1923e24-7fe9-4228-9301-facb8e37c9c7';

-- Patatas fritas (cheddar y bacon + 4 salsas): 2.00 €
UPDATE public.menu_productos SET precio = 2.00
WHERE id IN (
    '3ca1c5df-1578-419d-920d-25ba5012e8b6',
    '9ab4f327-1535-4103-b292-2068d18d3d50'
);

-- Nachos (ambas variantes por nombre): 2.00 €
UPDATE public.menu_productos SET precio = 2.00
WHERE categoria_id = '32447a0c-6911-4c3f-98d8-88a58d3e15ca'
AND nombre ILIKE '%nachos%';

-- Palomitas de pollo, Palomitas de queso gouda,
-- Salchichas 4 salsas, Cheesy Cheetos Bites: 2.50 €
UPDATE public.menu_productos SET precio = 2.50
WHERE id IN (
    'fb40b377-e42f-4d3d-b347-4dc1cf8ddcd',
    '17dc39bf-d4ba-4728-91eb-f7c4cba31442',
    'a83faca2-9849-4c16-a1ea-f682fd51b923',
    '0aa86602-8494-437b-9e60-5f95d71d8900'
);

-- — 5. BEBIDAS —————
-- Secciones confirmadas por imagen del menú real.
-- Orden de ejecución: de mayor a menor precio para evitar solapamientos.

-- Tardeo Premium – Ron Havana, Beefeater (gin y 0.0), Ballantine's,
-- Vodka Absolut, Petroni Spritz, Crema Ruavieja: 6.00 €
UPDATE public.menu_productos SET precio = 6.00
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND (
    nombre ILIKE '%havana%'
    OR nombre ILIKE '%beefeater%'

```

```

    OR nombre ILIKE '%ballantine%'
    OR (nombre ILIKE '%absolut%' AND nombre NOT ILIKE '%sprite%')
    OR nombre ILIKE '%petroni%'
    OR (nombre ILIKE '%ruavieja%' AND nombre NOT ILIKE '%café%' AND nombre NOT ILIKE '%cafe%')
);

-- Tardeo chill – Absolut & Sprite, Bacardi & Coke, Jack & Coke: 3.50 €
UPDATE public.menu_productos SET precio = 3.50
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND (
    (nombre ILIKE '%absolut%' AND nombre ILIKE '%sprite%')
    OR nombre ILIKE '%bacardi%'
    OR (nombre ILIKE '%jack%' AND nombre ILIKE '%coke%')
);

-- Energéticas – Monster (todos los sabores): 3.00 €
UPDATE public.menu_productos SET precio = 3.00
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND nombre ILIKE '%monster%';

-- Paulaner (excepción botellín): 3.50 €
UPDATE public.menu_productos SET precio = 3.50
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND nombre ILIKE '%paulaner%';

-- Botellines – Cruzcampo, Heineken 0.0, Desperados, El Águila: 2.50 €
-- (se excluyen Jarras y Paulaner)
UPDATE public.menu_productos SET precio = 2.50
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND (
    nombre ILIKE '%cruzcampo%'
    OR nombre ILIKE '%heineken%'
    OR nombre ILIKE '%desperados%'
    OR nombre ILIKE '%el águila%'
    OR nombre ILIKE '%el aguila%'
)
AND nombre NOT ILIKE '%quijote%'
AND nombre NOT ILIKE '%sancho%'
AND nombre NOT ILIKE '%paulaner%';

-- Jarras: lógica por tipo de cerveza + tamaño (Quijote / Sancho).
--
-- HELADAS (Cruzcampo Especial + Ladrón de Verano):
-- Quijote = 1.50 € | Sancho = 2.00 €
--
-- PREMIUM (Cruzcampo Radler, Ladrón de Manzanas, Heineken, El Águila):
-- Quijote = 2.00 € | Sancho = 2.50 €

-- Jarra Sancho premium (Radler, Ladrón de Manzanas, Heineken, El Águila): 2.50 €
UPDATE public.menu_productos SET precio = 2.50
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND nombre ILIKE '%sancho%'
AND (
    nombre ILIKE '%radler%'
    OR nombre ILIKE '%manzana%'
    OR nombre ILIKE '%heineken%'
    OR nombre ILIKE '%águila%'
    OR nombre ILIKE '%aguila%'
);

-- Jarra Quijote premium (Radler, Ladrón de Manzanas, Heineken, El Águila): 2.00 €
UPDATE public.menu_productos SET precio = 2.00
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND nombre ILIKE '%quijote%'
AND (
    nombre ILIKE '%radler%'
    OR nombre ILIKE '%manzana%'
    OR nombre ILIKE '%heineken%'
    OR nombre ILIKE '%águila%'
    OR nombre ILIKE '%aguila%'
);

-- Jarra Sancho heladas (Cruzcampo Especial + Ladrón de Verano): 2.00 €
UPDATE public.menu_productos SET precio = 2.00
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND nombre ILIKE '%sancho%'
AND (
    (nombre ILIKE '%cruzcampo%' AND nombre NOT ILIKE '%radler%')
    OR (nombre ILIKE '%ladrón%' AND nombre NOT ILIKE '%manzana%')
    OR (nombre ILIKE '%ladrón%' AND nombre NOT ILIKE '%manzana%')
);

-- Jarra Quijote heladas (Cruzcampo Especial + Ladrón de Verano): 1.50 €
UPDATE public.menu_productos SET precio = 1.50
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND nombre ILIKE '%quijote%'
AND (
    (nombre ILIKE '%cruzcampo%' AND nombre NOT ILIKE '%radler%')
    OR (nombre ILIKE '%ladrón%' AND nombre NOT ILIKE '%manzana%')
    OR (nombre ILIKE '%ladrón%' AND nombre NOT ILIKE '%manzana%')
);

```



```

-- Vinos – Ribera, Rioja, Verdejo, Rosado, Buenos Días: 2.00 €
UPDATE public.menu_productos SET precio = 2.00
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND (
    nombre ILIKE '%ribera%'
    OR nombre ILIKE '%rioja%'
    OR nombre ILIKE '%verdejo%'
    OR nombre ILIKE '%rosado%'
    OR nombre ILIKE '%buenos días%'
    OR nombre ILIKE '%buenos dias%'
);

-- Appletiser (acepta ambas grafías): 2.20 €
UPDATE public.menu_productos SET precio = 2.20
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND (nombre ILIKE '%appletis%' OR nombre ILIKE '%appletie%');

-- Clásicas – Refrescos (Coca-Cola, Fanta, Sprite, Nестea, etc.) y Agua: 2.00 €
-- Se usa seccion como filtro principal; si no está poblado, se aplica por nombre.
UPDATE public.menu_productos SET precio = 2.00
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND (
    seccion ILIKE '%clásic%'
    OR nombre ILIKE '%agua%'
    OR nombre ILIKE '%coca%'
    OR nombre ILIKE '%fanta%'
    OR nombre ILIKE '%sprite%'
    OR nombre ILIKE '%nестea%'
    OR nombre ILIKE '%aquarius%'
    OR nombre ILIKE '%tonica%'
    OR nombre ILIKE '%tónica%'
)
AND nombre NOT ILIKE '%appletis%'
AND nombre NOT ILIKE '%appletie%'
AND nombre NOT ILIKE '%monster%';

-- Café (excluye combinado con Ruavieja): 1.40 €
UPDATE public.menu_productos SET precio = 1.40
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND (nombre ILIKE '%café%' OR nombre ILIKE '%cafe%')
AND nombre NOT ILIKE '%ruavieja%';

-- Infusiones: 1.30 €
UPDATE public.menu_productos SET precio = 1.30
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND (
    nombre ILIKE '%infusión%'
    OR nombre ILIKE '%manzanilla%'
    OR nombre ILIKE '%tila%'
    OR nombre ILIKE '%poleo%'
);

-- Zumo: 1.80 €
UPDATE public.menu_productos SET precio = 1.80
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND nombre ILIKE '%zumo%';

-- Batido: 1.50 €
UPDATE public.menu_productos SET precio = 1.50
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND nombre ILIKE '%batido%';

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260613020000_alergenos_oficial.sql */
/* ===== */
-- =====
-- MIGRACIÓN: Alérgenos oficiales
-- Fuente: PDF 100M_Alergenos_MARZO_2026.pdf
-- Reemplaza: 20260612000000_alergenos_menu_activo.sql (datos inferidos)
-- Solo alérgenos confirmados (X). Trazas (*) excluidas.
-- Reglamento UE 1169/2011 – 14 alérgenos de declaración obligatoria
-- =====

-- 1. Limpiar entradas existentes para las 5 categorías gestionadas
DELETE FROM producto_alergenos
WHERE producto_id IN (
    SELECT id FROM menu_productos
    WHERE categoria_id IN (
        'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9', -- Montaditos
        '2a9b31b6-8752-4bbd-8950-3219458f7fa9', -- Bebidas
        '32447a0c-6911-4c3f-98d8-88a58d3e15ca', -- Aperitivos
        '1b1a043f-3ff3-4cdf-ad9f-3a2bab12ccf', -- Raciones
        '4170d84e-abbd-4310-a387-f6171017828a' -- Ensaladas
    )
)
AND disponible = true
);

-- =====
-- 2. MONTADITOS (01-100)

```

```

-- Estrategia: SELECT por numero+codigo evita hardcodear 100 UUIDs.
-- Salsa 100M contiene: huevos(X) + soja(X) + mostaza(X) – confirmado en #10 y #39.
-- Carrillera al vino tinto contiene: lacteos(X) + sulfitos(X) – confirmado en #05.
-- =====
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
AND p.disponible = true
AND (p.numero, a.codigo) IN (

-- — DE LA CASA (01-10) —————
-- 01 Jamón Gran Reserva y aceite de oliva
('01','gluten'),
-- 02 Tortilla de patatas y tomate
('02','gluten'), ('02','huevos'),
-- 03 Pulled pork BBQ [soja de pulled pork es trazas *]
('03','gluten'),
-- 04 Pollo y salsa alioli
('04','gluten'), ('04','huevos'),
-- 05 Carrillera al vino tinto
('05','gluten'), ('05','lacteos'), ('05','sulfitos'),
-- 06 Calamarcitos y mayonesa
('06','gluten'), ('06','huevos'), ('06','moluscos'),
-- 07 Pollo kebab y salsa BBQ [soja no confirmada en PDF; solo gluten+apio]
('07','gluten'), ('07','apio'),
-- 08 Bacon ahumado y queso madurado
('08','gluten'), ('08','lacteos'),
-- 09 Torreznos y salsa brava
('09','gluten'),
-- 10 Lomo al ajillo y salsa 100M
('10','gluten'), ('10','huevos'), ('10','soja'), ('10','mostaza'),

-- — CLÁSICOS (11-26) —————
('11','gluten'), ('11','huevos'), ('11','lacteos'),
('12','gluten'), ('12','huevos'),
('13','gluten'), ('13','huevos'),
('14','gluten'), ('14','huevos'),
-- 15 tiene salsa 100M
('15','gluten'), ('15','huevos'), ('15','soja'), ('15','mostaza'),
-- 16 tiene salsa BBQ → apio
('16','gluten'), ('16','huevos'), ('16','apio'),
('17','gluten'), ('17','lacteos'),
('18','gluten'),
('19','gluten'), ('19','mostaza'),
('20','gluten'), ('20','huevos'),
-- 21 tiene salsa BBQ → apio
('21','gluten'), ('21','apio'),
-- 22 kebab+tomate sin BBQ ni salsa100M
('22','gluten'),
('23','gluten'), ('23','lacteos'),
-- 24 tiene salsa 100M
('24','gluten'), ('24','huevos'), ('24','soja'), ('24','mostaza'),
-- 25 tiene salsa BBQ → apio
('25','gluten'), ('25','apio'),
('26','gluten'), ('26','huevos'),

-- — IMPRESCINDIBLES (27-47) —————
('27','gluten'), ('27','lacteos'),
('28','gluten'),
('29','gluten'),
('30','gluten'),
('31','gluten'),
('32','gluten'), ('32','lacteos'),
('33','gluten'), ('33','lacteos'),
('34','gluten'),
('35','gluten'),
('36','gluten'), ('36','huevos'),
('37','gluten'), ('37','huevos'),
('38','gluten'), ('38','huevos'), ('38','moluscos'),
-- 39 Calamarcitos y salsa 100M [corrección PDF: añadir huevos+soja+mostaza]
('39','gluten'), ('39','huevos'), ('39','moluscos'), ('39','soja'), ('39','mostaza'),
('40','gluten'), ('40','moluscos'),
('41','gluten'), ('41','huevos'), ('41','moluscos'),
('42','gluten'), ('42','huevos'), ('42','moluscos'),
('43','gluten'), ('43','huevos'),
-- 44 tiene salsa 100M
('44','gluten'), ('44','huevos'), ('44','soja'), ('44','mostaza'),
('45','gluten'), ('45','lacteos'),
('46','gluten'), ('46','huevos'),
('47','gluten'), ('47','lacteos'),

-- — ESPECIALES (48-67) —————
('48','gluten'), ('48','lacteos'),
('49','gluten'),
('50','gluten'),
('51','gluten'), ('51','huevos'),
('52','gluten'), ('52','huevos'), ('52','lacteos'), ('52','sulfitos'),
('53','gluten'), ('53','lacteos'), ('53','sulfitos'),
('54','gluten'), ('54','lacteos'), ('54','sulfitos'),
('55','gluten'), ('55','lacteos'), ('55','sulfitos'),

```

```

('56','gluten'), ('56','lacteos'), ('56','sulfitos'),
('57','gluten'), ('57','huevos'),
('58','gluten'), ('58','huevos'),
-- 59 tiene salsa 100M
('59','gluten'), ('59','huevos'), ('59','soja'), ('59','mostaza'),
('60','gluten'), ('60','pescado'), ('60','lacteos'),
('61','gluten'), ('61','pescado'),
('62','gluten'), ('62','pescado'), ('62','mostaza'),
('63','gluten'), ('63','pescado'),
('64','gluten'),
('65','gluten'), ('65','lacteos'),
-- 66 tiene salsa BBQ → apio
('66','gluten'), ('66','apio'),
('67','gluten'),

-- — MONTYCOOKIE (68-70) —————
('68','gluten'), ('68','huevos'), ('68','lacteos'), ('68','sesamo'),
-- 69: pistacho → frutas_cascara; sésamo no confirmado en PDF
('69','gluten'), ('69','huevos'), ('69','lacteos'), ('69','frutas_cascara'),
-- 70: sésamo es trazas (*) en PDF
('70','gluten'), ('70','huevos'), ('70','lacteos'),

-- — MONTYDINAS (71-75) — piadina (trigo+avena) —————
-- soja de pepperoni es trazas (*)
('71','gluten'), ('71','lacteos'),
('72','gluten'), ('72','lacteos'),
('73','gluten'), ('73','lacteos'),
('74','gluten'), ('74','lacteos'),
('75','gluten'), ('75','lacteos'),

-- — MONTYPERROS (76-80) — brioche con sésamo —————
-- huevos del brioche es trazas (*); sésamo y soja de salchicha confirmados
('76','gluten'), ('76','sesamo'), ('76','soja'),
('77','gluten'), ('77','sesamo'), ('77','soja'),
('78','gluten'), ('78','sesamo'), ('78','soja'), ('78','lacteos'),
('79','gluten'), ('79','sesamo'), ('79','soja'),
-- 80 tiene salsa 100M
('80','gluten'), ('80','sesamo'), ('80','soja'), ('80','huevos'), ('80','mostaza'),

-- — MONTYBURGERS (81-85) — pan mini burger —————
-- huevos y sésamo son trazas (*) en PDF; solo lacteos confirmado donde hay queso
('81','gluten'), ('81','lacteos'),
('82','gluten'), ('82','lacteos'),
('83','gluten'),
('84','gluten'), ('84','lacteos'),
('85','gluten'), ('85','lacteos'),

-- — MONTYPIZZAS (86-90) —————
-- soja de pulled pork/pepperoni es trazas (*)
('86','gluten'), ('86','lacteos'),
('87','gluten'), ('87','lacteos'),
('88','gluten'), ('88','lacteos'),
('89','gluten'), ('89','lacteos'),
('90','gluten'), ('90','lacteos'),

-- — MONTYGOURMETS (91-100) — pan gourmet con huevos —————
-- Pan gourmet confirma huevos(X) en todos
('91','gluten'), ('91','huevos'),
('92','gluten'), ('92','pescado'), ('92','huevos'),
('93','gluten'), ('93','pescado'), ('93','huevos'),
('94','gluten'), ('94','pescado'), ('94','huevos'),
('95','gluten'), ('95','pescado'), ('95','huevos'),
-- 96 y 100: huevos añadidos por pan gourmet (no estaban en migración anterior)
('96','gluten'), ('96','huevos'), ('96','lacteos'),
('97','gluten'), ('97','huevos'), ('97','lacteos'),
('98','gluten'), ('98','pescado'), ('98','huevos'),
('99','gluten'), ('99','pescado'), ('99','huevos'),
('100','gluten'), ('100','huevos'), ('100','lacteos')
)

```

ON CONFLICT DO NOTHING;

```

-- =====
-- 3. BEBIDAS
-- Sin cambios respecto a la migración anterior.
-- Destilados exentos por Regl. UE 1169/2011 Anexo II nota 8.
-- =====
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN alergenos a
WHERE p.categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
AND p.disponible = true
AND (p.numero, a.codigo) IN (
    ('B19','sulfitos'), -- Petroni Spritz
    ('B20','lacteos'), -- Crema Ruavieja
    ('B21','gluten'), -- Jarra Cruzcampo Especial
    ('B22','sulfitos'), -- Jarra Ladrón de Verano
    ('B23','gluten'), -- Jarra Cruzcampo Radler
    ('B24','sulfitos'), -- Jarra Ladrón de Manzanas
    ('B25','gluten'), -- Jarra Heineken
    ('B26','gluten'), -- Jarra El Águila Dorada

```

```

        ('B28','gluten'),      -- Heineken 0.0
        ('B29','gluten'),      -- Desperados
        ('B30','gluten'),      -- Paulaner
        ('B31','sulfitos'),     -- Buenos Días Ribera de Duero
        ('B32','sulfitos'),     -- Cune Rosado Navarra
        ('B33','sulfitos'),     -- Cune Blanco Verdejo
        ('B34','sulfitos'),     -- Cune Tinto Crianza Rioja
        ('B36','lacteos'),      -- Café con Ruavieja
        ('B39','lacteos')      -- Batido
    )
ON CONFLICT DO NOTHING;

-- =====
-- 4. APERITIVOS
-- Correcciones respecto a migración anterior:
-- - Patatas fritas 4 salsas: se añade gluten(X) confirmado en PDF
-- - Palomitas de pollo: se elimina huevos (era trazas *)
-- - Palomitas de queso gouda: se elimina huevos (era trazas *)
-- - Nachos con bacon: se añade sulfitos(X) confirmado en PDF
-- - Aceitunas manzanilla: cubierta por patrón ILIKE '%aceituna%'
-- =====

-- Gluten
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'gluten') a
WHERE p.id IN (
    '9ab4f327-1535-4103-b292-2068d18d3d50', -- Patatas fritas 4 salsas
    'fb40b377-e42f-4d3d-b347-4dcb1cf8ddcd', -- Palomitas de pollo
    '17dc39bf-d4ba-4728-91eb-f7c4cba31442', -- Palomitas de queso gouda
    'a83faca2-9849-4c16-a1ea-f682fd51b923', -- Salchichas 4 salsas
    '0aa86602-8494-437b-9e60-5f95d71d8900' -- Cheesy Cheetos Bites
)
ON CONFLICT DO NOTHING;

-- Huevos (solo productos con huevos confirmados X, no trazas)
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'huevos') a
WHERE p.id IN (
    '9ab4f327-1535-4103-b292-2068d18d3d50', -- Patatas fritas 4 salsas
    'a83faca2-9849-4c16-a1ea-f682fd51b923' -- Salchichas 4 salsas
)
ON CONFLICT DO NOTHING;

-- Lácteos – productos con UUID conocido
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'lacteos') a
WHERE p.id IN (
    '3ca1c5df-1578-419d-920d-25ba5012e8b6', -- Patatas fritas cheddar y bacon
    '17dc39bf-d4ba-4728-91eb-f7c4cba31442', -- Palomitas de queso gouda
    '0aa86602-8494-437b-9e60-5f95d71d8900' -- Cheesy Cheetos Bites
)
ON CONFLICT DO NOTHING;

-- Lácteos – nachos (ambas variantes: cheddar+bacon y cheddar+guacamole)
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'lacteos') a
WHERE p.categoria_id = '32447a0c-6911-4c3f-98d8-88a58d3e15ca'
    AND p.nombre ILIKE '%nachos%'
    AND p.disponible = true
ON CONFLICT DO NOTHING;

-- Sulfitos – patatas fritas cheddar y bacon
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'sulfitos') a
WHERE p.id = '3ca1c5df-1578-419d-920d-25ba5012e8b6'
ON CONFLICT DO NOTHING;

-- Sulfitos – nachos con bacon (confirmado en PDF)
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'sulfitos') a
WHERE p.categoria_id = '32447a0c-6911-4c3f-98d8-88a58d3e15ca'
    AND p.nombre ILIKE '%nachos%'
    AND p.nombre ILIKE '%bacon%'
    AND p.disponible = true
ON CONFLICT DO NOTHING;

-- Sulfitos – aceitunas (de la abuela + manzanilla)
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id

```

```

FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'sulfitos') a
WHERE p.categoria_id = '32447a0c-6911-4c3f-98d8-88a58d3e15ca'
      AND p.nombre ILIKE '%aceituna%'
      AND p.disponible = true
ON CONFLICT DO NOTHING;

-- Soja
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'soja') a
WHERE p.id = 'a83faca2-9849-4c16-a1ea-f682fd51b923' -- Salchichas 4 salsas
ON CONFLICT DO NOTHING;

-- Pescado – gildas (boquerón y anchoa)
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'pescado') a
WHERE p.categoria_id = '32447a0c-6911-4c3f-98d8-88a58d3e15ca'
      AND p.nombre ILIKE '%gilda%'
      AND p.disponible = true
ON CONFLICT DO NOTHING;

-- Cucurucho de patatas chips → sin alérgenos confirmados

-- =====
-- 5. RACIONES
-- Correcciones respecto a migración anterior:
-- - Croquetas de Jamón: se elimina huevos (trazas * en PDF)
-- - Alitas de pollo BBQ: se elimina huevos, se añade apio
-- - Patatas fritas Bravioli: se elimina huevos (trazas * en PDF)
-- - Tiras de pollo: se elimina huevos (no confirmado en PDF)
-- - Torreznos: sin alérgenos confirmados (igual que antes)
-- =====

-- Gluten
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'gluten') a
WHERE p.id IN (
  '3795cecd-4939-49fa-99a5-46b3acf1cb42', -- Croquetas de Jamón
  'f105cb80-8834-4b88-b8bd-a13e0fcb607', -- Croquetas de Mac&Cheese
  '2c747f14-7fad-440b-8947-7c913fbfc644', -- Alitas de pollo
  'a40bd701-89cc-4457-9a6b-f179eeb6ad20' -- Tiras de pollo
)
ON CONFLICT DO NOTHING;

-- Lácteos
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'lacteos') a
WHERE p.id IN (
  '3795cecd-4939-49fa-99a5-46b3acf1cb42', -- Croquetas de Jamón
  'f105cb80-8834-4b88-b8bd-a13e0fcb607' -- Croquetas de Mac&Cheese
)
ON CONFLICT DO NOTHING;

-- Apio – alitas de pollo sabor BBQ
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'apio') a
WHERE p.id = '2c747f14-7fad-440b-8947-7c913fbfc644'
ON CONFLICT DO NOTHING;

-- Patatas fritas Bravioli (52f15185): huevos son trazas (*) → sin alérgenos confirmados
-- Torreznos: sin alérgenos confirmados en PDF

-- =====
-- 6. ENSALADAS
-- Correcciones respecto a migración anterior:
-- - TexMex: se eliminan soja y apio (trazas * en PDF); solo gluten+mostaza
-- - Campera: se elimina sulfitos (trazas * en PDF); solo gluten+huevos+lacteos
-- =====

-- TexMex
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN alergenos a
WHERE p.id = 'bd45575b-709f-4f36-9e97-00c313cdb71f'
      AND a.codigo IN ('gluten', 'mostaza')
ON CONFLICT DO NOTHING;

-- Campera
INSERT INTO producto_alergenos (producto_id, alergeno_id)

```

```

SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN alergenos a
WHERE p.id = '3d61ba4a-c009-4d0c-bd74-aa20ea5720e4'
  AND a.codigo IN ('gluten', 'huevos', 'lacteos')
ON CONFLICT DO NOTHING;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260613035000_security_constraints.sql */
/* ===== */
-- Constraints de seguridad: impedir importes negativos o cero en pedidos
ALTER TABLE public.pedidos
  ADD CONSTRAINT pedidos_total_non_negative CHECK (total >= 0);

ALTER TABLE public.pedido_items
  ADD CONSTRAINT items_cantidad_positive CHECK (cantidad > 0),
  ADD CONSTRAINT items_precio_unitario_positive CHECK (precio_unitario > 0);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614000000_precios_jarras.sql */
/* ===== */
-- Corrección de precios de jarras – Junio 2026
--
-- Lógica de precios (precio = Quijote; Sancho = precio + 0.50€):
-- Cerveza Premium → Quijote 2.00 € / Sancho 2.50 € → precio = 2.00
-- Jarras Heladas → Quijote 1.50 € / Sancho 2.00 € → precio = 1.50

-- Cerveza Premium: precio base = 2.00 €
UPDATE public.menu_productos
SET precio = 2.00
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
  AND seccion = 'Cerveza Premium';

-- Jarras Heladas: precio base = 1.50 € (Quijote)
UPDATE public.menu_productos
SET precio = 1.50
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
  AND seccion = 'Jarras Heladas';

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614010000_precios_agua_infusion.sql */
/* ===== */
-- Corrección de precios puntuales – Junio 2026
-- Agua mineral: 2.20 € (antes 2.00 €)
-- Infusiones: 1.30 € (confirmación, sin cambio)

-- Agua
UPDATE public.menu_productos
SET precio = 2.20
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
  AND nombre ILIKE '%agua%';

-- Infusiones (manzanilla, tila, poleo...)
UPDATE public.menu_productos
SET precio = 1.30
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
  AND (
    nombre ILIKE '%infusi%'
    OR nombre ILIKE '%manzanilla%'
    OR nombre ILIKE '%tila%'
    OR nombre ILIKE '%poleo%'
  );

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614020000_alergenos_montaditos_1_9.sql */
/* ===== */
-- FIX: Alérgenos montaditos 01-09
-- La migración 20260613020000 usaba numero = '01'..'09' (con cero),
-- pero la BD almacena numero = '1'..'9' (sin cero), por lo que el INSERT
-- no hizo match en esos productos. Se repite usando numero::integer = N.
-- Fuente: PDF 100M_Alergenos_MARZO_2026.pdf
-- =====

-- 1. Limpiar posibles entradas huérfanas de 1-9 (idempotente)
DELETE FROM producto_alergenos
WHERE producto_id IN (
  SELECT id FROM menu_productos
  WHERE categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND disponible = true
    AND numero ~ '^\\d+$'
    AND numero::integer BETWEEN 1 AND 9
);

-- 01 Jamón Gran Reserva y aceite de oliva – gluten
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'

```

```

    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 1
    AND a.codigo IN ('gluten')
ON CONFLICT DO NOTHING;

-- 02 Tortilla de patatas y tomate - gluten, huevos
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 2
    AND a.codigo IN ('gluten','huevos')
ON CONFLICT DO NOTHING;

-- 03 Pulled pork BBQ - gluten
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 3
    AND a.codigo IN ('gluten')
ON CONFLICT DO NOTHING;

-- 04 Pollo y salsa alioli - gluten, huevos
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 4
    AND a.codigo IN ('gluten','huevos')
ON CONFLICT DO NOTHING;

-- 05 Carrillera al vino tinto - gluten, lacteos, sulfitos
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 5
    AND a.codigo IN ('gluten','lacteos','sulfitos')
ON CONFLICT DO NOTHING;

-- 06 Calamarcitos y mayonesa - gluten, huevos, moluscos
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 6
    AND a.codigo IN ('gluten','huevos','moluscos')
ON CONFLICT DO NOTHING;

-- 07 Pollo kebab y salsa BBQ - gluten, apio
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 7
    AND a.codigo IN ('gluten','apio')
ON CONFLICT DO NOTHING;

-- 08 Bacon ahumado y queso madurado - gluten, lacteos
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 8
    AND a.codigo IN ('gluten','lacteos')
ON CONFLICT DO NOTHING;

-- 09 Torreznos y salsa brava - gluten
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 9
    AND a.codigo IN ('gluten')
ON CONFLICT DO NOTHING;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614030000_local_restricciones.sql */
/* ===== */
-- =====
-- FEATURE: Restricciones por local
-- Permite a cada local desactivar productos o secciones, con soporte
-- opcional para restricciones horarias (hora_limite: no disponible desde esa hora).
-- =====

CREATE TABLE IF NOT EXISTS public.local_restricciones (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    local_id    UUID          NOT NULL REFERENCES public.locales(id) ON DELETE CASCADE,
    producto_id UUID          REFERENCES public.menu_productos(id) ON DELETE CASCADE,
    seccion     TEXT,
    -- null → siempre no disponible en este local
    -- valor → no disponible A PARTIR de esa hora (ej. '17:00')
    hora_limite TIME,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_restriccion_target CHECK (producto_id IS NOT NULL OR seccion IS NOT NULL)
);

CREATE UNIQUE INDEX IF NOT EXISTS local_restricciones_producto_uq
ON public.local_restricciones (local_id, producto_id)

```

```

WHERE producto_id IS NOT NULL;

CREATE UNIQUE INDEX IF NOT EXISTS local_restricciones_seccion_uq
ON public.local_restricciones (local_id, seccion)
WHERE seccion IS NOT NULL;

ALTER TABLE public.local_restricciones ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Anyone can view restricciones"
ON public.local_restricciones FOR SELECT USING (true);

CREATE POLICY "Admins can manage restricciones"
ON public.local_restricciones FOR ALL
USING (public.has_role(auth.uid(), 'admin'));

-- — Restricciones del local de Móstoles —————

DO $$
DECLARE
    v_local UUID;
BEGIN
    SELECT id INTO v_local FROM public.locales WHERE slug = 'mostoles-constitucion';
    IF v_local IS NULL THEN
        RAISE NOTICE 'Local mostoles-constitucion no encontrado, saltando restricciones.';
        RETURN;
    END IF;

    -- 1. Sección "Tardeo Chill" completa
    INSERT INTO public.local_restricciones (local_id, seccion)
    VALUES (v_local, 'Tardeo Chill')
    ON CONFLICT DO NOTHING;

    -- 2. "Café e Infusiones" no disponible a partir de las 17:00
    INSERT INTO public.local_restricciones (local_id, seccion, hora_limite)
    VALUES (v_local, 'Café e Infusiones', '17:00')
    ON CONFLICT DO NOTHING;

    -- 3. Appletiser
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true AND nombre ILIKE '%appletiser%'
    ON CONFLICT DO NOTHING;

    -- 4. Batidos que no sean de chocolate
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
        AND nombre ILIKE '%batido%'
        AND nombre NOT ILIKE '%chocolate%'
    ON CONFLICT DO NOTHING;

    -- 5. Monster que no sea Ultra White / Blanco
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
        AND nombre ILIKE '%monster%'
        AND nombre !~* 'ultra|blanco|white'
    ON CONFLICT DO NOTHING;

    -- 6. Heineken con jarra (no Heineken 0.0 / sin alcohol)
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
        AND nombre ILIKE '%heineken%'
        AND nombre !~* '0[.]0|sin\s*alcohol|00'
    ON CONFLICT DO NOTHING;

    -- 7. Tercios Cruzcampo (no Cruzcampo Sin Gluten)
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
        AND nombre ILIKE '%cruzcampo%'
        AND nombre NOT ILIKE '%sin gluten%'
    ON CONFLICT DO NOTHING;

    -- 8. Spritz y Petroni
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
        AND (nombre ILIKE '%spritz%' OR nombre ILIKE '%petroni%')
    ON CONFLICT DO NOTHING;

    -- 9. Desperados
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
        AND nombre ILIKE '%desperados%'
    ON CONFLICT DO NOTHING;

END;
$$;

```



```

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614040000_mostoles_seed_y_restricciones.sql */
/* ===== */
-- =====
-- Insertar local Mostoles (si no existe) y sus restricciones de carta.
-- La migración 20260614030000 detectó que el slug 'mostoles-constitucion'
-- no estaba en la BD; se re-inserta aquí de forma idempotente.
-- =====

-- — 1. Franchisee (idempotente) -----

INSERT INTO public.franchisees (nombre, email, application_fee_percent)
VALUES ('Mostoles', 'mostoles@hat3x.com', 0)
ON CONFLICT (email) DO NOTHING;

-- — 2. Local (idempotente) -----
-- franchisee_id es NOT NULL (añadido en migración 20260522).

INSERT INTO public.locales (nombre, ciudad, direccion, slug, lat, lng, activo, franchisee_id)
SELECT
  'Mostoles - Av. de la Constitución',
  'Mostoles',
  'Av. de la Constitución, 23, 28931 Mostoles, Madrid',
  'mostoles-constitucion',
  40.3223,
  -3.8652,
  true,
  f.id
FROM public.franchisees f
WHERE f.email = 'mostoles@hat3x.com'
ON CONFLICT (slug) DO NOTHING;

-- — 2. Restricciones -----

DO $$
DECLARE
  v_local UUID;
BEGIN
  SELECT id INTO v_local FROM public.locales WHERE slug = 'mostoles-constitucion';
  IF v_local IS NULL THEN
    RAISE EXCEPTION 'Local mostoles-constitucion no encontrado incluso después del INSERT';
  END IF;

  -- Sección "Tardeo Chill" completa
  INSERT INTO public.local_restricciones (local_id, seccion)
  VALUES (v_local, 'Tardeo Chill')
  ON CONFLICT DO NOTHING;

  -- "Café e Infusiones" no disponible a partir de las 17:00
  INSERT INTO public.local_restricciones (local_id, seccion, hora_limite)
  VALUES (v_local, 'Café e Infusiones', '17:00')
  ON CONFLICT DO NOTHING;

  -- Appletiser
  INSERT INTO public.local_restricciones (local_id, producto_id)
  SELECT v_local, id FROM public.menu_productos
  WHERE disponible = true AND nombre ILIKE '%appletiser%'
  ON CONFLICT DO NOTHING;

  -- Batidos que no sean de chocolate
  INSERT INTO public.local_restricciones (local_id, producto_id)
  SELECT v_local, id FROM public.menu_productos
  WHERE disponible = true
    AND nombre ILIKE '%batido%'
    AND nombre NOT ILIKE '%chocolate%'
  ON CONFLICT DO NOTHING;

  -- Monster que no sea Ultra White / Blanco
  INSERT INTO public.local_restricciones (local_id, producto_id)
  SELECT v_local, id FROM public.menu_productos
  WHERE disponible = true
    AND nombre ILIKE '%monster%'
    AND nombre !~* 'ultra|blanco|white'
  ON CONFLICT DO NOTHING;

  -- Heineken (no Heineken 0.0 / sin alcohol)
  INSERT INTO public.local_restricciones (local_id, producto_id)
  SELECT v_local, id FROM public.menu_productos
  WHERE disponible = true
    AND nombre ILIKE '%heineken%'
    AND nombre !~* '0[.]0|sin\s*alcohol|00'
  ON CONFLICT DO NOTHING;

  -- Tercios Cruzcampo (no Cruzcampo Sin Gluten)
  INSERT INTO public.local_restricciones (local_id, producto_id)
  SELECT v_local, id FROM public.menu_productos
  WHERE disponible = true
    AND nombre ILIKE '%cruzcampo%'
    AND nombre NOT ILIKE '%sin gluten%'
  ON CONFLICT DO NOTHING;

```

```

-- Spritz y Petroni
INSERT INTO public.local_restricciones (local_id, producto_id)
SELECT v_local, id FROM public.menu_productos
WHERE disponible = true
AND (nombre ILIKE '%spritz%' OR nombre ILIKE '%petroni%')
ON CONFLICT DO NOTHING;

-- Desperados
INSERT INTO public.local_restricciones (local_id, producto_id)
SELECT v_local, id FROM public.menu_productos
WHERE disponible = true
AND nombre ILIKE '%desperados%'
ON CONFLICT DO NOTHING;

END;
$$;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614050000_mostoles_restricciones_jarras.sql */
/* ===== */
-- =====
-- Mostoles – restricciones precisas de secciones de jarras/botella.
-- En cada sección se ocultan todos los productos EXCEPTO los aprobados.
-- =====

DO $$
DECLARE
    v_local UUID;
BEGIN
    SELECT id INTO v_local FROM public.locales WHERE slug = 'mostoles-constitucion';
    IF v_local IS NULL THEN
        RAISE EXCEPTION 'Local mostoles-constitucion no encontrado';
    END IF;

    -- — JARRAS HELADAS —————
    -- Solo: Jarra Cruzcampo Especial · Jarra Ladrón de Verano
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
    AND seccion = 'Jarras Heladas'
    AND NOT (
        nombre ILIKE '%cruzcampo%especial%'
        OR nombre ILIKE '%especial%cruzcampo%'
        OR nombre ILIKE '%ladr_n%verano%'
        OR nombre ILIKE '%verano%ladr_n%'
        OR nombre ILIKE '%ladron%verano%'
        OR nombre ILIKE '%verano%ladron%'
    )
    ON CONFLICT DO NOTHING;

    -- — CERVEZA PREMIUM —————
    -- Solo: Jarra Cruzcampo Radler · Jarra Ladrón de Manzanas · Jarra El Águila Dorada
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
    AND seccion = 'Cerveza Premium'
    AND NOT (
        nombre ILIKE '%cruzcampo%radler%'
        OR nombre ILIKE '%radler%cruzcampo%'
        OR nombre ILIKE '%ladr_n%manzana%'
        OR nombre ILIKE '%manzana%ladr_n%'
        OR nombre ILIKE '%ladron%manzana%'
        OR nombre ILIKE '%manzana%ladron%'
        OR nombre ILIKE '%_guila%dorada%'
        OR nombre ILIKE '%dorada%guila%'
    )
    ON CONFLICT DO NOTHING;

    -- — CERVEZA EN BOTELLA —————
    -- Solo: Heineken 0.0 · Paulaner
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
    AND seccion = 'Cerveza en Botella'
    AND NOT (
        nombre ILIKE '%heineken%'
        OR nombre ILIKE '%paulaner%'
    )
    ON CONFLICT DO NOTHING;

END;
$$;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614060000_activar_batido_chocolate.sql */
/* ===== */
-- Activa el batido de chocolate si estaba desactivado en la carta global.
UPDATE public.menu_productos
SET disponible = true
WHERE (nombre ILIKE '%batido%chocolate%' OR nombre ILIKE '%chocolate%batido%')

```

```

AND disponible = false;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614070000_mostoles_desbloquear_batido.sql */
/* ===== */
-- El producto se llama "Batido" (sin especificar sabor).
-- La restricción anterior lo bloqueó porque no contiene "chocolate" en el nombre.
-- Se elimina esa entrada para que el Batido vuelva a ser visible en Mostoles.

DELETE FROM public.local_restricciones
WHERE local_id = (SELECT id FROM public.locales WHERE slug = 'mostoles-constitucion')
AND producto_id IN (
    SELECT id FROM public.menu_productos
    WHERE nombre ILIKE '%batido%'
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614080000_mostoles_desbloquear_jarras_cruzcampo.sql */
/* ===== */
-- La restricción "tercios Cruzcampo" de la migración 040000 bloqueó también
-- las Jarras Cruzcampo (Especial y Radler), que SÍ deben estar disponibles.
-- Se eliminan solo las restricciones de productos Cruzcampo que son jarras.

DELETE FROM public.local_restricciones
WHERE local_id = (SELECT id FROM public.locales WHERE slug = 'mostoles-constitucion')
AND producto_id IN (
    SELECT id FROM public.menu_productos
    WHERE nombre ILIKE '%cruzcampo%'
    AND (
        nombre ILIKE '%jarra%'
        OR nombre ILIKE '%especial%'
        OR nombre ILIKE '%radler%'
    )
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260616090000_aperitivos_destino_bebidas.sql */
/* ===== */
--
-- FIX: aperitivos servidos en barra (aceitunas, gilda, cucurucho) deben ir a la
-- pista de "bebidas" (barra/caja), NO a cocina.
--
-- Causa raíz: resolve_pedido_item_destino() solo mandaba a 'bebidas' la categoría
-- 'Bebidas'. Los aperitivos están en la categoría 'Aperitivos', así que caían en
-- el ELSE → 'cocina'. Además el trigger set_pedido_item_destino_before_write pisa
-- el destino que envía el cliente con el resultado de esta función, por lo que
-- daba igual lo que mandara la app: el aperitivo siempre acababa en cocina.
--
-- Solución: añadir el reconocimiento de aperitivos por nombre en la función.
-- El trigger sync_pedido_sections (ya existente) recalcula estado_cocina /
-- estado_bebidas / tipo a partir del destino, así que el resto del flujo se
-- ajusta solo. No hace falta tocar nada más.
--
CREATE OR REPLACE FUNCTION public.resolve_pedido_item_destino(_producto_id uuid)
RETURNS text
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path TO 'public'
AS $$
    SELECT CASE
        WHEN mc.nombre = 'Bebidas' THEN 'bebidas'
        -- Aperitivos servidos en barra: se entregan en caja, igual que las bebidas.
        WHEN lower(mp.nombre) LIKE '%aceituna%'
            OR lower(mp.nombre) LIKE '%gilda%'
            OR lower(mp.nombre) LIKE '%cucurucho%' THEN 'bebidas'
        ELSE 'cocina'
    END
    FROM public.menu_productos mp
    LEFT JOIN public.menu_categorias mc ON mc.id = mp.categoria_id
    WHERE mp.id = _producto_id
$$;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260617120000_pedido_items_variante.sql */
/* ===== */
--
-- Variante del producto en el ítem del pedido (ej. "Jarra Quijote" / "Jarra
-- Sancho", "Boquerón" / "Anchoa", "BBQ" / "Brava", etc.)
--
-- Hasta ahora pedido_items solo guardaba producto_id, así que el dashboard
-- mostraba el nombre base y se perdía el tamaño/variante elegido por el cliente.
-- Añadimos una columna de texto libre para conservar esa elección.
--
ALTER TABLE public.pedido_items
ADD COLUMN IF NOT EXISTS variante text;

```

```

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260618120000_ingredientes_agotados.sql */
/* ===== */
--
-- SISTEMA DE INGREDIENTES Y "AGOTADOS" POR LOCAL
--
-- Objetivo: que el personal pueda marcar agotado:
-- (a) un INGREDIENTE (incluye los panes) -> los montaditos/ensaladas que lo
--     lleven aparecen en la app del cliente con aviso "AGOTADO" (no se ocultan).
-- (b) un PRODUCTO entero (aperitivos, raciones, bebidas, cookies, ruedas) ->
--     ese producto aparece con aviso "AGOTADO".
-- Todo es POR LOCAL (cada restaurante marca lo suyo).
--
-- Catálogo de ingredientes (tipo 'ingrediente' o 'pan')
CREATE TABLE IF NOT EXISTS public.ingredientes (
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    nombre      text NOT NULL UNIQUE,
    tipo        text NOT NULL DEFAULT 'ingrediente' CHECK (tipo IN ('ingrediente','pan')),
    created_at  timestamptz NOT NULL DEFAULT now()
);

-- Qué ingredientes lleva cada producto (solo montaditos + ensaladas)
CREATE TABLE IF NOT EXISTS public.producto_ingredientes (
    producto_id  uuid NOT NULL REFERENCES public.menu_productos(id) ON DELETE CASCADE,
    ingrediente_id uuid NOT NULL REFERENCES public.ingredientes(id) ON DELETE CASCADE,
    PRIMARY KEY (producto_id, ingrediente_id)
);

-- Ingredientes agotados por local (la fila existe = está agotado en ese local)
CREATE TABLE IF NOT EXISTS public.local_ingredientes_agotados (
    local_id     uuid NOT NULL REFERENCES public.locales(id) ON DELETE CASCADE,
    ingrediente_id uuid NOT NULL REFERENCES public.ingredientes(id) ON DELETE CASCADE,
    agotado_at   timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (local_id, ingrediente_id)
);

-- Productos enteros agotados por local (aperitivos/raciones/bebidas/cookies/ruedas)
CREATE TABLE IF NOT EXISTS public.local_productos_agotados (
    local_id     uuid NOT NULL REFERENCES public.locales(id) ON DELETE CASCADE,
    producto_id  uuid NOT NULL REFERENCES public.menu_productos(id) ON DELETE CASCADE,
    agotado_at   timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (local_id, producto_id)
);

CREATE INDEX IF NOT EXISTS idx_prod_ing_ingredient ON public.producto_ingredientes(ingrediente_id);
CREATE INDEX IF NOT EXISTS idx_local_ing_agot_local ON public.local_ingredientes_agotados(local_id);
CREATE INDEX IF NOT EXISTS idx_local_prod_agot_local ON public.local_productos_agotados(local_id);

-- --- RLS
ALTER TABLE public.ingredientes          ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.producto_ingredientes ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.local_ingredientes_agotados ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.local_productos_agotados  ENABLE ROW LEVEL SECURITY;

-- Lectura para todos (cliente anónimo + dashboard)
CREATE POLICY "ingredientes_read"          ON public.ingredientes          FOR SELECT USING (true);
CREATE POLICY "producto_ingredientes_read" ON public.producto_ingredientes FOR SELECT USING (true);
CREATE POLICY "local_ing_agotados_read"     ON public.local_ingredientes_agotados FOR SELECT USING (true);
CREATE POLICY "local_prod_agotados_read"    ON public.local_productos_agotados FOR SELECT USING (true);

-- Escritura (marcar/desmarcar agotado) solo para staff autenticado (dashboard)
CREATE POLICY "local_ing_agotados_write" ON public.local_ingredientes_agotados
    FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "local_prod_agotados_write" ON public.local_productos_agotados
    FOR ALL TO authenticated USING (true) WITH CHECK (true);

-- --- Realtime (para que la app del cliente se actualice al instante) ---
ALTER TABLE public.local_ingredientes_agotados REPLICA IDENTITY FULL;
ALTER TABLE public.local_productos_agotados    REPLICA IDENTITY FULL;
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_publication_tables WHERE pubname='supabase_realtime' AND tablename='local_ingredientes_agotados')
    THEN
        EXECUTE 'ALTER PUBLICATION supabase_realtime ADD TABLE public.local_ingredientes_agotados';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM pg_publication_tables WHERE pubname='supabase_realtime' AND tablename='local_productos_agotados') THEN
        EXECUTE 'ALTER PUBLICATION supabase_realtime ADD TABLE public.local_productos_agotados';
    END IF;
END $$;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260620120000_fix_precio_unitario_seguridad.sql */
/* ===== */
--
-- FIX DE SEGURIDAD (HIGH): manipulación de precios en el pago.
--
-- Problema: el cliente inserta pedido_items con `precio_unitario` arbitrario y
-- create-payment-intent recalcula el total a partir de ese valor. Un atacante

```

```

-- podía insertar un producto real con precio_unitario = 0.01 y pagar 1 céntimo.
--
-- Solución: forzar en la BD el precio real desde menu_productos.precio.
--   - Items SIN variante -> precio_unitario = precio del producto (autoritativo).
--   - Items CON variante -> no se permite por debajo del precio base
--                               (se conserva el recargo de la variante, p. ej. Sancho).
-- Se aplica en el servidor, así que ya no depende de lo que envíe el cliente.
--
CREATE OR REPLACE FUNCTION public.enforce_precio_unitario()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
    v_base numeric;
BEGIN
    SELECT precio INTO v_base FROM public.menu_productos WHERE id = NEW.producto_id;
    IF v_base IS NULL THEN
        RETURN NEW; -- producto inexistente: no debería ocurrir
    END IF;

    IF NEW.variante IS NULL THEN
        NEW.precio_unitario := v_base; -- precio autoritativo
    ELSE
        NEW.precio_unitario := GREATEST(COALESCE(NEW.precio_unitario, v_base), v_base); -- nunca por debajo del base
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_enforce_precio_unitario ON public.pedido_items;
CREATE TRIGGER trg_enforce_precio_unitario
BEFORE INSERT OR UPDATE OF precio_unitario, producto_id ON public.pedido_items
FOR EACH ROW
EXECUTE FUNCTION public.enforce_precio_unitario();

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260620130000_security_hardening.sql */
/* ===== */
-- SECURITY HARDENING - 2026-06-20
-- SEC-003: secreto de push fuera del código (tabla protegida app_secrets)
-- SEC-004: rate limit por sesión/IP en chat-menu
-- SEC-006: escrituras de "agotados" acotadas al local del staff
-- SEC-008: la consulta a franchisees devuelve [] en vez de error que filtra
--           el nombre de una función interna
-- =====
-- — SEC-003: secreto de push en tabla protegida (nunca en el repo) —
CREATE TABLE IF NOT EXISTS public.app_secrets (
    key    text PRIMARY KEY,
    value  text NOT NULL
);
ALTER TABLE public.app_secrets ENABLE ROW LEVEL SECURITY;
-- Sin políticas => anon/authenticated NO pueden leer ni escribir vía API.
REVOKE ALL ON public.app_secrets FROM anon, authenticated;

-- Placeholder. El valor REAL se pone con un UPDATE aparte (no versionado):
-- UPDATE public.app_secrets SET value = '<secreto-real>' WHERE key = 'push_notify_secret';
INSERT INTO public.app_secrets(key, value)
VALUES ('push_notify_secret', 'REEMPLAZAR_POR_SECRETO_REAL')
ON CONFLICT (key) DO NOTHING;

-- El trigger lee el secreto de la tabla (SECURITY DEFINER ignora RLS).
CREATE OR REPLACE FUNCTION public.notify_order_listo()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_secret TEXT;
    v_fired  BOOLEAN := FALSE;
BEGIN
    SELECT value INTO v_secret FROM public.app_secrets WHERE key = 'push_notify_secret';

    IF NEW.estado_cocina = 'listo' AND OLD.estado_cocina IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;
    IF NEW.estado_bebidas = 'listo' AND OLD.estado_bebidas IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;
    IF NEW.estado        = 'listo' AND OLD.estado        IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;

    IF v_fired THEN
        BEGIN
            PERFORM net.http_post(
                url      := 'https://znmhqnmmtkkaillwfscu.supabase.co/functions/v1/notify-order-ready',
                headers  := jsonb_build_object('Content-Type', 'application/json', 'x-push-notify-secret', v_secret),
                body     := jsonb_build_object('pedido_id', NEW.id, 'session_id', NEW.session_id, 'numero_pedido', NEW.numero_pedido)::text
            );
        EXCEPTION WHEN OTHERS THEN
            NULL; -- push best-effort: no debe abortar el UPDATE del pedido
        END;
    END IF;
END IF;

```

```

RETURN NEW;
END; $$;

-- -- SEC-004: rate limit (chat-menu) -----
CREATE TABLE IF NOT EXISTS public.chat_rate_limit (
    key          text PRIMARY KEY,
    count        int  NOT NULL DEFAULT 0,
    window_start timestamptz NOT NULL DEFAULT now()
);
ALTER TABLE public.chat_rate_limit ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON public.chat_rate_limit FROM anon, authenticated;

-- Devuelve TRUE si se permite la petición (y la contabiliza), FALSE si se supera.
CREATE OR REPLACE FUNCTION public.check_rate_limit(p_key text, p_limit int, p_window_secs int)
RETURNS boolean LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_count int;
    v_start timestamptz;
BEGIN
    SELECT count, window_start INTO v_count, v_start
    FROM public.chat_rate_limit WHERE key = p_key FOR UPDATE;

    IF NOT FOUND OR now() - v_start > make_interval(secs => p_window_secs) THEN
        INSERT INTO public.chat_rate_limit(key, count, window_start) VALUES (p_key, 1, now())
        ON CONFLICT (key) DO UPDATE SET count = 1, window_start = now();
        RETURN true;
    END IF;

    IF v_count >= p_limit THEN
        RETURN false;
    END IF;

    UPDATE public.chat_rate_limit SET count = count + 1 WHERE key = p_key;
    RETURN true;
END; $$;
REVOKE ALL ON FUNCTION public.check_rate_limit(text,int,int) FROM PUBLIC, anon, authenticated;
-- service_role (que usa la edge function) conserva permiso de ejecución.

-- -- SEC-006: escrituras de agotados acotadas al local del staff -----
DROP POLICY IF EXISTS "local_ing_agotados_write" ON public.local_ingredientes_agotados;
CREATE POLICY "local_ing_agotados_write" ON public.local_ingredientes_agotados
    FOR ALL TO authenticated
    USING (public.has_role_for_local(auth.uid(),'caja',local_id) OR public.has_role_for_local(auth.uid(),'cocina',local_id))
    WITH CHECK (public.has_role_for_local(auth.uid(),'caja',local_id) OR public.has_role_for_local(auth.uid(),'cocina',local_id));

DROP POLICY IF EXISTS "local_prod_agotados_write" ON public.local_productos_agotados;
CREATE POLICY "local_prod_agotados_write" ON public.local_productos_agotados
    FOR ALL TO authenticated
    USING (public.has_role_for_local(auth.uid(),'caja',local_id) OR public.has_role_for_local(auth.uid(),'cocina',local_id))
    WITH CHECK (public.has_role_for_local(auth.uid(),'caja',local_id) OR public.has_role_for_local(auth.uid(),'cocina',local_id));

-- -- SEC-008: franchisees -> [] en vez de "permission denied for function ..." --
-- La función devuelve NULL para anon (sin sesión), así que la policy no expone datos.
GRANT EXECUTE ON FUNCTION public.get_user_franchisee_id(uuid) TO anon;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260623200000_vista_pedidos_agregados.sql */
/* ===== */
--
-- FIX: el Histórico de caja consulta la vista public.v_pedidos_agregados que
-- no existía (error PGRST205 "Could not find the table ... in the schema cache").
-- Esta vista agrega los pedidos por hora/día/mes/año (en horario Europe/Madrid)
-- con nº de tickets e ingresos, que es justo lo que pide Historico.tsx.
--
CREATE OR REPLACE VIEW public.v_pedidos_agregados AS
SELECT
    date_trunc('hour', created_at AT TIME ZONE 'Europe/Madrid') AT TIME ZONE 'Europe/Madrid' AS hora,
    date_trunc('day', created_at AT TIME ZONE 'Europe/Madrid') AT TIME ZONE 'Europe/Madrid' AS dia,
    date_trunc('month', created_at AT TIME ZONE 'Europe/Madrid') AT TIME ZONE 'Europe/Madrid' AS mes,
    date_trunc('year', created_at AT TIME ZONE 'Europe/Madrid') AT TIME ZONE 'Europe/Madrid' AS anio,
    estado,
    count(*)::int AS tickets,
    coalesce(sum(total), 0)::numeric AS ingresos
FROM public.pedidos
GROUP BY 1, 2, 3, 4, 5;

-- Solo el personal autenticado (dashboard) puede leer los agregados.
GRANT SELECT ON public.v_pedidos_agregados TO authenticated;

```